HANDNOTE SERIES · SYSTEM DESIGN

High Level Design

Load balancers, CDNs, caches, database scaling, queues, object storage and observability — with the arithmetic.

118 topics · 11 parts · self-hosted and AWS, side by side
practice sets, worked answer keys, one-page revision cards

Contents

PART ONE

Foundations and the numbers

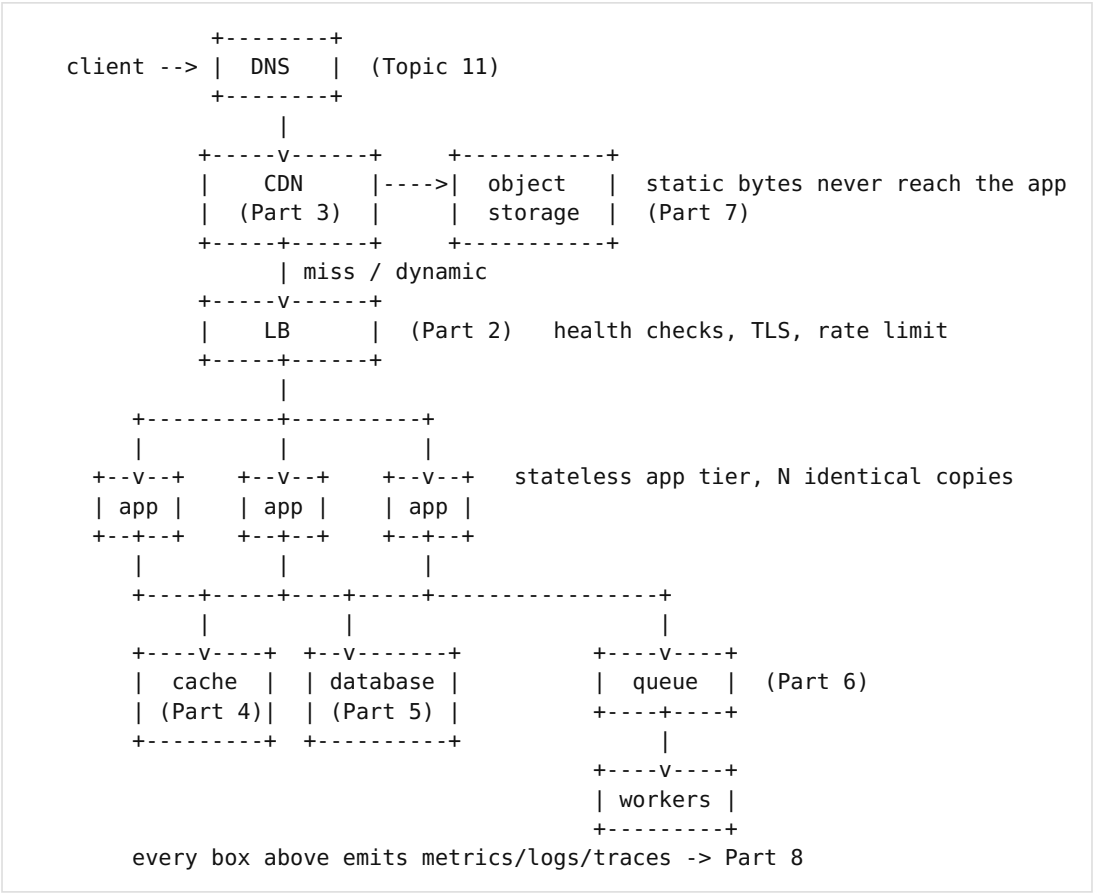
Every later part is an application of eight numbers: how long things take, how much traffic there is, how many bytes it becomes, what it costs to move them, and how often the whole thing is allowed to be down. A design without arithmetic is a drawing.

Part 1 · Foundations and the numbers

1. What high level design is (and is not)

High level design (HLD) decides *which boxes exist, what runs in each, and what flows between them*. Low level design (LLD) decides what happens inside one box: classes, interfaces, schemas at field level. HLD answers "where does this request go and what is the slowest thing it touches"; LLD answers "which class owns this method". You already do LLD daily. HLD is the same discipline applied to processes and machines instead of objects.

Almost every web system is a specialisation of one baseline. Learn the baseline, then learn what each part exists to fix.



Read that diagram as a cost gradient. Left to right, each hop is roughly 10× more expensive than the one before: a CDN hit is ~20 ms and $0.00, a cache hit ~1 ms of server time, a database query 5–50 ms and a finite connection slot, a cross-shard query 100 ms+. The whole book is about pushing work leftward.

> **THE KEY INSIGHT**
> HLD is not drawing boxes. It is deciding, with numbers, which box absorbs which load — and naming the one that fails first.

THE TRAP

Adding components because they appear in reference architectures. Symptom: a five-service, Kafka-backed, sharded design serving 40 requests per second, where one Postgres instance and a Valkey cache would have done it with 3% CPU. Every box you add multiplies failure modes (Topic 6) and on-call surface. The strongest answer in an interview is often "at this scale we do not need it yet, and here is the load at which we would".

```
HLD vs LLD — GAINS: shared vocabulary for capacity and failure · COSTS: nothing until you build a box · ACCEPTS: coarse granularity; per-class design deferred to LLD
```

2. The latency numbers every design obeys

Every design decision in this book reduces to moving work from a slow row of this table to a fast one. Memorise the orders of magnitude, not the digits.

Operation	Typical	What it implies
L1/L2 cache reference	0.5–7 ns	Free; never a design factor
Main memory reference	100 ns	A Valkey GET is ~200× this, all network
Mutex lock/unlock, uncontended	25 ns	Contention, not the lock, is the cost
NVMe SSD random read (4 KB)	50–150 µs	~10k–100k IOPS per device
Rotational disk seek	5–10 ms	Why B-trees exist; still true for cold archive
Same-AZ network round trip	0.25–0.5 ms	A cache call costs ~0.5 ms of pure RTT
Cross-AZ round trip	0.5–2 ms	Sync cross-AZ replication is affordable
Cross-region (Mumbai–Frankfurt)	110–130 ms	Sync cross-region writes are not
Round the world, fibre	~200 ms	Speed of light: 2/3 c in glass, hard floor
TLS 1.3 handshake (1-RTT)	1 × RTT	Topic 27: why the edge terminates it
Typical Postgres point query	0.2–2 ms	Plus pool wait, which is the real variable
Dhaka → Singapore RTT	~60 ms	Region choice costs more than most code

```
one user-visible page load, 12 assets, no CDN, Dhaka -> us-east-1 (RTT 230 ms)

  DNS 1xRTT | TCP 1xRTT | TLS 1xRTT | HTML 1xRTT | 12 assets / 6 conns = 2xRTT
  ------------------------------------------------------------------------
  = 6 x 230 ms = 1380 ms before the first pixel is painted

same page, CDN PoP in Dhaka (RTT 8 ms), assets cached at edge

  = 6 x 8 ms = 48 ms   (28x better, and the app tier served zero requests)
```

THE KEY INSIGHT

Latency is dominated by *round trips*, not by compute. Count RTTs first; optimise CPU last. Distance is the one variable no code change fixes.

THE TRAP

Optimising a 2 ms query while the request makes 30 sequential calls of 0.5 ms each (the N+1 pattern, in network form). Symptom: flame graph shows no hot function, yet p50 is 40 ms and CPU sits at 8%. Fix by batching — one call returning 30 rows, not 30 calls. See Topic 87 for how a trace makes this visible in one screen.

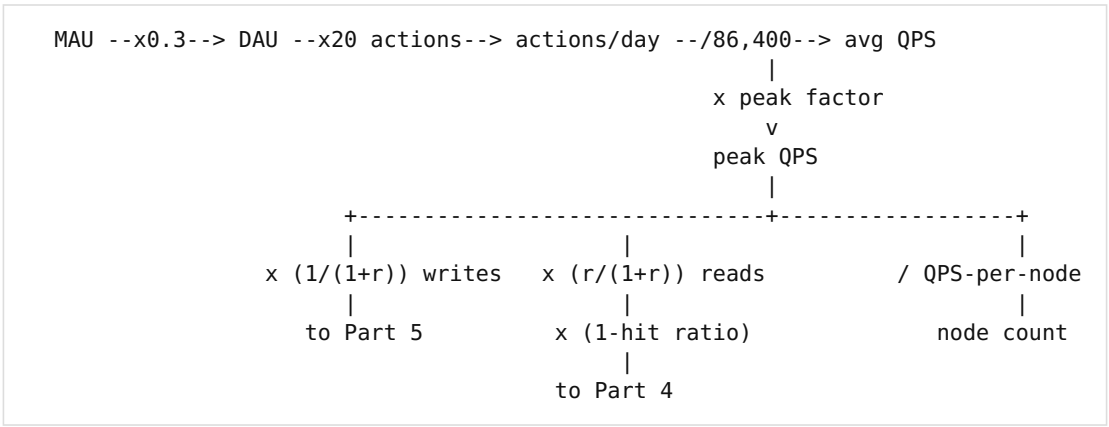
```
latency table — GAINS: rules out designs in seconds · COSTS: memorisation · ACCEPTS: numbers
drift ~2× per decade; orders of magnitude do not
```

3. Estimating QPS from users

Every capacity conversation starts by converting a business number (users) into an engineering number (queries per second). The chain is always the same, and you should be able to run it aloud in 60 seconds.

```
Given:  10,000,000 MAU
        DAU/MAU ratio                = 0.30      -> 3,000,000 DAU
        actions per active user/day  = 20        -> 60,000,000 actions/day
        read:write ratio             = 100:1

seconds per day = 86,400  (memorise as ~10^5)

average QPS = 60,000,000 / 86,400  = 694 QPS
peak factor = 3x  (diurnal; 5-10x for ticket-on-sale or flash events)
peak QPS    = 694 x 3              = 2,082 QPS

  reads  = 2,082 x (100/101)       = 2,062 QPS
  writes = 2,082 x (1/101)         =    21 QPS
```

Now size it. If one app instance handles 200 QPS at 60% CPU (measure this; do not guess), 2,082 / 200 = 11 instances, plus one for headroom and one for the rolling deploy = **13**. The write rate of 21 QPS is trivial for a single Postgres primary; the read rate of 2,062 QPS is not, which is exactly why Parts 3 and 4 exist — and why the first question after this estimate is "what fraction of those reads is cacheable?" At a 90% cache hit ratio the database sees 206 QPS, which one primary handles comfortably.



THE KEY INSIGHT

Peak QPS, not average, sizes the fleet; the cache hit ratio, not the request count, sizes the database. State both assumptions aloud — interviewers are grading the assumptions, not the arithmetic.

THE TRAP

Using a 2–3× diurnal peak factor for a system with a scheduled event. A bus-ticket platform selling Eid-period seats does not have a diurnal curve; it has a step function at 09:00 on release day — 40× average for eight minutes. Symptom: an autoscaler that reacts in 90 s (Topic 104) while the entire sale completes in 300 s. Size for the event, or queue it (Topic 60).

```
QPS estimation — GAINS: instance count and DB load in one minute · COSTS: 3 assumptions you must defend · ACCEPTS: ±2× error, which is fine — the decision boundaries are 10× apart
```

4. Estimating storage and its growth

Storage is the one number that only goes up. The formula is `bytes/record × records/day × retention days × replication factor`, and the replication factor is what people forget.

```
Ticketing platform, per booking row:
    ids + timestamps + status + fk columns   ~ 200 B
    index overhead (3 secondary indexes)     ~ 150 B
    row total                                ~ 350 B

50,000 bookings/day x 350 B          = 17.5 MB/day
x 365                                = 6.4 GB/year   <-- OLTP, tiny

Same platform, per booking PDF ticket + QR:
    120 KB x 50,000/day              = 6.0 GB/day
    x 365                            = 2.19 TB/year  <-- object storage
    x 3 (S3 stores across >=3 AZs)   = provider's problem, you pay for 2.19 TB

Audit/event log, 1 KB per event, 12 events per booking:
    50,000 x 12 x 1 KB               = 600 MB/day
    x 90-day retention               = 54 GB hot
    x 3 (Kafka replication factor)   = 162 GB of broker disk
```

The lesson is visible immediately: the transactional database is 6 GB after a year and will never need sharding; the blobs are 2 TB and belong in object storage (Part 7); the event stream is the only thing whose replication factor you pay for directly. Three different growth curves, three different homes. Deciding this on day one is most of what "designing for scale" means.

THE KEY INSIGHT

Separate the data by growth rate, not by subject. Rows grow linearly and slowly; blobs grow 100–1000× faster per event; logs grow fastest and are worth the least. Put each where its curve is cheapest.

THE TRAP

Storing files as `LONGBLOB`/`bytea` in the relational database "for transactional consistency". Symptom: a 40 GB table where `mysqldump` takes four hours, every replica carries every byte, and the buffer pool is evicted by one user downloading their PDFs. Store the bytes in object storage and the key in the row — then handle the orphan problem honestly (Topic 81).

```
storage estimation — GAINS: names which store each dataset belongs in · COSTS: must know bytes/record, which requires a schema · ACCEPTS: index overhead estimates are ±50%
```

5. Bandwidth, egress and what it costs

Bandwidth is QPS × payload size, and it is the number that turns into an invoice. Compute and storage are cheap; moving bytes out of a cloud region is not.

```
peak 2,082 QPS x 40 KB average response
    = 83.3 MB/s = 666 Mbps sustained at peak
    = 2,082 x 40 KB x 86,400 x (1/3 duty)  ~ 2.4 TB/day  ~ 72 TB/month

AWS egress to internet, first 10 TB tier ~ $0.09/GB (varies by region):
    72,000 GB x $0.09                       = $6,480/month
Same bytes served from CloudFront (~$0.085/GB, and origin->CDN is free):
    at 92% cache hit ratio, origin egress   = 5,760 GB x $0.09 = $518
    CDN egress                              = 72,000 x $0.085  = $6,120
    -> CDN saves latency, not much money, unless commitments apply

Cross-AZ traffic (both directions billed, ~$0.01-0.02/GB):
    a chatty app <-> cache <-> db mesh at 3 hops x 40 KB x 2,082 QPS
    = 250 MB/s cross-AZ = 648 TB/month = $6,480-$13,000/month
    -> AZ-affinity routing is a five-figure decision, not a nicety
```

Self-hosted comparison: a 1 Gbps unmetered dedicated line in Dhaka or Singapore costs $200–800/month flat and carries the same 72 TB. That gap — roughly 10× — is why bandwidth-heavy businesses (video, image hosting, backups) either self-host, use a flat-rate provider, or negotiate committed-use discounts. It is also why Cloudflare's zero-egress pricing on R2 is a genuine architectural input, not marketing.

THE KEY INSIGHT

Ingress is free almost everywhere; egress is where the bill lives. Design so that bytes leave the region once and are cached everywhere after (Part 3), and so that chatty service-to-service traffic stays inside one AZ.

THE TRAP

Sizing links on average throughput. Symptom: a 1 Gbps link at 30% average that drops packets for six minutes every evening, appearing in dashboards as random p99 spikes and TCP retransmits — never as "bandwidth". Size on peak plus 40%, and alert on 95th-percentile utilisation, not mean.

```
bandwidth — GAINS: predicts the invoice and the link size · COSTS: needs a payload-size
measurement · ACCEPTS: compression changes it 2–5×; measure post-gzip
```

6. Availability arithmetic: nines and dependencies

Availability is a percentage of time the system meets its contract. The reason to do the arithmetic is that dependencies *multiply,* and most engineers are surprised by how fast a chain of good components becomes a bad system.

Nines	Downtime/year	Per month	Reality
99% (two)	3.65 days	7.2 hours	A hobby project
99.9% (three)	8.77 hours	43.8 min	Typical single-region SaaS
99.95%	4.38 hours	21.9 min	Common paid SLA (ALB, RDS Multi-AZ)
99.99% (four)	52.6 min	4.4 min	Multi-AZ, automated failover, no manual step
99.999% (five)	5.26 min	26 s	Multi-region active-active; very expensive

```
SERIAL dependencies (request needs all of them) -- probabilities multiply:
    LB 99.99% x app 99.95% x cache 99.9% x DB 99.95%
    = 0.9999 x 0.9995 x 0.999 x 0.9995 = 0.99790  = 99.79%
    -> 18.4 hours/year, worse than every component in it

PARALLEL redundancy (any one suffices) -- failure probabilities multiply:
    two DB replicas at 99.9% each:  1 - (0.001 x 0.001) = 99.9999%
    ...but only if failover is automatic AND correlated failure is excluded.
    Same rack, same AZ, same bad deploy => correlation ~1, and you have 99.9%.

Make the cache non-critical (serve on miss instead of erroring):
    LB x app x DB = 0.9999 x 0.9995 x 0.9995 = 99.89%   (+0.10 pt, ~9 h/year)
```

THE KEY INSIGHT

Every component on the critical path subtracts availability. The cheapest availability win is not adding redundancy — it is moving a component *off* the critical path, so its failure degrades the response instead of ending it (Topic 99).

THE TRAP

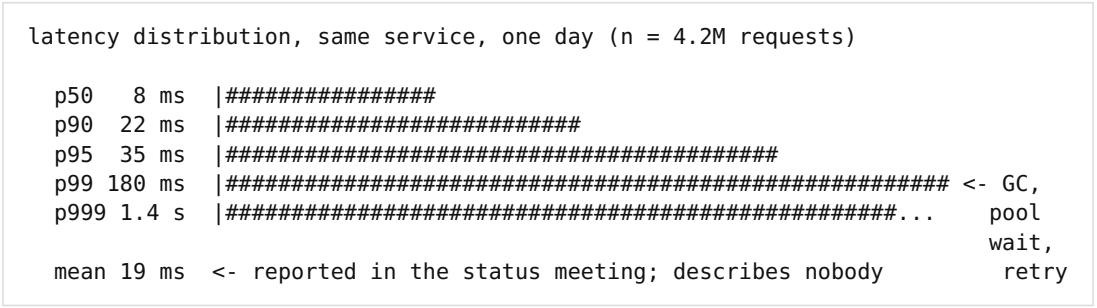
Assuming independence. Two replicas behind one power feed, one switch, one config push or one expiring certificate fail together, so the parallel formula lies by three orders of magnitude. Symptom: the postmortem sentence "both replicas were unavailable simultaneously, which our model said was a 1-in-1000 year event." It was a shared-fate event, and it was your model that was wrong.

```
availability math — GAINS: predicts real uptime from component SLAs · COSTS: each nine roughly doubles infrastructure and process cost · ACCEPTS: correlated failure breaks the parallel formula entirely
```

7. Tail latency and fan-out amplification

The average is a lie told by the fast requests. What users experience — and what SLOs are written against (Topic 92) — is p95, p99 and p99.9. The arithmetic that makes this urgent is fan-out: if one page needs 20 backend calls and each has a 1% chance of being slow, the chance the page is slow is not 1%.

```
P(page is fast) = (1 - 0.01)^20 = 0.99^20 = 0.8179
P(page hits at least one p99 tail)      = 18.2%

fan-out 100 (a feed assembling 100 items):  1 - 0.99^100 = 63.4%
    -> nearly two in three page loads touch a p99 backend response

Fix 1: reduce fan-out                 20 calls -> 1 batch call: 1%
Fix 2: hedged request (send a 2nd copy at p95, take the first answer)
       tail becomes ~p95 of p95; costs ~5% extra load
Fix 3: return partial results at a deadline (Topic 96, 99)
```



THE KEY INSIGHT

Tail latency is amplified by fan-out and by retries, and it is caused by queueing — connection pool waits, GC pauses, disk contention — not by slow code. Look for a queue before you look at a profiler.

THE TRAP

Averaging percentiles across instances or over time. The mean of ten servers' p99 values is not the fleet p99, and it can be off by 3×. Symptom: the dashboard says p99 = 120 ms while a single bad host serves 900 ms to 8% of users. Aggregate from histograms (Topic 85), never from pre-computed percentiles.

```
tail latency — GAINS: matches the number users feel · COSTS: histograms cost cardinality (Topic
84); hedging costs ~5% extra load · ACCEPTS: p99.9 is expensive to move and often not worth it
```

8. Stateless services, and CAP as a decision tool

A service is stateless when any instance can serve any request, because everything needed is in the request or in a shared store. This single property is what makes horizontal scaling, rolling deploys, autoscaling and instance death boring instead of dangerous. Almost every scaling technique later in the book assumes it.

```
STATEFUL (breaks at instance #2)          STATELESS (scales to N)
  $_SESSION on local disk                   session in Valkey / signed JWT
  in-process job scheduler                  queue with visibility timeout (T66)
  uploaded file on local /var/www           object storage key (T77)
  in-memory rate-limit counter              shared counter with TTL (T103)
  SSE/WebSocket bound to one node           connection registry + pub/sub (T116)
```

CAP: when a network partition (P) splits your nodes, you must choose between consistency (C: refuse the request rather than serve stale or divergent data) and availability (A: answer anyway). It is a choice you make *per operation*, not per company. PACELC adds the part that

matters more in practice: *else*, when there is no partition, you still trade Latency against Consistency — that is the entire content of Topics 46–48.

SCENARIO

A ferry-booking cluster splits: Dhaka nodes cannot reach the Chattogram database. **Seat reservation** chooses C — refusing to sell is recoverable, double-selling seat 14A is not; the API returns 503 and the customer retries in 30 s. **Schedule browsing** chooses A — serving a 5-minute-stale timetable from cache is obviously better than an error page. Same partition, same system, opposite answers, decided by the cost of being wrong.

THE KEY INSIGHT

"Is it CP or AP?" is the wrong question. Ask it per operation: what does a wrong answer cost here versus what does no answer cost? Reads usually pick A, money and inventory usually pick C.

THE TRAP

Believing you are stateless because the code has no globals. Symptom: after scaling from one instance to three, users are randomly logged out and uploads "disappear" — PHP sessions on local disk and files in `storage/app/public` on one machine's filesystem. Test for it by killing the instance that served the last request and replaying it elsewhere.

```
statelessness — GAINS: horizontal scale, rolling deploys, cheap instance death · COSTS: one network hop per session/state read (~0.5 ms), a shared store to run · ACCEPTS: the shared store becomes the new critical dependency (Topic 6)
```

PRACTICE SET 1 — Topics 1–8

A. Conceptual

1. State the difference between HLD and LLD in one sentence that mentions neither "high" nor "low".
2. Why does adding a cache to the critical path *lower* availability, and what single change reverses that?
3. A colleague says "we are CP". What is wrong with the statement as an architectural claim?
4. Why does peak QPS size the fleet while the cache hit ratio sizes the database?
5. Explain why cross-AZ synchronous replication is affordable and cross-region synchronous replication is not, using one number.
6. Give two reasons the mean latency of a service can improve while user experience gets worse.
7. Name three things that make an apparently stateless PHP application stateful in practice.
8. Why is ingress free and egress expensive at every major cloud? Answer in terms of incentives, not physics.
9. What is the difference between a 3× diurnal peak factor and the peak factor for a scheduled ticket release, and what does that change in the design?
10. When is a 99.99% component behind a 99.9% component a waste of money?

B. Pen-and-paper estimation

1. A platform has 4,000,000 MAU, DAU/MAU 0.25, 12 actions per active user per day, read:write 50:1, peak factor 4. Compute average QPS, peak QPS, peak read QPS and peak write QPS.
2. Continuing (B1): each app instance sustains 250 QPS at 65% CPU. How many instances, including one for headroom and one for a rolling deploy?
3. Continuing (B1): the cache achieves an 88% hit ratio on reads. What read QPS reaches the database? If one replica handles 800 QPS, how many replicas?
4. Each response averages 25 KB post-gzip. Compute peak Mbps and monthly TB, assuming peak traffic for 8 hours a day and 25% of peak for the other 16.
5. Compute the serial availability of LB 99.99% × gateway 99.95% × app 99.95% × cache 99.9% × primary DB 99.95%. Convert to minutes of downtime per month. Then recompute with the cache made non-critical.
6. A page issues 35 backend calls. Each backend has p99 = 250 ms and p50 = 6 ms. What fraction of page loads touches at least one p99 response? What does batching into 4 calls change it to?
7. Bookings: 80,000/day, 400 B per row including index overhead, plus one 180 KB PDF each, plus 15 audit events of 900 B. Give one-year row storage, one-year object storage, and 60-day audit-log storage at Kafka replication factor 3.
8. A request path makes 4 cross-AZ hops of 45 KB each at 1,500 QPS. Compute monthly cross-AZ GB and the bill at $0.02/GB (both directions billed).

C. Design tasks (build → break → observe → fix)

1. Draw the five-box baseline from Topic 1 for a bus-ticket search page, and annotate every arrow with a latency from Topic 2. Total the critical path.

2. **Break it:** run a single-instance app with file-based sessions behind two containers and an nginx round-robin upstream. Log in, then refresh repeatedly. Observe the logout. Now move sessions to Valkey and repeat. Record exactly how many refreshes it took to see the failure and why it was not every refresh.
3. **Break it:** take any endpoint and add a hard dependency on a cache with no fallback (throw on connection error). Stop the cache. Observe the error rate go to 100% and compute the availability you just bought yourself. Change the code to fall back to the database on cache error, stop the cache again, and record the new latency and error rate.
4. Measure your own numbers: on your machine, time 10,000 sequential Valkey GETs and 10,000 sequential single-row MySQL selects. Report both means and the ratio. Compare against Topic 2's table and explain any difference greater than 3×.
5. Write the QPS estimate for a system you have actually built, then instrument it and compare. Report the ratio of estimate to measurement and which assumption was most wrong.
6. **Break it:** configure a service with an average-sized network link (use `tc qdisc` to cap egress at your computed average Mbps rather than peak). Drive peak traffic. Observe retransmits and p99 without any change in CPU, then raise the cap to peak × 1.4.

ANSWER KEY 1 — Topics 1–8

A. Conceptual

1. **HLD decides which processes exist and what flows between them; LLD decides what happens inside one process.** The boundary is the network call.
2. **Because serial dependencies multiply** (Topic 6): a 99.9% cache on the critical path caps the system at 99.9% regardless of everything else. Reverse it by catching cache errors and reading through to the database — the cache becomes a latency optimisation, not an availability dependency.
3. CAP applies **per operation during a partition**, not to a company or a database product. The same system is CP for seat reservation and AP for timetable browsing (Topic 8).
4. Every request hits an app instance, so the fleet must absorb peak. Only cache *misses* hit the database, so **DB load = peak read QPS × (1 − hit ratio)**; a hit ratio change from 90% to 95% halves database load.
5. **Cross-AZ RTT is ~0.5–2 ms; cross-region is 110–130 ms.** Adding 1 ms to a write is invisible; adding 120 ms to every write is a redesign.
6. (i) A flood of fast cache hits or 304s pulls the mean down while p99 is unchanged; (ii) fan-out means each page touches many requests, so per-request mean is irrelevant to per-page experience (Topic 7).
7. Local-disk sessions, uploads on local filesystem, in-process schedulers or in-memory counters/rate-limits (Topic 8).
8. Egress pricing raises switching costs: data is cheap to put in and expensive to take out, which discourages multi-cloud and migration. Physics does not distinguish direction on a duplex link.
9. Diurnal peak is a smooth 3× over hours, absorbable by autoscaling. A release is a step function — 40× for minutes — faster than autoscaler reaction (Topic 104). You must pre-provision or place a queue in front (Topic 60), and often both.
10. When the 99.99% component sits **serially behind** the 99.9% one: the product is 99.89%, so you paid for a nine you cannot observe. Spend it on the weakest serial link instead.

B. Estimation — worked

1. DAU = 4,000,000 × 0.25 = **1,000,000**. Actions/day = 12M. Average QPS = 12,000,000 / 86,400 = **138.9**. Peak = ×4 = **555.6**. Reads = 555.6 × 50/51 = **544.7**; writes = 555.6/51 = **10.9**.
2. 555.6 / 250 = 2.22 → 3 instances to serve, + 1 headroom + 1 deploy = **5**.
3. 544.7 × 0.12 = **65.4 QPS** to the database. 65.4 / 800 = 0.08 → **1 replica** is ample; you are nowhere near needing read scaling, which is the correct conclusion to state out loud.
4. Peak bytes/s = 555.6 × 25 KB = 13.9 MB/s = **111 Mbps**. Daily = 13.9 MB/s × (8×3600) + 3.47 MB/s × (16×3600) = 400 GB + 200 GB = 600 GB/day → **18 TB/month**.
5. 0.9999 × 0.9995 × 0.9995 × 0.999 × 0.9995 = **0.99740 = 99.74%** → 0.0026 × 43,800 min/month = **113.9 min/month**. Without the cache in the chain: 0.99840 = **99.84%** → **70.1 min/month**. One code change bought 44 minutes a month.
6. $1 - 0.99^{35} = 1 - 0.7034 =$ **29.7%**. With 4 calls: $1 - 0.99^{4} =$ **3.9%**. Batching cut tail-exposure 7.6× without making any backend faster.
7. Rows: 80,000 × 400 B × 365 = **11.7 GB/year**. Objects: 80,000 × 180 KB = 14.4 GB/day × 365 = **5.26 TB/year**. Audit: 80,000 × 15 × 900 B = 1.08 GB/day × 60 = 64.8 GB × 3 = **194 GB of broker disk**.

8. 45 KB × 1,500 × 4 = 270 MB/s. × 86,400 = 23.3 TB/day → **700 TB/month** → at $0.02/GB = **~$14,000/month**. This is why AZ-affinity exists.

C. What to verify

1. Critical path should total roughly: DNS 0 (cached) + TLS 1×RTT + LB 0.3 ms + app 4 ms + cache 0.6 ms + DB 3 ms ≈ **8–10 ms server-side**, dominated by client RTT.
2. Expect the logout **on the first request routed to the other container** — with round-robin and browser keep-alive, that is often the 2nd or 3rd refresh, not the 1st, because one TCP connection stays pinned to one upstream. That pinning is exactly the illusion that hides the bug in staging. After moving to Valkey: **zero logouts over 100 refreshes.**
3. Error rate **100%**, availability now capped at the cache's — you converted an optimisation into a dependency. After the fallback: error rate **0%**, p50 rises from ~1 ms to the database figure (commonly 3–15 ms), and database QPS rises by 1/(1−hit ratio). Watch for the stampede in Topic 38 — you may see the database saturate for the first few seconds.
4. Expect Valkey **0.15–0.4 ms** per GET over loopback and MySQL **0.3–1.5 ms** per indexed point select, so a ratio of roughly **2–5×**, not the 100× people expect — over loopback both are dominated by round trip and parsing, not by storage. The 100× gap appears only when the query is not a point lookup or the buffer pool misses to disk.
5. Most commonly wrong assumption: **actions per user per day**, usually overestimated 2–5×. Second: DAU/MAU.
6. Expect throughput to plateau at the cap while CPU stays flat, TCP retransmits climb, and p99 rises 5–20× with p50 nearly unchanged — the signature of a queue, not of slow code (Topic 7).

HANDNOTE SUMMARY CARD — PART 1

```
FOUNDATIONS: THE EIGHT NUMBERS
================================================================================
ESTIMATION CHAIN
  MAU x (DAU/MAU) x actions/day / 86,400        = avg QPS
  avg QPS x peak factor (3x diurnal, 10-40x event) = peak QPS
  peak QPS x r/(1+r) = reads    x 1/(1+r) = writes
  DB read load = peak reads x (1 - hit ratio)
  instances    = peak QPS / QPS-per-node + 1 headroom + 1 deploy
  storage      = bytes/record x records/day x retention x replication
  bandwidth    = QPS x payload;  egress $/GB is the invoice

LATENCY TABLE (memorise orders of magnitude)
  L1 0.5 ns | RAM 100 ns | NVMe 4K read 50-150 us | disk seek 5-10 ms
  same-AZ RTT 0.5 ms | cross-AZ 1-2 ms | cross-region 110-130 ms
  round-world 200 ms | TLS1.3 1 RTT | PG point query 0.2-2 ms
  RULE: count round trips first, optimise CPU last

AVAILABILITY
  serial:   A1 x A2 x A3 ...        (worse than every part)
  parallel: 1 - (1-A1)(1-A2)        (only if failures are independent)
  99.9% = 43.8 min/mo | 99.95% = 21.9 | 99.99% = 4.4 | 99.999% = 26 s
  cheapest nine = move a component OFF the critical path

TAIL
  P(slow page) = 1 - (1 - p)^fanout
    fanout 20 @ 1% = 18.2% | fanout 100 @ 1% = 63.4%
  fixes: batch (fanout down) | hedge at p95 (+5% load) | deadline + partial
  never average percentiles across hosts; aggregate histograms

STATELESS CHECKLIST
  session -> shared store    upload -> object store    cron -> queue
  counter -> shared+TTL      socket -> registry+pubsub
  test: kill the node that served the last request, replay it elsewhere

CAP / PACELC
  partition -> choose C (refuse) or A (answer stale), PER OPERATION
  no partition -> still trading Latency vs Consistency (Topics 46-48)
  money/inventory -> C     browsing/feeds -> A

TRAPS
  boxes without arithmetic | average instead of peak | assuming independence
  blobs in the RDBMS | cache as a hard dependency | mean instead of p99
================================================================================
```

PART TWO

Load balancer

The box that makes "add another server" possible, and the box that decides which server dies first. Everything here is a scheduling decision made 2,000 times a second with incomplete information.

Part 2 · Load balancer

9. What a load balancer actually does

A load balancer accepts client connections and forwards the work to one of N backends, choosing which one per *packet*, per *connection*, or per *request*. Which of those three it chooses is the single most important fact about it, because it determines what the balancer can see, what it can retry, and how evenly it can spread load.

Granularity	Example	Consequence
Per packet	ECMP in a router, Maglev	Stateless, line rate; needs consistent hashing or TCP breaks
Per connection	NLB, HAProxy in TCP mode, LVS	One slow backend holds a keep-alive connection for minutes
Per request	ALB, nginx, Envoy, HAProxy HTTP mode	Even spread, retries, routing by path/header; costs parsing

```
# nginx: per-request balancing over three app containers
upstream app {
    least_conn;
    server 10.0.1.11:9000 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:9000 max_fails=3 fail_timeout=10s;
    server 10.0.1.13:9000 max_fails=3 fail_timeout=10s backup;
    keepalive 64;            # reuse upstream conns: saves ~1 RTT per request
}
server {
    listen 443 ssl http2;
    location / {
        proxy_pass http://app;
        proxy_next_upstream error timeout http_502 http_503;
        proxy_connect_timeout 2s;    # Topic 96: budget, not a default
        proxy_read_timeout   10s;
    }
}
```

```
AWS equivalent, same three backends

  Route 53 --> ALB (L7, per request)  --> target group --> 3 EC2/ECS tasks
                |  health check /healthz every 15 s, 2 fails = out
                |  deregistration delay 30 s = connection draining (T18)
                +- WAF, TLS cert from ACM, access logs to S3

  NLB (L4, per connection) sits below it when you need static IPs,
  non-HTTP protocols, or millions of idle connections (WebSocket, T116).
```

THE KEY INSIGHT

A load balancer is a scheduler with stale information. It cannot know a backend's current queue depth — only what it has sent and what has come back. Every algorithm in Topic 13 is a different guess about that hidden state.

THE TRAP

Believing an L4 balancer distributes *requests*. With HTTP keep-alive, one connection carries hundreds of requests, so an L4 balancer that spreads connections evenly can still send 70% of the *work* to one backend. Symptom: two of six containers at 90% CPU, four at 15%, with a perfectly flat connection-count graph. Move to L7, or disable upstream keep-alive and pay an RTT per request.

```
load balancer — GAINS: horizontal scale, failure isolation, one TLS termination point · COSTS:
0.2–1 ms added latency, a new dependency at 99.99% · ACCEPTS: it is now the single point every
request passes (Topic 19)
```

10. L4 versus L7

L4 balancers make decisions from the TCP/UDP header only: source IP, source port, destination IP, destination port, protocol. L7 balancers terminate the connection, parse HTTP, and can decide from method, path, host, headers, cookies and body. The trade is visibility versus cost.

	L4 (NLB / LVS / HAProxy tcp)	L7 (ALB / nginx / Envoy)
sees	5-tuple only	method, path, host, headers
does	forward packets/connections	terminate, parse, re-originate
TLS	passes through (backend does it)	terminates (or re-encrypts)
retry	cannot (bytes already sent)	yes, idempotent requests
routing	none	/api/* -> svc-a, /img/* -> svc-b
latency	~0.05-0.1 ms	~0.3-1.0 ms
throughput	millions of conns/node	~50-200k req/s per node
sticky	by source IP hash	by cookie
health	TCP connect / port open	GET /healthz, status + body

```
# HAProxy: the same frontend in both modes
frontend tcp_in                     frontend http_in
    mode tcp                            mode http
    bind :443                           bind :443 ssl crt /etc/ssl/site.pem
    default_backend pool                acl is_api path_beg /api
                                        use_backend api if is_api
backend pool                            default_backend web
    mode tcp
    balance source                  backend api
    server a 10.0.1.11:443 check        mode http
    server b 10.0.1.12:443 check        balance leastconn
                                        option httpchk GET /healthz
                                        http-check expect status 200
                                        server a 10.0.1.11:9000 check inter 3s
```

Use L4 when the protocol is not HTTP (Postgres, Redis, SMTP, game UDP), when you need extreme connection counts at low cost, or when regulation forbids decrypting. Use L7 for anything where you want path routing, per-request fairness, retries, header-based canaries (Topic 109) or a WAF. Most real systems run both: NLB for static IPs and raw throughput, with L7 proxies behind it.

THE KEY INSIGHT

L7 costs you ~0.5 ms and buys you every routing, retry and observability feature in this book. If you cannot decrypt, you cannot decide.

THE TRAP

Losing the client IP at L7. Once the proxy re-originates the connection, the backend sees the proxy's address, so rate limits (Topic 103), geo rules and audit logs all key on one IP. Symptom: every request in the log comes from 10.0.0.5, and your per-IP limiter blocks the entire site at once. Fix with `X-Forwarded-For` plus a trusted-proxy list, or PROXY protocol at L4.

```
L4 vs L7 — GAINS(L7): routing, retries, per-request balance · COSTS: ~0.5 ms, CPU for TLS and
parsing, client IP handling · ACCEPTS(L4): no retries, connection-level skew (Topic 9)
```

11. DNS load balancing and the TTL trap

DNS is the cheapest load balancer: return several A records, or return a different one per client, and traffic spreads with no box in the path at all. It is also the least controllable, because the decision is cached by resolvers and by the client for a duration you only *suggest*.

```
$ dig +noall +answer api.example.com
api.example.com.  60  IN  A  203.0.113.10
api.example.com.  60  IN  A  203.0.113.11
api.example.com.  60  IN  A  203.0.113.12
                  ^^
                  TTL in seconds -- an advisory, not a guarantee

Failure timeline when 203.0.113.10 dies, TTL = 60:
  t+0s    host dies; 1/3 of new connections still sent to it
  t+0s    health check notices (Topic 15), record pulled from the zone
  t+60s   well-behaved resolvers expire the entry
  t+300s  ISP resolvers that clamp minimum TTL to 300 expire it
  t+???   JVMs with networkaddress.cache.ttl=-1 NEVER expire it
          browsers pin per tab; mobile carriers cache aggressively
  observed: 3-8% of traffic still hitting a dead IP 30 minutes later
```

```
DNS balancing            vs   virtual IP / anycast (Topic 12, 19)
  failover time = TTL + resolver lag      failover = health check interval
  cost = $0                               cost = an LB or BGP session
  granularity = per resolver              granularity = per connection
  good for: region selection, GSLB        good for: instance-level failure
```

THE KEY INSIGHT

DNS is for choosing a *region or edge*, not for choosing an instance. Instance failure must be handled by something that reacts in seconds — a VIP, an LB, or anycast withdrawal.

THE TRAP

Lowering TTL to 5 s and assuming fast failover. Symptom: DNS query volume rises 12×, resolver latency appears in every page load, some resolvers silently clamp to their own minimum anyway, and failover is still not fast. Set TTL to 60–300 s for stability and put a real balancer behind the name. Lower TTL a day *before* a planned migration, then raise it again.

```
DNS balancing — GAINS: free, global, no traffic path · COSTS: no per-instance control, no health
awareness at the client · ACCEPTS: minutes of traffic to dead endpoints; unusable as the only
failover mechanism
```

12. Anycast, BGP and global traffic steering

Anycast announces the same IP prefix from many locations; the internet's own routing (BGP) delivers each client to the topologically nearest announcement. The load balancing happens in other people's routers, for free, and failover is a route withdrawal that propagates in seconds. Every large CDN and public DNS resolver works this way.

```
one IP, 1.2.3.4, announced from four PoPs

   Dhaka client  --\                      BGP picks fewest AS hops,
   Delhi client  ---+--> nearest PoP      not lowest latency -- these
   Berlin client ---+                     differ maybe 10-20% of the time
   Sao Paulo     --/

   PoP failure: withdraw the announcement -> traffic reconverges in
   5-60 s, no DNS change, no client involvement, no TTL to wait out.

   The catch: reconvergence RESETS in-flight TCP connections.
   Fine for HTTP/DNS. Bad for a 4-hour database session.
```

Layer above it: **GSLB** (global server load balancing) steers by policy — geo, latency measurements, weighted round-robin, or health — usually implemented in DNS. Route 53 latency-based routing and health-checked failover records are the managed version; PowerDNS with a geo backend, or a pair of anycast BGP sessions from your own AS, is the self-hosted one.

Mechanism	Failover time	Use for
Anycast BGP withdrawal	5–60 s	Edge/PoP failure, DDoS absorption
DNS health-check failover	TTL + 60–300 s	Region failure, DR cutover (Topic 111)
LB health check	3–30 s	Instance failure (Topic 15)
Client-side retry	Immediate	Single failed request (Topic 97)

THE KEY INSIGHT

Failover mechanisms form a ladder by reaction time: retry (ms) → LB (s) → anycast (tens of s) → DNS (minutes). Pick the rung that matches the blast radius; using DNS for an instance failure is the classic mismatch.

THE TRAP

Anycast with long-lived stateful connections. A BGP reconvergence mid-session sends your packets to a different PoP that has never heard of the connection, and the client sees an RST. Symptom: unexplained connection resets clustered by minute, correlating with nothing in your logs because the event happened in someone else's network. Terminate TLS at the PoP and keep sessions short, or use unicast IPs for stateful protocols.

```
anycast/GSLB — GAINS: geographic latency reduction, DDoS spread, seconds-scale PoP failover · COSTS: an ASN, IP allocation and BGP operations, or a CDN vendor · ACCEPTS: route changes reset TCP; routing is by AS hops, not latency
```

13. Balancing algorithms and their failure signatures

Every algorithm is a guess at which backend has the shortest queue. They differ in what evidence they use and how they fail when one backend goes bad.

Algorithm	Decides by	Fails by
Round robin	Position in a list	Ignores request cost; a slow backend keeps its share
Weighted RR	Static capacity weights	Weights go stale after a resize
Least connections	In-flight count per backend	A backend erroring in 1 ms looks idle → gets everything
Least response time / EWMA	Observed latency average	Oscillates: everyone piles onto the fastest, which slows
Power of two choices	Pick 2 at random, take the shorter queue	Almost nothing; near-optimal with O(1) state
IP hash / consistent hash	Hash of a key	Hot key → hot backend (Topic 39)
Random	Nothing	Surprisingly decent at high N; poor at N < 5

```
The "least connections" failure, in numbers
-------------------------------------------
3 backends, 900 QPS. Backend C's database credentials expire; it now
returns HTTP 500 in 0.8 ms instead of 200 in 40 ms.

  in-flight(A) = 300 QPS x 0.040 s = 12       (Little's law: L = lambda x W)
  in-flight(B) = 12
  in-flight(C) = 300 x 0.0008 s   = 0.24

  least_conn now sends nearly EVERY new request to C.
  Observed: error rate jumps from 0% to ~95%, latency graph IMPROVES.

Fix: eject on error rate, not on connection count.
  envoy outlier detection: consecutive_5xx: 5, base_ejection_time: 30s
  haproxy:  observe layer7  error-limit 10  on-error mark-down
  nginx:    max_fails=3 fail_timeout=10s  (passive; blunt but works)
```

THE KEY INSIGHT

Power of two choices — sample two backends at random and send to the shorter queue — gets you within a few percent of perfect balance with no global state. It is the default worth using when you have no reason to pick something else.

THE TRAP

Fast failures attract traffic. Any algorithm that rewards low latency or low in-flight count will route *toward* a broken backend, because broken is fast. Symptom: aggregate latency improves at the exact moment the error rate explodes. Always pair a load metric with an error metric (Topic 15).

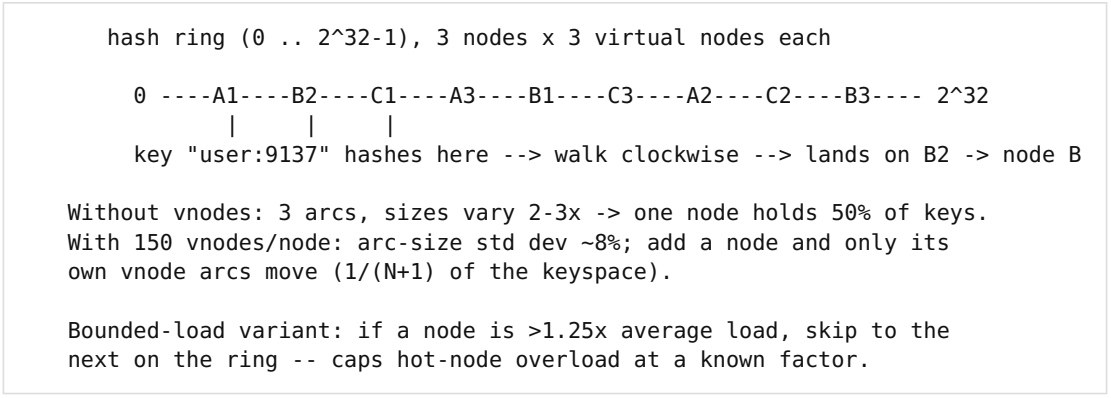
```
algorithms — GAINS: 10–40% better tail than round robin · COSTS: per-backend state; EWMA needs
tuning · ACCEPTS: all of them mis-route toward fast failures unless error-aware
```

14. Consistent hashing and virtual nodes

Sometimes you need the *same* key to reach the same backend — a cache shard (Topic 42), a sticky session, a per-user rate limiter. Naive `hash(key) % N` works until N changes, at which

point almost every key moves. Consistent hashing places both nodes and keys on a ring, so a node change moves only the keys in that node's arc.

```
N = 4 -> 5 with modulo:      fraction of keys remapped = 1 - 1/5   = 80%
N = 4 -> 5 with ring:        fraction remapped         = 1/5       = 20%
general: modulo ~ (N-1)/N remapped;  ring ~ 1/(N+1) remapped

Cache impact of an 80% remap at 95% hit ratio, 2,000 read QPS:
  hit ratio collapses to ~19% for the refill window
  DB load: 100 QPS -> 1,620 QPS for tens of seconds  = the outage
```



This is the same mechanism used by Valkey Cluster's 16,384 hash slots (explicit slots instead of a ring, same effect), by Maglev's lookup table in Google's L4 balancers, and by shard routing in Topic 52. Recognising it three times in one book is the point.

THE KEY INSIGHT

Consistent hashing trades perfect balance for *stability under membership change*. Virtual nodes buy back the balance; bounded loads cap the damage from a hot key.

THE TRAP

Assuming the ring protects you from hot keys. It does the opposite: it guarantees the hot key always lands on the same node. Symptom: one cache node at 100% CPU while five idle, during a flash sale on one route. Fix by splitting the key (`seat:route7:shard{0..9}`) or replicating it (Topic 39).

```
consistent hashing — GAINS: 1/(N+1) keys move on membership change instead of ~all · COSTS: ring
state, ~150 vnodes per node in memory, uneven arcs without them · ACCEPTS: hot keys are pinned,
not spread
```

15. Health checks and outlier ejection

A health check is the balancer's only source of truth about a backend. Its design decides how fast you shed a dead node and how often you shed a healthy one by mistake.

```
TYPES
  liveness   "is the process up?"          TCP connect, or GET /healthz -> 200
  readiness  "should it receive traffic?"  checks deps: DB, cache, migrations
  active     LB probes on a timer          detects idle failure; costs QPS
  passive    LB watches real responses     free, instant, but only under load

Detection time = interval x unhealthy_threshold  (+ probe timeout)
  ALB default:  30 s x 2 = 60 s   <-- one minute of 5xx by default
  tuned:        5 s x 2  = 10 s
  aggressive:   2 s x 2  = 4 s    <-- now flapping risk (see the trap)

Recovery must be slower than detection: healthy_threshold 3-5,
plus slow-start so a cold JIT/opcache node is not hit at full rate.
```

```
# The endpoint itself -- the deep/shallow split that matters
GET /healthz    -> 200 {"ok":true}          shallow: process is alive
GET /readyz     -> 200 or 503               deep: DB + cache reachable

// Laravel: keep the deep check cheap and cached for 5 s so 3 LBs
// x 20 nodes x 1/s do not add 60 QPS of SELECT 1 to the primary.
Route::get('/readyz', function () {
    return Cache::remember('readyz', 5, function () {
        DB::select('SELECT 1');
        Cache::store('redis')->get('ping');
        return response()->json(['ok' => true]);
    });
});
```

THE KEY INSIGHT

Liveness must never check dependencies. If `/healthz` fails when the database is slow, every node fails at once and the balancer removes the entire fleet — converting a degraded database into a total outage.

THE TRAP

Correlated deep health checks. Symptom: database blips for 4 seconds; all 12 backends report unhealthy simultaneously; ALB has no healthy targets and returns 503 to 100% of traffic for 90 seconds, long after the database recovered. The usual fixes: shallow liveness, deep readiness with hysteresis, and a "minimum healthy targets" / fail-open rule so that *all* unhealthy is treated as *all* healthy.

```
health checks — GAINS: dead nodes out in 4–60 s · COSTS: probe QPS (n_LB × n_nodes / interval),
false ejections · ACCEPTS: detection speed and flap resistance trade directly against each other
```

16. Sticky sessions and what they cost

Session affinity pins a client to one backend, by cookie at L7 or by source-IP hash at L4. It exists to make stateful servers survivable. It is almost always a sign that Topic 8 was skipped.

```
# nginx cookie affinity (open-source: ip_hash; NGINX Plus/HAProxy: cookie)
upstream app { ip_hash; server 10.0.1.11:9000; server 10.0.1.12:9000; }

# HAProxy, cookie-based, which survives NAT
backend web
    balance roundrobin
    cookie SRV insert indirect nocache
    server a 10.0.1.11:9000 check cookie a
    server b 10.0.1.12:9000 check cookie b

# ALB: target group attribute
stickiness.enabled = true
stickiness.type    = lb_cookie      # or app_cookie
stickiness.lb_cookie.duration_seconds = 3600
```

```
what stickiness costs, in one picture

  deploy node A ->  all A-pinned users lose session state
  scale out     ->  new node gets 0 traffic until new sessions arrive
  hot user      ->  one heavy client cannot be spread
  ip_hash + NAT ->  a whole office/carrier CGNAT lands on ONE backend
                    (observed: 40% of traffic on 1 of 6 nodes)

  the alternative, one hop away:
  session in Valkey  ->  any node serves any request  ->  none of the above
```

THE KEY INSIGHT

Stickiness converts an application design problem into a traffic distribution problem, and traffic distribution problems are worse: they appear only at scale, under NAT, or during a deploy.

THE TRAP

Sticky sessions plus autoscaling. Symptom: you scale from 4 to 8 nodes during a sale; the four new nodes sit at 5% CPU while the original four stay at 95%, because every existing session is pinned. The graph looks like autoscaling worked. It did not. Use stickiness only for genuinely per-connection state (WebSocket, Topic 116), and drain rather than cut (Topic 18).

```
stickiness — GAINS: makes stateful backends usable, improves local cache hit rate · COSTS: uneven load, painful deploys, session loss on node death · ACCEPTS: new capacity is not used until sessions rotate
```

17. TLS termination, offload and mTLS

Terminating TLS at the balancer means one place holds certificates, one place gets the CPU cost of the handshake, and everything behind it speaks plaintext or a cheaper internal TLS. The handshake, not the bulk encryption, is what costs.

```
Handshake cost (one core, modern x86)
  RSA-2048 private key op       ~ 1,200-1,800 ops/s   <- the bottleneck
  ECDSA P-256 private key op    ~ 8,000-15,000 ops/s  <- 6-8x cheaper
  AES-GCM bulk, AES-NI          ~ 1-4 GB/s            <- effectively free

At 2,000 new connections/s with RSA: ~1.3 cores just for handshakes.
Session resumption (tickets / PSK) turns most of them into 0-RTT or
1-RTT resumptions costing ~1/20th. Measure resumption rate; a rate
below 60% usually means tickets are not shared across LB nodes.

$ openssl s_client -connect api.example.com:443 -tls1_3 </dev/null 2>&1 \
    | grep -E "Protocol|Cipher|Verify return code"
  Protocol  : TLSv1.3
  Cipher    : TLS_AES_128_GCM_SHA256
  Verify return code: 0 (ok)
```

Mode	Backend sees	Use when
Termination	Plaintext HTTP	Trusted network, max performance
Re-encryption	TLS from the LB	Compliance, shared/zero-trust network
Passthrough (L4)	The original TLS	LB must not hold keys; no L7 features
mTLS	Verified client cert	Service-to-service auth, partner APIs

THE KEY INSIGHT

Switching certificates from RSA-2048 to ECDSA P-256 multiplies handshake capacity by roughly 6–8× on the same hardware. It is the single cheapest capacity change in this book.

THE TRAP

Certificate expiry as a correlated failure (Topic 6). Every node has the same certificate, so it expires everywhere at the same second. Symptom: total outage at a round timestamp, with healthy CPU, healthy database and a flood of `SSL_ERROR_EXPIRED`. Automate renewal (ACM, cert-manager, certbot) and alert at 30 *and* 7 days — a renewal that silently fails is invisible until it is total.

```
TLS at the edge — GAINS: one cert store, ~1 RTT saved by resumption, L7 features unlocked · COSTS: 0.5–1.5 cores per 2k handshakes/s on RSA · ACCEPTS: plaintext inside the perimeter unless you re-encrypt
```

18. Connection draining and zero-downtime deploys

Removing a backend is not "stop the process". It is: stop receiving new work, finish in-flight work, then exit. Every zero-downtime deploy is this sequence, and every "deploys cause a 502 spike" incident is a missing step of it.

```
the correct shutdown sequence (and where each system implements it)

 1. mark unready        readiness -> 503        k8s: readinessProbe fails
 2. WAIT propagation    3-15 s                  <- the step everyone omits
 3. LB stops new conns  deregistration          ALB: deregistration_delay
 4. finish in flight    up to drain timeout     nginx: worker_shutdown_timeout
 5. close keep-alives   send Connection: close  or GOAWAY for HTTP/2
 6. process exits       SIGTERM -> SIGKILL      k8s terminationGracePeriod

 Omit step 2 and the LB is still routing to a process that already
 called exit(): the 502 burst lasts exactly as long as your propagation
 delay, appears only in production, and never reproduces in staging.
```

```
# Kubernetes: the drain that actually works
spec:
  terminationGracePeriodSeconds: 45         # must exceed 2 + 4 below
  containers:
  - name: app
    readinessProbe: { httpGet: { path: /readyz }, periodSeconds: 3 }
    lifecycle:
      preStop:
        exec: { command: ["sh","-c","sleep 10"] }   # step 2, explicitly

# ALB target group
deregistration_delay.timeout_seconds = 30   # step 3-4

# Laravel Horizon / queue workers: same idea, different verb
php artisan horizon:terminate    # finish the current job, then exit
# NEVER SIGKILL a worker mid-job unless every job is idempotent (Topic 63)
```

THE KEY INSIGHT

Between "I am unready" and "the balancer knows" there is a propagation delay. Sleeping through that delay before shutting down is the whole trick.

THE TRAP

Grace period shorter than the longest request. Symptom: a 502/504 for exactly the slowest 0.5% of requests during every deploy — typically report generation or a bulk import — while p50 is untouched. Either raise the grace period above your p99.9 request duration, or move long work to a queue (Topic 60) so no request outlives a deploy.

```
draining — GAINS: deploys without 5xx · COSTS: 30–60 s longer rollouts, more pods alive at once ·
ACCEPTS: requests longer than the grace period are still killed
```

19. The load balancer as a single point of failure

Everything in Part 2 exists to remove single points of failure — and puts one new box in front of all traffic. The question is not whether it can fail, but what happens in the seconds after it does.

```
SELF-HOSTED: active-passive VIP          SELF-HOSTED: active-active ECMP

  VIP 203.0.113.5                          router ECMP-hashes 5-tuple
   |                                        across 4 LB nodes
   +-- lb1 (MASTER, keepalived/VRRP)        |    |    |    |
   +-- lb2 (BACKUP)                        lb1  lb2  lb3  lb4
                                            \    \    /    /
  failover: VRRP misses 3 adverts            all announce the same /32
  (~3 s), lb2 takes the VIP via              node loss -> rehash -> only
  gratuitous ARP. Existing TCP               that node's connections reset
  connections RESET unless state             (Maglev-style consistent hash
  is synced (conntrackd).                    reduces this to ~1/N).

MANAGED (AWS): ALB/NLB is already N nodes across >=2 AZs behind one DNS
name; the "LB SPOF" becomes an AZ or control-plane failure instead. You
still choose: cross-zone balancing on (even spread, cross-AZ $) or off
(cheap, but one AZ's targets can be overloaded).
```

Failure	Detection	Blast radius
LB process crash	VRRP 1–3 s	All in-flight connections on that node
LB node network loss	VRRP / BGP 3–30 s	Same, plus split-brain risk (two masters)
Whole AZ	Health checks 10–60 s	Everything in that AZ; needs cross-AZ targets
Config push error	Immediate, global	**Everything** — the real top cause

THE KEY INSIGHT

The most common load balancer outage is not hardware. It is a configuration change applied everywhere at once. Treat LB config like code: validate (`nginx -t`, `haproxy -c`), canary one node, then roll.

THE TRAP

An active-passive pair where the passive node has never served traffic. Symptom: failover works, then the passive node falls over in 40 seconds because its file-descriptor limit, TLS session cache or worker count was never tuned. Force regular failovers — a monthly scheduled flip (Topic 105) is the only proof that the standby works.

```
LB redundancy — GAINS: removes the last SPOF on the request path · COSTS: 2× LB nodes, VRRP/BGP
operations, or a managed LB bill · ACCEPTS: failover resets in-flight connections; config errors
remain global
```

20. Admission control and edge rate limiting

A load balancer can also decide *not* to admit work. This is the cheapest protection you have: a request rejected at the edge costs ~0.05 ms and touches no database. The full algorithm treatment is Topic 103; here it is where to put it and what the numbers look like.

```
# nginx: two zones -- a per-IP request limit and a global connection cap
limit_req_zone  $binary_remote_addr zone=perip:10m  rate=20r/s;
limit_conn_zone $binary_remote_addr zone=conns:10m;

server {
    limit_req   zone=perip burst=40 nodelay;   # token bucket: 20/s, burst 40
    limit_conn  conns 24;
    limit_req_status 429;
    location /api/booking { limit_req zone=perip burst=5; }   # tighter
}
# 10m of zone memory holds ~160,000 IPv4 states (64 B each).

# HAProxy: stick-table version, same idea, with visibility
backend api
    stick-table type ip size 1m expire 60s store http_req_rate(10s)
    http-request track-sc0 src
    http-request deny deny_status 429 if { sc_http_req_rate(0) gt 200 }
```

```
where to reject, and what it costs you to say no

  CDN/WAF edge     0.01 ms   free      <- best: never enters your network
  LB               0.05 ms   ~0 CPU    <- good: no app, no DB
  app middleware   3-10 ms   1 worker  <- occupies a PHP-FPM slot
  database         20+ ms    1 conn    <- worst: the thing you protect is hit

  concurrency limit is often better than a rate limit:
    max_in_flight = target_p99 x capacity  (Little's law again)
    at 500 QPS capacity and a 200 ms target: cap in-flight at 100
```

THE KEY INSIGHT

Reject as early and as cheaply as possible, and always return `429` with `Retry-After` — a well-formed rejection lets clients back off (Topic 97); a timeout teaches them to retry harder.

THE TRAP

Rate limiting by IP behind a proxy or CGNAT. Symptom: a whole mobile carrier or a corporate office is blocked as one "attacker", while the actual abuser rotates across a botnet and is never limited. Key on authenticated user or API key where you have one, IP only as a fallback, and always exempt your own health checks.

```
edge admission control — GAINS: protects everything downstream at ~0.05 ms per rejection · COSTS: per-key state (~64 B), false positives behind NAT · ACCEPTS: local counters are per-LB-node, so the real limit is n_nodes × configured rate (Topic 103)
```

PRACTICE SET 2 — Topics 9–20

A. Conceptual

1. Why can an L4 balancer with a perfectly even connection count still send most of the *work* to one backend?
2. Give the one-sentence rule for what DNS load balancing is for and what it is not for.
3. Why does least-connections routing send traffic *toward* a broken backend, and what metric fixes it?
4. What does power-of-two-choices buy over plain random, and what state does it require?
5. Explain why liveness probes must not check the database, in terms of Topic 6.
6. What is the propagation gap in a rolling deploy, and which step of the shutdown sequence closes it?
7. Name three ways sticky sessions defeat autoscaling.
8. Why is ECDSA the cheapest capacity upgrade available at the edge?
9. What is the difference in blast radius between an LB node failure and an LB config push, and which is more common?
10. Why does a 429 protect you better than a timeout?
11. Anycast fails over in seconds without DNS. Name the class of traffic for which that failover is destructive.
12. A cache cluster grows from 8 nodes to 9. Compare keys remapped under modulo hashing versus a consistent hash ring.

B. Pen-and-paper

1. An ALB uses interval 30 s, unhealthy threshold 2. A backend dies at t=0. How long does it keep receiving traffic? Recompute with interval 5 s, threshold 2, and state what you traded away.
2. 3 backends, 1,200 QPS, mean service time 50 ms. Use Little's law to give in-flight requests per backend. Backend C starts failing in 1 ms. What is C's in-flight count, and what share of new requests does least-connections give it?
3. A cache ring has 6 nodes. Compute the fraction of keys remapped by adding a 7th under (a) `hash % N`, (b) a consistent hash ring. At 4,000 read QPS and a 94% hit ratio, give database QPS during the refill window in both cases.
4. TLS: 3,500 new connections/s, RSA-2048 at 1,500 handshakes/s/core, resumption rate 0%. How many cores? Recompute with ECDSA P-256 at 10,000/s/core, and again with a 70% resumption rate (resumptions cost 1/20th).
5. nginx `limit_req_zone` with a 10 MB zone at 64 B per state: how many distinct IPs fit? Your service has 4 LB nodes each configured at 20 r/s per IP. What is the effective global limit per IP?
6. Deploy math: 12 pods, rolling update maxUnavailable 1, readiness delay 3 s, preStop sleep 10 s, drain 30 s, container start 20 s. How long is the full rollout?
7. A page with 20 assets is served from an L7 LB adding 0.6 ms per request, versus an L4 LB adding 0.08 ms. Compute the added server-side latency per page. Then state why the L7 answer is still usually right.
8. Cross-zone balancing is off; you have 3 AZs with 4, 4 and 2 targets, and the ALB spreads evenly across AZ nodes. What share of traffic does each target in the 2-target AZ receive relative to the others?

C. Lab tasks (build → break → observe → fix)

1. Stand up nginx with three identical containers as an upstream. Confirm even distribution with 1,000 requests and per-backend access-log counts.
2. **Break it:** make backend #3 return 500 in under 1 ms (e.g. an endpoint that immediately aborts). Switch nginx to `least_conn` and drive 500 QPS. Record the error rate and the p50 latency graph. Then add `max_fails=3 fail_timeout=10s` (or move to Envoy outlier detection) and record both again.
3. **Break it:** point your liveness probe at an endpoint that runs `SELECT 1`. Stop the database. Observe every backend being ejected and the LB returning 503 with zero healthy targets. Fix by splitting `/healthz` (shallow) from `/readyz` (deep) and repeat — record what the client now sees instead.
4. **Break it:** deploy with `terminationGracePeriodSeconds: 5` and no preStop sleep while a load generator runs. Count the 502s. Add the 10 s preStop sleep and raise the grace period above your p99.9, and count again.
5. Enable file-based sticky sessions with `ip_hash`, then send traffic from two machines through one NAT gateway. Record the resulting distribution across backends.
6. Measure TLS: run `openssl speed rsa2048 ecdsap256` on your machine and compute the new-connection capacity per core for each. Compare with the figures in Topic 17.
7. Apply an nginx `limit_req` of 20 r/s with burst 40. Drive 100 r/s from one IP. Record the 429 rate, then confirm from the access log that rejected requests never reached the app (no application log line).

ANSWER KEY 2 — Topics 9–20

A. Conceptual

1. **HTTP keep-alive.** One connection carries many requests, so connection-level fairness says nothing about request-level fairness.
2. **DNS chooses a region or edge; something that reacts in seconds chooses an instance.** Resolver and client caching make TTL advisory.
3. Broken backends answer fast, so their in-flight count collapses and they look idle (Little's law: L = λW). Fix with **error-rate-based ejection** — consecutive 5xx, or success-rate outlier detection.
4. It cuts the maximum queue length from O(log n / log log n) to O(log log n) — in practice near-perfect balance — using **only the two sampled backends' in-flight counts**, no global state.
5. A deep liveness probe makes every backend fail **at the same moment**, which is a correlated failure: the parallel-redundancy formula does not apply and availability collapses to the database's.
6. The gap between the pod declaring itself unready and the LB acting on it. **Step 2 — sleep through the propagation delay before exiting.**
7. New nodes receive no existing sessions; load stays on the old nodes; and scale-in kills sessions outright. (Also: deploys drop state.)
8. Handshake capacity per core rises from ~1,500/s to ~10,000/s — **6–8× more connections on the same hardware**, for a certificate change.
9. Node failure: one node's in-flight connections. Config push: **global, instant, and it is the more common cause of real LB outages.**
10. A 429 with `Retry-After` is a signal the client can obey; a timeout is indistinguishable from packet loss and triggers **more** retries (Topic 97).
11. **Long-lived stateful connections** — database sessions, long WebSockets. Reconvergence sends packets to a PoP with no connection state, producing RSTs.
12. Modulo: 1 − 1/9 = **88.9%** of keys move. Ring: ≈1/9 = **11.1%**.

B. Worked

1. 30 × 2 = **60 s** (plus up to one interval of skew). At 5 s: **10 s**. Traded: 6× the probe QPS and a much higher chance of ejecting a healthy node during a GC pause or brief blip.
2. Per backend: 400 QPS × 0.050 s = **20 in flight**. C: 400 × 0.001 = **0.4**. Least-connections will send C **essentially every new request** until its in-flight count reaches 20, which at 1 ms service time never happens — so ~100% of new requests, i.e. the error rate approaches the full traffic rate.
3. (a) 1 − 1/7 = **85.7%**; (b) ≈1/7 = **14.3%**. Baseline DB load = 4,000 × 0.06 = **240 QPS**. During refill, effective miss rate ≈ 0.06 + 0.857 × 0.94 = 0.866 → **3,464 QPS** (modulo), versus 0.06 + 0.143 × 0.94 = 0.194 → **777 QPS** (ring). The first number is an outage; the second is a busy afternoon.
4. RSA: 3,500 / 1,500 = **2.33 → 3 cores**. ECDSA: 3,500 / 10,000 = **0.35 core**. With 70% resumption on ECDSA: full handshakes 1,050/s → 0.105 core, resumptions 2,450/s × 1/20 → 0.012 core, total **~0.12 core** — a 20× reduction from two configuration choices.
5. 10 MB / 64 B = **163,840 states**. Effective global limit = 4 × 20 = **80 r/s per IP**, assuming the LB spreads that IP's requests — the classic reason local limits under-protect (Topic 103).

6. Per pod: 3 + 10 + 30 (drain overlaps) + 20 start ≈ 33–63 s; with maxUnavailable 1 the pods go one at a time, so 12 × ~45 s ≈ **9 minutes**. This is why maxSurge exists.
7. 20 × (0.6 − 0.08) = **10.4 ms per page**. Still worth it: retries, per-request balance, path routing, canaries, WAF and per-request logs — and 10 ms is 4% of one cross-region RTT.
8. Each AZ gets 1/3 of traffic. Targets in the 2-target AZ get 33.3%/2 = **16.7% each**, versus 33.3%/4 = **8.3% each** elsewhere — **2× the load**. Keep target counts balanced per AZ or enable cross-zone balancing and pay the cross-AZ transfer.

C. What to verify

1. Expect within ±2% of 333 requests each; a larger skew means keep-alive is pinning connections (re-run with `Connection: close` to confirm).
2. Expect the error rate to climb toward **>90%** while **p50 latency falls** — the signature failure of Topic 13. After ejection is enabled: errors drop to roughly `fail_timeout/(fail_timeout+detection)` of the previous level, in bursts of 3 every 10 s.
3. Expect **0 healthy targets and 503 for 100% of traffic**, lasting past database recovery by `interval × healthy_threshold`. After the split: the app returns **200 for cached/static paths and 500 only for database-backed paths** — partial availability instead of total outage (Topic 99).
4. Expect one 502 burst per pod, sized `propagation_delay × QPS` — at 200 QPS and a 4 s gap, roughly **800 failed requests per pod**. After the fix: **0**.
5. Expect both machines on **one backend** (same NAT source IP), so a 50/50 client split becomes 100/0 at the backend.
6. `openssl speed` typically reports RSA-2048 sign around **1,000–2,000/s** and ECDSA P-256 sign around **25,000–45,000/s** per core (ECDSA *verify* is the slower half); TLS server capacity tracks the sign operation, so expect the same 6–10× ratio as Topic 17.
7. Expect **~20 r/s served, ~80 r/s rejected** after the burst of 40 is spent, and **zero application log lines** for rejected requests — the proof that rejection cost you nothing downstream.

HANDNOTE SUMMARY CARD — PART 2

```
LOAD BALANCER
================================================================================
GRANULARITY        packet (ECMP/Maglev) | connection (NLB/LVS) | request (ALB/nginx)
  keep-alive means connection-even != work-even  -> use L7 for real fairness

L4 vs L7   L4: 5-tuple, ~0.08 ms, millions of conns, no retry, TLS passthrough
           L7: path/header, ~0.5 ms, retries, canary, WAF, needs XFF for client IP

ALGORITHMS
  round robin      ignores cost         least_conn   fast failures attract load
  EWMA/least-time  oscillates           P2C          near-optimal, O(1) state
  consistent hash  pins hot keys        random       fine at high N
  RULE: pair any load metric with an ERROR metric or you route toward broken

CONSISTENT HASHING
  modulo remap = (N-1)/N     ring remap = 1/(N+1)     vnodes ~150/node
  bounded load: skip node if > 1.25x mean.  Valkey Cluster = 16,384 slots

HEALTH CHECKS
  liveness  = process only (NEVER checks the DB -- correlated ejection)
  readiness = deps, cached 5 s, hysteresis
  detect = interval x unhealthy_threshold  (ALB default 30x2 = 60 s)
  fail-open when ALL targets are unhealthy

DRAIN SEQUENCE (memorise)
  unready -> SLEEP for propagation -> LB stops new -> finish in flight
  -> Connection: close / GOAWAY -> exit
  grace period MUST exceed p99.9 request duration

TLS
  RSA-2048 ~1.5k handshakes/s/core | ECDSA P-256 ~10k | resumption ~1/20 cost
  AES-GCM bulk is free. Cert expiry = correlated total outage; alert 30 d + 7 d

DNS / ANYCAST / FAILOVER LADDER
  client retry ms -> LB 3-30 s -> anycast 5-60 s -> DNS TTL+minutes
  anycast resets long-lived TCP; DNS TTL is advisory (JVM cache = forever)

ADMISSION CONTROL
  reject at: CDN 0.01 ms > LB 0.05 ms > app 3-10 ms > DB 20 ms+
  429 + Retry-After, never a timeout.  local limit x n_LB = real limit
  concurrency cap = capacity x target_p99   (Little: L = lambda x W)

TRAPS
  ip_hash + CGNAT | sticky + autoscale | deep liveness | grace < p99.9
  config push = global blast radius | passive standby never tested
================================================================================
```

PART THREE

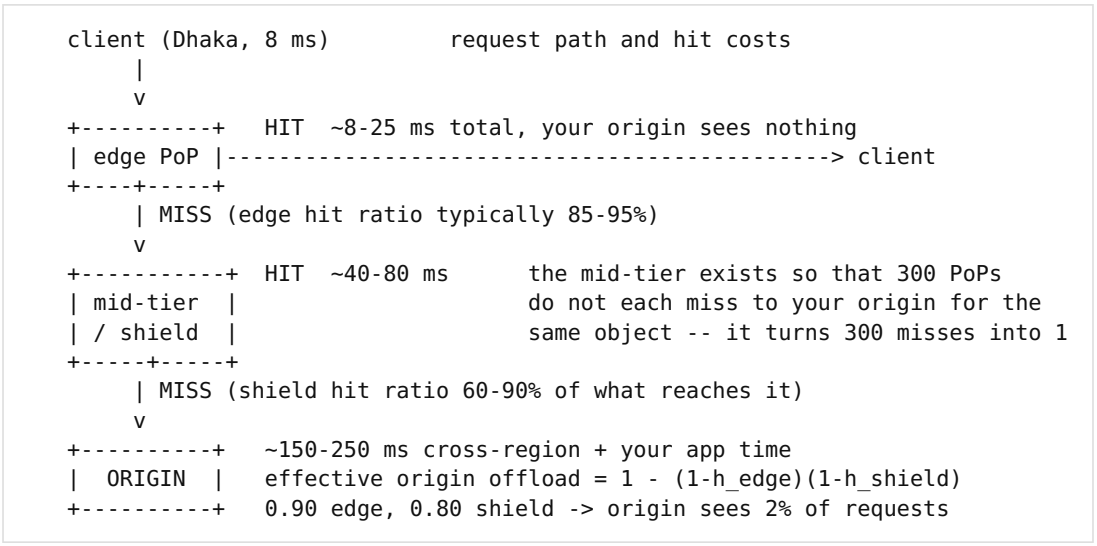
CDN and the edge

The cheapest request is the one your servers never see. A CDN is a cache with a geography problem attached — everything here is Part 4's theory applied at 300 locations you do not operate.

Part 3 · CDN and the edge

21. CDN anatomy: PoPs, tiers and origin shield

A CDN is a fleet of caching reverse proxies placed near users, sharing one anycast IP range (Topic 12). The client resolves your hostname to the nearest point of presence (PoP); the PoP serves from disk if it has the object, and otherwise fetches from a mid-tier cache, and only then from your origin.




```
Offload arithmetic for 72 TB/month of asset traffic (Topic 5):
  no CDN:              72,000 GB from origin      = $6,480 egress
  90% edge hit:         7,200 GB from origin      =   $648
  + 80% shield hit:     1,440 GB from origin      =   $130
  CDN egress:          72,000 GB x ~$0.085        = $6,120  (or $0 on R2)
  origin CPU saved:  ~97% of asset requests never touch nginx or PHP
```

THE KEY INSIGHT

Origin offload compounds across tiers: $1 - (1-h_1)(1-h_2)$. Two 90% caches in series leave your origin serving 1% of requests. This is the same composition as Topic 32's hierarchy, just spread over continents.

THE TRAP

Assuming a global CDN gives you a global hit ratio. Each PoP has its own cache; an object requested once per hour in 200 PoPs may be evicted between requests everywhere, giving a 0% hit ratio and *more* origin load than no CDN at all (200 misses instead of 1). Symptom: origin request count rises after enabling the CDN. Fix by enabling the shield/mid-tier tier so long-tail objects are cached once, centrally.

```
CDN topology — GAINS: 5–25× latency reduction, 95–99% origin offload · COSTS: $/GB egress, cache-control discipline, an extra system to debug · ACCEPTS: per-PoP cache fragmentation on long-tail content
```

22. Push CDN versus pull CDN

A **pull** CDN fetches from your origin on the first miss. A **push** CDN holds content you uploaded, and has no origin to fall back to. Pull is the default because it needs no publishing pipeline; push wins when the first request must not be slow, or when there is no origin worth having.

	Pull	Push
First request	Slow: full origin RTT + app time	Fast: already at the edge
Storage cost	Only what is requested	Everything, in every region you push to
Publishing	None — just change the origin	An upload step in CI/CD
Deletion	TTL expiry or purge	Explicit delete
Best for	Websites, images, APIs, long tail	Large releases, video libraries, firmware

```
Pull (CloudFront + S3 origin, or CloudFront + ALB origin):
  GET /img/bus-12.jpg -> edge MISS -> shield MISS -> origin 200 + headers
  Cache-Control: public, max-age=31536000, immutable

Push-like pattern with object storage as the origin (the common hybrid):
  CI builds assets with content hashes, uploads once:
    aws s3 sync ./dist s3://assets-prod/ \
      --cache-control "public,max-age=31536000,immutable" \
      --exclude "index.html"
    aws s3 cp ./dist/index.html s3://assets-prod/index.html \
      --cache-control "no-cache"     # the one file that must revalidate
  Then a single warm request per PoP (or the CDN's prefetch API) removes
  the first-hit penalty for the launch.
```

THE KEY INSIGHT

The hybrid everyone actually runs: object storage as origin, pull CDN in front, content-hashed filenames with a one-year immutable TTL, and exactly one non-cacheable file (the HTML entry point) that points at the hashed names.

THE TRAP

A pull CDN in front of an origin that generates content on the fly. The first request after each release misses in every PoP simultaneously — a thundering herd (Topic 100) at your origin at exactly the moment you deployed. Symptom: 5xx spike 30 seconds into every release. Use the shield, and pre-warm the top 100 objects.

```
push vs pull — GAINS(pull): zero publishing work · COSTS: cold-miss latency for the first user in
each PoP · ACCEPTS(push): storage in every region and an explicit deletion problem
```

23. The cache-control contract

Every cache between your server and the user — browser, corporate proxy, CDN edge, shield — obeys the same header contract. Getting these four directives right is 80% of CDN work.

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=31536000, immutable
ETag: "9f2b1c3a"
Last-Modified: Tue, 11 Aug 2026 09:14:02 GMT
Vary: Accept-Encoding

  public              any shared cache may store it
  private             browser only -- CDN must not store (user-specific)
  no-cache            store it, but revalidate before every use
  no-store            never write it to disk anywhere
  max-age=N           fresh for N seconds (browser AND CDN unless overridden)
  s-maxage=N          fresh for N seconds in SHARED caches only (CDN)
  immutable           do not revalidate even on reload -- for hashed names
  stale-while-revalidate=N   serve stale for N s while refreshing in background
  stale-if-error=N           serve stale for N s if the origin is down

Revalidation, which costs 1 RTT and ~200 bytes instead of the whole body:
  GET /style.css
  If-None-Match: "9f2b1c3a"
  --> HTTP/1.1 304 Not Modified          (no body)
```

Content	Header
Hashed asset (app.9f2b1c.js)	`public, max-age=31536000, immutable`
HTML entry point	`no-cache` (store + revalidate)
Public API list, tolerates 60 s	`public, s-maxage=60, stale-while-revalidate=300`
Logged-in user page	`private, no-store`
Seat availability	`no-store` — never cache money

THE KEY INSIGHT

`stale-while-revalidate` is the single highest-value directive: users always get an instant response, and exactly one request per object per window goes to your origin. It converts the stampede of Topic 38 into a background refresh.

THE TRAP

A missing `private` on a personalised response. Symptom: a user reports seeing *someone else's* name or booking list. A shared cache stored a page built for user A and served it to user B, and the bug is invisible in development where no shared cache exists. Default to `private, no-store` for anything behind authentication, and enforce it in middleware, not per controller.

```
cache headers — GAINS: correct offload with zero infrastructure · COSTS: a policy decision per
route · ACCEPTS: a wrong header is a data-leak or a stale-forever bug, and both are silent
```

24. Cache keys, query strings and Vary

The cache key decides what counts as "the same object". By default it is scheme + host + path, and every additional component you include multiplies the number of stored copies — which divides your hit ratio.

```
Requests for one image, all logically identical:
  /img/bus-12.jpg
  /img/bus-12.jpg?utm_source=fb&utm_campaign=eid
  /img/bus-12.jpg?fbclid=IwAR3x9...          (unique per user!)

If query string is in the cache key: unbounded distinct keys, hit ratio -> 0.
  CloudFront:  cache policy QueryStringsConfig = none (or an allow-list)
  nginx:       proxy_cache_key "$scheme$host$uri";        # drops the query
  Varnish:     unset req.url query params in vcl_recv

Vary multiplies too -- one copy per distinct header value:
  Vary: Accept-Encoding                2-3 copies (gzip, br, identity)  OK
  Vary: Accept-Language                x 40 languages                   risky
  Vary: User-Agent                     x thousands of UA strings        fatal
  Vary: Cookie                         x every session                  fatal
    -> effectively disables shared caching, silently
```

```
hit ratio vs key cardinality, same 1M requests over 10k logical objects

  key = path                      10,000 keys   hit ratio ~ 99%
  + query string (utm/fbclid)    ~600,000 keys  hit ratio ~ 12%
  + Vary: User-Agent            ~4,000,000 keys hit ratio ~ 2%
                                                origin load 50x higher
```

THE KEY INSIGHT

Hit ratio is inversely proportional to key cardinality. Every dimension you add to the key — a query parameter, a Vary header, a cookie — splits the cached population. Normalise the key to the smallest set that still returns a correct response.

THE TRAP

`Vary: Cookie` arriving by accident, usually from a framework that sets a session cookie on every response. Symptom: CDN hit ratio reported as 3%, origin traffic unchanged after enabling the CDN, and the cache dashboard shows millions of unique keys for a few thousand URLs. Strip cookies on asset routes at the edge, and do not start sessions for anonymous static requests.

```
cache keys — GAINS: normalisation can move hit ratio from 12% to 99% · COSTS: care; over-
normalising serves the wrong variant · ACCEPTS: personalisation and caching are fundamentally in
tension (Topic 26)
```

25. Invalidation versus versioned URLs

There are exactly two ways to stop a cache serving an old object: tell every cache to forget it (purge), or change its name so the old one is never requested again (versioning). The second is strictly better wherever you control the URL.

```
PURGE (fights 300 PoPs + browsers you do not control)
  aws cloudfront create-invalidation --distribution-id E123 \
      --paths "/img/logo.png"
  propagation 5-90 s; first 1,000 paths/month free, then $0.005 each
  browsers still hold their copy until max-age expires -- you cannot purge those

VERSIONING (nothing to invalidate)
  /assets/app.9f2b1c3a.js        immutable, max-age=31536000
  deploy -> /assets/app.7d4e01b2.js   new URL, new object, no purge, no wait
  old object simply stops being requested and ages out of the cache

Laravel/Vite already do this:  @vite('resources/js/app.js')
  emits /build/assets/app-DhK3n2.js from a content hash in the manifest.
```

Purge remains necessary for content you do not name — a legally required takedown, a leaked file, a user's profile image at a stable URL. For those, prefer **surrogate keys / cache tags**: tag responses with `Surrogate-Key: route-12 operator-8` and purge by tag, so one schedule change invalidates 400 related pages in one call.

THE KEY INSIGHT

Invalidation is a distributed-systems problem; renaming is a build-step problem. Convert the first into the second whenever you own the URL — this is the same "make it immutable and version it" move as event sourcing and content-addressed storage.

THE TRAP

Caching `/index.html` for a year alongside hashed assets. Symptom: after a deploy, some users get a blank page or a JS error, because their cached HTML references `app.OLDHASH.js` which no longer exists — and it persists for however long you set max-age, with no way to reach those browsers. The entry point must always be `no-cache`.

```
invalidation — GAINS(versioning): instant, free, no propagation window · COSTS: a build step and
a manifest · ACCEPTS(purge): 5–90 s propagation, $0.005/path, and browser copies you cannot reach
at all
```

26. Edge compute and dynamic acceleration

Not everything cacheable is static, and not everything dynamic must travel to the origin. Two distinct tools: **edge compute** runs your code in the PoP; **dynamic acceleration** keeps the request uncached but routes it over the CDN's warm, optimised backbone.

```
// Edge function: personalise at the edge without splitting the cache key
// (CloudFront Functions / Cloudflare Workers / Fastly Compute)
function handler(event) {
  const req = event.request;
  const country = req.headers['cloudfront-viewer-country'].value;  // BD, IN...
  // rewrite to a per-country variant: 5 cache keys, not 5 million
  req.uri = `/${country === 'BD' ? 'bd' : 'intl'}${req.uri}`;
  return req;
}

Latency of dynamic acceleration, Dhaka -> us-east-1 origin:
  direct:  TCP 1 RTT + TLS 1 RTT + request 1 RTT = 3 x 230 ms = 690 ms
  via CDN: client-PoP 3 x 8 ms  = 24 ms
           PoP-origin over a warm, pooled, keep-alive TLS connection
           = 1 x 230 ms (no handshake -- already established)
  total ~254 ms vs 690 ms:  2.7x faster for an UNCACHEABLE request
```

```
where to put the logic

  edge function    <1 ms, no state, no DB     redirects, headers, A/B split,
                                              auth token checks, URL rewrites
  edge worker      ~5 ms, KV/cache access     personalisation, geo pricing
  regional origin  full app, full DB          anything transactional
  RULE: the edge is for DECIDING; the origin is for CHANGING STATE.
```

THE KEY INSIGHT

Even with a 0% hit ratio, a CDN is worth having: it terminates TLS near the user and reuses a warm connection to your origin, removing two of three round trips from every dynamic request.

THE TRAP

Putting authorisation logic in an edge function that reads no shared state. Symptom: a revoked token keeps working for up to the edge cache's TTL, because revocation lives in a database the edge never consults. Validate signatures and expiry at the edge (cheap, stateless), but always check revocation at the origin for anything that can move money.

```
edge compute — GAINS: 2–3× faster dynamic requests, personalisation without key explosion ·
COSTS: per-invocation billing, a second deployment target, limited runtime · ACCEPTS: no strongly
consistent state at the edge
```

27. TLS 1.3 and HTTP/3 at the edge

Protocol choices at the edge are pure round-trip arithmetic, and the edge is where they pay off most because the client-to-PoP RTT is small but the number of handshakes is enormous.

```
connection setup cost, in round trips (RTT = client to PoP)

  TCP + TLS 1.2      1 (TCP) + 2 (TLS) = 3 RTT before the first byte
  TCP + TLS 1.3      1 + 1             = 2 RTT
  TCP + TLS 1.3 resumed (PSK, 0-RTT)   = 1 RTT (data rides the first packet)
  QUIC/HTTP3 new     1 RTT (crypto is inside the transport handshake)
  QUIC/HTTP3 resumed 0 RTT

  at 8 ms PoP RTT:  TLS1.2 24 ms -> HTTP/3 resumed 0 ms of setup
  at 230 ms origin RTT: the same saving is 690 ms -> 0 ms (Topic 26)
```

Feature	HTTP/2	HTTP/3 (QUIC over UDP)
Transport	TCP	UDP, congestion control in user space
Head-of-line block	At TCP level: one lost packet stalls all streams	Per stream only — the real win on lossy mobile
Connection migration	Breaks on IP change (WiFi→cellular)	Survives via connection ID
Setup	2 RTT (1 resumed)	1 RTT (0 resumed)
CPU cost	Lower (kernel TCP, TLS offload)	~2× higher per byte at the server

THE KEY INSIGHT

HTTP/3's advantage is not bandwidth, it is *loss tolerance*. On a 2% packet-loss mobile link, HTTP/2 stalls every stream on each loss; HTTP/3 stalls one. That is where the measured 10–30% page-load improvements on mobile networks come from.

THE TRAP

0-RTT data is replayable by design. An attacker who captures a 0-RTT packet can resend it, so a 0-RTT `POST /bookings` can create two bookings. Symptom: rare duplicate writes with identical payloads and no client-side retry to explain them. Restrict 0-RTT to idempotent methods, or require an idempotency key (Topic 63).

```
TLS 1.3 / HTTP3 — GAINS: 1–3 RTT saved per connection; per-stream loss recovery · COSTS: ~2× server CPU per byte for QUIC; UDP blocked on some networks · ACCEPTS: 0-RTT replay risk on non-idempotent requests
```

28. Range requests and video delivery

Large files are not fetched whole. Clients ask for byte ranges, and CDNs cache those ranges as separate chunks, so a user who watches 10 seconds of a 2 GB video causes only a few megabytes to move.

```
GET /video/eid-promo.mp4 HTTP/1.1
Range: bytes=0-1048575

HTTP/1.1 206 Partial Content
Content-Range: bytes 0-1048575/2147483648
Accept-Ranges: bytes
Content-Length: 1048576
Cache-Control: public, max-age=604800

Seeking to 12:30 in a 2 GB file:
  player computes the byte offset from the index, issues
  Range: bytes=1073741824-1075838976  -> only 2 MB fetched, not 1 GB.
CDN "segmented caching" stores each 1-2 MB block as its own cache object.
```

```
adaptive bitrate (HLS/DASH) -- what the player actually does

  master.m3u8  ---> 240p/480p/720p/1080p variant playlists
                       |
                   seg0001.ts (6 s, ~1.2 MB at 1.6 Mbps)
                   seg0002.ts   each segment is an independent cache object
                   ...          with a long TTL; only the LIVE playlist is
                                short-TTL (2-6 s)

  bandwidth math, 10,000 concurrent viewers at 1080p (5 Mbps):
    10,000 x 5 Mbps = 50 Gbps sustained
    per hour: 50 Gbps x 3600 / 8 = 22.5 TB/hour = at $0.085/GB, ~$1,900/hour
  -> video economics are bandwidth economics; everything else rounds to zero
```

THE KEY INSIGHT

Segment your large media into independently cacheable objects with long TTLs and keep only the manifest short-lived. Then a live stream is just a rapidly changing list of very cacheable files — which is why HLS scales to millions of viewers on ordinary HTTP caches.

THE TRAP

An origin that does not support `Range`. Symptom: seeking in a video is impossible or restarts the download, mobile users burn their data plan, and origin egress is 50× the expected figure because every seek re-fetches the whole file. Serve media from object storage (which supports ranges natively), never from a PHP controller that `readfile()`s the whole object.

```
range/video — GAINS: fetch only what is watched; segments cache perfectly · COSTS: an encoding
pipeline, 4–6× storage for renditions · ACCEPTS: bandwidth is the dominant cost and scales
linearly with viewers
```

29. Security at the edge: signed URLs, WAF, bots

The edge is the only place where you can reject an attack before it consumes your capacity. Three mechanisms cover most needs: signed URLs for private content, a WAF for known attack patterns, and bot management for automated abuse.

```
SIGNED URL -- authorisation without a database lookup at the edge
  https://cdn.example.com/tickets/9f2b.pdf
      ?Expires=1786800000
      &Signature=Q3RhY2s...
      &Key-Pair-Id=K2JCJMDEHXQW5F

  edge verifies the signature and expiry locally: ~0.05 ms, no origin call.
  TTL discipline: 60-300 s for a download link; anything longer is a
  shareable capability token (Topic 77 covers the upload direction).

WAF rules that earn their cost:
  - request body size cap (10 MB)     - SQLi/XSS managed rule sets
  - per-URI rate limit (Topic 20)     - geo rules for admin paths
  - block known-bad ASNs on /login    - challenge on >N failed logins

BOT REALITY (typical public site): 30-50% of raw requests are automated.
  good bots (search crawlers)  -> allow, but rate limit
  scrapers (fare aggregators)  -> challenge or serve cached-only
  credential stuffing          -> JS/managed challenge at the edge
  Blocking these at the edge is ~0.05 ms; at the app it is a PHP-FPM worker
  and a database query, i.e. ~200x more expensive per rejected request.
```

THE KEY INSIGHT

A signed URL moves authorisation from a request your servers handle to a signature the edge verifies. It is the only way to serve private content at CDN speed without putting your auth system on the critical path.

THE TRAP

Signed URLs with a long expiry on cacheable content. Symptom: a ticket PDF link shared in a group chat still works three weeks later, and the CDN happily serves the cached object to everyone who has the URL. Keep expiry short, include the object path in the signature, and set `Cache-Control: private` for per-user objects so shared caches never store them.

```
edge security — GAINS: attacks rejected at ~0.05 ms and $0 origin cost · COSTS: WAF per-request
fees, false positives on legitimate traffic · ACCEPTS: signed URLs are bearer tokens — anyone
holding the URL is authorised
```

30. CDN economics, and when not to use one

CDNs are sold on speed and bought on bandwidth. Do the arithmetic before assuming one is free money.

```
Case A -- image-heavy site, 72 TB/month, global users
  origin-only egress   72,000 GB x $0.09  = $6,480  + origin CPU + p95 800 ms
  CDN (92% hit)        origin 5,760 GB    =   $518
                       CDN 72,000 GB x $0.085 = $6,120
                       total $6,638, p95 -> 90 ms
  -> cost roughly flat, latency 9x better, origin fleet ~4x smaller. Buy it.

Case B -- internal admin tool, 40 users in one office, 30 GB/month
  CDN cost ~$3, added complexity: cache-key bugs, purge steps, one more
  vendor in every incident. -> Do not buy it. Put nginx in front.

Case C -- Bangladesh-only ticketing site, one region, 6 TB/month
  users are 8-40 ms from Dhaka/Singapore already; CDN saves ~20 ms on
  assets, and $0. Worth it for asset offload and DDoS absorption, NOT
  for the latency story. Judge it on origin CPU saved, not on ms.

Zero-egress providers (Cloudflare R2, some flat-rate CDNs) change Case A
from "cost-neutral" to "-$6,000/month", which is why bandwidth-heavy
businesses migrate storage rather than optimise code.
```

Do not use a CDN when	Because
All users are in one metro, all content dynamic and private	Nothing is cacheable; you pay for a proxy hop
Content changes per request (dashboards, carts)	Hit ratio ~0; use edge acceleration only (Topic 26)
Strict data-residency rules	Copies land in jurisdictions you did not choose
Very low traffic	The operational overhead exceeds the benefit

THE KEY INSIGHT

Justify a CDN with the two numbers it actually moves: origin requests avoided (fleet size, Topic 3) and p95 for users far from your region (Topic 2). "It is faster" is not a justification; "it removes 94% of asset requests and 7 of 11 app instances" is.

THE TRAP

The CDN becoming an availability dependency you did not plan for. Symptom: a provider incident takes your site down even though your origin is healthy, and you cannot fail away because DNS points at their anycast IPs with a 300 s TTL. Keep an origin hostname that works without the CDN, tested monthly, and keep the failover TTL low on that record specifically.

```
CDN economics — GAINS: latency, origin offload, DDoS absorption · COSTS: $0.08–0.12/GB, cache-correctness bugs, a vendor in the request path · ACCEPTS: a new global SPOF unless you keep a tested bypass
```

PRACTICE SET 3 — Topics 21–30

A. Conceptual

1. Explain how enabling a CDN can *increase* origin load, and the single setting that prevents it.
2. Why is `stale-while-revalidate` more valuable than a longer `max-age`?
3. State the difference between `max-age` and `s-maxage`, and give one route where you would set them differently.
4. Why does adding `Vary: Cookie` effectively disable shared caching?
5. Why is renaming an object better than purging it, in one sentence about distributed systems?
6. A request is completely uncacheable. Give two reasons to still route it through a CDN.
7. Why must the HTML entry point be `no-cache` when the assets are `immutable`?
8. What does HTTP/3 fix that HTTP/2 could not, and on which network does it matter most?
9. Why is 0-RTT unsafe for a booking POST?
10. Explain why a signed URL is a bearer token, and what follows for its expiry.
11. Name two situations where a CDN is the wrong choice.
12. Why do video systems make the manifest short-lived and the segments long-lived?

B. Pen-and-paper

1. Edge hit ratio 0.88, shield hit ratio 0.75. What fraction of requests reaches the origin? At 9,000 asset requests/s, what is origin RPS with and without the shield?
2. Monthly asset egress is 55 TB. Compute origin egress at a 94% total offload, then the bill at $0.09/GB origin and $0.085/GB CDN. Repeat with a zero-egress CDN.
3. 10,000 logical objects, 1 M requests. Adding `fbclid` to the cache key produces on average 40 distinct keys per object. Estimate the new hit ratio and the multiple by which origin requests increase.
4. Client RTT to PoP is 12 ms; origin RTT is 210 ms. Compute time-to-first-byte for an uncacheable request (a) direct with TLS 1.3, (b) via a CDN with a warm origin connection.
5. A 90-minute film at 5 Mbps: give the file size, the bytes fetched by a viewer who watches 4 minutes then seeks to the end and watches 2 minutes, assuming 6-second segments.
6. 25,000 concurrent viewers at 3 Mbps: compute sustained Gbps, TB per hour, and hourly cost at $0.085/GB.
7. CloudFront invalidation: you purge 4,000 paths per month. Compute the cost given 1,000 free and $0.005 per path thereafter. Compare with the cost of content-hashed filenames.
8. A TLS 1.2 client on a 40 ms link loads a page with 18 assets over 6 connections. Count round trips for connection setup only, then recompute for TLS 1.3 with session resumption.

C. Lab tasks (build → break → observe → fix)

1. Put nginx in front of a static directory as a caching proxy (`proxy_cache_path`). Serve 1,000 requests and record `$upstream_cache_status` counts (HIT/MISS/EXPIRED).
2. **Break it:** include `$args` in `proxy_cache_key`, then replay the same 1,000 requests with a random `?fbclid=` on each. Record the hit ratio and the number of files created in the cache directory. Fix by dropping the query string from the key and re-measure.
3. **Break it:** add `Vary: Cookie` to a cached asset response while your app sets a session cookie. Observe the hit ratio collapse and inspect the stored variants. Fix by stripping cookies on the asset location.

4. **Break it:** set `Cache-Control: public, max-age=600` on a page that renders the logged-in user's name, then request it as user A and again as user B through the shared cache. Record what user B sees. Fix with `private, no-store` enforced in middleware and verify.
5. Set `max-age=31536000, immutable` on `/index.html`, ship a "new release" with different asset hashes, and observe the stale HTML requesting a missing asset (404 in the console). Fix the header and record how long a browser kept the bad copy.
6. Implement `stale-while-revalidate=60` in nginx (`proxy_cache_use_stale updating` plus `proxy_cache_background_update on`). Expire an object and drive 200 concurrent requests. Count how many reached the origin, with and without the setting.
7. Serve a 200 MB file from object storage and from a PHP controller using `readfile()`. Issue `curl -r 100000000-100001000` against both and record the bytes transferred and the time.

ANSWER KEY 3 — Topics 21–30

A. Conceptual

1. Each PoP caches independently, so a long-tail object requested once per hour misses in **every** PoP. **Enable the shield / mid-tier** so 300 misses become 1.
2. Because it decouples freshness from user-visible latency: the user always gets an instant response and **exactly one** refresh request per window reaches the origin. A longer max-age just serves staler data with the same stampede when it finally expires.
3. `max-age` applies to all caches including the browser; `s-maxage` applies to shared caches only. Example: an API list with `max-age=0, s-maxage=60` — the CDN absorbs the load, the browser always revalidates.
4. It creates **one cached variant per distinct cookie value**, and session cookies are unique per user, so the cache stores a private copy per user and never serves a hit.
5. Purging requires coordinating hundreds of independent caches plus browsers you cannot reach; **renaming requires no coordination at all**.
6. (i) TLS terminates at the PoP, removing 2 of 3 setup round trips; (ii) the PoP→origin hop uses a warm pooled connection over an optimised backbone. Also DDoS absorption and WAF.
7. The HTML is the only file that names the hashed assets. If it is cached for a year, browsers keep requesting the **old hashes**, which no longer exist → blank page.
8. **Per-stream loss recovery** (no TCP head-of-line blocking) plus connection migration; it matters most on **lossy mobile networks**, where measured gains are 10–30%.
9. 0-RTT data is **replayable**: an attacker resends the captured packet and creates a duplicate booking. Restrict to idempotent methods or require an idempotency key (Topic 63).
10. Possession of the URL **is** the authorisation — there is no identity check. Therefore expiry must be short (60–300 s) and the signature must bind the path.
11. Single-metro fully dynamic private traffic; and very low traffic where operational complexity dominates. (Also strict data residency.)
12. Segments never change once written, so they cache for days; the manifest is the only thing that changes as new segments appear, so it must be revalidated every few seconds.

B. Worked

1. Origin fraction = $(1-0.88)(1-0.75) = 0.12 \times 0.25 =$ **0.03 = 3%**. With shield: 9,000 × 0.03 = **270 RPS**. Without: 9,000 × 0.12 = **1,080 RPS** — the shield removed 810 RPS.
2. Origin = 55,000 × 0.06 = **3,300 GB → $297**. CDN = 55,000 × $0.085 = **$4,675**. Total **$4,972**. Zero-egress CDN: **$297** plus request fees — a $4,675/month difference.
3. Keys go from 10,000 to ~400,000; average requests per key falls from 100 to 2.5, so the hit ratio falls from ~99% to roughly **60%** for repeat-heavy traffic and far lower for one-shot links — origin requests rise from 1% to 40% of traffic, i.e. **~40×**.
4. (a) TCP 1 + TLS 1.3 1 + request 1 = 3 × 210 = **630 ms**. (b) 3 × 12 = 36 ms to the PoP, plus one 210 ms origin round trip on a warm connection = **246 ms** — **2.6× faster**.
5. Size = 5 Mbps × 5,400 s / 8 = **3.375 GB**. Watched 6 minutes total = 360 s = 60 segments × (5 Mbps × 6 s / 8 = 3.75 MB) = **225 MB** — 6.7% of the file.
6. 25,000 × 3 Mbps = **75 Gbps**. Per hour: 75/8 × 3,600 = 33,750 GB = **33.75 TB/hour** → **$2,869/hour**.
7. 3,000 billable × $0.005 = **$15/month**, plus a 5–90 s propagation wait per deploy and browser copies you still cannot reach. Content hashing costs **$0** and propagates instantly.

8. TLS 1.2: 6 connections × 3 RTT = 18 RTT × 40 ms = **720 ms** of setup. TLS 1.3 resumed: 6 × 1 RTT = **240 ms**; with HTTP/2 on one connection it is **40 ms**.

C. What to verify

1. Expect 1 MISS then ~999 HIT per object; `$upstream_cache_status` should show HIT > 99%.
2. Expect hit ratio ≈ **0%** and **one cache file per request** (check with `find cache/ -type f | wc -l`) — disk fills, eviction thrashes. After removing `$args`: back to >99%.
3. Expect the cache to store a variant per session cookie; hit ratio falls to the rate at which one browser repeats a request. nginx will show MISS for every new visitor.
4. Expect **user B to see user A's name** within the max-age window — a real data leak reproducible in 30 seconds. After the fix: MISS/no-store on every request and no shared copy.
5. Expect a JS 404 and a blank page; the browser keeps the bad HTML for the **full max-age** with no way to invalidate it — commonly observed as "clear your cache" support tickets lasting days.
6. Without the setting: expect **~200 origin requests** (a stampede). With `proxy_cache_use_stale updating`: expect **1** origin request and 199 stale-but-instant responses.
7. Object storage: **~1 KB transferred, 206 status, tens of ms**. PHP `readfile()`: typically **200 MB transferred, 200 status**, seconds of transfer and one PHP-FPM worker occupied throughout — the Topic 28 trap, measured.

HANDNOTE SUMMARY CARD — PART 3

```
CDN AND THE EDGE
================================================================================
TOPOLOGY        client -> edge PoP -> mid-tier/shield -> origin
  origin fraction = (1-h_edge)(1-h_shield)   0.90/0.80 -> 2% reaches origin
  no shield = 300 PoPs each miss for the same object (CDN can RAISE load)

HEADERS (the whole contract)
  public | private | no-cache (store+revalidate) | no-store (never)
  max-age=N   all caches        s-maxage=N   shared caches only
  immutable   never revalidate  ETag/If-None-Match -> 304, ~200 B, 1 RTT
  stale-while-revalidate=N  instant response + ONE background refresh
  stale-if-error=N          serve stale when the origin is down

  hashed asset  public,max-age=31536000,immutable
  index.html    no-cache                      <- ALWAYS
  public list   public,s-maxage=60,swr=300
  authed page   private,no-store              <- middleware, not per route
  money/seats   no-store

CACHE KEY       default = scheme+host+path
  every added dimension divides hit ratio: query string, Vary, cookie
  Vary: Accept-Encoding OK | Accept-Language risky | User-Agent/Cookie fatal
  strip utm/fbclid; strip cookies on asset routes

INVALIDATION    rename > purge
  content hash = instant, free, no propagation | purge = 5-90 s, $0.005/path
  browsers cannot be purged at all -> that is why index.html is no-cache
  surrogate keys / cache tags for content you do not name

EDGE COMPUTE    edge DECIDES, origin CHANGES STATE
  even at 0% hit ratio: TLS at PoP + warm origin conn = 3 RTT -> 1 RTT
  verify signature+expiry at edge; check revocation at origin

PROTOCOL RTT    TLS1.2 3 | TLS1.3 2 | resumed 1 | QUIC 1 | QUIC resumed 0
  HTTP/3 wins on LOSS (per-stream), not bandwidth; ~2x server CPU
  0-RTT is replayable -> idempotent methods only

MEDIA           Range: bytes=a-b -> 206 Partial Content
  HLS/DASH: segments long TTL, manifest 2-6 s
  10k viewers x 5 Mbps = 50 Gbps = 22.5 TB/h = ~$1,900/h

ECONOMICS       justify with origin requests avoided + p95 for distant users
  do NOT use: one metro + all-private, ~0% hit ratio, data residency, tiny site
  keep a tested origin bypass hostname -- the CDN is a global SPOF

TRAPS
  missing `private` = cross-user leak | Vary: Cookie | index.html cached
  long-lived signed URL = shareable capability | origin without Range support
================================================================================
```

PART FOUR

The cache layer

A cache is a bet that the next request wants what the last one wanted. The bet pays 20–60× on latency and turns 20,000 QPS into 1,000. It is also the component most likely to serve a wrong answer confidently, so half this part is about invalidation and the other half about what breaks when the bet is off.

Part 4 · The cache layer

31. Why caching works: skew, hit ratio and the latency it buys

Caching is not "memory is fast." Caching works because request distributions are skewed: a small set of keys absorbs most of the traffic. On a bus-ticketing search endpoint with 400 routes, the 12 busiest routes are typically 75–80% of searches. Cache those 12 and you have paid for the whole layer.

The distribution is usually Zipf-like: request frequency of the n-th most popular key falls roughly as $1/n$. That is what makes a 5 GB cache in front of a 2 TB database sensible — 0.25% of the data serves 95% of the reads.

```
Mean latency, cache-aside (Topic 33), checked first:

  T = t_cache + (1 - h) * t_db          h = hit ratio

  t_cache = 0.4 ms (same-AZ Valkey), t_db = 25 ms (indexed query + pool wait)

  h = 0.00   T = 25.4 ms      backend QPS at 20k front QPS = 20,000
  h = 0.80   T =  5.4 ms                                    4,000
  h = 0.90   T =  2.9 ms                                    2,000
  h = 0.95   T =  1.65 ms                                   1,000
  h = 0.99   T =  0.65 ms                                     200
  h = 0.999  T =  0.43 ms                                      20

The interesting column is the right one. Latency improves 2.5x from h=0.90
to h=0.99; backend load improves 10x. Caching is a capacity technique that
happens to also be a latency technique.
```

```
requests    key rank (Zipf, s=1)          cumulative share of traffic
  ####################  rank 1-12    ......................  78%
  ######                rank 13-50   ......................  91%
  ##                    rank 51-400  ......................  100%
                        |            |
                        cache these  everything below here is a miss you
                        (5 GB)       pay 25 ms for, at 9% of traffic
```

THE KEY INSIGHT

Quote hit ratio and backend QPS together, never latency alone. "95% hit ratio" means nothing until you say "so the database sees 1,000 QPS instead of 20,000, and it can do 3,000." That sentence is the design.

THE TRAP

Measuring hit ratio over the whole keyspace and declaring victory. Symptom: a 94% overall hit ratio and a p99 that is still 180 ms. The misses are not random — they concentrate on one expensive query whose rebuild takes 600 ms. Measure hit ratio *per key prefix*, and weight it by the cost of the miss (Topic 38).

```
caching — GAINS: 10× backend headroom at h=0.99, 20× latency · COSTS: RAM at ~$5/GB/month, a
second consistency domain · ACCEPTS: every value served may be up to one TTL old
```

32. The cache hierarchy: six places a value can already be

"Add a cache" is under-specified — there are six caches in a normal request path, and they differ by 5 orders of magnitude in latency and by everything in invalidation control. Choose the layer by how much staleness the data tolerates and by who else needs the same value.

```
Layer              Latency      Shared by     You invalidate by
-----------------  -----------  ------------  --------------------------
1 Browser/app      0 ms         one user      you cannot -- TTL only (T23)
2 CDN edge         10-30 ms     one region    purge API, seconds (T25)
3 App in-process   100 ns-1 us  one process   process restart / pub-sub
4 Distributed      0.3-1 ms     all processes DEL, instantly (T40)
5 DB buffer pool   50-200 us    one DB node   automatic, LRU
6 Disk (NVMe)      80-300 us    --            n/a

A 20 ms API response, decomposed:
  0.4 ms  Valkey GET (layer 4, hit)
  1.2 ms  app render
  0.3 ms  TLS + LB (Topic 17)
 18.1 ms  network to a user 1,400 km away
                     <-- layer 2 attacks this one, nothing else can
```

THE KEY INSIGHT

Layers 3 and 4 solve different problems. In-process is 400× faster and has no network, but N app servers hold N divergent copies you cannot purge. Use in-process only for data whose staleness window can be a full TTL — feature flags, route metadata, currency tables — never for anything a user just edited.

THE TRAP

Caching at layer 4 what layer 5 already caches. Symptom: you add Valkey in front of a 4 GB table on a database with 32 GB of buffer pool, and p50 gets *worse* — you replaced a 100 µs in-process page read with a 400 µs network round trip. Cache the *result* of expensive work (a 12-table join, a rendered fragment), not rows the buffer pool already holds.

```
layering — GAINS: each layer removes a whole class of work · COSTS: N invalidation stories, N
staleness windows · ACCEPTS: the layers disagree with each other during every write
```

33. Cache-aside: the default pattern, and the four lines that matter

Cache-aside (lazy loading) puts the application in charge: check cache, on a miss load from the source, then write it back. The cache knows nothing about the database. It is the default because it fails safe — if the cache is down, every request is a miss and the system is slow but correct.

```
function getTrip(int $id): array {
    $key = "trip:v3:{$id}";
    $hit = $redis->get($key);
    if ($hit !== false) {
        return json_decode($hit, true);            // ~0.4 ms
    }
    $row = $db->selectOne('SELECT ... WHERE id = ?', [$id]);   // ~25 ms
    if ($row === null) {
        // negative cache -- see Topic 39, short TTL on purpose
        $redis->setex($key, 30, 'null');
        return [];
    }
    $ttl = 300 + random_int(0, 60);                // jitter -- Topic 35
    $redis->setex($key, $ttl, json_encode($row));
    return $row;
}
```

```
$ redis-cli -h cache01 --stat
------- data ------ --------------------- load -------------------
keys       mem      clients blocked requests            connections
1284431    4.21G    312     0       88213441 (+12043)    41288

$ redis-cli info stats | grep keyspace
keyspace_hits:83914022
keyspace_misses:4299419        -> h = 0.951
```

THE KEY INSIGHT

Cache-aside populates only what is actually requested, so the cache automatically converges on the working set of Topic 31 with no configuration. That is why it beats pre-warming everything: the access pattern chooses the contents.

THE TRAP

The version-less key. Symptom: you change the shape of the cached array, deploy, and half your servers crash on `$row['seats_left']` because the old shape is still in Valkey for another 300 s. Put a schema version in the key (`trip:v3:`) and bump it in the same commit that changes the shape — a deploy then invalidates exactly what it broke, at zero cost.

```
cache-aside — GAINS: simple, cache-outage-tolerant, self-tuning contents · COSTS: every miss pays
cache RTT + DB, first request is always slow · ACCEPTS: the read-write race of Topic 41
```

34. Read-through, write-through, write-behind, write-around

The other four patterns move responsibility from the application into the cache layer or change what happens on writes. They differ in exactly two dimensions: who fetches on a miss, and whether the write goes to cache, store, or both.

```
Pattern        Read miss             Write path            Data loss window
-------------  --------------------  --------------------  ----------------
Cache-aside    app loads, app sets   app writes DB, DEL    none
Read-through   cache loads (plugin)  --                    none
Write-through  --                    app writes cache,     none
                                     cache writes DB sync
Write-behind   --                    app writes cache,     up to flush
                                     cache flushes async   interval (!)
Write-around   --                    app writes DB only,   none
                                     cache untouched

Latency of a write, measured (p50):
  write-through   0.4 (cache) + 12 (DB fsync) = 12.4 ms
  write-behind    0.4 ms                       -> 30x faster, and lossy
  write-around    12 ms, and the next read is a guaranteed miss
```

```
WRITE-BEHIND, the failure that defines it

 t=0    app --SET--> cache [ok, 0.4 ms]      user sees "Booking saved"
 t=0.2s cache buffer: 1,840 dirty keys
 t=0.9s cache process killed (OOM, Topic 36)
 t=0.9s  -> 1,840 bookings that were acknowledged never reached MySQL
        -> no error anywhere; the data simply does not exist
```

THE KEY INSIGHT

Write-around is the right default for write-heavy, read-rarely data (audit logs, raw event rows). Writing them through a cache pollutes it with entries that will be evicted before anyone reads them, dropping the hit ratio of the keys that matter — you spend RAM to make Topic 31 worse.

THE TRAP

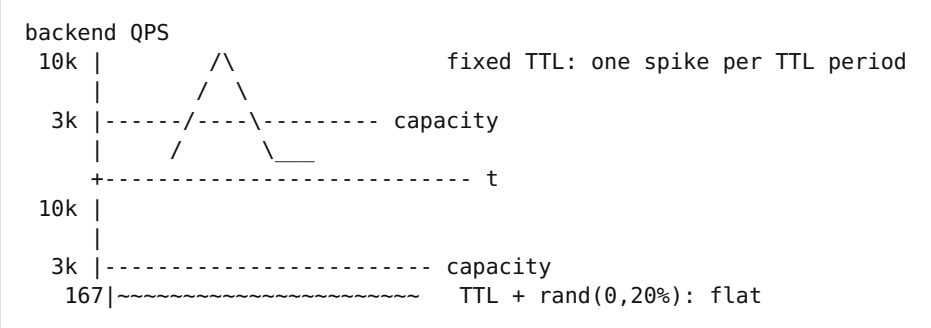
Write-behind for anything a user is told succeeded. Symptom: a customer has a payment SMS and a booking reference, and your database has no row. Write-behind is legitimate only for counters, view tallies and analytics — data where losing the last 5 seconds is a rounding error, not a dispute. Everything with money in it is write-through or DB-first.

```
write patterns — GAINS: write-behind absorbs 30× write bursts · COSTS: write-through doubles
write latency · ACCEPTS: write-behind trades durability for throughput, explicitly
```

35. TTL design, and why every TTL needs jitter

The TTL is the staleness contract: it is the maximum time a user can see an old value when nothing else invalidates. Pick it from tolerance, not habit — and then break the synchronisation that a fixed TTL creates.

```
Data                          TTL      Why
---------------------------  -------  --------------------------------
Route/stop metadata           24 h     changes monthly, staleness harmless
Rendered search results       60 s     users retry; 60 s of staleness = ok
Seat availability             0        never cache -- Topic 43
User profile                  300 s    plus explicit DEL on save (T40)
Auth/permission               60 s     revocation must take effect fast
Negative (key not found)      30 s     short: the row may appear (T39)

WHY JITTER, with numbers:
  10,000 keys are populated during a 09:00 traffic ramp, TTL = 300 s.
  At 09:05:00 all 10,000 expire in the same second.
  Backend sees 10,000 rebuilds in ~1 s against a 3,000 QPS database.
  -> 3.3 s of queueing, pool exhaustion (Topic 45), 504s at the LB.

  With TTL = 300 + rand(0,60):
  the same 10,000 expiries spread over 60 s = 167 QPS. Nothing happens.
```



THE KEY INSIGHT

Any two cache entries created together will expire together unless you add noise. Jitter of ±10–20% costs one line and removes an entire class of periodic outage — the same defence as backoff jitter in Topic 68 and health-check jitter in Topic 15.

THE TRAP

A very long TTL used as a substitute for invalidation. Symptom: support reports "the price is wrong for some users" for a whole day and you cannot reproduce it, because the wrong value lives in one of 8 app-local caches with TTL 86400. Long TTLs are only safe with an explicit invalidation path (Topic 40); without one, cap TTL at what you are willing to be wrong for.

```
TTL + jitter — GAINS: bounded staleness, no synchronised expiry · COSTS: one line of randomness,
slightly variable freshness · ACCEPTS: values are wrong for up to TTL by design
```

36. Eviction: what happens when memory runs out

A cache is a fixed-size box, so the interesting behaviour is what it discards. In Valkey/Redis this is one setting, and the wrong value turns a cache into a database that refuses writes.

```
# valkey.conf
maxmemory 6gb                    # ALWAYS set. Default 0 = unlimited = OOM kill
maxmemory-policy allkeys-lru     # default is noeviction -- see the trap
maxmemory-samples 5              # approximated LRU: sample 5, evict oldest

  noeviction      reject writes with OOM error once full   (correct for a
                  queue or a source-of-truth store, wrong for a cache)
  allkeys-lru     evict least-recently-used of ALL keys    <-- cache default
  allkeys-lfu     evict least-FREQUENTLY-used              <-- skewed traffic
  volatile-lru    evict LRU among keys that have a TTL     <-- the trap
  volatile-ttl    evict the soonest-to-expire first
  allkeys-random  cheapest, surprisingly ok for uniform access
```

```
$ redis-cli info stats | grep evicted
evicted_keys:2841193            # rising fast = cache is too small
$ redis-cli info memory | grep -E 'used_memory_human|maxmemory_human'
used_memory_human:5.98G
maxmemory_human:6.00G           # pinned at the ceiling: always-evicting

Sizing check: 1.28M keys / 4.21 GB = 3.4 KB average value.
Working set from Topic 31 was 5 GB -> 6 GB ceiling is 20% headroom.
Hit ratio fell 0.951 -> 0.902 when evicted_keys started climbing; each
0.01 of hit ratio here is +200 QPS on MySQL.
```

THE KEY INSIGHT

LFU beats LRU exactly when traffic is Zipf-skewed and occasionally scanned. One analytics job that reads 500,000 cold keys will flush an LRU cache completely; LFU keeps the 12 hot routes of Topic 31 because they have counters, not just recency. Symptom of the wrong choice: hit ratio collapses on a schedule matching a batch job.

THE TRAP

`volatile-lru` with keys that have no TTL. Symptom: writes start failing with `OOM command not allowed when used memory > 'maxmemory'` even though the box has free RAM, because no key is eligible for eviction. Use `allkeys-*` for a cache; reserve `volatile-*` for an instance that mixes cache entries with data you must not lose — which is itself an anti-pattern (Topic 42).

```
eviction — GAINS: bounded memory, automatic working-set tracking · COSTS: approximated LRU is a
sample, not exact · ACCEPTS: a full cache silently drops the tail of your keyspace
```

37. What Valkey/Redis actually is: one thread, and the commands that stop it

Command execution is single-threaded. Every command runs to completion before the next begins, which is why `INCR` is atomic without locks — and why one `O(n)` command over a 3-million-element key stalls every other client on the node.

```
O(1)     GET SET INCR EXPIRE HSET LPUSH SETNX ZADD
O(log n) ZRANGEBYSCORE (+k), ZRANK
O(n)     KEYS *  SMEMBERS  HGETALL  LRANGE 0 -1   <-- n = size of the key
O(n)     FLUSHALL, DEBUG SLEEP, and any Lua script you write badly

$ redis-cli --latency
min: 0, max: 214, avg: 0.42 (1443 samples)   <-- max 214 ms = someone
                                                  ran KEYS on 1.2M keys

Instead:  SCAN 0 MATCH 'trip:v3:*' COUNT 200    # cursor, 200 keys per call,
                                                    never blocks > ~1 ms

Memory overhead per key is real:
  1,000,000 keys x (key string +
  ~50-90 B of dict/robj/expire overhead) = 60-90 MB before any values.
  Storing 1M counters as 1M keys: ~110 MB.
  Storing them in one HASH (ziplist-encoded below 128 fields
  per hash, so shard it): ~16 MB. 7x less.
```

```
Pipelining, the other 100x

 without:  [GET][RTT][GET][RTT]...  100 keys x 0.4 ms RTT = 40 ms
 with:     [GET GET GET ... GET][RTT]            = 0.4 ms + 100 x ~2 us
 MGET k1..k100 is better still: one command, one round trip.

 Valkey 8+/Redis 8 add I/O threads: parsing and socket work go
 multi-core, command execution stays single-threaded. It raises
 throughput ~2-3x on large payloads; it does not make KEYS safe.
```

THE KEY INSIGHT

Because execution is serial, the cache's p99 is set by the worst command anyone runs, not by your command. A cache "being slow" is nearly always one client doing something `O(n)` — find it with `SLOWLOG GET 10` before you resize anything.

THE TRAP

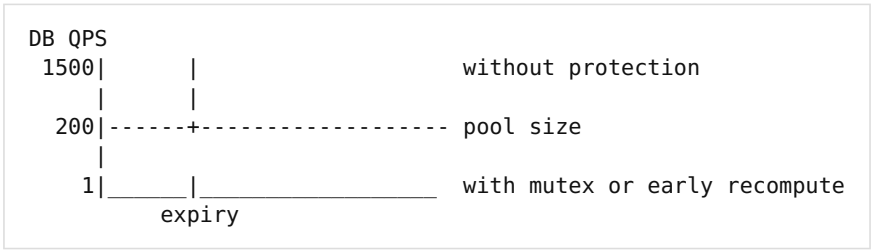
`KEYS pattern` in production code, usually inside a "clear the cache for this user" helper. Symptom: p99 across every endpoint jumps to 200 ms+ at exactly the times an admin saves a record, and the cache CPU shows one thread at 100%. Replace it with a version-key bump (Topic 40) or `SCAN`.

```
the single thread — GAINS: free atomicity, no lock bugs, predictable 0.4 ms · COSTS: one core per
shard, hostile to O(n) commands · ACCEPTS: any slow command is a full-node outage for its
duration
```

38. Cache stampede: 1,500 clients rebuilding the same key

A popular key expires. Every request that arrives before the first rebuild finishes also misses, and also rebuilds. The database receives one expensive query multiplied by concurrency — the single most common way a cache layer causes the outage it was installed to prevent.

```
Key: search:dhaka-chittagong:2026-08-14   -- 5,000 QPS on this key alone
Rebuild cost: 300 ms (12-table join + availability calc)

  t=0.000  TTL expires
  t=0.000  request 1 misses  -> starts rebuild
  t=0.001  requests 2-6      -> also miss, also rebuild
  ...
  t=0.300  request 1500      -> 1,500 identical 300 ms queries in flight
           MySQL: 1,500 of 200 pool slots -> queue -> all endpoints stall

FIX A -- mutex (one rebuilder, others serve stale or wait):
  if (!$redis->set("lock:$key", $id, ['NX', 'EX' => 10])) {
      return $stale ?: retryAfter(50);   // 1 rebuild, 1499 cheap paths
  }

FIX B -- probabilistic early expiry (XFetch); no lock, no thundering herd:
  now + delta * beta * log(rand())  >  expiry   ->  rebuild early
  delta = measured rebuild time (0.3 s), beta = 1.0
  Each request has a small, rising chance of refreshing BEFORE expiry, so
  the rebuild happens while the old value is still servable.

FIX C -- never let it expire: background job refreshes every 30 s; the
  cached entry has TTL 300 s purely as a safety net.
```



THE KEY INSIGHT

Stampede protection is only needed where rebuild cost × key popularity is large. Compute that product per key prefix; for most prefixes it is under 1 and a plain TTL is fine. This is the same "one refill per window" discipline as `stale-while-revalidate` at the CDN (Topic 23).

THE TRAP

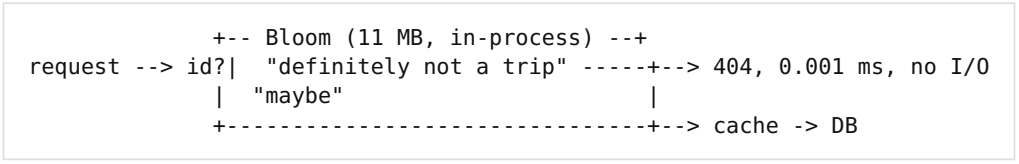
A mutex with no timeout and no fallback. Symptom: the rebuilder crashes mid-rebuild, the lock lives until its EX expires, and for 10 seconds every request returns an error instead of the perfectly good stale value you threw away. Always keep the stale value (`SET` a second key with 2× TTL), and always give the lock an expiry.

```
stampede control — GAINS: backend load at expiry drops 1500→1 · COSTS: a lock key, or serving
stale for ~300 ms · ACCEPTS: someone reads a slightly stale value so everyone else stays up
```

39. Penetration, hot keys and negative caching

Three miss-side failures that a hit-ratio dashboard hides: keys that never exist, keys that are too popular for one node, and the interaction between them.

```
PENETRATION -- requests for keys that do not exist anywhere.
  Attack or bug: GET /trip/999999991, /trip/999999992, ...
  Cache misses (nothing to cache), DB misses, 100% of traffic reaches MySQL.
  20,000 QPS of "SELECT ... WHERE id = ?" returning 0 rows -> DB down.

  Fix 1: negative cache. setex($key, 30, 'null'). Cost: one entry per bogus
         id -- so bound it, or an attacker fills your cache instead.
  Fix 2: Bloom filter of valid ids, in front of everything.
         n = 10,000,000 ids, target false-positive p = 1%
         m = -n ln p / (ln 2)^2  = 95.8 Mbit = 11.4 MB
         k = (m/n) ln 2          = 6.6 -> 7 hash functions
         11 MB answers "definitely not present" for 99% of bogus ids,
         with zero false negatives. A miss on the filter never touches MySQL.

HOT KEY -- one key exceeding a single node's capacity.
  Valkey node: ~120,000 ops/s. Flash sale key: 300,000 GET/s.
  Sharding does not help: one key lives in one slot on one node (Topic 42).
  Fix: replicate the value under N suffixed keys (key:0..key:9) and pick
       at random -> 10 nodes share it; or cache it in-process (layer 3,
       Topic 32) with a 5 s TTL, which removes the network entirely.
```



THE KEY INSIGHT

A Bloom filter answers "no" with certainty and "maybe" with a tunable error rate. That asymmetry is exactly what a cache miss path needs, and 11 MB of it replaces 10 GB of negative cache entries. Same structure appears in LSM-tree storage engines to skip SSTable reads.

THE TRAP

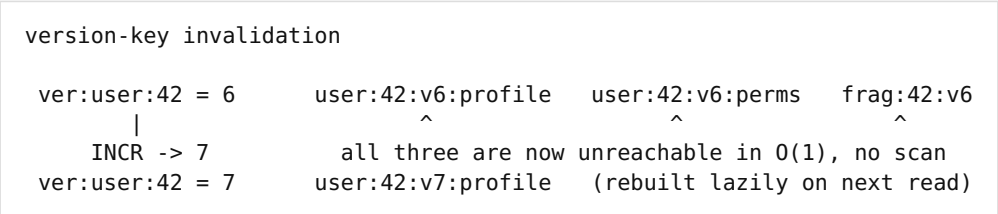
Negative caching with the same TTL as positive entries. Symptom: a user creates a booking, immediately opens its page, and gets 404 for five minutes. The `null` written when the row genuinely did not exist is now outliving the creation. Negative TTL should be 10–60 s, and the write path must `DEL` the negative entry.

```
miss-side defence — GAINS: bogus traffic costs 0.001 ms instead of 25 ms · COSTS: 11 MB/10M ids,
a rebuild job when ids are deleted · ACCEPTS: 1% of bogus ids still reach the database
```

40. Invalidation: TTL, event, and the version key

There are exactly three ways to make a cached value stop being served, and choosing among them is most of cache design. They compose: nearly every production system uses TTL as the floor and one of the other two on top.

```
1. TTL ONLY          value dies on a clock. Simplest, always safe as a
                     backstop. Staleness = up to TTL. Topic 35.

2. EVENT-DRIVEN      write path deletes the key.
     $db->update(...);
     $redis->del("user:v2:{$id}");        // AFTER commit -- Topic 41
   Precise, instant. Fails when: you forget one write path (an admin
   tool, a migration, a cron), or the DEL is lost in a cache failover.

3. VERSION KEY       never delete anything; change the name.
     $v = $redis->incr("ver:user:{$id}");        // 7
     key = "user:{$id}:v{$v}:profile"
   One INCR invalidates every derived key for that entity at once --
   profile, permissions, rendered fragments, all of them -- without
   knowing their names. Old entries are never read again and die by TTL.
   Cost: one extra GET per read (pipeline it), and dead entries occupy
   memory until eviction (Topic 36).

WHAT NOT TO DO:  $redis->keys("user:{$id}:*")  -- Topic 37, blocks the node.
```



THE KEY INSIGHT

Version keys convert invalidation from "find and delete N keys" into "write one integer." That is the only invalidation strategy that stays correct as the number of derived caches grows, because the write path does not need to know what exists downstream.

THE TRAP

Event-driven invalidation with no TTL. Symptom: one path forgot to invalidate — typically a bulk import or an admin action — and a wrong value is served *forever,* surviving deploys, with no expiry to save you. Always keep a TTL as the backstop even when you invalidate explicitly. The TTL is what bounds your worst bug.

```
invalidation — GAINS: version keys are O(1) regardless of derived-key count · COSTS: one extra
read per request, dead memory until eviction · ACCEPTS: TTL remains the true upper bound on
wrongness
```

41. The read-write race: how a correct-looking cache serves stale data forever

Cache-aside has one interleaving that writes a stale value *after* the invalidation and pins it for a full TTL. It is rare per request and inevitable at scale, and it is why "delete, don't update" and "delete after commit" are rules rather than preferences.

```
READER R (miss path)                WRITER W (update path)
-----------------------------       -----------------------------
t0  GET trip:9  -> miss
t1  SELECT ... -> price = 500
                                    t2  UPDATE trips SET price=650
                                    t3  COMMIT
                                    t4  DEL trip:9        (deletes nothing)
t5  SET trip:9 = 500 (TTL 300)
                                    -- cache now holds 500 for 300 s
                                    -- the database holds 650
                                    -- no error, no log line, no alert
```

```
WHY "DELETE" NOT "UPDATE" ON WRITE:
  Two concurrent writers doing SET cache = their own value can commit to
  MySQL in one order and to the cache in the other. DEL has no ordering
  problem: whoever reads next reloads the committed truth.

WHY "AFTER COMMIT" NOT BEFORE:
  DEL before COMMIT lets a reader repopulate from the pre-commit snapshot
  (readers do not see uncommitted rows). You have then cached the old
  value with a fresh TTL -- the exact bug above, made routine.

MITIGATIONS, in increasing order of strength:
  a) Short TTL. Bounds damage to 300 s. Free. Usually enough.
  b) Delayed double delete: DEL, commit, sleep ~500 ms, DEL again.
     Kills the value the racing reader wrote. Ugly, effective, common.
  c) Set-if-not-exists on the read path: SET key val NX EX 300
     The racing reader's write loses to any newer entry.
  d) CDC-driven invalidation (Topic 59): the binlog is the single
     ordered source of invalidations, so no application path can miss
     one and no interleaving can invert. This is the real fix.
```

THE KEY INSIGHT

The window is exactly "how long a reader holds a value between reading the database and writing the cache" — typically 1–30 ms. At 20,000 QPS with 100 writes/s that is a handful of poisoned keys per hour, each stale for a full TTL. Design for it or measure it; it will not announce itself.

THE TRAP

Believing the race is fixed because it never appears in staging. Symptom in production: one specific row is wrong, refreshing does not help, and it corrects itself after exactly the TTL — then reappears days later. That signature (self-healing after TTL) is this race and nothing else.

```
the read-write race — GAINS: none; it is a cost of cache-aside · COSTS: bounded by TTL, or a CDC
pipeline to remove entirely · ACCEPTS: a small number of entries are stale for one full TTL
```

42. Topology: replicas, Cluster, slots and hash tags

One cache node has a memory ceiling (RAM on the box) and a throughput ceiling (one core for commands, Topic 37). You pass them differently, and the choice changes what your application can express.

```
SINGLE + REPLICAS (Sentinel)
  primary handles writes, N replicas serve reads, Sentinel promotes on
  failure. Capacity: 1 x RAM. Reads scale, writes do not.
  Failover: 10-30 s of write unavailability, and replicas are ASYNC --
  acknowledged writes can be lost on promotion (same physics as Topic 47).

CLUSTER (sharded)
  16,384 hash slots spread over N primaries. slot = CRC16(key) mod 16384.
  Capacity: N x RAM, N x throughput. 3 primaries + 3 replicas is the
  smallest sane deployment.

  $ redis-cli -c -h cache01 get trip:v3:88
  -> MOVED 12182 10.0.3.7:6379          # client is redirected, then caches
                                          the slot map (1 extra RTT, once)

  THE CONSTRAINT: multi-key commands must be in ONE slot.
  MGET trip:v3:1 trip:v3:2   -> CROSSSLOT Keys in request don't hash
                                to the same slot
  Fix -- hash tags: only the text inside {} is hashed.
  MGET {trip:88}:price {trip:88}:seats     -> same slot, one node, legal
  Cost of hash tags: you have chosen the shard by hand, so a hot entity
  is a hot node (Topic 39).

CAPACITY ARITHMETIC
  Working set 5 GB, peak 400,000 ops/s, value 3.4 KB.
  Per node: 120,000 ops/s, 16 GB usable.
  Throughput-bound: ceil(400k/120k) = 4 primaries. Memory needs 1.
  -> 4 primaries + 4 replicas, r6g.xlarge-class. Throughput, not RAM,
     sized this cluster -- check both, the binding one varies.
```

```
client --slot map--> [P1 slots 0-4095 ] --async--> [R1]
                     [P2 slots 4096-8191] -------> [R2]
                     [P3 slots 8192-12287] ------> [R3]
                     [P4 slots 12288-16383] -----> [R4]
    MOVED redirect teaches the client; steady state is 1 RTT, 1 hop.
```

THE KEY INSIGHT

Cluster mode removes the memory ceiling but takes away cross-key atomicity: no multi-key transaction, no Lua script, no MGET across slots unless you tag them. Design keys for slot locality *before* you need the cluster, or migrating into one becomes a rewrite.

THE TRAP

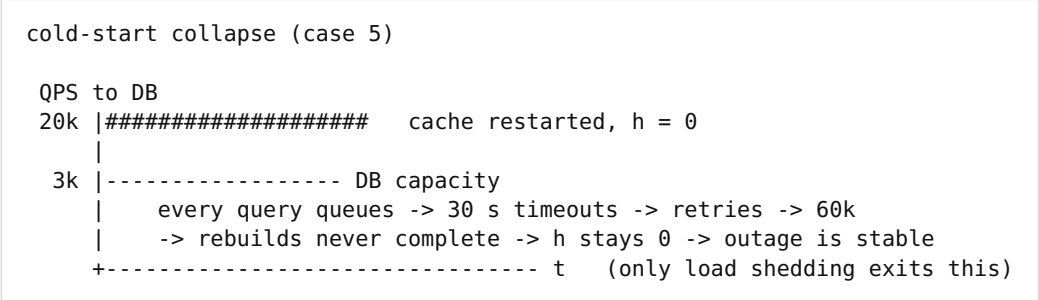
Sharing one cache instance between cache entries and things you cannot lose — sessions, queues, rate-limiter state. Symptom: an eviction storm (Topic 36) logs users out en masse, or the rate limiter resets and lets a flood through. Separate instances, separate `maxmemory-policy`: cache is `allkeys-lru`, state is `noeviction` and monitored.

```
cluster — GAINS: N× RAM and N× ops/s, automatic slot failover · COSTS: no cross-slot atomicity, a
slot map in every client · ACCEPTS: async replication, so failover can lose the last few writes
```

43. When not to cache

The strongest sentence in this part. A cache is a second copy of the truth with its own failure modes, and there are five situations where it is a clear net loss.

```
1. NO REUSE. Hit ratio is bounded by how often a key is read twice within
   its TTL. Per-user, per-request keys (a filtered search with 9 free-text
   parameters) are read once. h ~ 0.02: you added 0.4 ms and a bug surface
   to 98% of requests.
   Test first: log key hashes for an hour, count the repeats.

2. WRITE-HEAVY, READ-RARE. A key written 10x per read spends its life being
   invalidated. Write-around (Topic 34) or nothing.

3. MONEY AND CONTENTION. Seat availability, inventory, wallet balance. A
   500 ms stale "3 seats left" sells the same seat twice (Topic 118). Cache
   the trip metadata, never the count. The correct pattern is a
   transactional read of the source with row locks -- Topic 54.

4. THE DATABASE ALREADY HAS IT. 4 GB table, 32 GB buffer pool, single-row
   lookups at 100 us. Adding a 400 us network hop makes p50 four times
   worse (Topic 32). Cache computation, not storage.

5. THE CACHE BECAME LOAD-BEARING. If your database cannot serve the traffic
   at h = 0, a cache restart is an outage: 20,000 QPS hits a 3,000 QPS
   database, everything times out, retries make it 60,000 (Topic 97), and
   the cache cannot refill because every rebuild query is queued behind
   the stampede. This is the cache-layer outage nearly everyone eventually
   has.
   -> Requirement: either the origin survives h=0 at reduced quality
      (load shedding, Topic 99), or you warm the cache before taking
      traffic, or you accept a documented cold-start outage window.
```



THE KEY INSIGHT

Ask "what is my p99 with the cache empty?" during design, not during the incident. If the honest answer is "we are down," the cache is not an optimisation — it is an undeclared dependency with no failover, and it belongs in your capacity plan (Topic 104) and your DR test.

THE TRAP

Caching to hide a missing index. Symptom: hit ratio 0.97 and everything is fine until a deploy invalidates a prefix, whereupon the 2.5 s query behind it takes the site down. Fix the query first (Topic 44); a cache in front of an unindexed query is a loaded gun with a TTL for a trigger.

```
not caching — GAINS: one fewer consistency domain, no stale-data class of bug · COSTS: the
database must actually be sized for the load · ACCEPTS: higher steady-state cost in exchange for
a system that degrades instead of collapsing
```

PRACTICE SET 4 — Topics 31–43

A. Conceptual

1. Why is backend QPS a better argument for a cache than mean latency?
2. Give one type of data that belongs in an in-process cache and one that must never be there, with the reason in terms of invalidation.
3. Why does cache-aside fail safe when the cache is down, and which pattern does not?
4. Explain why write-behind is acceptable for a page-view counter and unacceptable for a booking.
5. Two keys are created in the same second with the same TTL. What happens 300 seconds later, and what one change prevents it?
6. Why does `volatile-lru` cause write failures on a box with free RAM?
7. An analytics job runs at 02:00 and the cache hit ratio collapses at 02:04 every night. Name the eviction policy in use and the one you would switch to.
8. Why is `KEYS` a full-node outage rather than a slow query?
9. State the difference between cache penetration and cache stampede in one sentence each.
10. Why does a Bloom filter never need to be consulted about false negatives?
11. Why is "delete the key" safer than "update the key" on a write path?
12. What is the exact signature, in symptoms, of the Topic 41 read-write race?
13. Why can a hot key not be fixed by adding cache nodes?
14. Name two things that must not share a cache instance with your cache entries, and why.

B. Pen-and-paper

1. Front-end traffic is 34,000 QPS. Cache RTT 0.4 ms, database 25 ms. Compute mean latency and backend QPS at h = 0.85, 0.95 and 0.99. If the database sustains 2,500 QPS, what is the minimum acceptable hit ratio?
2. A cache holds 1.28 M keys in 4.21 GB. You expect 4× the keys next year at the same average value size. Size the instance with 25% headroom, and say whether memory or throughput binds if peak is 450,000 ops/s and a node does 120,000.
3. 12,000 keys are populated between 08:58 and 09:00 with TTL 600 s. The database sustains 3,000 QPS. Show the expiry spike, then recompute with TTL = 600 + rand(0,120).
4. A key receives 4,000 QPS and takes 250 ms to rebuild. How many concurrent rebuilds occur on expiry without protection? Your pool has 200 connections — what is the queue depth?
5. Size a Bloom filter for 40 M valid ids at p = 0.5%. Give m in MB and k. How much RAM would the equivalent negative cache need at 80 B/entry for 5 M distinct bogus ids?
6. Storing 2 M counters: (a) as 2 M top-level keys at ~80 B overhead each, (b) sharded into hashes of 100 fields. Estimate both, and the ratio.
7. Traffic 34,000 QPS, values 2 KB, working set 9 GB, node capacity 120,000 ops/s and 16 GB usable. How many primaries? How many nodes in total with one replica each?
8. Write orders: a reader misses at t=0 and reads price 500 from the database; a writer commits 650 at t=3 ms and deletes at t=4 ms; the reader sets at t=9 ms with TTL 300. For how long is the wrong value served, and what does the user see at t=310 s?

C. Lab tasks (build → break → observe → fix)

1. **Build.** Put Valkey in front of one slow endpoint using cache-aside with TTL 300 + jitter. Record `keyspace_hits` and `keyspace_misses` before and after 5 minutes of load (`hey -z 60s -c 50` or similar), and compute h.
2. **Break: the stampede.** Set TTL to 30 s on a key you hit with 200 concurrent clients and make the rebuild sleep 300 ms. Watch `SHOW PROCESSLIST` (or `pg_stat_activity`) at the moment of expiry and record the peak concurrent query count and the p99. Then add the `SET NX EX` mutex and repeat. Expect the peak to fall to 1.
3. **Break: the read-write race.** Add a 200 ms sleep between the database read and the `SET` in your miss path. In another terminal, update the row and `DEL` the key inside that window. Read the key afterwards. Record what is in the cache, what is in the database, and how long the divergence lasts. Fix it with `SET ... NX` and show the race lose.
4. **Break: the eviction cliff.** Set `maxmemory 64mb` and `maxmemory-policy volatile-lru`, then write keys with no TTL until it fills. Capture the exact error string. Switch to `allkeys-lru`, repeat, and record `evicted_keys` and the hit ratio instead.
5. **Break: the O(n) command.** Load 500,000 keys, run `redis-cli --latency` in one terminal and `KEYS 'trip:*'` in another. Record max latency. Repeat with `SCAN ... COUNT 200`. Then find the offender with `SLOWLOG GET 5`.
6. **Break: cold start.** With load running at your normal rate, `FLUSHALL`. Record time-to-recover, database CPU, error rate, and whether the hit ratio ever climbs back on its own. This is Topic 43 case 5 — if it does not recover, write down the load-shedding rule that would save it.
7. **Fix and measure.** Convert one entity's invalidation from per-key `DEL` to a version key. Verify with `MONITOR` that a save now issues one `INCR` instead of N deletes, and that stale keys become unreachable immediately.

ANSWER KEY 4 — Topics 31–43

A. Conceptual

1. Latency improves 2.5× between h=0.90 and h=0.99; backend load improves 10×. **The cache is bought to create database headroom** — latency is the visible side effect, capacity is the reason.
2. In-process: feature flags, route metadata, currency tables — **tolerate a full TTL of staleness and are identical for all users**. Never: anything a user just edited, because **you cannot purge N process copies** (Topic 32).
3. Cache-aside treats a cache error as a miss, so the system degrades to database-only. **Write-behind does not** — its buffered writes are the only copy, so cache loss is data loss (Topic 34).
4. A lost counter increment is a rounding error in a metric; **a lost booking is a customer with a receipt and no reservation**. Durability requirement, not performance requirement.
5. They expire in the same second, producing a synchronised miss storm. **Add jitter: TTL + rand(0, 10–20%)**.
6. Only keys *with a TTL* are eviction candidates. If none has one, nothing is evictable, so the instance returns `OOM command not allowed` despite the box having RAM (Topic 36).
7. It is **allkeys-lru**: the batch job's cold keys are the most recently used, so the hot set is evicted. **Switch to allkeys-lfu**, which keeps keys with high access counters.
8. Command execution is single-threaded (Topic 37), so an O(n) command over 1.2 M keys **blocks every other client for its whole duration** — measured at 214 ms. Every endpoint's p99 rises together.
9. Penetration: **requests for keys that exist nowhere**, so the cache can never help and 100% of that traffic reaches the database. Stampede: **one popular key expires and N concurrent requests rebuild it simultaneously**.
10. A Bloom filter can say "maybe present" for an absent element, but **never "absent" for a present one** — setting bits is monotone. So a "no" is authoritative and safe to serve as 404.
11. Two writers can reach the database in one order and the cache in the other, leaving the cache holding the older value. **DEL has no ordering problem**: the next reader reloads committed truth (Topic 41).
12. **One row is wrong, refreshes do not help, it self-corrects after exactly the TTL, then recurs days later.** Self-healing on the TTL boundary is the fingerprint.
13. A key maps to one slot, which lives on one primary (Topic 42) — **adding nodes adds slots, not copies of that key**. Fix by replicating the value under N suffixed keys or caching it in-process.
14. **Sessions and rate-limiter/queue state.** An eviction storm under `allkeys-lru` logs users out or resets limiter counters; those need `noeviction` and their own instance.

B. Worked

1. T = 0.4 + (1−h)×25. h=0.85: **4.15 ms, 5,100 QPS**. h=0.95: **1.65 ms, 1,700 QPS**. h=0.99: **0.65 ms, 340 QPS**. Need (1−h)×34,000 ≤ 2,500 → 1−h ≤ 0.0735 → **h ≥ 0.927**; design for 0.95 so a partial invalidation does not cross the line.
2. 4×1.28 M = 5.12 M keys × 3.29 KB ≈ **16.8 GB**; with 25% headroom **21 GB**. Throughput: 450,000/120,000 = 3.75 → **4 primaries**. Memory: 21/16 = 1.3 → 2. **Throughput binds; 4 primaries + 4 replicas.**

3. 12,000 expiries in one second against 3,000 QPS capacity = **4 s of saturation** plus queueing, pool exhaustion and 504s. With rand(0,120): 12,000/120 = **100 QPS** extra — 3.3% of capacity, invisible.
4. 4,000 QPS × 0.25 s = **1,000 concurrent rebuilds**. Pool is 200, so **800 requests queue**; at 250 ms of service time the last one waits ≈1.0 s, which exceeds most client timeouts and triggers the retry amplification of Topic 97.
5. m = −n ln p/(ln2)² = −40e6×ln(0.005)/0.4805 = **441 Mbit = 55 MB**; k = (m/n) ln2 = 11.0×0.693 = **7.6 → 8 hashes**. Negative cache: 5 M × 80 B = **400 MB**, and it only covers ids already seen — 7× the memory for strictly worse coverage.
6. (a) 2 M × 80 B ≈ **160 MB**. (b) 20,000 hashes × 100 fields, small-hash encoding ≈ **18–24 MB**. Ratio **≈7×** (Topic 37).
7. Throughput: 34,000/120,000 = 0.28 → 1. Memory: 9 GB with headroom → 12 GB → 1 node holds it. **Minimum 3 primaries anyway** (Cluster's smallest sane deployment, so no single node holds a third of the keyspace alone) **+ 3 replicas = 6 nodes**.
8. The DEL at t=4 removes nothing; the SET at t=9 writes 500 with a fresh TTL. **The wrong price is served for 300 s**. At t=310 s the entry has expired and the next reader loads **650, correctly** — the self-healing signature from A.12.

C. What to verify

1. h = hits/(hits+misses) should reach **0.90–0.98** within a minute on a repeating workload. If it is under 0.3, you have the no-reuse case of Topic 43.1 — check whether your key includes a per-request parameter.
2. Unprotected peak concurrency ≈ **QPS × rebuild time** (200 clients → up to 200 in flight; p99 jumps to **≥300 ms and often to the client timeout**). With the mutex: **concurrent rebuilds = 1**, others return stale or 202. The database CPU graph should show a spike replaced by a flat line.
3. Expected: cache holds the **old** value, database holds the new one, divergence lasts **the full TTL**. With `SET ... NX` the late writer's SET returns **0/false** and the key stays absent, so the next read reloads truth. Log the NX failure — in production its rate is your race rate.
4. Exact string: `OOM command not allowed when used memory > 'maxmemory'`. After switching, writes succeed, `evicted_keys` climbs monotonically, and hit ratio settles lower — that drop is the true cost of an undersized cache.
5. `--latency` max should jump from **<2 ms to 100–300 ms** during KEYS on 500 k keys, and stay under ~2 ms with SCAN. `SLOWLOG GET 5` prints the command, its microseconds and the client address — that is how you find it in production.
6. Expect database CPU to saturate immediately and hit ratio to climb slowly. If your database cannot serve the offered load, **the hit ratio does not recover** and errors persist — a stable failure. Record time-to-h=0.9 and compare it to your SLO burn rate (Topic 93); this number belongs in your runbook.
7. `MONITOR` should show a single `INCR ver:user:42` per save. Verify a stale key is unreachable by name immediately, and confirm memory does not grow unbounded — old versions must still be evictable (`allkeys-lru`).

HANDNOTE SUMMARY CARD — PART 4

```
================================================================================
 CACHE LAYER -- ONE PAGE                                          Topics 31-43
================================================================================
 MATH
   T = t_cache + (1-h) * t_db          backend_qps = (1-h) * front_qps
   0.4 + (1-h)*25 ms:   h=.80 5.4ms  .90 2.9  .95 1.65  .99 0.65
   34k QPS, DB does 2.5k  ->  h must be >= 0.927. Design 0.95.
   Bloom:  m = -n ln p / (ln2)^2      k = (m/n) ln 2
           10M ids @ 1% = 11.4 MB, 7 hashes
   Nodes:  max(ceil(peak_ops/120k), ceil(working_set*1.25/16GB)), min 3
--------------------------------------------------------------------------------
 PATTERNS                    read miss          write            loses data?
   cache-aside      app loads + SET      DB then DEL           no  <-- default
   read-through     cache plugin loads   --                    no
   write-through    --                   cache -> DB sync      no  (2x latency)
   write-behind     --                   cache, flush later    YES (counters
                                                                   only)
   write-around     --                   DB only               no  (write-heavy)
--------------------------------------------------------------------------------
 KEYS AND TTL
   trip:v3:{id}          version in key = free deploy-time invalidation
   ver:user:{id} INCR    O(1) invalidation of ALL derived keys
   TTL = base + rand(0, 20%)             ALWAYS. Fixed TTL = synchronised herd.
   metadata 24h | search 60s | profile 300s+DEL | authz 60s | negative 30s
   seats/inventory/balance: DO NOT CACHE
--------------------------------------------------------------------------------
 valkey.conf
   maxmemory 6gb                 # never leave at 0
   maxmemory-policy allkeys-lru  # allkeys-lfu if batch jobs scan cold keys
   maxmemory-samples 5           # volatile-* only if TTLs are guaranteed
--------------------------------------------------------------------------------
 STAMPEDE / PENETRATION / HOT KEY
   stampede    SET lock:k NX EX 10 -> 1 rebuilder, others serve stale
               or XFetch: now + delta*beta*ln(rand) > expiry -> refresh early
               or background refresher; TTL is only a safety net
   penetration negative cache 30s + Bloom filter of valid ids
   hot key     key:0..key:9 fanout, or in-process cache 5s (network removed)
--------------------------------------------------------------------------------
 TRAPS (symptom -> cause)
   p99 spikes when admin saves            -> KEYS in cache-clear helper (T37)
   OOM error with free RAM                -> volatile-lru, no TTLs      (T36)
   one row wrong, self-heals after TTL    -> read-write race           (T41)
   hit ratio collapses nightly at 02:04   -> LRU flushed by batch scan (T36)
   404 for 5 min after creating a record  -> negative TTL too long     (T39)
   users logged out during eviction storm -> sessions share the cache  (T42)
   CROSSSLOT ... don't hash to same slot  -> need {hash tags}          (T42)
   site down after cache restart          -> cache is load-bearing     (T43)
--------------------------------------------------------------------------------
 CLUSTER
   slot = CRC16(key) mod 16384        MOVED = client refreshes slot map
   multi-key ops must share a slot -> MGET {trip:88}:price {trip:88}:seats
   replicas are ASYNC: failover can lose acknowledged writes
   cache and session/limiter state NEVER share an instance
--------------------------------------------------------------------------------
 DO NOT CACHE WHEN
   read-once keys (h < 0.1)   | writes >> reads    | money + contention (T118)
   4GB table with 32GB pool   | DB cannot survive h=0 without shedding (T99)
================================================================================
```

PART FIVE

Database scaling

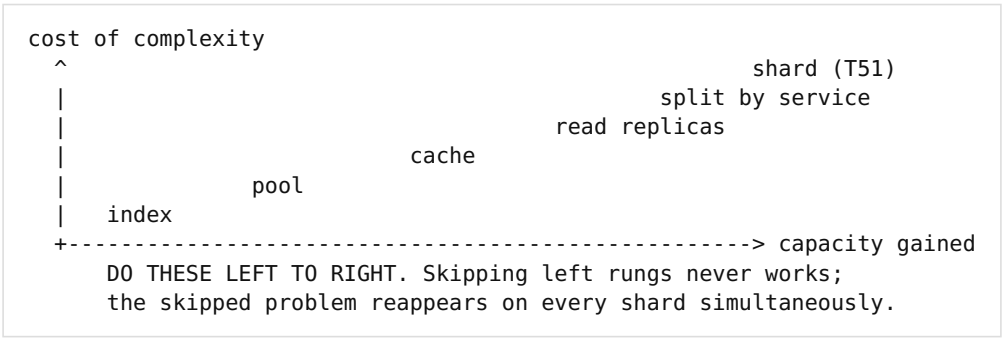
Every other component in this book is stateless or can be rebuilt. The database cannot, which is why it is the last thing you scale and the first thing that limits you. The order of the moves matters more than any single move: teams that shard before indexing end up with eight slow databases.

Part 5 · Database scaling

44. The ladder: the order in which you are allowed to scale

There is a fixed order, and each rung costs 10× more operational complexity than the one below it. Almost every "we need to shard" conversation is actually rung 1 or rung 3 unfinished.

```
Rung  Move                    Typical gain     Cost to you
----  ----------------------  ---------------  ------------------------
 1    Fix the query + index    10-1000x         an afternoon
 2    Bigger box (vertical)    2-8x             a restart, a bill
 3    Connection pooling       2-5x effective   a proxy process (T45)
 4    Cache in front           5-20x read       Part 4, staleness
 5    Read replicas            Nx read          replication lag (T46)
 6    Split by table/service   linear-ish       cross-service joins die
 7    Shard by key             linear           everything gets harder

RUNG 1, with the numbers that justify the rule:

mysql> EXPLAIN SELECT * FROM bookings
    -> WHERE user_id = 4821 AND status = 'confirmed' ORDER BY created_at DESC;
+----+-------------+----------+------+---------------+------+---------+
| id | select_type | table    | type | possible_keys | key  | rows    |
+----+-------------+----------+------+---------------+------+---------+
|  1 | SIMPLE      | bookings | ALL  | NULL          | NULL | 8412330 |
+----+-------------+----------+------+---------------+------+---------+
Extra: Using where; Using filesort            -> 8.4M rows, 2.1 s

mysql> CREATE INDEX idx_user_status_created
    ->   ON bookings (user_id, status, created_at DESC);

|  1 | SIMPLE      | bookings | ref  | idx_user_...  | idx_ |      23 |
Extra: Using index condition                  -> 23 rows, 0.9 ms

2,333x fewer rows examined. No new machine, no new failure mode.
A shard split would have given 8x -- and made this query worse.
```



THE KEY INSIGHT

Sharding multiplies capacity but divides nothing about query cost. A query examining 8.4 M rows on one node examines 1.05 M rows on each of eight — still 400 ms, now with a scatter-gather and eight failure domains. Fix the access pattern first; sharding is a multiplier applied to whatever you already have, including the bad parts.

THE TRAP

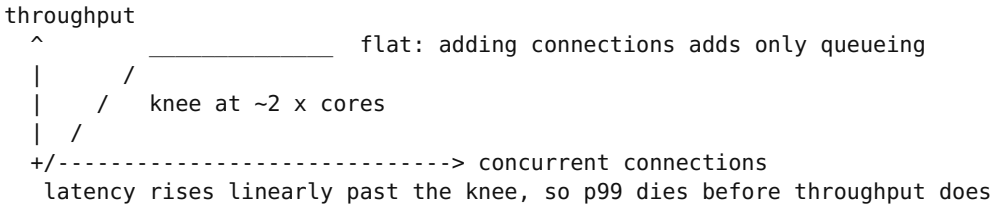
Vertical scaling as a permanent answer. Symptom: an 8×-larger instance gives 1.3× the throughput, because the bottleneck was a single write-lock hot spot or fsync latency, neither of which cares about core count. Measure what saturates — CPU, IOPS, lock waits, or connection slots — before buying; each has a different rung.

```
the ladder — GAINS: the cheapest capacity you own is an index · COSTS: discipline, and resisting
the interesting problem · ACCEPTS: rungs 1–4 delay but do not remove the eventual shard decision
```

45. Connection limits: the wall nobody plans for

A database connection is not free: in PostgreSQL it is an OS process, in MySQL a thread, each with several megabytes of memory and a share of the lock manager. Application servers, however, are scaled horizontally without asking, and each one opens a pool. The multiplication is what takes you down.

```
32 app containers x 20 pool connections = 640 connections wanted
PostgreSQL max_connections = 200            -> 440 rejected:
  FATAL: sorry, too many clients already
MySQL max_connections = 500, but at 500 threads:
  context switching + per-thread buffers (sort_buffer, join_buffer)
  = 500 x ~4 MB = 2 GB of RAM that is no longer buffer pool.

Useful connections is NOT max_connections. It is roughly:
  cores x 2 + effective_spindles      (Postgres rule of thumb)
  16 cores -> ~35 concurrently ACTIVE connections do useful work.
Above that, throughput is flat and latency rises linearly (queueing).

THE FIX -- a pooler that multiplexes:

# pgbouncer.ini
[databases]
app = host=db-primary port=5432 dbname=app
[pgbouncer]
pool_mode = transaction        # session|transaction|statement
max_client_conn = 5000         # what the app fleet may open
default_pool_size = 40         # what the database actually sees
reserve_pool_size = 10

640 client connections -> 40 server connections. The database never
knows the fleet grew.
```



THE KEY INSIGHT

Transaction-mode pooling is what makes the numbers work, because a web request holds a server connection only for the milliseconds it is inside a transaction, not for the seconds it is alive. That is a 10–50× reduction in server-side connections for the same traffic.

THE TRAP

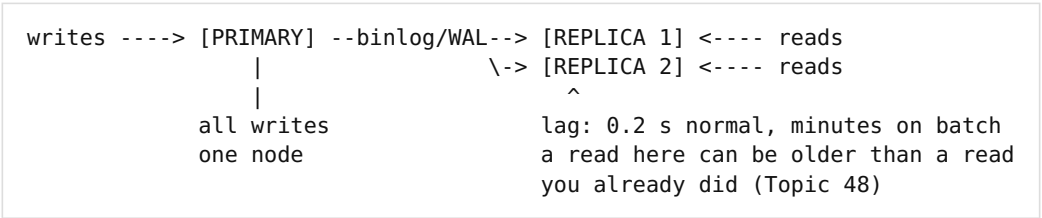
Transaction-mode pooling with session state. Symptom: intermittent, impossible bugs — a `SET search_path`, an advisory lock, a `PREPARE`, or a temporary table vanishes because the next statement landed on a different server connection. Session-scoped features require `pool_mode = session`, which forfeits most of the benefit. Keep requests stateless (Topic 8) and this never arises.

```
pooling — GAINS: 640→40 server connections, stable p99 · COSTS: a process in the path, session
semantics restricted · ACCEPTS: the pooler is now a component that can queue, saturate and fail
```

46. Read replicas and the lag you must design around

A replica applies the primary's write stream and serves reads. It multiplies read capacity linearly and does nothing for writes — and it is always behind, by an amount that varies with write volume, network, and how parallel the apply is.

```
mysql> SHOW REPLICA STATUS\G
      Replica_IO_Running: Yes
     Replica_SQL_Running: Yes
   Seconds_Behind_Source: 0            -- steady state
       ...during a 40-minute bulk import of 12M rows:
   Seconds_Behind_Source: 847          -- 14 minutes stale

postgres=# SELECT now() - pg_last_xact_replay_timestamp() AS lag;
  lag
-----------------
 00:00:00.214              -- 214 ms is normal
                           -- 00:14:07 during a batch job is also normal

WHAT CAUSES LAG
  single-threaded apply on the replica vs N parallel writers on the primary
  long transactions (one 9-minute UPDATE = 9 minutes of lag at the tail)
  replica also serving 4,000 QPS of reads -> apply competes for I/O
  network (cross-region: +80 ms floor, and any hiccup accumulates)

CAPACITY
  primary: 3,000 QPS mixed. Writes are 600 QPS and must stay on it.
  Each replica: ~2,800 QPS read.
  34,000 QPS front x (1 - 0.95 cache hit) = 1,700 QPS to the DB tier.
  1,700 - 600 writes = 1,100 read QPS -> 1 replica covers it; run 2
  so that losing one during a deploy is not an outage.
```



THE KEY INSIGHT

Replicas scale reads only. If your workload is 30% writes, replicas cap out at about 3× total capacity no matter how many you add — every replica must apply 100% of the writes. That ceiling is precisely when sharding (Topic 51) becomes the only remaining move.

THE TRAP

Routing all reads to replicas by default. Symptom: a user updates their profile, the page reloads, and the old value is back — reproducible only under load, never in staging where lag is 0 ms. This is Topic 48; the fix is a routing rule, not a bigger replica.

```
read replicas — GAINS: N× read capacity, a warm failover candidate · COSTS: a full copy per
replica, lag in every read path · ACCEPTS: reads are stale by an unbounded amount during batch
work
```

47. Synchronous, asynchronous, semi-synchronous

The replication mode decides exactly one thing: whether a committed write can disappear when the primary dies. Everything else — latency, availability during a replica outage — follows from it.

```
ASYNC (MySQL default, Postgres synchronous_commit=off/local)
  commit = fsync locally, reply to client, ship to replicas whenever.
  write latency: 8-12 ms      data loss on primary death: yes, the tail
  A failover promotes a replica missing the last N transactions.

SEMI-SYNC (MySQL rpl_semi_sync, Postgres synchronous_commit=remote_write)
  commit = local fsync + at least one replica ACKNOWLEDGES RECEIPT.
  write latency: 8-12 ms + 1 RTT (0.5 ms same-AZ, 1.5 ms cross-AZ, 70 ms
  cross-region)
  data loss: none for the acknowledged replica, unless both die together.

SYNC / QUORUM (Postgres synchronous_commit=on with N replicas, Galera,
Aurora's 4-of-6 quorum)
  commit = durable on a majority before the client hears "ok".
  write latency: local + slowest-of-quorum RTT
  data loss: none. Availability: if the quorum cannot be met, WRITES STOP.

# postgresql.conf
synchronous_standby_names = 'ANY 1 (replica_a, replica_b)'
synchronous_commit = on          # 'remote_apply' also waits for visibility

Cross-region sync writes, measured honestly:
  Dhaka -> Singapore RTT = 62 ms. Every write becomes 70+ ms.
  600 writes/s x 70 ms = 42 concurrent write transactions permanently
  in flight -- which is where your connection pool goes (Topic 45).
```

THE KEY INSIGHT

This is CAP made operational (Topic 8): synchronous replication chooses consistency and will refuse writes during a partition; asynchronous chooses availability and will lose the tail. There is no configuration that gives both, and every managed service is one of these three with a marketing name.

THE TRAP

Semi-sync with a timeout that silently degrades to async. Symptom: your runbook says "no data loss," but the primary's log shows `Semi-sync replication switched OFF` during the exact minute of the incident, because no replica acknowledged within the timeout. Alert on that log line; a durability guarantee that disables itself under load is not a guarantee.

```
replication mode — GAINS: sync = zero RPO · COSTS: +1 RTT per write, and write unavailability
without a quorum · ACCEPTS: async trades a few seconds of durability for latency and uptime
```

48. Read-your-writes, monotonic reads, and how to actually get them

Replica lag breaks two guarantees users assume without knowing their names. Both are fixed in the routing layer, cheaply, and both are commonly discovered in production instead.

```
READ-YOUR-WRITES   after I write X, I never read an older X.
MONOTONIC READS    I never see time go backwards across two reads.
                   (request 1 -> replica A, lag 50 ms: sees new value
                    request 2 -> replica B, lag 900 ms: sees old value)

FOUR FIXES, cheapest first

1. STICKY WINDOW. After any write by user U, route U's reads to the
   PRIMARY for N seconds.
     $redis->setex("rmw:$userId", 5, 1);       // Part 4 again
     $conn = $redis->exists("rmw:$userId") ? $primary : $replica;
   Covers ~99% of cases for 5 s of extra primary load. Most common fix.

2. STICKY REPLICA. Hash the user to one replica so at least their view
   never moves backwards (monotonic reads, not read-your-writes).

3. LSN / GTID FENCING -- the correct one.
   After commit, capture the position and require the replica to have it:
     postgres: SELECT pg_current_wal_lsn();     -> '3/9A2C1F8'
               (on replica) SELECT pg_last_wal_replay_lsn() >= '3/9A2C1F8'
     mysql:    SELECT @@gtid_executed; then on the replica
               SELECT WAIT_FOR_EXECUTED_GTID_SET('...', 1);   -- 1 s timeout
   Exact, per-request, no blanket primary load. Costs a token in the
   session/cookie and a branch in the router.

4. WRITE-THROUGH THE CACHE. On write, update the cache with the new value
   (not DEL). The user's next read hits the cache and never touches a
   replica. Beware Topic 41's ordering issue for concurrent writers.
```

```
t=0  POST /profile ----------------> PRIMARY   (commit, LSN 3/9A2C1F8)
t=40ms GET /profile --> REPLICA B (replayed to 3/9A2B00, BEHIND)
        -> user sees their old name; support ticket says "it didn't save"
     with fix 3: router sees replica LSN < required, falls back to primary
```

THE KEY INSIGHT

These are *session* guarantees, not global ones. You do not need the whole system to be linearizable — you need one user not to see their own history reversed. That is why an LSN token in the session costs almost nothing and a globally synchronous database costs everything (Topic 47).

THE TRAP

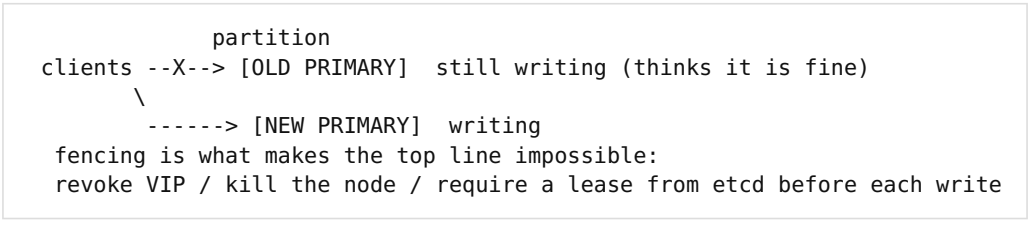
Fixing it by "waiting 200 ms before redirecting." Symptom: works for a year, then fails every day at 03:00 when the nightly import pushes lag to 14 minutes (Topic 46). A sleep is not a synchronisation primitive; use the LSN.

```
session guarantees — GAINS: correct user-visible behaviour without global consistency · COSTS: a token per session, a branch in the router · ACCEPTS: other users still see stale data, which is usually fine
```

49. Failover: promotion, split-brain and fencing

The primary dies. Something must notice, choose a replica, promote it, and redirect every client — without ever having two primaries accepting writes. That last clause is the whole problem.

```
TIMELINE OF A REAL FAILOVER (Patroni/Orchestrator-class tooling)
  t+0 s     primary stops responding
  t+10 s    3 consecutive health-check failures (Topic 15) -> suspicion
  t+12 s    leader election among managers; quorum of 3 agrees
  t+13 s    pick replica with highest LSN/GTID (least data loss)
  t+14 s    FENCE the old primary: revoke its VIP, or STONITH,
            or demote via etcd key so it self-demotes on write
  t+16 s    promote replica; it stops read-only mode
  t+18 s    update service discovery / VIP / proxy routing
  t+25 s    application pools reconnect (they must retry -- Topic 68)
  ------------------------------------------------------------
  ~25 s of write unavailability, plus whatever async writes were lost.

SPLIT BRAIN, the failure mode that eats data:
  the old primary was not dead, only unreachable (network partition,
  or a 40 s GC pause). It still holds the VIP for some clients.
    node A: accepts bookings 9001-9040
    node B (promoted): accepts bookings 9001-9040   <- same ids
  When the partition heals, one set must be discarded. There is no
  merge for "two people bought seat 14A."

WHY QUORUM: with 3 managers, a partitioned minority cannot elect. With
2, both sides think they are right. Always odd, always >= 3.
```



THE KEY INSIGHT

Failover is not "promote a replica" — it is "make it impossible for the old primary to accept another write, then promote." Fencing comes first. Every split-brain post-mortem is a fencing step that was skipped, timed out, or depended on the dead node's cooperation.

THE TRAP

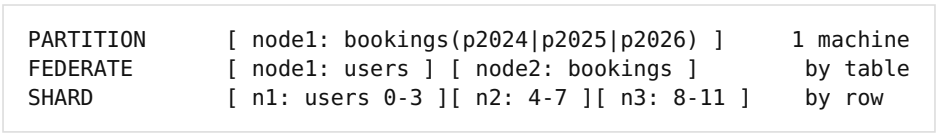
Untested failover. Symptom: at 02:00 the promotion works, and the application stays down anyway because the connection pool caches DNS for 300 s (Topic 11), or the app opened connections at boot and never retries. Practise it — a game day (Topic 105) with a real `kill -9` on the primary, in business hours, quarterly.

```
failover — GAINS: minutes of downtime instead of hours · COSTS: a consensus layer, a fencing
mechanism, quarterly drills · ACCEPTS: ~25 s of write unavailability and async tail loss
```

50. Partitioning, sharding, federation: three different words

These get used interchangeably and mean different things operationally. The distinction that matters is *how many machines the query planner can see*.

```
PARTITIONING (one database, one node, many physical sub-tables)
  ALTER TABLE bookings PARTITION BY RANGE (YEAR(created_at)) (
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p2026 VALUES LESS THAN (2027));
  Buys: partition pruning (scan 1 of 3), instant DROP PARTITION for
        retention instead of a 6-hour DELETE.
  Does not buy: capacity. Same disk, same CPU, same connection limit.

FEDERATION / VERTICAL SPLIT (different tables on different nodes)
  users+auth -> db1     bookings+trips -> db2     analytics -> db3
  Buys: independent scaling and blast radius per domain.
  Costs: JOIN across db1 and db2 no longer exists. You now do two
         queries and join in the application, or duplicate a column.

SHARDING / HORIZONTAL SPLIT (same table, rows spread over N nodes)
  bookings where user_id % 8 = 0 -> shard0 ... = 7 -> shard7
  Buys: linear write capacity, linear storage, linear connections.
  Costs: every cross-shard question becomes scatter-gather (T54),
         and the shard key is nearly unchangeable (T51).

WHICH ONE DO YOU NEED?
  disk full, old data cold        -> partitioning + retention drop
  one domain is 90% of load       -> federation
  writes exceed one primary       -> sharding, and only then
```



THE KEY INSIGHT

Partitioning is a storage-layout optimisation that runs inside one server; sharding is a distributed system with a routing layer, a rebalancing story, and no cross-node transactions. Calling both "partitioning" is how teams accidentally sign up for the second while planning the first.

THE TRAP

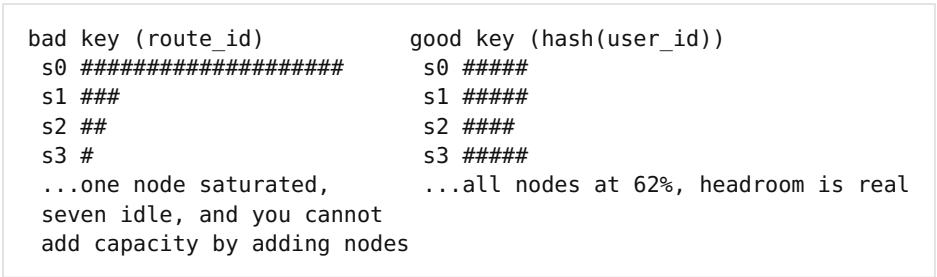
Partitioning without including the partition key in the WHERE clause. Symptom: queries get *slower* after partitioning, because the planner cannot prune and now scans 12 partitions with 12 separate index lookups. Check for `partitions: p2026` (MySQL) or "Sub-plans Removed" in the plan; if every partition is listed, pruning is not happening.

```
the three splits — GAINS: pruning, isolation, or capacity — pick which you need · COSTS:
federation kills joins, sharding kills transactions · ACCEPTS: only sharding adds write capacity,
and it costs the most
```

51. Choosing a shard key: the decision you cannot take back

The shard key determines which node holds a row. Choose it so that (a) the common query knows the key, and (b) traffic spreads evenly. Getting this wrong does not degrade gracefully — it produces one node at 100% and seven at 12%, and changing it later means rewriting every row.

```
CANDIDATE KEYS for a bus-ticketing system, 8 shards:

  route_id      queries know it (search by route). Distribution:
                Dhaka-Chittagong is 22% of all bookings.
                -> one shard gets 22% of writes; 8 shards give you
                   1/0.22 = 4.5x capacity, not 8x. Hot shard.

  created_at    every new booking goes to the newest shard.
                -> 100% of writes on one node. The classic disaster.

  user_id       even by construction (hash of an opaque id).
                Queries by user: single-shard. Good.
                Queries by route (the search page): scatter to all 8.

  booking_id    even, but "list my bookings" hits all 8 shards.

DECISION: user_id, because writes and per-user reads dominate; the
route search is served from cache + a read model (Topic 55), not from
the sharded table.

HOTSPOT MATH -- what "even" must mean:
  N shards, largest tenant share s of traffic.
  usable capacity = min(N, 1/s) x per-node capacity
  s = 0.22 -> 4.5x from 8 nodes (56% wasted)
  s = 0.03 -> 8.0x from 8 nodes (fully used)
  Rule: no single shard-key value may exceed 1/N of traffic. If one
  does, that value needs its own sub-sharding (key = user_id:bucket).
```



THE KEY INSIGHT

The shard key is chosen by the *write* path and the highest-QPS read path together. Any query that does not contain the key becomes a scatter-gather whose latency is the slowest of N shards (Topic 7's tail amplification, applied to your database).

THE TRAP

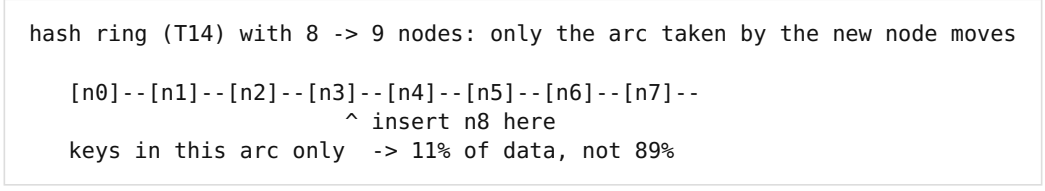
A monotonically increasing shard key — timestamp, auto-increment id, sequential invoice number. Symptom: the newest shard runs at 100% CPU while the others idle, and adding a shard moves the problem rather than fixing it. If you must range-shard on time, prefix the key with a hash bucket.

```
shard key — GAINS: near-linear capacity when it is even · COSTS: every keyless query becomes N
queries · ACCEPTS: the key is effectively permanent; changing it is a full data migration
```

52. Range, hash and directory sharding

Given a key, three ways to map it to a node. They differ in range-query support, rebalancing cost, and how badly a hot key hurts.

```
RANGE      shard = f(key interval)      users A-F | G-M | N-S | T-Z
  + range scans are single-shard (WHERE created_at BETWEEN ...)
  + trivial to split a hot range in two
  - skewed by nature: "S" surnames, or today's date
  used by: HBase, Bigtable, CockroachDB (auto-splitting ranges)

HASH      shard = hash(key) mod N       or consistent hashing (T14)
  + even distribution for free
  - range queries scatter to every shard
  - "mod N" rehashes ~ (N-1)/N of keys when N changes: going 8 -> 9
    moves 89% of rows. Consistent hashing moves ~1/9 = 11% instead.
  used by: Cassandra (with vnodes), most hand-rolled sharding

DIRECTORY  shard = lookup(key) in a mapping service
  shard_map: user 4821 -> shard 3   (a table, or etcd, or Vitess's
                                     topology service)
  + arbitrary placement: move one noisy tenant to its own node
  + rebalancing = update rows in the map, no rehash
  - the directory is a lookup on every query (cache it, Part 4)
  - the directory is a new SPOF and must be replicated
  used by: multi-tenant SaaS, Vitess-style topologies

REBALANCE COST, 8 -> 9 shards, 400 GB total:
  mod N            356 GB moved     hours of double-writing (T53)
  consistent hash   44 GB moved
  directory          0 GB forced; move exactly the tenants you choose
```



THE KEY INSIGHT

Directory sharding is underrated for business systems: it is the only scheme that lets you say "this one tenant is 22% of traffic, give it a dedicated node" without redesigning the key. You trade a cached lookup for total placement freedom — usually the right trade in multi-tenant products.

THE TRAP

`hash(key) mod N` written into application code. Symptom: the day you need a ninth shard, 89% of rows are on the wrong node and there is no incremental path — you are facing a full migration with double writes for weeks. Use consistent hashing or a directory from the first shard.

```
mapping scheme — GAINS: hash = even, range = scans, directory = control · COSTS: hash breaks
ranges, range skews, directory adds a lookup · ACCEPTS: whichever rebalancing bill you chose on
day one
```

53. Resharding a live system without downtime

Splitting shards while serving traffic is the highest-risk routine operation in this book. The shape is always the same five phases, and each is individually reversible — which is the entire reason it is survivable.

```
PHASE 1  DUAL WRITE       app writes to old shard AND new shard.
                          Reads still old. Deploy behind a flag.
PHASE 2  BACKFILL         copy historical rows in batches, oldest first.
                          400 GB at 40 MB/s = 2.8 hours; throttle to
                          keep replica lag (T46) under 5 s.
PHASE 3  VERIFY           compare checksums per range; count mismatches.
                          Run for at least 24 h to cover daily jobs.
PHASE 4  CUTOVER READS    flip reads to new shard for 1% -> 10% -> 100%,
                          watching error rate and p99 at each step.
PHASE 5  STOP DUAL WRITE  remove old path, then drop old data after a
                          retention window (never on the same day).

VERIFICATION QUERY that catches the usual bug:
  SELECT COUNT(*), MD5(GROUP_CONCAT(id, ':', updated_at ORDER BY id))
  FROM bookings WHERE user_id BETWEEN 4000000 AND 4009999;
  -- run on both sides, compare. Mismatch = your dual-write dropped
  -- updates (typically DELETEs, which people forget to mirror).

VITESS / CITUS do phases 1-4 as a supervised operation:
  vtctldclient MoveTables --workflow=split create ...
  vtctldclient VDiff ...          # phase 3 built in
  vtctldclient SwitchTraffic --tablet-types=rdonly,replica
Use the tool if you can. Hand-rolled resharding is where data goes.
```

```
      writes          reads
P1    old + new       old            reversible: stop writing new
P2    old + new       old            reversible: delete new data
P3    old + new       old            reversible
P4    old + new       new (1%..100%) reversible: flip reads back
P5    new             new            NOT reversible after old is dropped
```

THE KEY INSIGHT

Dual write plus backfill plus verify is not specific to databases — it is the general shape of every live data migration: cache format changes, search index rebuilds, storage-bucket moves (Topic 82). Learn it once; the risk lives entirely in phase 5, so stay in phase 4 until the verification is boring.

THE TRAP

Dual writes that are not transactional with the primary write. Symptom: after cutover, roughly 0.1% of rows are missing or stale — exactly the ones written during a brief error on the new shard. Either write both inside one transaction (impossible across nodes) or, better, drive the second write from the change log (Topic 59) so it inherits the primary's ordering and retries.

```
resharding — GAINS: capacity without downtime · COSTS: weeks of dual-write, 2× storage, a
verification harness · ACCEPTS: a small window where two copies of truth exist and can diverge
```

54. Life after sharding: cross-shard queries and transactions

Once rows live on different machines, three things you took for granted stop working: the join, the transaction, and the aggregate. Each has a replacement, and each replacement is worse.

```
THE JOIN
  SELECT b.*, u.name FROM bookings b JOIN users u ON u.id = b.user_id
  -- fine if both are sharded by user_id (co-location). Otherwise:
  1. query shard for bookings, collect user_ids
  2. multi-get users (cache first)
  3. join in application memory
  Cost: 2 round trips instead of 1, and no database-side optimiser.

THE AGGREGATE (scatter-gather)
  SELECT COUNT(*) FROM bookings WHERE created_at > ?    -- 8 shards
  latency = max(shard latencies), not the mean. With p99 = 120 ms per
  shard, P(at least one slow) = 1 - 0.99^8 = 7.7% -- your p99 is now
  their p99 (Topic 7). Fix: maintain counters (T55) instead of counting.

THE TRANSACTION
  Two-phase commit (2PC): prepare on all, then commit on all.
    + atomic across shards
    - blocking: if the coordinator dies after PREPARE, participants
      hold locks until it returns. Latency 2 RTT + slowest node.
    - throughput drops 3-10x. Avoid in the hot path.

  Saga: a sequence of local transactions, each with a compensator.
    reserve_seat -> charge_card -> issue_ticket
    failure at charge_card triggers release_seat.
    + no distributed locks, scales
    - NO ISOLATION. Someone can see a seat reserved-but-unpaid.
      You must design that intermediate state into the product
      ("held for 10 minutes"), which is exactly what real booking
      systems do (Topic 118).
```

```
SAGA with compensation
  [reserve seat] --ok--> [charge card] --FAIL--> [release seat]
        |                                              ^
        +----------------- compensator ----------------+
  the intermediate state is visible to users. Name it in the UI.
```

THE KEY INSIGHT

A saga replaces isolation with an explicit business state. That is not a workaround — "seat held for 10 minutes," "payment pending," "order processing" are sagas made visible, and they are how every large system that takes money actually works.

THE TRAP

An unbounded scatter-gather in a user-facing path, typically an admin filter someone added. Symptom: one endpoint's p99 is 8× everything else and gets worse each time you add a shard. Either give it the shard key, or serve it from a read model (Topic 55) or a search index — never from all shards synchronously.

```
cross-shard work — GAINS: the system still functions after sharding · COSTS: application-side
joins, sagas instead of transactions · ACCEPTS: no global isolation; intermediate states are
visible to users
```

55. Denormalisation, materialised views and CQRS

When the read shape and the write shape disagree, stop trying to satisfy both with one table. Maintain a second representation optimised for reading, and accept that it lags.

```
THE PROBLEM
  "Trips leaving Dhaka tomorrow with seats available, sorted by price"
  = 5 joins + a live seat count + a sort. 380 ms, 40 QPS ceiling,
  and it is the home page.

FIX 1 -- COUNTER COLUMN (cheapest denormalisation)
  ALTER TABLE trips ADD seats_left INT NOT NULL;
  UPDATE trips SET seats_left = seats_left - 1 WHERE id = ?
    AND seats_left > 0;              -- atomic, and it is the reservation
  Read cost: 0 joins. Write cost: contention on hot rows (Topic 118).

FIX 2 -- MATERIALISED VIEW (database maintains it)
  CREATE MATERIALIZED VIEW trip_search AS SELECT ...;
  REFRESH MATERIALIZED VIEW CONCURRENTLY trip_search;   -- Postgres
  Read: 4 ms. Staleness: refresh interval. MySQL has no native MV --
  you build it with a summary table + triggers or a cron job.

FIX 3 -- CQRS (separate write model and read model)
  writes -> normalised OLTP tables (source of truth)
       -> CDC (Topic 59) -> read model: one denormalised row per card,
          or Elasticsearch, or a Redis sorted set
  reads  -> read model only. 4 ms, no joins, independently scalable.

  Cost: two schemas, a pipeline, and eventual consistency that users
  can see ("I edited it and search still shows the old title for 2 s").
```

```
       writes                          reads
client ---> [OLTP normalised] --CDC--> [read model] <--- client
                truth, small            denormalised, big, rebuildable
                constraints here        no constraints, no joins
  rebuild is a feature: replay the log and you have a new read model
```

THE KEY INSIGHT

A read model is a cache you are willing to maintain, with two properties a cache does not have: it survives restart, and it can be queried in ways the source cannot (full-text, geo, ranking). The invalidation discipline of Part 4 still applies — it just moved into a pipeline you own.

THE TRAP

Denormalised copies with no rebuild path. Symptom: a bug corrupts `seats_left` for 400 trips and there is no authoritative way to recompute it, because the events that produced it were never stored. Always keep the derivation: either the source rows or the event log. A read model you cannot rebuild from scratch is a second source of truth, and two sources of truth is zero.

```
read models — GAINS: 380 ms → 4 ms, reads scale independently · COSTS: a pipeline, two schemas,
rebuild tooling · ACCEPTS: visible eventual consistency between write and read side
```

56. SQL or NoSQL: choose by access pattern, not by volume

The useful question is not "how much data" but "what shapes of query must be fast, and what must be atomic." Volume decides how many nodes; access pattern decides the engine.

```
Need                                    Choose
--------------------------------------  ------------------------------
Multi-row atomicity, FKs, constraints   relational (Postgres/MySQL)
Ad-hoc queries you have not thought of  relational
Known key, huge write rate, no joins    KV / wide-column (Cassandra,
                                        DynamoDB, ScyllaDB)
Documents with varying shape, read by   document (MongoDB) -- but note
their own id                            Postgres JSONB does this too
Relationship traversal 3+ hops deep     graph (Neo4j)
Time-ordered append + range scans       time-series (Timescale,
                                        ClickHouse, InfluxDB)
Full-text, faceting, relevance ranking  search index (Elasticsearch)
                                        -- never as the source of truth

THE HONEST DEFAULT: PostgreSQL. It is a relational database, a JSON
document store (JSONB + GIN indexes), a queue (SKIP LOCKED), a
time-series store (with Timescale), a geo database (PostGIS) and a
vector database (pgvector). One operational skill set, one backup
story, and it takes you to roughly 50-100k QPS with the ladder of
Topic 44 fully climbed.

WHAT NOSQL ACTUALLY BUYS (Cassandra, 8 nodes):
  linear write scaling: 8 nodes = 8x write throughput, no primary
  no single write bottleneck, no failover pause
  tunable consistency: QUORUM reads+writes = strong-ish (R+W > N)
WHAT IT COSTS:
  you must know every query BEFORE designing tables -- the table IS
  the query. New access pattern = new table + backfill.
  no joins, no multi-partition transactions, no ORDER BY outside
  the clustering key.
```

THE KEY INSIGHT

Wide-column stores are not "SQL without the schema" — they are the opposite: the schema is more rigid, because it is the query plan frozen into the table definition. Choosing one means promising you know your access patterns in advance, which is a product statement, not a technical one.

THE TRAP

Picking a document store for "flexibility" and then needing a report across collections. Symptom: application code doing three queries and a nested loop to produce a join, with no transaction around it, and a nightly job that reconciles the drift. If you cannot state your top ten queries, you need the engine that tolerates queries you have not invented yet.

```
engine choice — GAINS: matching the engine to the access pattern is worth 10–100× · COSTS: each
non-relational engine forfeits joins or transactions or both · ACCEPTS: a wrong choice is a
rewrite, not a config change
```

57. Distributed SQL: what the newer engines actually change

CockroachDB, Spanner, YugabyteDB, TiDB and Aurora DSQL sit between the two worlds: SQL, transactions and secondary indexes, with automatic sharding and replication underneath. They remove the operational burden of Topics 51–53 and charge for it in write latency.

```
WHAT THEY GIVE YOU
  automatic range splitting and rebalancing (no shard key decision)
  distributed ACID transactions across all rows (no saga required)
  survive a node or AZ loss with no manual failover
  SQL, and usually Postgres wire compatibility

WHAT IT COSTS -- always the same thing, consensus latency:
  every write is a Raft round to a majority of replicas.
  same AZ:      +1-2 ms       three AZs:   +2-5 ms
  three regions: +60-120 ms per write, and it cannot be optimised away
  because it is the speed of light and a majority quorum.

  single-row read: 1-3 ms (leaseholder, local)
  contended row (a counter everyone increments):
     throughput ~ 1 / (consensus RTT) per row = a few hundred/s.
     Same hot-row limit as Topic 118, moved into the storage engine.

CLOCK: Spanner uses TrueTime (GPS + atomic clocks, uncertainty ~7 ms)
and WAITS OUT the uncertainty on commit. Cockroach uses HLC and
retries transactions that read within the uncertainty window --
visible to you as retryable serialization errors. Your application
MUST implement a retry loop; this is not optional.

WHEN IT IS THE RIGHT ANSWER
  multi-region write availability is a hard requirement, and
  you would otherwise be hand-building Topics 51-54.
WHEN IT IS NOT
  single-region, < 20k QPS, latency-sensitive writes -> a well-tuned
  Postgres primary with replicas is faster and cheaper.
```

THE KEY INSIGHT

Distributed SQL does not repeal the trade-off; it relocates it. You stop making a shard-key decision and start paying a consensus RTT on every write. That is an excellent deal at 3 regions and a poor one at 1 AZ.

THE TRAP

Porting an application that assumes transactions never fail. Symptom: intermittent `restart transaction: TransactionRetryWithProtoRefreshError` under load, surfacing as random 500s. Optimistic concurrency means retries are part of the contract — wrap every transaction in a bounded retry with jitter (Topic 68) before you migrate anything.

```
distributed SQL — GAINS: ACID + automatic sharding + AZ/region survival · COSTS: +2–120 ms per write, mandatory retry logic · ACCEPTS: hot single rows are still limited by one consensus round each
```

58. Multi-region writes and conflict resolution

One region is one earthquake, one BGP mistake, one cloud-provider AZ event. Going multi-region for reads is easy (Topic 46 plus geography); going multi-region for *writes* forces a choice with no comfortable option.

```
OPTION A -- SINGLE WRITE REGION (active-passive)
  all writes go to region 1; region 2 has async replicas, read-only.
  write latency for far users: +RTT (Dhaka -> Frankfurt = 130 ms)
  failover: promote region 2, accept losing the async tail (T47).
  RPO: seconds-minutes. RTO: minutes. Simple, and correct.
  This is the right answer for 90% of systems that ask the question.

OPTION B -- PARTITIONED WRITES (each region owns a key range)
  EU users' rows are writable only in EU, APAC's only in APAC.
  No conflicts by construction -- it is sharding with geography as
  the shard key (T51). Also the shape data-residency laws want.
  Cost: a user who moves regions needs a migration; global entities
  (inventory, seat maps) still need one owner.

OPTION C -- ACTIVE-ACTIVE (both regions accept writes for any row)
  Conflicts are now guaranteed. Resolution strategies:
    LWW (last write wins by timestamp): simple, and silently DISCARDS
      one user's write. Clock skew of 40 ms decides who loses.
    Version vectors: detect the conflict, hand both versions to the
      application to merge. Correct, and someone must write the merge.
    CRDTs: types that merge deterministically -- G-Counter (sum of
      per-replica counters), OR-Set, LWW-Register.
      Great for: like counts, presence, collaborative text.
      Impossible for: "only 40 seats exist." No CRDT prevents
      overselling, because the constraint is global.

ARITHMETIC that ends most active-active discussions:
  cross-region RTT 130 ms. Strong consistency needs a majority round.
  Any write path that must be globally correct costs >= 130 ms.
  If your product cannot pay that, you are choosing eventual
  consistency for that data -- say so explicitly, in the design doc.
```

```
seat 14A, active-active, 130 ms apart

 EU  t=0   sell 14A to Alice ------\
 APAC t=20ms sell 14A to Bob ------ \-- replicate, t=130ms --> CONFLICT
 LWW: Bob wins, Alice has a receipt and no seat, and no error was raised.
 Correct design: seat inventory has ONE owning region. Always.
```

THE KEY INSIGHT

Split your data by whether it has a global constraint. Counters, profiles, preferences, drafts and likes are safe active-active. Anything with "only N exist" or "must not go negative" needs a single owner — and can still be read everywhere.

THE TRAP

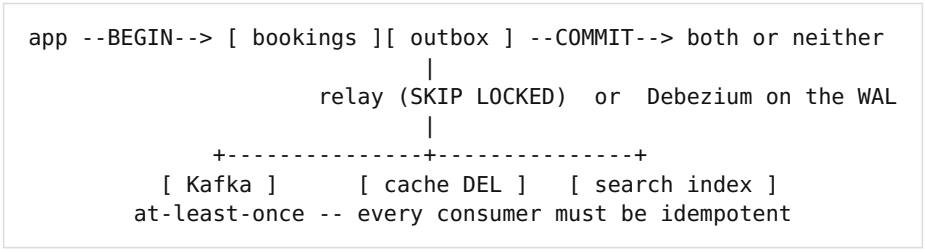
LWW conflict resolution chosen by default because it is the vendor's default. Symptom: silent, unexplainable data loss — a support ticket saying "my edit disappeared" with no error in any log, at a rate proportional to concurrent cross-region editing. LWW is a decision to discard writes; make it deliberately or not at all.

```
multi-region writes — GAINS: regional failure survival, local write latency · COSTS: 130 ms for
global correctness, or a merge function per data type · ACCEPTS: with LWW, some acknowledged
writes are silently discarded
```

59. Change data capture and the transactional outbox

Two systems must agree: the database and something else — a cache, a search index, another service, a queue. Writing to both from application code is the dual-write problem of Topic 41 in general form. CDC solves it by making the database's own log the source of the second write.

```
THE BROKEN VERSION
  $db->commit();                       // succeeds
  $kafka->publish($event);             // process dies here
  -> the booking exists and no downstream system will ever know.
  Reversing the order is equally broken (event without a booking).

THE OUTBOX -- one local transaction, two effects
  BEGIN;
    INSERT INTO bookings (...) VALUES (...);
    INSERT INTO outbox (id, topic, payload, created_at)
      VALUES (uuid(), 'booking.created', '{"id":9001,...}', now());
  COMMIT;                              -- atomic: both or neither

  A relay reads the outbox and publishes:
    SELECT * FROM outbox WHERE published_at IS NULL
      ORDER BY id LIMIT 500 FOR UPDATE SKIP LOCKED;
  SKIP LOCKED lets N relay workers run concurrently without blocking.
  Delivery is AT-LEAST-ONCE (the relay may crash after publish,
  before marking) -> consumers must be idempotent (Topic 63).

CDC -- skip the outbox table, read the log itself
  Debezium tails the MySQL binlog / Postgres WAL and publishes every
  row change to Kafka, in commit order, with before+after images.
  $ SHOW BINARY LOG STATUS;  -- MySQL 8.4+ (was SHOW MASTER STATUS)
  Lag: 50-500 ms typical. Ordering: per-table, per-key, guaranteed.

WHAT CDC UNLOCKS (each is a topic elsewhere in this book)
  cache invalidation that cannot be forgotten          (Topic 41)
  read models / CQRS projections                       (Topic 55)
  dual-write during resharding, driven by the log      (Topic 53)
  search index sync, audit trail, analytics replication
```



THE KEY INSIGHT

The database's commit log is the only totally ordered record of what happened. Anything derived from it — caches, indexes, replicas, projections, other services — is correct by construction and rebuildable by replay. Deriving state from the log instead of from application code removes an entire family of bugs.

THE TRAP

An outbox table that is never pruned. Symptom: the outbox becomes the largest table in the database, the relay's `WHERE published_at IS NULL` query degrades to a scan, and event lag climbs from 200 ms to minutes. Delete published rows on a schedule (or partition by day and drop partitions, Topic 50), and alert on outbox depth — it is your best early warning that a downstream consumer has stopped.

```
CDC / outbox — GAINS: atomic write + publish, replayable derived state · COSTS: a relay or
Debezium cluster, outbox retention, at-least-once semantics · ACCEPTS: consumers lag by 50–500 ms
and must be idempotent
```

PRACTICE SET 5 — Topics 44–59

A. Conceptual

1. Why does sharding not fix a query that examines 8.4 M rows?
2. An 8× bigger instance gave 1.3× the throughput. Name two bottlenecks that would explain it.
3. Why is `max_connections` not the same as useful concurrency?
4. Which pooling mode breaks `SET search_path`, and why?
5. Your workload is 30% writes. What is the approximate ceiling on total capacity from read replicas alone, and why?
6. Give the one-line difference between semi-synchronous and synchronous replication in terms of what stops when a replica is unreachable.
7. Why is "sleep 200 ms before redirecting" not a fix for read-your-writes?
8. State what fencing is and why it must happen before promotion.
9. Why must a consensus/manager quorum be odd and at least three?
10. Distinguish partitioning from sharding in one sentence about the query planner.
11. Why is `created_at` the worst possible shard key?
12. Why does `hash(key) mod N` make adding a shard so expensive, and what moves instead under consistent hashing?
13. In a five-phase reshard, which phase is irreversible and what should you do before entering it?
14. Why does a scatter-gather's latency equal the slowest shard rather than the average?
15. What does a saga give up that 2PC provides, and how do real products make that acceptable?
16. Why is a read model you cannot rebuild worse than no read model?
17. Why is "the table is the query" a fair description of Cassandra schema design?
18. Distributed SQL removes the shard-key decision. What does it charge you instead, and why can it not be optimised away?
19. Name two data types that are safe under active-active writes and one that never is.
20. Why does the outbox pattern fix the dual-write problem when reordering the two writes does not?

B. Pen-and-paper

1. 48 app containers × 25 pool connections against Postgres with `max_connections = 300` on 16 cores. How many connections are wanted, how many are useful, and what `default_pool_size` would you set in PgBouncer? What is the multiplexing ratio?
2. Front traffic 60,000 QPS, cache hit ratio 0.96, write share 18%. Compute DB-tier QPS, write QPS and read QPS. Each replica serves 2,800 QPS. How many replicas, and how many would you actually run?
3. Primary fsync 9 ms. Same-AZ RTT 0.5 ms, cross-AZ 1.6 ms, cross-region 62 ms. Give write latency under async, semi-sync (cross-AZ) and sync (cross-region), and the concurrent in-flight write count at 600 writes/s in each case.
4. Eight shards. The largest route is 22% of writes, the largest user is 0.4%. Compute usable capacity multiplier when sharding by route_id and by user_id.
5. 640 GB across 8 shards, moving to 10. How much data moves under `mod N`, under consistent hashing, and under a directory where you choose to move two tenants totalling 30 GB?

6. Backfill 640 GB at 55 MB/s with a 40% throttle to protect replica lag. How long? If verification must cover a weekly job, what is the minimum phase-3 duration?
7. Per-shard p99 is 90 ms and p99.9 is 400 ms. For a scatter-gather across 12 shards, what is the probability at least one shard is above its p99, and what does that make the fan-out p99?
8. A hot row is incremented under distributed SQL with a 4 ms consensus round. What is the maximum sustainable increment rate for that row, and what happens to the 1,000 req/s that want it?
9. An outbox receives 900 rows/s, average 1.2 KB, and the relay is down for 3 hours. How large is the backlog in rows and GB, and how long does a relay draining at 2,500 rows/s take to catch up?

C. Lab tasks (build → break → observe → fix)

1. **Build.** Load 5 M rows into a `bookings` table. Run a filtered, sorted query and capture `EXPLAIN ANALYZE`. Add the composite index and capture it again. Record rows examined and wall time for both.
2. **Break: the connection wall.** Set `max_connections = 40`. Run a load generator with 200 concurrent clients. Capture the exact error and the p99. Put PgBouncer in `transaction` mode with `default_pool_size = 25` in front and repeat. Record throughput and p99 both ways.
3. **Break: replication lag and read-your-writes.** Set up one replica. Route reads to it. Start a bulk `UPDATE` of 2 M rows, then, while it runs, update your own profile and immediately read it. Record `Seconds_Behind_Source` (or `pg_last_wal_replay_lsn`) and what you saw. Fix with an LSN/GTID check and prove the fallback to primary fires.
4. **Break: split brain.** On a three-node test cluster, partition the primary with `iptables -A INPUT -s <peer> -j DROP` instead of killing it. Let the manager promote a replica, then write to both sides. Heal the partition. Record what happened to the writes on the demoted node — this is the data you would have lost.
5. **Break: the wrong shard key.** Simulate 8 shards in one database as 8 tables. Route 100,000 synthetic bookings by `route_id`, then by `hash(user_id)`. Plot rows per shard for both and compute the usable-capacity multiplier from B.4.
6. **Break: the dual write.** Write a job that inserts a booking and then publishes an event, and kill it between the two with a deliberate exception. Count orphaned bookings. Reimplement with an outbox in one transaction, kill it in the same place, and show the relay publishing the event after restart.
7. **Build and verify a reshard.** Dual-write one table to a second "shard", backfill it, then run the checksum query from Topic 53 on both sides. Deliberately drop `DELETE` from the dual-write path first, and show the checksum catching it.

ANSWER KEY 5 — Topics 44–59

A. Conceptual

1. Sharding divides rows, not work per row. Eight shards examine **1.05 M rows each** — still hundreds of milliseconds, now with scatter-gather. **The index is 2,333×; the shard is 8×.**
2. **A single hot write lock (one row or one index tail) and fsync/IOPS latency.** Neither improves with cores; both are serial.
3. Useful concurrency is about **2× cores + spindles** (~35 on 16 cores). Past the knee, throughput is flat and **only queueing latency grows**.
4. **Transaction mode.** The next statement may land on a different server connection, so session state — search_path, temp tables, advisory locks, prepared statements — is gone.
5. **About 3×** (1/0.30): every replica must apply 100% of writes, so writes consume capacity on every node. That ceiling is when sharding becomes the only move.
6. Semi-sync: if no replica acknowledges, it **degrades to async and keeps accepting writes**. Sync/quorum: **writes stop** until a quorum returns.
7. A sleep assumes a lag bound that does not exist. Lag is **0.2 s normally and 14 minutes during a bulk import** (Topic 46). Use the LSN, which is a fact rather than a guess.
8. Fencing is **making it impossible for the old primary to accept another write** — revoking its VIP, STONITH, or requiring an etcd lease. Before promotion, because after promotion any old-primary write is a **divergence with no merge**.
9. Odd so a partition cannot produce two equal halves; ≥3 so a **minority cannot elect**. With 2, both sides can believe they are the majority.
10. Partitioning: **one planner, one node, many sub-tables**. Sharding: **N independent planners and a routing layer**, with no cross-node transaction.
11. It is monotonically increasing, so **100% of writes land on the newest shard**; the other N−1 nodes are idle and adding nodes does not help.
12. `mod N` remaps **(N−1)/N = 89%** of keys going 8→9. Consistent hashing moves only the new node's arc, **≈1/9 = 11%**.
13. **Phase 5**, when the old data is dropped. Before it: run verification long enough to cover every periodic job, and keep the old data for a full retention window after cutover.
14. The result needs every shard, so it completes when the last one does: **max, not mean**. With p99 90 ms and 12 shards, P(any slow) = $1-0.99^{12}$ = **11.4%**.
15. It gives up **isolation** — intermediate states are visible. Products make it acceptable by **naming the state in the UI**: "held for 10 minutes", "payment pending".
16. Because a corrupted read model then has **no authoritative recovery path**, making it a second source of truth. Keep the source rows or the event log so it can be replayed from scratch.
17. Tables are defined by partition key and clustering columns, which fix **exactly which queries are possible and in what order**. A new access pattern requires a new table plus a backfill.
18. **A consensus round-trip to a majority on every write** (+2–5 ms across AZs, +60–120 ms across regions). It is the speed of light plus a quorum — not an implementation inefficiency.
19. Safe: **like counters (G-Counter), presence, preferences/drafts**. Never: **anything with a global constraint such as seat inventory** — no CRDT can enforce "only 40 exist".
20. Because the outbox insert is **in the same local transaction** as the business row: both commit or neither. Reordering two independent writes only changes which one is orphaned.

B. Worked

1. Wanted **1,200**; the database allows 300; useful is **~35** (2×16 + spindles). Set `default_pool_size` ≈ **40** with `max_client_conn = 2000`. Ratio **1,200:40 = 30×**.
2. DB tier = 60,000×0.04 = **2,400 QPS**; writes **432**, reads **1,968**. 1,968/2,800 = 0.7 → **1 replica** mathematically; **run 3** so one can be lost to failure and one to a deploy without touching the primary.
3. Async **9 ms** → 600×0.009 = **5.4 in flight**. Semi-sync cross-AZ **10.6 ms** → **6.4**. Sync cross-region **71 ms** → **42.6 concurrent write transactions** permanently occupying pool slots (Topic 45).
4. route_id: min(8, 1/0.22) = **4.5×** (44% of the fleet wasted). user_id: min(8, 1/0.004) = **8.0×**, fully usable.
5. `mod N`: 9/10 of keys remap → **576 GB**. Consistent hashing: ≈1/10 → **64 GB**. Directory: **30 GB**, chosen deliberately.
6. Effective rate 55×0.6 = 33 MB/s → 640,000/33 = 19,394 s = **5.4 hours**. Phase 3 must run **at least 7 days** to observe a weekly job at least once (8 days is safer).
7. P(any above p99) = $1-0.99^{12}$ = **11.4%**, so the fan-out's p99 is roughly the shards' **p99.9 ≈ 400 ms**. Fan-out converts your tail into your median experience (Topic 7).
8. One row = one consensus round at a time: **1/0.004 = 250 increments/s**. The other 750 req/s **queue or return retry errors**; the fix is sharded counters (Topic 39) or a queue in front (Topic 118).
9. 3 h × 900/s = **9.72 M rows ≈ 11.7 GB**. Drain surplus is 2,500−900 = 1,600/s → 9.72 M/1,600 = **1.7 hours** to catch up, during which every downstream consumer is also that far behind.

C. What to verify

1. Expect `Seq Scan` / `type: ALL` with rows in the millions and **1–3 s**, becoming `Index Scan`/`ref` with **tens of rows and under 5 ms**. If the index is not used, check column order — equality columns first, then the sort column.
2. Direct: `FATAL: sorry, too many clients already` (or MySQL `ER_CON_COUNT_ERROR`) plus p99 in seconds. Through PgBouncer: **no errors, higher throughput, p99 several times lower**, with waiting visible in `SHOW POOLS` rather than as failures.
3. Lag should climb into **tens of seconds to minutes** during the bulk update, and your own profile read should return the **pre-update value**. After the fix, the router must fall back to the primary; log how often it fires — that rate is your real lag exposure.
4. Both nodes accept writes while partitioned. On heal, one side's writes are **rolled back or must be discarded manually**; primary keys may collide. Confirm your tooling fenced the old primary — if it did not, record the exact number of writes that would have been lost.
5. route_id: one bucket near **22,000 of 100,000**, others far smaller → multiplier **≈4.5**. hash(user_id): all buckets within a few percent of 12,500 → **≈8.0**.
6. Expect **a non-zero count of bookings with no event**. With the outbox, the count is **zero**: the row and the outbox entry commit together, and the relay publishes after restart — possibly **twice**, which is why consumers must be idempotent (Topic 63).
7. Checksums must match exactly. With `DELETE` omitted, the counts diverge by the number of deletions and the MD5 differs — **this is precisely the bug that verification exists to catch**, and the reason phase 3 runs for days rather than minutes.

HANDNOTE SUMMARY CARD — PART 5

```
================================================================================
 DATABASE SCALING -- ONE PAGE                                    Topics 44-59
================================================================================
 THE LADDER (left to right, never skip)
   index -> vertical -> pool -> cache -> replicas -> federate -> shard
   index 10-1000x (afternoon)   shard Nx (quarters)   pick the left one
--------------------------------------------------------------------------------
 CONNECTIONS
   useful_conns ~ 2 x cores + spindles      (16 cores -> ~35)
   48 apps x 25 pool = 1200 wanted -> pgbouncer transaction mode -> 40 seen
   pool_mode=transaction breaks: search_path, temp tables, advisory locks,
   PREPARE.  session mode keeps them and forfeits the multiplexing.
--------------------------------------------------------------------------------
 REPLICATION
   async     +0 RTT   tail loss on failover        MySQL default
   semi-sync +1 RTT   no loss if one replica lives  ALERT on "switched OFF"
   sync      +1 RTT   WRITES STOP without quorum    Aurora 4/6, PG sync_commit
   lag: 0.2 s normal, minutes during batch. replicas scale READS only:
   write share w -> capacity ceiling ~ 1/w  (30% writes = 3x, then shard)
   read-your-writes: sticky-to-primary 5 s | LSN/GTID fence (correct) | cache
--------------------------------------------------------------------------------
 FAILOVER
   detect(10s) -> elect -> pick highest LSN -> FENCE -> promote -> reroute
   ~25 s write downtime. Fence BEFORE promote or you get split brain.
   quorum odd and >= 3.  Test with kill -9 quarterly (T105).
--------------------------------------------------------------------------------
 SHARDING
   partition = 1 node, sub-tables, pruning + DROP PARTITION retention
   federate  = tables on different nodes, joins die
   shard     = rows on N nodes, transactions die
   usable = min(N, 1/largest_key_share) x node.   22% tenant on 8 shards=4.5x
   key: created_at WORST | route_id hot | hash(user_id) good
   map: hash (even, no ranges, mod N moves 89%) | range (scans, skews)
        | directory (place anything, cached lookup, SPOF)
   RESHARD: dual-write -> backfill -> verify(days) -> cutover 1/10/100%
            -> stop dual-write.  Only the last step is irreversible.
--------------------------------------------------------------------------------
 AFTER SHARDING
   join      -> app-side, 2 round trips, no optimiser
   aggregate -> scatter-gather; latency = MAX of shards (1-0.99^N slow)
   txn       -> 2PC (blocking, 3-10x slower) or SAGA (no isolation;
                name the intermediate state: "held 10 min")
--------------------------------------------------------------------------------
 DERIVED STATE
   dual write is broken in both orders -> OUTBOX in the same transaction
     SELECT ... FROM outbox WHERE published_at IS NULL FOR UPDATE SKIP LOCKED
   or CDC (Debezium on binlog/WAL, 50-500 ms, commit-ordered)
   feeds: cache invalidation, CQRS read models, reshard, search, audit
   at-least-once ALWAYS -> idempotent consumers (T63). Prune the outbox.
--------------------------------------------------------------------------------
 MULTI-REGION
   A single write region  (right answer 90% of the time; RPO seconds)
   B partition by region  (no conflicts by construction; residency laws)
   C active-active        LWW = silent write loss | CRDT for counters/sets
     never for "only N seats exist" -- global constraint needs one owner
--------------------------------------------------------------------------------
 TRAPS
   partitioned but no key in WHERE -> all partitions scanned, slower
   mod N in app code -> 89% moves on shard #9
   sleep() as read-your-writes fix -> breaks at 03:00 batch time
   outbox never pruned -> largest table, relay query degrades to a scan
   distributed SQL without retry loop -> random 500s under contention
================================================================================
```

PART SIX

Queue systems

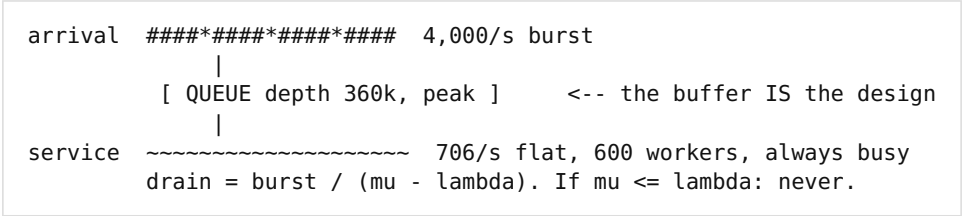
A queue converts a load problem into a latency problem, which is usually a trade you want: 40,000 arriving requests become a 6-minute backlog instead of an outage. Everything hard about queues follows from one fact — delivery is at-least-once, so your consumer will run twice.

Part 6 · Queue systems

60. Why async: load levelling, with the arithmetic

Synchronous systems must be sized for the peak. Asynchronous systems can be sized for the mean, and pay for the difference in queue depth and delay. That substitution — capacity for latency — is the entire reason queues exist.

```
TICKET RELEASE, 09:00. Bookings arrive at 4,000/s for 90 seconds,
then settle to 120/s. Each booking needs: PDF ticket, SMS, email,
seat-map update, analytics event -- 850 ms of work.

SYNCHRONOUS
  concurrency needed = 4,000/s x 0.85 s = 3,400 workers
  provisioned all day for 90 seconds of use. Cost: 28x the mean.
  If under-provisioned, the request path itself times out and the
  booking fails -- the peak breaks the thing users care about.

ASYNCHRONOUS
  request path does: INSERT booking + enqueue (4 ms). Returns 202.
  workers = mean rate x work = 120/s x 0.85 = 102 workers, +50%
  headroom = 150.

  Drain time for the burst:
    enqueued in burst   = 4,000 x 90        = 360,000 jobs
    drain rate          = 150 workers/0.85s = 176 jobs/s
    surplus over arrival (120/s)            = 56 jobs/s
    backlog clears in   360,000 / 56        = 6,428 s = 1.8 hours (!)

  1.8 hours to send an SMS is not acceptable, so size for the tail:
  600 workers -> 706/s drain -> surplus 586/s -> backlog gone in
  10 minutes. Still 5.7x cheaper than 3,400 synchronous workers.

LITTLE'S LAW is the whole of queue capacity planning:
  L = lambda x W        (items in system = arrival rate x time in system)
  Wanted: W <= 60 s at lambda = 4,000/s -> L <= 240,000 in flight
  Required service rate mu > lambda, or the queue grows without bound.
  There is no configuration that drains a queue whose mu < lambda.
```



THE KEY INSIGHT

A queue does not add throughput. It only lets you serve a peak with mean-sized capacity, at the cost of delay. If your *average* arrival rate exceeds your service rate, the queue is not a buffer — it is a slowly failing system with an alert that fires three hours late.

THE TRAP

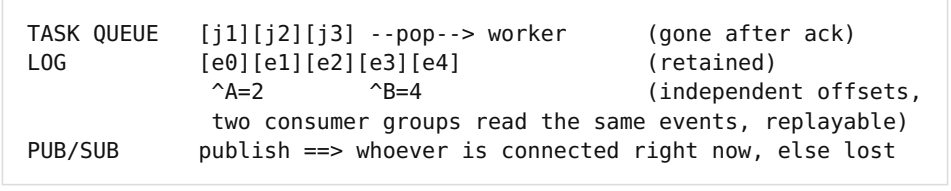
Putting a user-visible step behind a queue without telling the user. Symptom: support tickets saying "I paid and got no ticket," 40 minutes into a backlog that your dashboards call healthy because the API is returning 202 at 3 ms. Anything asynchronous needs a visible state ("processing"), a completion signal, and an alert on *age of oldest message*, not just depth.

```
async processing — GAINS: peak absorbed with 5.7× less capacity, request path stays at 4 ms · COSTS: a broker to operate, eventual completion, duplicate handling · ACCEPTS: work is delayed by the backlog, which users may see
```

61. Four different things called "a queue"

Task queue, log, pub/sub and stream have genuinely different semantics. Picking the wrong one shows up months later as "we cannot replay" or "we cannot scale consumers."

```
TASK QUEUE (RabbitMQ, SQS, Redis+Sidekiq/Horizon, Beanstalkd)
  message is CONSUMED and deleted. One worker gets each job.
  Per-message ack, per-message retry, per-message delay.
  Ordering: not guaranteed once you have > 1 consumer.
  Best for: send email, resize image, charge card, generate PDF.

LOG (Kafka, Pulsar, Redpanda)
  message is APPENDED and retained for a period (7 days, or forever).
  Consumers track an offset; reading does not delete.
  Ordering: total, within a partition.
  Replayable: a new consumer can start from offset 0 in the past.
  Best for: event streams, CDC (T59), analytics, multi-consumer fanout.

PUB/SUB (Redis PUBLISH, NATS core, WebSocket fanout)
  fire and forget. No storage. A subscriber that is offline
  misses the message forever.
  Best for: cache-invalidation broadcasts, live presence, chat fanout
  where history is stored elsewhere.

STREAM PROCESSING (Kafka Streams, Flink)
  a log plus stateful operators: windows, joins, aggregations.

THE DECIDING QUESTIONS
  Does a second team need the same events later?    -> log
  Must a message be retried individually?           -> task queue
  Do you need to replay history to rebuild state?   -> log
  Is missing a message acceptable?                  -> pub/sub
  Do you need per-message delay/priority?           -> task queue
```



THE KEY INSIGHT

The log's superpower is that consumption is a read, not a destructive pop. That single property gives you replay, multiple independent consumers, and the ability to rebuild a read model (Topic 55) from scratch — none of which a task queue can offer at any price.

THE TRAP

Using Redis pub/sub for work that must not be lost. Symptom: during a deploy, the subscriber restarts and the messages published in those 8 seconds are gone — no error, no dead letter, no trace. Redis Streams or a real broker if the message matters; pub/sub is for things you can miss.

```
choosing the shape — GAINS: the log gives replay and fanout, the task queue gives per-message
control · COSTS: logs need partition design, task queues lose ordering · ACCEPTS: pub/sub
silently drops anything for a disconnected subscriber
```

62. Delivery semantics: why exactly-once is not a setting

Three guarantees are advertised; only two exist end to end. The third is assembled by you, from at-least-once delivery plus idempotent processing.

```
AT-MOST-ONCE   ack before processing. Crash = message lost.
               Use for: metrics samples, telemetry, presence pings.

AT-LEAST-ONCE  ack after processing. Crash after work, before ack
               = redelivery. THE DEFAULT EVERYWHERE.
               Duplicate rate in practice: 0.01-1% of messages,
               concentrated during deploys and consumer crashes.

EXACTLY-ONCE   what everyone wants. Impossible as a pure transport
               guarantee, because the consumer's side effect (charge
               a card, send an SMS) is not inside the broker's
               transaction. Kafka's "exactly-once semantics" is real
               but bounded: it covers read -> process -> write
               ENTIRELY WITHIN KAFKA (transactional producer +
               read_committed). It does not cover your HTTP call to
               a payment gateway.

THE WINDOW, concretely:
  t=0    consumer receives "charge booking 9001"
  t=120  gateway charges the card, returns 200
  t=121  consumer process is OOM-killed before committing the offset
  t=130  another consumer receives the SAME message
         -> the card is charged twice unless the consumer is
            idempotent (Topic 63). The broker did nothing wrong.

WHAT YOU ACTUALLY BUILD
  at-least-once delivery + idempotent consumer = effectively-once
  This phrase should appear in your design docs instead of
  "exactly-once", because it names where the work lives: your code.
```

THE KEY INSIGHT

Ack timing *is* the semantic. Ack first and you have chosen to lose messages; ack last and you have chosen to duplicate them. There is no third position, so choose per queue based on which failure is cheaper for that job.

THE TRAP

A framework that acks on receipt by default. Symptom: after a worker OOM, several hundred jobs simply never ran, with no dead-letter entry and no error — the work vanished silently. Check your library's default (`auto_ack`, `enable.auto.commit`, `--no-ack`) before you trust it with anything.

```
delivery semantics — GAINS: at-least-once never loses work · COSTS: every consumer must tolerate
a repeat · ACCEPTS: 0.01–1% duplicates, clustered around deploys and crashes
```

63. Idempotency: the technique that makes duplicates harmless

An idempotent operation produces the same state whether it runs once or five times. Since Topic 62 guarantees repeats, idempotency is not a nice property — it is the correctness requirement for every consumer you write.

```
NATURALLY IDEMPOTENT              NOT IDEMPOTENT
  SET status = 'paid'               UPDATE balance = balance - 100
  PUT /object (full replace)        INSERT INTO ledger (...)
  DELETE by id                      POST /charge
  cache SET                         send SMS

PATTERN 1 -- IDEMPOTENCY KEY + UNIQUE INDEX (the workhorse)
  CREATE TABLE processed (
    idem_key   CHAR(36) PRIMARY KEY,
    result     JSON,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

  BEGIN;
    INSERT INTO processed (idem_key) VALUES (?);   -- duplicate key =
                                                   -- already done, return
    ... the real side effect ...
  COMMIT;
  The unique index is the lock. No race, no distributed coordination.
  Retention: 7-30 days, then prune (same discipline as Topic 59).

PATTERN 2 -- CONDITIONAL UPDATE (state machines)
  UPDATE bookings SET status='paid', paid_at=now()
   WHERE id=? AND status='pending';
  affected_rows = 0 means someone already did it. Cheap, no extra table.

PATTERN 3 -- PASS THE KEY DOWNSTREAM
  POST /v1/charges
  Idempotency-Key: 3f9a-booking-9001-attempt
  Every serious payment API accepts this. The duplicate charge is
  then prevented by the gateway, which is the only place it can be
  prevented correctly.

DEDUPE WINDOW SIZING
  redelivery happens within seconds normally, but a DLQ replay
  (Topic 68) can be days later. Size the window for your worst
  replay, not your typical retry: 30 days x 900 msg/s x 36 B key
  = 70 GB if you store every key -- so store keys only for
  operations with irreversible side effects.
```

THE KEY INSIGHT

A unique constraint in the database is a distributed lock that already exists, is already replicated, and already survives failover. Reach for it before any lock service; nine idempotency problems in ten are a `PRIMARY KEY` away from solved.

THE TRAP

An idempotency key derived from the message body's hash. Symptom: two legitimately identical actions — the same user buying the same ticket twice on purpose — and the second is silently swallowed. The key must identify the *attempt*, generated by the client or the producer, not the content.

```
idempotency — GAINS: duplicates become free; retries become safe everywhere · COSTS: a key table
and its retention, one extra write per job · ACCEPTS: dedupe is bounded by the window you keep
```

64. Kafka's model: partitions, offsets and consumer groups

Kafka is a partitioned, replicated, append-only log. Four concepts carry all of its behaviour, including its one famous limitation.

```
$ kafka-topics.sh --create --topic bookings \
    --partitions 12 --replication-factor 3 --config min.insync.replicas=2

TOPIC        a named stream
PARTITION    an ordered, immutable log. Ordering exists HERE and
             nowhere else. partition = hash(key) % 12
OFFSET       a consumer's position. Committed, not deleted.
CONSUMER GROUP  a set of consumers sharing the work: each partition
             is assigned to EXACTLY ONE consumer in the group.

  -> the classic constraint: parallelism <= partition count.
     12 partitions means at most 12 useful consumers; a 13th sits idle.

$ kafka-consumer-groups.sh --describe --group ticket-workers
TOPIC     PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID
bookings  0          8841203         8841203         0    worker-3
bookings  1          8839114         8841990         2876 worker-7   <--
bookings  2          8840551         8840551         0    worker-1

DURABILITY KNOBS, and what they cost
  acks=0    fire and forget            0 ms      lose everything on crash
  acks=1    leader only                ~2 ms     lose on leader failure
  acks=all  + min.insync.replicas=2    ~5 ms     survives one broker loss
                                                 <-- use this for anything
                                                     you care about

RETENTION is time or size, not consumption:
  retention.ms=604800000        (7 days -- replay window)
  A consumer down for 8 days has lost data. Alert on lag in TIME
  (records-lag-max plus consumer_lag_seconds), not just in records.

THROUGHPUT REALITY
  one partition sustains ~10-50 MB/s. 12 partitions x 1 KB messages
  ~ 120,000-600,000 msg/s per topic on modest hardware. Kafka is
  almost never the bottleneck; your consumer's 850 ms of work is.
```

```
topic bookings
 P0 [e][e][e][e][e][e]<- worker-1        ordering guaranteed INSIDE
 P1 [e][e][e][e]      <- worker-7        a partition only
 P2 [e][e][e][e][e]   <- worker-1
 group ticket-workers: 12 partitions over 8 workers = some own 2
 add a 13th worker -> it is idle (until share groups, Topic 65)
```

THE KEY INSIGHT

Partition count is a capacity *and* an ordering decision made at topic creation, and increasing it later changes `hash(key) % N` — so messages for one key start landing in a different partition and ordering breaks across the boundary. Over-provision partitions modestly at the start (2–4× expected consumers), because the fix later is a new topic and a migration.

THE TRAP

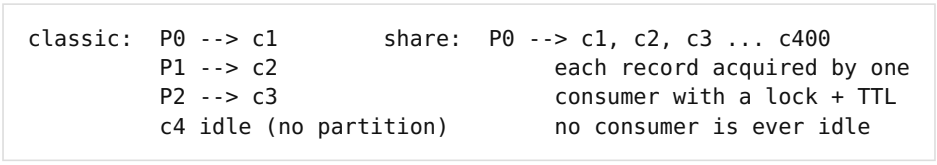
`acks=1` with automatic leader election. Symptom: after a broker restart, a few thousand records that the producer logged as delivered are simply absent downstream. Reconciliation finds them missing days later. Use `acks=all` plus `min.insync.replicas=2` for anything with business meaning, and accept the 3 ms.

```
Kafka log — GAINS: replay, multi-consumer fanout, ordered per key, 100k+ msg/s · COSTS: partition planning, ZooKeeper-free but still a cluster to run · ACCEPTS: consumer parallelism capped by partitions, ordering only within one
```

65. Share groups: Kafka finally does queues

The partition-equals-consumer constraint of Topic 64 is exactly what makes Kafka awkward as a job queue. Share groups (KIP-932, production-ready in Apache Kafka 4.2) remove it: many consumers may read the same partition cooperatively, with per-record acknowledgement.

```
CLASSIC CONSUMER GROUP        SHARE GROUP
  1 partition -> 1 consumer     N consumers may share 1 partition
  offset commit (a position)    per-record acquire + ack + delivery count
  scale = add partitions        scale = add consumers, instantly
  ordered per partition         NOT ordered -- it is a queue
  a poison message blocks       per-record reject -> release/archive
  the partition (T69)

$ kafka-features.sh --bootstrap-server localhost:9092 \
    upgrade --feature share.version=1

$ kafka-console-share-consumer.sh --topic pdf-jobs --group pdf-workers
# the broker acquires a record for a consumer with a time-limited
# lock (an acquisition lock); on ack it is finished, on timeout or
# release it becomes available to another consumer -- exactly the
# visibility-timeout model of SQS (Topic 66), inside Kafka.

WHEN THIS CHANGES YOUR DESIGN
  before: "we need 400 concurrent PDF workers" -> 400 partitions,
          each with its own replica set, memory and rebalance cost.
  after:  4 partitions, 400 share-group consumers, no repartitioning
          when the burst of Topic 60 arrives.

WHAT IT DOES NOT CHANGE
  ordering is gone by construction. If your jobs must run in order
  per booking, use a classic consumer group keyed by booking_id.
  Consolidation is the point: one system for streams AND queues,
  rather than Kafka plus RabbitMQ.
```



THE KEY INSIGHT

Ordering and elastic parallelism are mutually exclusive, and share groups let you choose per topic rather than per cluster. Ask, for each stream: does this need order, or does it need workers? Almost no system needs both for the same messages.

THE TRAP

Assuming share groups make Kafka a drop-in RabbitMQ. Symptom: a team enables them on a topic whose consumers assumed per-key ordering, and state machines start processing "cancelled" before "created." Share groups deliver *concurrently from the same partition* — the ordering you relied on is gone the moment you switch.

```
share groups — GAINS: consumers scale past partition count, per-record ack and retry · COSTS: Kafka 4.2+, an internal state topic, new client APIs · ACCEPTS: no ordering at all within a share group
```

66. Task-queue mechanics: ack, visibility timeout, prefetch

RabbitMQ and SQS share one model with different names. Three settings determine whether your queue behaves or duplicates everything.

```
VISIBILITY TIMEOUT (SQS) / consumer ack timeout (RabbitMQ)
  how long a message stays invisible after delivery, before it is
  redelivered because no ack arrived.
  aws sqs receive-message --visibility-timeout 300

  THE RULE: visibility timeout > p99.9 of job duration.
  Job p50 = 800 ms, p99.9 = 240 s. Timeout at 30 s means every slow
  job is delivered again WHILE THE FIRST IS STILL RUNNING. Two
  workers, one job, one card charged twice.
  Symptom to recognise: duplicates that correlate with job DURATION,
  not with crashes. That is always this setting.
  Fix: raise the timeout, or heartbeat/extend it
  (ChangeMessageVisibility) from a long-running worker.

PREFETCH / max in flight (RabbitMQ basic.qos, Kafka max.poll.records)
  channel.basic_qos(prefetch_count=1)
  prefetch=1     perfect fairness, one extra RTT per message
  prefetch=100   throughput, but a worker holding 100 messages that
                 dies redelivers all 100, and a slow worker starves
                 its peers while its 100 sit idle.
  Rule of thumb: prefetch ~ ceil(1 / (job_time x consumers)) ... in
  practice: 1 for slow jobs (> 1 s), 10-50 for fast ones (< 50 ms).

MANUAL ACK, always:
  channel.basic_consume(queue='pdf', auto_ack=False)
  ... do the work ...
  channel.basic_ack(delivery_tag=tag)     # after, never before (T62)

$ rabbitmqctl list_queues name messages messages_unacknowledged consumers
name      messages  messages_unacknowledged  consumers
pdf-jobs  184203    600                      600
          ^backlog  ^in flight = prefetch x consumers
```

THE KEY INSIGHT

`messages_unacknowledged` equals prefetch × consumers when healthy. If it is pinned at that ceiling while the backlog grows, your consumers are saturated; if it is near zero while the backlog grows, they are not consuming at all — two different incidents that the depth graph alone cannot distinguish.

THE TRAP

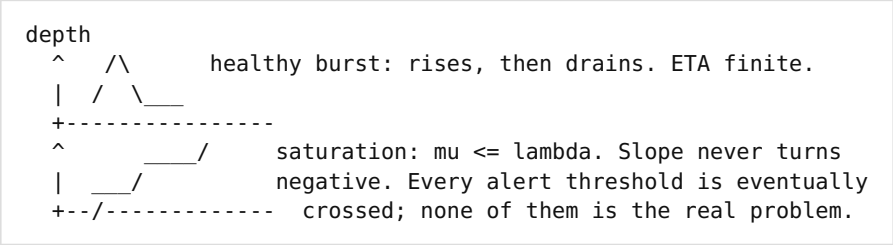
High prefetch with heterogeneous job durations. Symptom: one worker sits idle with 100 buffered messages while another grinds through a 10-minute job, and the queue drains at half the rate you provisioned. Prefetch buffers commit work to a worker before it knows what the work costs.

```
task-queue tuning — GAINS: fair distribution, bounded in-flight work · COSTS: one RTT per message
at prefetch=1 · ACCEPTS: a timeout shorter than p99.9 duration guarantees duplicate execution
```

67. Lag, backpressure and the depth graph

Queue depth is the most misread metric in production. Depth alone tells you nothing; depth plus its derivative, plus the age of the oldest message, tells you everything.

```
THE THREE SIGNALS
  depth            L, items waiting. Useless alone -- 200k is fine if
                   it drains in 2 minutes.
  oldest_age       W, how long the front item has waited. THIS is the
                   user-visible number. Alert on it.
  drain rate       mu - lambda. Negative means the backlog is
                   permanent and no amount of waiting fixes it.

  ETA = depth / (mu - lambda)          undefined when mu <= lambda

WORKED
  depth 184,203, consumers drain 706/s, arrivals 120/s
  ETA = 184,203 / 586 = 314 s = 5.2 min       -> healthy burst
  Same depth, drain 130/s, arrivals 120/s
  ETA = 184,203 / 10 = 18,420 s = 5.1 hours   -> incident,
  and the depth graph looks identical at this instant.

BACKPRESSURE -- what to do when mu cannot reach lambda
  1. shed at the edge: reject or 429 the producer (Topic 99, 103).
     A bounded queue that rejects is healthier than an unbounded
     one that lies.
  2. degrade the work: skip the thumbnail, defer analytics, send
     the SMS without the PDF.
  3. spill: write overflow to object storage and process later
     (Topic 79) -- keeps the broker healthy.
  4. autoscale consumers on oldest_age, not CPU. Workers waiting
     on a 200 ms API call are 5% CPU and 100% busy, so CPU-based
     scaling never fires (Topic 104).

WHAT NEVER WORKS: raising the broker's memory limit. That converts
a fast, visible failure into a slow, expensive one.
```



THE KEY INSIGHT

Alert on the age of the oldest message and on the sign of the drain rate. Depth thresholds page you at 3am for a burst that would have cleared, and stay silent through a slow saturation that is already breaking your SLO.

THE TRAP

Autoscaling consumers on CPU. Symptom: the backlog grows for an hour while the autoscaler reports 8% CPU and does nothing, because the work is I/O-bound. Scale on queue age or depth per consumer — the only signals that reflect the actual work.

```
lag management — GAINS: an incident you can see 40 minutes early · COSTS: three metrics and an
autoscaling rule per queue · ACCEPTS: when mu < lambda you must shed or degrade — there is no
third option
```

68. Retries, backoff, jitter and the dead-letter queue

Failures are normal; the retry policy decides whether they stay contained or become the outage. Two settings (backoff and jitter) and one component (the DLQ) cover almost all of it.

```
NAIVE: retry immediately, 5 times.
  A downstream service returns 503 for 60 s. 900 jobs/s x 5 attempts
  = 4,500 req/s at a service that is already failing. You have
  built a denial-of-service tool aimed at your own dependency
  (Topic 97).

EXPONENTIAL BACKOFF WITH FULL JITTER
  delay = random(0, min(cap, base x 2^attempt))
  base = 1 s, cap = 300 s
  attempt 1: 0-2 s   2: 0-4 s   3: 0-8 s   4: 0-16 s   5: 0-32 s
  Without jitter, all 900 failed jobs retry at exactly t+2, t+4, t+8
  -- a synchronised herd, the same pathology as the fixed TTL of
  Topic 35 and the health-check storm of Topic 15.

BUDGET, not just count: cap total retries per unit time, e.g.
  "retries may not exceed 10% of successful requests" -- so a broad
  outage cannot multiply load at all.

CLASSIFY BEFORE RETRYING
  retry:        503, 429, timeout, connection reset, deadlock,
                serialization failure (Topic 57)
  do not retry: 400, 401, 403, 404, 422 -- the input is wrong and
                will be wrong on attempt 5. Straight to DLQ.

DEAD-LETTER QUEUE
  after N attempts the message moves to a DLQ, out of the hot path.
  It must carry: original payload, failure reason, attempt count,
  first-seen timestamp, trace id (Topic 87).
  aws sqs set-queue-attributes --attributes RedrivePolicy=
    '{"maxReceiveCount":"5","deadLetterTargetArn":"...dlq"}'
  DLQ depth > 0 is ALWAYS an alert. A DLQ nobody watches is a
  silent data-loss queue with extra steps.
  Replay must be deliberate, rate-limited, and idempotent (T63) --
  a 40,000-message replay at full speed is the original incident,
  repeated.
```

THE KEY INSIGHT

Full jitter is not a refinement of backoff — it is the part that works. Exponential backoff alone still synchronises every client onto the same retry instants; randomness is what spreads the load, and it costs one function call.

THE TRAP

Retrying a non-idempotent job. Symptom: a customer is charged three times for one booking, and the logs show three successful gateway calls with three different request ids. Retry safety is a property of the *consumer* (Topic 63), not of the retry policy. Never enable retries on a handler you have not made idempotent.

```
retry policy — GAINS: transient failures disappear without human involvement · COSTS: a DLQ, an
alert, a replay tool · ACCEPTS: retries multiply load exactly when the system is least able to
take it
```

69. Partition keys, ordering and head-of-line blocking

Ordering exists only within a partition (Topic 64), so the partition key is a correctness decision: it declares which events must be processed in sequence, and therefore which cannot be parallelised.

```
KEY CHOICE, same trade-off as the shard key of Topic 51
  key = booking_id   all events for one booking are ordered.
                     Parallelism = number of distinct bookings. Good.
  key = null         round-robin, maximum spread, NO ordering.
  key = "global"     everything in one partition: total order and
                     1/12th of your throughput. Occasionally correct,
                     usually a mistake.
  key = tenant_id    ordered per tenant -- and one big tenant is a
                     hot partition (22% on 12 partitions, Topic 51).

HEAD-OF-LINE BLOCKING, the failure this creates
  partition 3: [ok][ok][POISON][e][e][e][e]... 40,000 waiting
  The poison message fails, is retried, fails, is retried. Because
  the consumer must preserve order, NOTHING behind it is processed.
  One malformed record halts one twelfth of your pipeline.

  Symptom: lag on exactly one partition while the other eleven read
  zero, and a consumer log repeating the same offset every 30 s.
  Diagnose with: kafka-consumer-groups.sh --describe (per-partition
  LAG column) -- one row huge, the rest zero.

THREE FIXES
  1. bounded retries then SIDELINE the record to an error topic
     and advance the offset. Order is broken for that key only,
     deliberately and visibly.
  2. share groups (Topic 65) where order is not required -- a
     poison record is released and archived per record.
  3. per-key retry queues: failed key K goes to a delay topic,
     the main partition keeps moving for all other keys.
```

THE KEY INSIGHT

Ordering is a cost you pay in throughput and in blast radius. Ask which events truly require it — usually only same-entity state transitions — and key on the narrowest entity that satisfies that, never on something broader "to be safe."

THE TRAP

Infinite retries on a partitioned consumer. Symptom: one partition's lag climbs linearly for hours while overall throughput looks fine, because the other eleven absorb the load. Always bound retries and always have somewhere to put the message afterwards.

```
partition keys — GAINS: per-key ordering with parallelism across keys · COSTS: hot keys become
hot partitions; poison records block a whole partition · ACCEPTS: any ordering guarantee is also
a serialisation point
```

70. Effectively-once, assembled: outbox plus idempotent consumer

This topic composes Topics 59, 62, 63 and 68 into the pattern you will actually deploy. Nothing here is new — that is the point.

```
PRODUCER SIDE (atomicity: the event exists iff the row exists)
  BEGIN;
    INSERT INTO bookings ...;
    INSERT INTO outbox (idem_key, topic, payload) VALUES (uuid(), ...);
  COMMIT;                                                    -- Topic 59
  relay: SELECT ... FOR UPDATE SKIP LOCKED -> publish -> mark sent
  (may publish twice if it dies after publishing: at-least-once)

CONSUMER SIDE (idempotence: running twice equals running once)
  handle(msg):
    BEGIN;
      INSERT INTO processed (idem_key) VALUES (msg.idem_key);
        -- duplicate key -> ROLLBACK, ack, return. Done already.
      apply_side_effect(msg);        -- must itself be transactional
                                     -- or carry the key downstream
    COMMIT;
    ack(msg);                        -- after commit, always (Topic 62)

RETRY POLICY                                                 -- Topic 68
  transient -> exponential backoff + full jitter, max 5
  permanent -> DLQ immediately, alert on DLQ depth > 0

END-TO-END GUARANTEE YOU CAN WRITE IN A DESIGN DOC
  "Every committed booking produces exactly one charge, because the
   event is committed with the booking, delivery is at-least-once,
   and the consumer deduplicates on idem_key for 30 days. Worst
   case is delay, not loss or duplication."

WHAT STILL BREAKS IT
  a side effect outside the transaction and without a key --
  e.g. an SMS gateway with no idempotency support. Then you choose:
  send twice (annoying) or risk not sending (worse). Write down
  which you chose. It is a product decision, not a technical one.
```

```
[tx: row + outbox] -> relay -> broker -> consumer [tx: idem + effect] -> ack
       atomic          >=1x     ordered      dedupe            durable
   loss impossible   dup possible          dup absorbed      = effectively once
```

THE KEY INSIGHT

Correctness comes from two transactions, one on each side, joined by a key. The broker in the middle is deliberately allowed to be sloppy — that is why this pattern survives every broker failure mode, including redelivery, reordering across keys and replay.

THE TRAP

Deduplicating in memory or in a cache with a short TTL. Symptom: a DLQ replay three days after the incident re-charges 4,000 customers, because the dedupe set only held an hour. The dedupe store must outlive your worst replay window (Topic 63) and must be as durable as the side effect it protects.

```
effectively-once — GAINS: no loss, no duplicate side effects, replay-safe · COSTS: two extra
tables, retention jobs, discipline in every handler · ACCEPTS: latency, and duplicates for side
effects that cannot take a key
```

71. Delay, priority and fairness

Three requirements that arrive after the first queue is running, each with a standard implementation and a standard way of going wrong.

```
DELAYED / SCHEDULED
  "remind 24 h before departure", "retry in 5 minutes"
  SQS:      DelaySeconds up to 900 s only (15 min ceiling)
  RabbitMQ: TTL + dead-letter exchange, or the delayed-message plugin
  Redis:    ZADD due_jobs <unix_ts> job_id ; poller does
            ZRANGEBYSCORE due_jobs 0 now LIMIT 0 500
  Beyond ~15 min, store the schedule in the DATABASE and enqueue when
  due. Brokers are bad calendars: a 3-month delay in a broker is
  3 months of retained state you cannot query, cancel or audit.

PRIORITY
  Separate queues beat priority flags: high / default / low, with
  workers weighted 6:3:1. Explicit, observable, and each queue has
  its own depth graph.
  Single queue with priority numbers -> starvation of low priority
  (nothing ever drains) and no visibility into which class is late.

FAIRNESS / MULTI-TENANCY
  One tenant enqueues 400,000 jobs. Every other tenant waits behind
  them -- the queueing version of the hot shard (Topic 51).
  Fixes:
   a) per-tenant queues + round-robin over queues (or Kafka share
      groups with a queue per tenant class)
   b) admission quota at enqueue: max N in-flight per tenant,
      reject or defer above it (Topic 103)
   c) weighted fair queueing: pick the tenant with least service
      received, like a scheduler's virtual time.

  MEASURE fairness as p99 wait time BY TENANT. A global p99 hides
  one tenant waiting 40 minutes behind another's bulk import.
```

THE KEY INSIGHT

Separate queues are almost always better than priority levels inside one queue. They give per-class metrics, per-class scaling, per-class retry policy and an obvious failure boundary — four things a priority integer cannot provide.

THE TRAP

Using the broker as a scheduler for far-future work. Symptom: a marketing job scheduled six weeks out cannot be cancelled after the campaign is called off, because there is no way to query or delete individual delayed messages. Keep schedules in a table; enqueue only what is due within minutes.

```
delay/priority/fairness — GAINS: latency guarantees per class and per tenant · COSTS: more
queues, a scheduler table, quota tracking · ACCEPTS: strict priority starves the bottom class
unless you weight it
```

72. When not to use a queue

Queues add a broker, a consumer fleet, duplicate handling, a DLQ, and a whole class of "it is still processing" support tickets. Five cases where the synchronous call is the better engineering.

```
1. THE USER IS WAITING FOR THE RESULT.
   A search, a price quote, a login. Making it async only adds
   polling and a spinner. If the answer must be on screen in 200 ms,
   a queue cannot help -- only faster work can.

2. THE WORK IS SHORTER THAN THE OVERHEAD.
   Job takes 8 ms; enqueue + broker + poll + ack costs 6-15 ms and
   a second failure domain. Do it inline.

3. ORDERING AND ATOMICITY MATTER MORE THAN THROUGHPUT.
   "Deduct stock, then reserve seat" inside one transaction is
   simple and correct. Split across a queue it becomes a saga
   (Topic 54) with visible intermediate states.

4. YOU CANNOT TOLERATE DUPLICATES AND CANNOT BE IDEMPOTENT.
   A third-party API with no idempotency key and irreversible
   effects. At-least-once will eventually double it (Topic 62). A
   synchronous call with a strict timeout at least fails visibly.

5. THE QUEUE IS HIDING A CAPACITY PROBLEM.
   If mu < lambda in steady state (Topic 60), the queue converts an
   obvious overload into a growing backlog, a stale dashboard and an
   incident discovered by customers. Buy capacity or shed load; a
   buffer in front of a permanently undersized consumer is a
   deferred outage with worse observability.

ALSO CONSIDER, BEFORE A BROKER:
   your database as a queue -- SELECT ... FOR UPDATE SKIP LOCKED
   handles thousands of jobs/s, gives you transactional enqueue for
   free (Topic 59), and uses infrastructure you already operate and
   back up. Outgrow it at ~5,000 jobs/s, not before.
```

THE KEY INSIGHT

The right question is not "should this be async" but "does the caller need the result to continue." If yes, it is synchronous and must be fast. If no, it is asynchronous and must be observable. Everything else follows from that one answer.

THE TRAP

Introducing a queue to fix p99 latency. Symptom: the API's p99 drops to 5 ms and the user-perceived latency gets worse, because the work still takes 850 ms and now the user also polls for it. You moved the latency out of your graphs, not out of the product.

```
not queueing — GAINS: one fewer system, no duplicates, no backlog to explain · COSTS: the request
path must be sized for peak · ACCEPTS: a slow dependency directly slows your API, which is at
least honest
```

PRACTICE SET 6 — Topics 60–72

A. Conceptual

1. In one sentence, what does a queue actually convert, and into what?
2. Why is a queue useless if the mean arrival rate exceeds the service rate?
3. State Little's Law and say which term you control when you add workers.
4. Give the property a log has that a task queue cannot have at any price.
5. Why does ack timing *define* the delivery semantic?
6. Explain why Kafka's transactional "exactly-once" does not cover a payment gateway call.
7. Why is a unique index a better idempotency mechanism than a lock service?
8. Why must an idempotency key identify the attempt rather than the content?
9. What caps consumer parallelism in a classic Kafka consumer group, and what does that number cost you if set too high?
10. What exactly do share groups give up in exchange for elastic consumers?
11. You see duplicates that correlate with job duration, not with crashes. Name the setting.
12. Why can two queues with identical depth be in completely different health states?
13. Why does autoscaling consumers on CPU fail for I/O-bound work?
14. Why is exponential backoff without jitter still a synchronised herd?
15. Which HTTP status codes should never be retried, and why?
16. Describe the symptom of head-of-line blocking as it appears in `kafka-consumer-groups.sh --describe`.
17. In the effectively-once assembly, which two transactions carry the correctness, and what joins them?
18. Why are separate priority queues better than a priority integer?
19. Why is a broker a poor scheduler for work due in six weeks?
20. Give two situations where a database table with `SKIP LOCKED` is the correct queue.

B. Pen-and-paper

1. Bookings arrive at 6,000/s for 120 s, then 200/s. Post-processing takes 1.1 s per booking. Compute synchronous worker count for the peak, then the async worker count for the mean. With 900 workers, how long to clear the burst backlog?
2. Same burst. You require the oldest message never to exceed 5 minutes. Using Little's Law, what service rate and worker count does that demand?
3. A topic has 12 partitions and messages average 1.4 KB at 90,000 msg/s. What is the per-partition throughput in MB/s, and is that within a single partition's practical limit?
4. Job p50 is 700 ms, p99 is 40 s, p99.9 is 260 s. Visibility timeout is 60 s and you process 900 jobs/s. Roughly how many duplicate executions per hour does the timeout alone cause?
5. Depth 240,000, arrivals 300/s. Case A: drain 950/s. Case B: drain 340/s. Compute ETA for both, and state what each should page.
6. Full jitter with base 1 s, cap 300 s. List the delay range for attempts 1–6 and the expected total elapsed time before the DLQ, using the mean of each range.
7. A downstream service fails for 90 s. You process 1,200 jobs/s with 5 immediate retries. Compute the peak request rate you inflict. Then recompute with a retry budget capped at 10% of successful requests.

8. One tenant enqueues 500,000 jobs; normal traffic is 40 jobs/s from 900 tenants. Workers drain 700/s FIFO. How long does an ordinary tenant's job wait? What does a per-tenant in-flight cap of 50 change?
9. Outbox relay throughput is 2,000 rows/s; producers write 1,500 rows/s. The relay is down 45 minutes. Compute the backlog and the catch-up time, and the maximum end-to-end lag any consumer sees.

C. Lab tasks (build → break → observe → fix)

1. **Build.** Stand up RabbitMQ or Redis + a worker. Enqueue 50,000 jobs of ~50 ms each. Record depth, in-flight and drain rate every 5 s, then compare measured drain time against the Little's Law prediction.
2. **Break: at-most-once.** Set `auto_ack=True` (or `enable.auto.commit=true` with a long interval) and `kill -9` a worker mid-batch. Count how many jobs were never performed and never redelivered. Switch to manual ack after processing and repeat — the count must become zero and duplicates must appear instead.
3. **Break: the visibility timeout.** Give the handler a `sleep(90)` and set the ack/visibility timeout to 30 s. Log a line with the message id and process id at the start of every execution. Show the same id being executed by two workers simultaneously. Fix by raising the timeout, then by heartbeat-extending it, and show the overlap disappear.
4. **Break: the non-idempotent consumer.** Have the handler `INSERT` a ledger row and crash after the insert but before the ack. Count rows after redelivery. Add the `processed(idem_key)` table and show the second attempt rolling back on the duplicate key while still acking.
5. **Break: head-of-line blocking.** On a keyed topic (or a single-consumer FIFO queue), inject one message the handler always throws on. Watch the per-partition lag while the other partitions stay at zero. Fix with bounded retries plus an error topic, and show the offset advancing.
6. **Break: the retry storm.** Point the handler at a local service you can stop. Stop it for 60 s with immediate retries enabled and record requests/s hitting it (`nginx` access log count). Re-run with exponential backoff plus full jitter and compare the peak.
7. **Break: the silent DLQ.** Route permanent failures to a DLQ with no alert, run for 10 minutes with 2% poison messages, then count what accumulated. Add an alert on DLQ depth > 0 and a rate-limited replay tool; verify the replay is idempotent by running it twice.

ANSWER KEY 6 — Topics 60–72

A. Conceptual

1. It converts **a capacity problem into a latency problem**: the peak is absorbed as delay instead of as provisioned workers.
2. Because depth grows without bound — **ETA = depth/(μ−λ) is undefined when μ ≤ λ**. Only more capacity, shedding or degradation exits that state.
3. **L = λW.** Adding workers raises μ, which lowers **W** (wait time) and therefore L; it does not change λ.
4. **Replay.** Consumption is a read, not a destructive pop, so history can be re-consumed by a new consumer group to rebuild state (Topic 55).
5. Ack before processing = **at-most-once (loss)**; ack after = **at-least-once (duplicates)**. There is no third placement, so the semantic is literally where you put the ack.
6. Kafka's transaction spans **reads and writes within Kafka**. The gateway call is an external side effect outside that transaction, so a crash after charging still redelivers.
7. It is **already distributed, replicated, durable and survives failover**, and it makes the race impossible rather than unlikely — no lease renewal, no clock assumptions.
8. Because **two legitimate identical actions** (the same user buying the same ticket twice deliberately) must both succeed. A content hash would swallow the second.
9. **Partition count** — one partition per consumer maximum. Too many partitions cost replica sets, memory, open files and longer rebalances.
10. **Ordering**, entirely. Records from one partition are delivered concurrently to many consumers, so no sequence is preserved.
11. **Visibility timeout / ack timeout shorter than p99.9 job duration.** The first execution is still running when the message is redelivered.
12. Because health is the **derivative and the age**, not the level: 240,000 draining at +650/s clears in minutes; the same depth draining at +10/s is a five-hour incident.
13. A worker blocked on a 200 ms API call is **5% CPU and 100% busy**, so the CPU signal never crosses the threshold while the backlog grows.
14. Because every failed client computes **the same delay** (t+2, t+4, t+8) and retries in lock-step. Jitter is what decorrelates them.
15. **400, 401, 403, 404, 422** — the request is wrong and will be equally wrong on attempt five. Retrying wastes capacity and delays the DLQ signal.
16. **One partition's LAG column is huge while the others read zero**, and the consumer log repeats the same offset on a fixed interval.
17. The **producer's local transaction** (row + outbox) and the **consumer's local transaction** (idem key + side effect), joined by the **idempotency key** carried in the message.
18. Separate queues give **per-class depth metrics, scaling, retry policy and failure isolation**; a priority integer gives none of those and starves the lowest class.
19. Delayed messages **cannot be queried, cancelled or audited**, and they hold broker state for the entire period. A table can be queried and a row can be deleted.
20. **Under ~5,000 jobs/s**, and **when the enqueue must be transactional with the business write** (Topic 59) — you get atomicity free and operate one fewer system.

B. Worked

1. Sync: 6,000×1.1 = **6,600 workers**. Async mean: 200×1.1 = **220 workers**. With 900 workers, μ = 900/1.1 = 818/s; surplus over 200/s = 618/s; backlog = 6,000×120 = 720,000 → 720,000/618 = **1,165 s ≈ 19.4 minutes**.
2. W ≤ 300 s at λ = 6,000/s → L ≤ **1.8 M in flight**, which the burst (720,000) satisfies; the binding constraint is draining 720,000 within 300 s → surplus ≥ 2,400/s → μ ≥ 2,600/s → **2,860 workers**. Note it is still 2.3× cheaper than synchronous.
3. 90,000×1.4 KB = 126 MB/s total → **10.5 MB/s per partition**, comfortably inside the 10–50 MB/s practical range. **Kafka is not the bottleneck**; consumer work is (Topic 64).
4. Jobs exceeding 60 s are those above roughly the p99 mark: ≈1% of 900/s = 9/s → **≈32,400 duplicate executions per hour**. Each is a concurrent double-run, not a sequential retry — the worst kind.
5. A: 240,000/650 = **369 s ≈ 6 minutes** → healthy burst, **do not page**. B: 240,000/40 = **6,000 s ≈ 1.7 h → page on oldest-message age**, and check whether μ is about to fall below λ.
6. Ranges: **0–2, 0–4, 0–8, 0–16, 0–32, 0–64 s**. Means 1+2+4+8+16+32 = **63 s** of expected elapsed time before the DLQ.
7. Naive: 1,200×(1+5) = **7,200 req/s** aimed at a failing service. With a 10% budget: 1,200 successful-equivalent → retries capped at **120/s**, total ≈**1,320 req/s** — 5.5× less, and the dependency can recover.
8. FIFO: the 500,000 backlog drains at 700−40 = 660/s → **≈12.6 minutes** of wait for every other tenant's jobs. With a 50-in-flight cap per tenant, the bulk tenant occupies at most 50 of the workers, so ordinary jobs wait **under a second** and the bulk import simply takes longer.
9. 45 min × 1,500/s = **4.05 M rows**. Surplus 2,000−1,500 = 500/s → **8,100 s = 2.25 h** to catch up. Max end-to-end lag ≈ **45 min + 2.25 h = 3 hours** — which is why outbox depth is an alert, not a dashboard (Topic 59).

C. What to verify

1. Measured drain time should match **depth/(μ−λ)** within ~10%. If it does not, look for prefetch buffering or a connection limit — the model is rarely wrong, the assumed μ usually is.
2. With auto-ack: a **non-zero count of jobs that vanished**, no DLQ entry, no error. With manual ack: **zero lost** and **≥1 duplicate** (the in-flight job at kill time). That swap is Topic 62 in one experiment.
3. Log must show the **same message id in two process ids with overlapping timestamps**. After raising the timeout above p99.9, overlaps go to zero; with heartbeat extension, they stay zero even for a 10-minute job.
4. Expect **2 ledger rows** for one logical event. With the `processed` table: **1 row**, and the second attempt should log a duplicate-key rollback and still ack — verify it acks, or you have built an infinite loop.
5. One partition's lag climbs linearly, the rest stay at 0, and the consumer logs the **same offset repeatedly**. After the fix, lag drains and the poison record appears in the error topic **with its attempt count and trace id**.
6. Immediate retries should show roughly **(1+N)× normal rate** at the failing service. With backoff and full jitter, the peak should fall by **4–6×** and the arrival pattern should look scattered rather than spiked at fixed intervals.

7. The DLQ should hold **≈2% of everything processed** and nothing should have alerted — that is the point of the exercise. The double replay must produce **the same final state**; if row counts double, your handler is not idempotent (Topic 63) and the replay tool is dangerous.

HANDNOTE SUMMARY CARD — PART 6

```
================================================================================
 QUEUE SYSTEMS -- ONE PAGE                                        Topics 60-72
================================================================================
 MATH
   L = lambda * W                 ETA = depth / (mu - lambda)
   mu <= lambda  ->  backlog is permanent. No setting fixes it.
   sync workers = peak_rate * job_time      async = mean_rate * job_time
   6,000/s burst x 1.1 s: sync 6,600 workers | async 900 -> drains in 19 min
--------------------------------------------------------------------------------
 FOUR SHAPES
   task queue  consumed+deleted, per-msg ack/retry/delay, no order  (SQS/Rabbit)
   log         appended+retained, offsets, REPLAYABLE, ordered/part (Kafka)
   pub/sub     no storage; offline subscriber loses it forever      (Redis)
   stream      log + windows/joins/state                            (Flink)
   need replay or 2nd consumer -> log | per-message retry -> task queue
--------------------------------------------------------------------------------
 SEMANTICS (ack timing IS the semantic)
   ack BEFORE work = at-most-once (loss)   ack AFTER = at-least-once (dups)
   exactly-once = at-least-once + idempotent consumer = "effectively once"
   idempotency:  INSERT processed(idem_key) PK   -> dup key = already done
                 UPDATE ... WHERE status='pending' -> affected_rows=0
                 pass Idempotency-Key downstream
   key identifies the ATTEMPT, never the content. Window >= worst replay.
--------------------------------------------------------------------------------
 KAFKA
   partition = hash(key) % N ; ordering exists only inside a partition
   classic group: 1 partition -> 1 consumer  (parallelism <= partitions)
   share group (4.2, KIP-932): N consumers per partition, per-record ack,
     acquisition lock -- queue semantics, NO ordering
   acks=all + min.insync.replicas=2 for anything that matters (+3 ms)
   retention is TIME, not consumption. Alert on lag in seconds.
--------------------------------------------------------------------------------
 TASK QUEUE KNOBS
   visibility/ack timeout > p99.9 job duration   (else concurrent double-run;
     symptom: duplicates correlate with DURATION, not crashes)
   prefetch 1 for jobs > 1 s, 10-50 for < 50 ms
   unacked == prefetch x consumers when healthy
--------------------------------------------------------------------------------
 RETRIES
   delay = random(0, min(cap, base * 2^attempt))     FULL JITTER, always
   1s base: 0-2, 0-4, 0-8, 0-16, 0-32 -> ~63 s expected before DLQ
   retry: 503 429 timeout deadlock serialization | never: 400 401 403 404 422
   retry budget <= 10% of successes. DLQ depth > 0 = ALERT. Replay rate-limited
   and idempotent.
--------------------------------------------------------------------------------
 ALERT ON
   oldest_message_age (user-visible)   sign of (mu - lambda)   DLQ depth
   per-partition lag (one hot row = head-of-line block)   outbox depth
   NOT on: depth alone, consumer CPU (I/O-bound work never trips it)
--------------------------------------------------------------------------------
 TRAPS
   auto-ack default            -> silent loss on OOM
   infinite retries + ordering -> one poison record halts a partition
   broker as scheduler         -> cannot cancel a 6-week delay
   dedupe in cache with 1h TTL -> 3-day DLQ replay re-charges everyone
   queue to "fix p99"          -> latency left your graph, not the product
--------------------------------------------------------------------------------
 DO NOT QUEUE WHEN
   user waits for the result | job < broker overhead | needs one transaction
   non-idempotent + irreversible | mu < lambda in steady state
   under ~5k jobs/s: SELECT ... FOR UPDATE SKIP LOCKED in your DB is enough
================================================================================
```

PART SEVEN

File and object storage

Object storage is the one component you can treat as infinite, and the one whose bill grows without anybody noticing. The engineering is in three places: keeping bytes out of your application servers, keeping the database and the bucket in agreement, and knowing what a retrieval actually costs.

Part 7 · File and object storage

73. Block, file, object: three different contracts

All three store bytes; they differ in what operations exist and therefore in what scales. Choosing wrongly is expensive because migration means rewriting every access path.

```
BLOCK (EBS, a SAN volume, a bare disk)
  contract: read/write fixed-size blocks at offsets. No names.
  a filesystem is what gives it names. Attached to ONE machine
  (mostly), and its size is fixed at provisioning.
  latency: 0.1-1 ms.  Use for: database files, VM disks.

FILE (NFS, EFS, SMB)
  contract: POSIX -- directories, rename, partial writes, locks,
  permissions. Shared by many machines.
  Costs: metadata operations are chatty and become the bottleneck.
  A directory listing of 200,000 entries is a slow, serialised call.
  latency: 1-10 ms.  Use for: legacy apps that expect a filesystem,
  shared config, home directories.

OBJECT (S3, GCS, R2, MinIO)
  contract: PUT/GET/DELETE a whole object by key, over HTTP.
  No partial write. No rename (copy + delete). No directories --
  "folders" are just a prefix convention with a slash in the key.
  Unlimited capacity, 11 nines durability, and it is reachable by
  every machine and every browser.
  latency: 20-100 ms first byte (5-10 ms on express/one-zone tiers).
  Use for: user uploads, images, video, backups, logs, data lakes.

THE TEST
  Do you need to modify 40 bytes in the middle of a 2 GB file?  block
  Does an existing binary expect open()/seek()/write()?          file
  Is it write-once, read-many, by key, from anywhere?           object

COST, same 10 TB stored for a month:
  EBS gp3     ~$800     (provisioned, whether used or not)
  EFS         ~$3,000   (per GB, POSIX is expensive)
  S3 Standard ~$230     (plus requests and egress -- Topic 79)
```

```
block   [ 0101 | 0110 | 1100 ]   offsets, one host
file    /a/b/c.jpg  (open, seek, write, rename, chmod)  many hosts
object  PUT /bucket/2026/08/9001/original.jpg   HTTP, everyone, whole objects
```

THE KEY INSIGHT

Object storage scales because it removed the operations that force coordination: no rename, no partial write, no locks, no directory tree. Every capability it dropped is one you must reimplement if you need it — and usually you do not.

THE TRAP

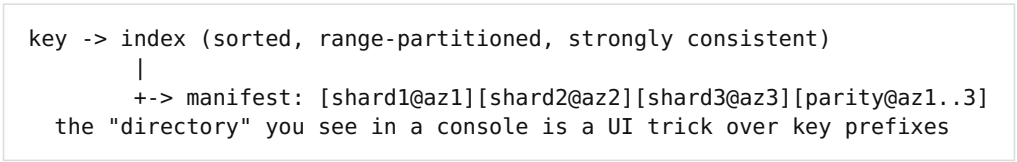
Treating a bucket like a filesystem. Symptom: a "list the folder" endpoint takes 40 s and times out, because `LIST` is a paginated scan over a sorted keyspace, at 1,000 keys per request, with no shortcut for counting. Keep the file *index* in your database (Topic 81) and use the bucket only to fetch bytes by exact key.

```
storage class choice — GAINS: object storage is unlimited, cheap and globally reachable · COSTS:
no rename, no partial write, 20–100 ms latency · ACCEPTS: you maintain the index yourself, in the
database
```

74. Inside an object store: flat keyspace and key design

There are no directories. There is one enormous sorted key–value index mapping key to a manifest of data shards, partitioned across an index fleet. Key design is therefore partition design, and it has visible performance consequences.

```
PUT s3://tickets/2026/08/13/booking-9001/original.jpg
  1. authenticate, resolve bucket -> index partition by key prefix
  2. split payload into shards, erasure-code them (Topic 75)
  3. write shards across >= 3 AZs, wait for quorum
  4. commit the key -> manifest mapping in the strongly consistent
     index. THIS commit is what makes the object visible.
  5. return 200 with an ETag

WHY PREFIXES MATTER
  the index is sorted and range-partitioned. Sequential prefixes
  concentrate writes on one partition until it splits.
  bad:  2026-08-13T09:00:01-booking-9001     (timestamp first --
        every write in the same second is adjacent, one partition)
  good: 9f2b1c/2026/08/13/booking-9001.jpg   (hash prefix spreads)
  S3 now auto-scales to ~3,500 PUT/s and 5,500 GET/s PER PREFIX, and
  splits partitions automatically -- but splitting takes 30-60 min,
  during which you get 503 SlowDown on a launch-day burst.

KEY DESIGN RULES
  1. keys are immutable identifiers -- include a version or content
     hash so a re-upload never overwrites (Topic 80, 25)
  2. never put mutable state in the key (no "approved/" prefix that
     you would need to move; a move is copy + delete, and 2 TB of
     copying)
  3. keep the database's row id in the key so an orphan is traceable
     back to a row (Topic 81)
  4. flat and predictable beats deep and clever

  tickets/{hash4}/{booking_id}/{uuid}-original.jpg
  tickets/{hash4}/{booking_id}/{uuid}-thumb-320.webp
```



THE KEY INSIGHT

The commit of the key→manifest mapping in a strongly consistent index is what gives modern object stores read-after-write consistency (Topic 76). The bytes were durable before that; the object was not *visible* until the index said so.

THE TRAP

Timestamp-prefixed keys on a launch day. Symptom: `503 SlowDown` during exactly the hour that matters, from a service with no throughput limits in its marketing. Put entropy in the first 4–8 characters of the key.

```
key design — GAINS: even partition load, traceable objects, safe re-uploads · COSTS: keys become opaque and must be stored in the database · ACCEPTS: renaming or reorganising later means copying every byte
```

75. Durability: what 11 nines means and how it is bought

"99.999999999% durability" is an annual expected-loss figure, not a promise. It is bought with erasure coding across independent failure domains, and it is worth knowing the arithmetic because it tells you what durability does *not* cover.

```
REPLICATION (3 copies)         storage overhead 3.0x
  survives 2 simultaneous device losses.

ERASURE CODING (k data + m parity, e.g. 6+3)
  object split into 6 data shards + 3 parity shards, spread over
  >= 3 AZs. Any 6 of the 9 reconstruct the object.
  storage overhead = 9/6 = 1.5x        <-- half the cost of 3x
  survives 3 simultaneous shard losses.
  cost: reconstruction reads 6 shards, so a degraded read is slower
  and repair traffic is heavy after a rack failure.

WHAT 11 NINES BUYS
  10,000,000 objects x 1e-11 = 0.0001 objects lost per year.
  Equivalently: store 10 million objects and expect to lose one
  every 10,000 years.

WHAT IT DOES NOT COVER -- and this is the real content of the topic:
  you deleting the object                  (versioning, Topic 82)
  your code overwriting it with garbage    (versioning + checksums)
  a ransomware or compromised credential   (object lock, MFA delete)
  a region-wide outage (availability,      (cross-region replication)
    which is 99.9-99.99%, six orders of
    magnitude weaker than durability)
  the bucket being public                  (that is not durability)

  Every real-world "we lost the files" incident is one of these,
  never a disk failure. Design against the list, not the nines.
```

```
object -> [d1][d2][d3][d4][d5][d6] + [p1][p2][p3]     6+3 erasure code
           AZ-a   AZ-b   AZ-c        spread across 3 AZs
 lose any 3 shards -> still reconstructable
 lose the whole key by DELETE -> 11 nines did nothing for you
```

THE KEY INSIGHT

Durability protects against hardware; versioning and access control protect against you. The provider's number is about disks failing, which is the failure mode that essentially never causes data loss in practice.

THE TRAP

A single-AZ storage class chosen for price. Symptom: an AZ event takes your entire media library offline for hours, and if the AZ is lost permanently, the data is gone — One Zone classes trade the multi-AZ spread for about 20% savings. Fine for regenerable derivatives (thumbnails), never for originals.

```
durability — GAINS: 1.5× overhead for 3-shard tolerance across AZs · COSTS: slower degraded
reads, heavy repair traffic · ACCEPTS: no protection at all against deletes, overwrites or
credential compromise
```

76. Consistency and conditional writes

Object stores were eventually consistent for years, and a generation of workarounds exists because of it. That era ended in December 2020; what remains is a per-key guarantee with a specific gap that conditional writes now close.

```
WHAT YOU GET (S3, GCS, and MinIO in modern versions)
  strong read-after-write for PUT, overwrite and DELETE
  strong list-after-write
  no extra cost, no latency penalty
  -> the old "sleep 200 ms then read" and S3Guard-style consistency
     layers are obsolete. Delete them if you still have them.

WHAT YOU DO NOT GET
  multi-object atomicity. Writing manifest.json and 40 segments is
  40+1 independent operations; a crash leaves a partial set.
  Concurrent PUTs to the same key: LAST WRITER WINS, silently.

  t=0    process A PUT profile.json  {name: "Ali", phone: X}
  t=5ms  process B PUT profile.json  {name: "Ali", phone: Y}
  -> whichever lands second wins; A's write is gone with a 200 OK.

CONDITIONAL WRITES = compare-and-swap on a key
  # create only if absent (a lock, a claim, a first-writer-wins)
  PUT /bucket/lock/job-9001   If-None-Match: *
    -> 412 Precondition Failed if it already exists

  # overwrite only if unchanged since I read it
  GET  /bucket/state.json           -> ETag: "9f2b1c3a"
  PUT  /bucket/state.json   If-Match: "9f2b1c3a"
    -> 412 if someone else wrote in between; re-read and retry

  aws s3api put-object --bucket b --key state.json \
      --if-match '"9f2b1c3a"' --body state.json

WHAT THIS UNLOCKS
  optimistic concurrency without a database (Topic 57's model,
  applied to a bucket), leader election, and a strongly consistent
  event log or table format built directly on object storage.
```

THE KEY INSIGHT

Strong consistency removed the stale-read problem but not the lost-update problem. `If-Match` is what converts last-writer-wins into an explicit CAS, and it is the same discipline as the version-key in Part 4 and the conditional `UPDATE ... WHERE status='pending'` of Topic 63.

THE TRAP

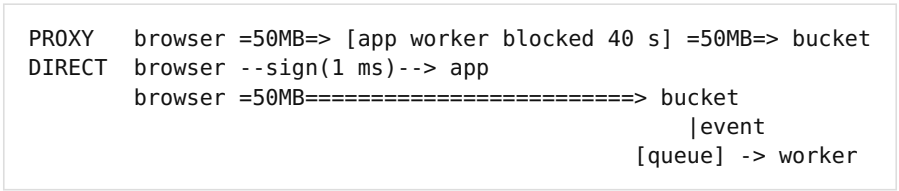
Two workers processing the same job because "the object was not there when I checked." Symptom: duplicate derivative files, doubled billing rows, two thumbnails with different sizes. `HEAD`-then-`PUT` is a race; `PUT ... If-None-Match: *` is not.

```
consistency — GAINS: read-after-write everywhere, CAS via preconditions · COSTS: a retry loop on
412, an ETag to carry · ACCEPTS: no atomicity across multiple objects — that must be designed
around
```

77. Upload paths: presigned direct upload versus proxying

The single highest-leverage decision in this part. If bytes travel through your application servers, your application servers are sized by upload traffic; if they do not, uploads cost you one signature.

```
PROXY (the default in most frameworks -- and usually wrong)
  browser --50 MB--> your app --50 MB--> bucket
  For 400 concurrent 50 MB uploads over 10 Mbit connections:
    each occupies a worker for 40 s
    400 workers blocked doing nothing but copying bytes
    2 GB of memory or temp disk churn
    bandwidth billed twice (in and out)
  Your API's p99 collapses for reasons unrelated to your API.

PRESIGNED URL (direct to storage)
  1. browser: POST /api/uploads {filename, size, content_type}
  2. app: validate quota/type/size, INSERT a pending row,
     return a URL signed for 15 minutes, scoped to ONE key
     and ONE content-type, with a size limit
  3. browser: PUT the bytes straight to the bucket
  4. bucket -> event notification -> queue -> worker marks the row
     complete and generates derivatives (Topic 80)

  $ aws s3 presign s3://tickets/9f2b/9001/uuid-original.jpg \
      --expires-in 900
  https://tickets.s3.amazonaws.com/9f2b/...&X-Amz-Expires=900
      &X-Amz-Signature=...

  App cost per upload: one signature, ~1 ms, zero bytes.

CONSTRAINTS TO SET IN THE POLICY, not in the client
  content-length-range 0-52428800     (server enforces the max)
  starts-with $Content-Type image/
  exact key -- never let the client choose it, or one user
  overwrites another's object (Topic 74)
```



THE KEY INSIGHT

A presigned URL is a capability: whoever holds it can perform exactly one operation on exactly one key until it expires. That is why the constraints must be baked into the signature — the client is not trusted, and after signing there is no further check.

THE TRAP

A presigned PUT whose key comes from the client's filename. Symptom: user B uploads `avatar.jpg` and user A's avatar changes, or worse, a key like `../../config/app.json` is accepted. Generate the key server-side from a UUID and the row id, always.

```
direct upload — GAINS: app servers do 1 ms of work instead of 40 s, no double bandwidth · COSTS:
an event-driven completion path, signature policy to get right · ACCEPTS: a leaked URL is a valid
write until it expires
```

78. Multipart, resumable and integrity

Large uploads over mobile networks fail. Multipart turns one fragile 50-minute transfer into many independently retryable pieces, and it is also how you get parallelism and checksums.

```
MULTIPART
  parts: 5 MB minimum (except the last), 5 GB maximum, 10,000 max
  -> maximum object 5 TB. Single PUT is capped at 5 GB.
  1. CreateMultipartUpload            -> uploadId
  2. UploadPart x N (parallel, retry each independently) -> ETags
  3. CompleteMultipartUpload {uploadId, [{partNumber, etag}...]}
     -- this is the atomic commit; the object appears here or not
        at all (Topic 76)

  $ aws s3 cp big.mp4 s3://media/big.mp4 \
      --expected-size 8589934592
  # the CLI does multipart automatically above 8 MB, 8 parallel parts

  2 GB file, 100 Mbit link:
    single PUT      160 s, and a disconnect at 95% restarts from 0
    multipart 16 MB 128 parts, 8 in parallel -> ~40 s wall clock,
                    a failed part costs 16 MB of retransmission

RESUMABLE = list what already landed, upload the rest
  $ aws s3api list-parts --upload-id ... --key big.mp4
  A client that stores the uploadId can resume after an hour offline.

INTEGRITY
  ETag is MD5 only for single-part uploads. For multipart it is
  "md5-of-concatenated-part-md5s-N" -- NOT the file's MD5. Comparing
  it to a local MD5 will fail and confuse you.
  Use explicit checksums instead:
    --checksum-algorithm SHA256      (per part and for the whole)
  and store the digest in your database row so you can verify later.

ABANDONED UPLOADS COST MONEY
  incomplete multipart parts are stored and billed forever.
  Lifecycle rule, non-optional:
    AbortIncompleteMultipartUpload: DaysAfterInitiation: 7
```

THE KEY INSIGHT

`CompleteMultipartUpload` is the commit point. Everything before it is invisible to readers, which means an upload interrupted at 99% leaves no partial object for anyone to read — the same all-or-nothing property a transaction gives you, achieved without one.

THE TRAP

No abort-incomplete lifecycle rule. Symptom: a storage bill that exceeds the total size of every object you can list, with no way to see the difference in the console's object count. Failed uploads accumulate invisibly; the rule costs one line of JSON.

```
multipart — GAINS: 4× faster via parallelism, per-part retry, 5 TB objects · COSTS: three API calls, an abort lifecycle rule, ETag is no longer an MD5 · ACCEPTS: abandoned parts are billed until a rule removes them
```

79. Storage classes, lifecycle and what retrieval really costs

Storage price per GB is the number everyone compares and the smallest part of the bill for anything accessed. Requests, retrieval fees, minimum durations and egress dominate, and archive tiers are where surprise invoices are made.

```
Class              $/GB/mo   retrieval    min days  first byte
-----------------  --------  -----------  --------  ------------
Standard           0.023     free         --        ms
Intelligent-Tier   0.023*    free         --        ms  (*+monitoring
                                                        per object)
Standard-IA        0.0125    $0.01/GB     30        ms
One Zone-IA        0.010     $0.01/GB     30        ms (1 AZ, T75)
Glacier Instant    0.004     $0.03/GB     90        ms
Glacier Flexible   0.0036    $0.01/GB     90        1-5 min to 12 h
Deep Archive       0.00099   $0.02/GB     180       12-48 h
+ requests: PUT ~$0.005/1,000, GET ~$0.0004/1,000
+ EGRESS to the internet: ~$0.09/GB  <-- usually the largest line

WORKED: 400 TB of ticket PDFs, 2% read per month
  All Standard:            400,000 x 0.023          = $9,200/mo
  Standard-IA:             400,000 x 0.0125         = $5,000
    + retrieval 8,000 GB x 0.01                     =    $80
                                                    = $5,080/mo
  Deep Archive:            400,000 x 0.00099        =   $396
    + retrieval 8,000 x 0.02                        =   $160
    + 12-48 h wait -- unusable for a customer download
  DECISION: originals to Standard-IA after 30 days, Deep Archive
  after 1 year (regulatory copies only, never on the read path).

THE TRAPS IN THIS TABLE
  minimum duration: delete an IA object after 3 days and you are
  billed for 30. A 90-day churn pattern in Glacier costs MORE
  than Standard.
  small objects: IA and Glacier bill a 128 KB minimum per object.
  4 KB thumbnails in IA are billed at 128 KB -- 32x the bytes.
  transition requests: moving 10 M objects to IA costs
  10,000 x $0.01 per 1,000 = $100 one-off, plus per-object overhead.

LIFECYCLE POLICY (the whole configuration is this small)
  { "Rules": [
    { "ID": "originals", "Filter": {"Prefix": "originals/"},
      "Transitions": [
        {"Days": 30,  "StorageClass": "STANDARD_IA"},
        {"Days": 365, "StorageClass": "DEEP_ARCHIVE"}],
      "Expiration": {"Days": 2555} },
    { "ID": "derivatives", "Filter": {"Prefix": "thumbs/"},
      "Expiration": {"Days": 90} },
    { "ID": "aborts",
      "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7} }]}
```

THE KEY INSIGHT

Derivatives should expire, not archive. A thumbnail is a pure function of the original, so deleting it after 90 days and regenerating on demand costs one CPU second instead of years of storage — and the regeneration path is already built (Topic 80).

THE TRAP

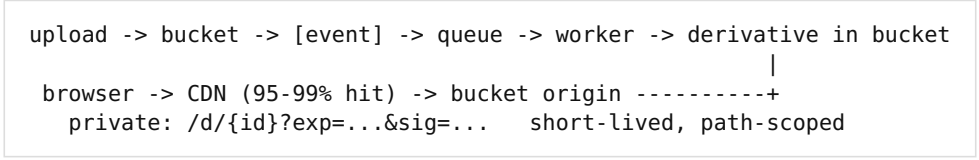
Lifecycle-transitioning millions of small objects to a colder class. Symptom: the bill goes *up* after a cost-optimisation project, from the 128 KB minimum billing size plus one transition request per object. Compute the break-even before writing the rule: it is roughly object size > 128 KB and retention > the minimum duration.

```
tiering — GAINS: 45–95% off storage for cold data · COSTS: retrieval fees, minimum durations,
per-object minimums, transition requests · ACCEPTS: archive tiers are hours away and cannot serve
users
```

80. Serving media: CDN, signed URLs and derivative pipelines

Serving bytes from a bucket to a browser is where Parts 3 and 7 meet. Three rules cover it: never serve originals, never serve from the bucket directly, and never let the client name the transformation for free.

```
THE PATH
  browser -> CDN (Part 3) -> origin: bucket (or an image service)
  CDN cache hit ratio for media is typically 0.95-0.99, so origin
  request volume is 1-5% of user volume, and egress is billed at
  CDN rates (~$0.08/GB) instead of bucket egress ($0.09/GB) with
  free origin-to-CDN transfer on most providers.

DERIVATIVES: generate on upload, or on first request?
  ON UPLOAD (eager)     predictable latency, wasted work for
                        variants nobody requests. 6 sizes x 4M
                        images = 24M objects you now store.
  ON FIRST REQUEST      one slow request per variant (200-800 ms),
  (lazy, then cached)   then CDN-cached. Storage only for what is
                        actually used. Preferred.
  Either way the worker is driven by the bucket event -> queue
  (Topic 77), and the derivative key encodes the transform:
    thumbs/{hash4}/{id}/{uuid}-w320-q80.webp

ACCESS CONTROL
  public assets      long max-age + immutable + content-hash key
  private documents  signed CDN URL, short expiry (5-15 min)
    - the signature must cover the path AND the expiry
    - a signed URL is a bearer capability: anyone who has it, has
      the file until it expires (same as Topic 77)
    - never sign a long-lived URL for a document that must be
      revocable; issue short ones from an authorised endpoint
  DO NOT make the bucket public and rely on obscure keys. Bucket
  enumeration is the single most common source of leaked user
  documents.

  Content-Disposition: attachment; filename="ticket-9001.pdf"
  set it at PUT time or via a signed response-header override, or
  browsers will render PDFs inline when you wanted a download.
```



THE KEY INSIGHT

The original is a source artifact, not a delivery artifact. Every user-facing byte should be a derivative with a content-addressed key, which makes it permanently cacheable (Topic 25) and makes re-processing everything a matter of changing a version segment in the key.

THE TRAP

An image endpoint that accepts arbitrary width and quality parameters. Symptom: a scraper requests 4,000 distinct widths, your worker fleet saturates, and the CDN hit ratio collapses because every URL is unique. Allow a fixed allow-list of sizes and reject the rest with 400.

```
media delivery — GAINS: 95–99% of bytes served from the edge, originals never exposed · COSTS: a derivative pipeline, signing endpoint, key discipline · ACCEPTS: signed URLs remain valid until expiry, even after access is revoked
```

81. Two stores, one truth: metadata rows and orphaned objects

The database holds the index; the bucket holds the bytes. They are separate systems with no shared transaction, which is Topic 59's dual-write problem in its most common form — and the reason every mature system has a reconciliation job.

```
FOUR STATES, only one of which is correct
  row + object        correct
  row, no object      BROKEN LINK -- user sees a 404 on their ticket
  object, no row      ORPHAN -- invisible, unbilled to any user,
                      grows forever, may contain personal data you
                      are legally required to delete
  row + wrong object  worst: a key collision or a client-chosen key

THE SEQUENCE THAT MINIMISES DAMAGE
  UPLOAD                          DELETE
  1. INSERT row status=pending    1. UPDATE row status=deleting
  2. PUT object                   2. DELETE object
  3. UPDATE row status=ready      3. DELETE row (or keep tombstone)
  Crash between 1 and 2 leaves a pending row: harmless, and a
  sweeper removes it after 24 h.
  Crash between 2 and 3 on delete leaves a row-less object: the
  reconciliation job finds it.
  ALWAYS create the row first and delete the row last. The failure
  then leans towards "a row with no bytes", which is visible and
  fixable, instead of "bytes nobody knows about".

RECONCILIATION JOB (weekly, and it is not optional)
  1. S3 Inventory delivers a daily manifest of every key (CSV/Parquet
     in a bucket) -- do NOT LIST 40 million objects yourself
  2. load it into a temp table, LEFT JOIN against your files table
  3. keys with no row and older than 48 h  -> orphan  -> delete
     rows with no key and status=ready     -> broken -> alert, restore
                                              from versioning (T82)
  Report both counts as a metric. A rising orphan count is a bug in
  a delete path; a rising broken-link count is a bug in an upload
  path, and users are seeing it.

GDPR/erasure: an orphan containing personal data is a compliance
problem, because you cannot prove it was deleted if you do not know
it exists.
```

THE KEY INSIGHT

You cannot make the two writes atomic, so choose which inconsistency you prefer and enforce it with ordering. Rows first on create, rows last on delete — the system then only fails in the direction you have a job for.

THE TRAP

Deleting the database row inside the same request that deletes the object, with no tombstone. Symptom: the object delete fails (throttled, permissions, network), the row is gone, and nothing in the system will ever reference those bytes again. You now pay for them forever and cannot prove what they were.

```
reconciliation — GAINS: bounded, measurable divergence between index and bytes · COSTS: a weekly job, an inventory feed, two metrics · ACCEPTS: the two stores are always briefly inconsistent by design
```

82. Versioning, soft delete, object lock and the restore you have never tested

Topic 75 established that durability does not protect you from yourself. These four features do, and each has a cost that is easy to underestimate.

```
VERSIONING
  every PUT creates a new version id; DELETE creates a delete marker
  and hides the object without removing it.
  $ aws s3api list-object-versions --bucket tickets --prefix 9f2b/
  $ aws s3api delete-object --version-id <marker>   # undelete
  COST: you now store every version of every object. A nightly job
  that re-uploads 4 M unchanged files creates 4 M versions per night.
  Non-negotiable lifecycle rule:
    NoncurrentVersionExpiration: NoncurrentDays: 30
    ExpiredObjectDeleteMarker: true

OBJECT LOCK (WORM)
  governance mode: privileged users can override
  compliance mode: NOBODY can delete before the retention date --
    not you, not the root account, not support. That is the point,
    and it is also the risk: a wrong 7-year retention on 400 TB is
    a 7-year bill you cannot cancel. Test in governance first.

REPLICATION
  same-region: another bucket, another account -> protects against
    a compromised credential in the primary account
  cross-region: protects against a region event, adds transfer cost
    and is asynchronous (typically seconds to 15 minutes)
  Replication does NOT protect against a bad write: it faithfully
  replicates your corruption. Versioning is the protection; that is
  why replication requires versioning to be enabled.

THE 3-2-1 CHECK, applied
  3 copies, 2 media/classes, 1 off-account or off-region.
  A bucket with versioning in one account is ONE copy with undo.

THE RESTORE DRILL -- the part everybody skips
  quarterly: pick 20 random objects, delete them, restore from
  versions, verify checksums against the database rows (T78).
  Measure: time to restore ONE object, and time to restore a whole
  prefix of 1 M objects. The second number is usually a shock --
  restoring 1 M objects at 1,000 requests/s is 17 minutes of API
  calls, and from Deep Archive it is 12-48 h before the first byte.
```

THE KEY INSIGHT

Versioning turns delete into a soft delete, which converts your worst incident class — a bad script deleting a prefix — from unrecoverable to a 30-minute recovery. It is the highest value-per-line configuration in this part, and it must be paired with a noncurrent-version expiry or it becomes your largest bill.

THE TRAP

Versioning enabled, expiry rule missing. Symptom: storage grows steadily while the object count in the console stays flat, because noncurrent versions and delete markers are not shown by default. Teams discover this at 4–10× the expected bill, often a year later.

```
protection — GAINS: undo for deletes and overwrites, WORM for compliance · COSTS: every version
stored and billed, replication transfer, drill time · ACCEPTS: compliance-mode locks cannot be
undone, and archive restores take hours
```

PRACTICE SET 7 — Topics 73–82

A. Conceptual

1. Which operations did object storage drop in order to scale, and which one do you most often have to reimplement?
2. Why is a "list the folder" endpoint over a bucket a design error?
3. Why does a timestamp-first key prefix cause `503 SlowDown`?
4. What exactly makes an object *visible* after a PUT?
5. State the storage overhead and failure tolerance of 3× replication versus 6+3 erasure coding.
6. Name three data-loss causes that 11 nines of durability does nothing about.
7. What problem did strong read-after-write consistency *not* solve?
8. Why is `HEAD`-then-`PUT` not a lock, and what is?
9. Give the two costs of proxying uploads through your application servers.
10. Why must the object key in a presigned URL be generated server-side?
11. Why is a multipart ETag not the file's MD5, and what should you store instead?
12. Which single lifecycle rule prevents an invisible, permanent bill?
13. Why can moving small objects to a colder class increase the total bill?
14. Why should derivatives expire rather than be archived?
15. Why is a public bucket with unguessable keys not access control?
16. In the upload and delete sequences, why create the row first and delete the row last?
17. What does a rising orphan count indicate, and what does a rising broken-link count indicate?
18. Why does cross-region replication not protect against a bad write?
19. Why is versioning without a noncurrent-version expiry rule dangerous?
20. Why is "time to restore one object" the wrong number to rehearse?

B. Pen-and-paper

1. 620 TB of PDFs, 3% read per month, average object 240 KB. Compute the monthly cost in Standard, in Standard-IA including retrieval, and in Deep Archive including retrieval. State which you would choose for the customer-facing copy and why.
2. Same corpus. Compute the number of objects, then the one-off cost of transitioning them to IA at $0.01 per 1,000 requests. How many months of savings does that transition take to repay?
3. 4.2 M thumbnails averaging 9 KB. Compute billed storage in Standard versus IA given IA's 128 KB per-object minimum. Which is cheaper, and by how much?
4. Uploads: 900 per minute, average 12 MB, over connections averaging 6 Mbit/s. Compute the concurrent worker count required if you proxy the bytes, and the memory at 8 MB of buffer per transfer.
5. A 3 GB video on a 120 Mbit link. Compute single-PUT wall time, then multipart with 32 MB parts and 8 parallel streams (assume the link is the only limit). How many parts, and what does one failed part cost in retransmission?
6. 6+3 erasure coding on a 40 MB object. Give the size of each shard, the total stored bytes, the overhead factor, and how many shard losses are survivable.

7. Media egress is 90 TB/month. Compare serving entirely from the bucket at $0.09/GB against a CDN at $0.08/GB with a 0.97 hit ratio and free origin-to-CDN transfer. What is the monthly difference?
8. Your bucket receives 5,200 PUT/s on a single prefix during a launch. Given per-prefix limits of ~3,500 PUT/s, what happens, and how many hash prefixes do you need to stay under the limit with 40% headroom?
9. A nightly sync re-uploads 4.1 M unchanged objects (average 300 KB) into a versioned bucket. Compute the storage added per night and per 30 days if noncurrent versions never expire.

C. Lab tasks (build → break → observe → fix)

1. **Build.** Run MinIO locally. Implement the presigned-upload flow end to end: pending row, signed PUT scoped to one key with a size limit, bucket event, worker marks the row ready. Time the application's involvement per upload and confirm it is milliseconds regardless of file size.
2. **Break: last writer wins.** Launch two processes that PUT different content to the same key with a 5 ms offset. Read it back and record which won and whether either received an error. Re-implement with `If-Match` on the ETag and show the loser receiving **412** and retrying against the current value.
3. **Break: the client-chosen key.** Sign an upload using the client's filename. From a second account, upload `avatar.jpg` and show it overwriting the first user's object. Fix by generating `{hash4}/{row_id}/{uuid}` server-side and prove the second upload lands elsewhere.
4. **Break: the orphan.** Delete 50 database rows without deleting their objects, and delete 50 objects without their rows. Then write the reconciliation job (inventory listing LEFT JOIN files) and verify it finds exactly 50 orphans and 50 broken links. Alert on both counts.
5. **Break: the interrupted multipart.** Start a large multipart upload and kill the client at ~60%. Confirm with `list-multipart-uploads` that parts are stored and billed while the object does not exist. Add the abort-incomplete lifecycle rule and re-verify.
6. **Break: the delete you cannot undo.** With versioning *disabled*, delete a prefix of 20 objects and attempt recovery. Then enable versioning, repeat, and restore by removing delete markers. Record the wall-clock time of both attempts.
7. **Fix and measure.** Put a CDN (or an nginx proxy cache simulating one) in front of the bucket. Measure origin request count and bytes for 1,000 requests to 20 distinct objects, before and after. Then add a client-supplied `?w=` parameter, request 200 random widths, and watch the hit ratio collapse — then enforce an allow-list.

ANSWER KEY 7 — Topics 73–82

A. Conceptual

1. It dropped **rename, partial write, locks and the directory tree** — all coordination points. The one you must rebuild is the **index**, which lives in your database (Topic 81).
2. Because `LIST` is a **paginated scan of a sorted keyspace at 1,000 keys per request** with no count shortcut — 40 s and a timeout for a large prefix.
3. The index is **range-partitioned and sorted**, so sequential keys concentrate every write on one partition until it splits, which takes **30–60 minutes**.
4. **The commit of the key→manifest mapping in the strongly consistent index.** Shards are durable before that; the object is not readable until the index commits.
5. 3× replication: **3.0× overhead, survives 2 losses**. 6+3 erasure coding: **1.5× overhead, survives 3 losses** — better on both axes, at the cost of slower degraded reads.
6. **Your own DELETE, a bad overwrite, and a compromised credential / ransomware** (also: region-level availability events, which are ~6 orders of magnitude more likely than durability loss).
7. **The lost-update problem.** Concurrent PUTs to one key are still last-writer-wins, silently, with a 200 OK to the loser.
8. They are two separate operations with a gap between them, so two workers can both see "absent". `PUT ... If-None-Match: *` makes creation itself conditional and atomic.
9. **A worker blocked for the whole transfer** (400 workers × 40 s for 50 MB uploads) and **bandwidth billed twice**, in and out.
10. Because the signature is a capability with no further checks. A client-chosen key permits **overwriting another user's object** or path traversal into keys you never intended to expose.
11. For multipart it is the **MD5 of the concatenated part MD5s, suffixed with the part count**. Store an explicit **SHA-256** in the database row instead.
12. `AbortIncompleteMultipartUpload` — without it, abandoned parts are stored and billed forever and never appear in the object count.
13. Colder classes bill a **128 KB minimum per object** and charge a **transition request per object**, so millions of small objects cost more after "optimisation".
14. A derivative is a **pure function of the original**: regenerating costs about one CPU second, while archiving costs storage for years and still cannot be served quickly.
15. Because **bucket and key enumeration is routine**, and an unguessable key is still a permanent, unrevocable capability once it leaks. It is obscurity, not authorisation.
16. So that failure always leans towards **a row with no bytes** — visible, alertable, fixable — rather than **bytes with no row**, which are invisible, billed forever, and a compliance risk.
17. Rising orphans = **a bug in a delete path**. Rising broken links = **a bug in an upload path, and users are already seeing 404s**.
18. Because it **faithfully replicates the corruption**. Only versioning provides undo — which is why replication requires versioning to be enabled.
19. Every PUT retains the previous version, so a nightly re-upload of unchanged files **multiplies storage indefinitely** while the console's object count stays flat.
20. Because incidents restore **prefixes, not objects**. One object is seconds; 1 M objects is **~17 minutes of API calls at 1,000 req/s**, and from Deep Archive **12–48 hours before the first byte**.

B. Worked

1. 620,000 GB. Standard: **$14,260/mo**. IA: 620,000×0.0125 = $7,750 + retrieval 18,600 GB×0.01 = $186 → **$7,936**. Deep Archive: $614 + 18,600×0.02 = $372 → **$986**, but **12–48 h to first byte**. Customer-facing copy: **Standard-IA** — 44% cheaper than Standard and still millisecond access.
2. 620 TB / 240 KB = **2.71 M objects** → transition cost 2,710×$0.01 = **$27.10** one-off. Savings are $14,260−$7,936 = $6,324/mo, so it repays in **about 2 minutes of a month** — transitions are trivial for large objects and only matter for small ones (see 3).
3. Standard: 4.2 M×9 KB = 37.8 GB → **$0.87/mo**. IA billed at 128 KB each: 4.2 M×128 KB = 537.6 GB → **$6.72/mo** plus transition requests. **Standard is 7.7× cheaper** — the classic cost-optimisation backfire.
4. Transfer time = 12 MB×8 / 6 Mbit = 16 s. Arrival 15/s → concurrency = 15×16 = **240 blocked workers**, and 240×8 MB = **1.9 GB** of buffers doing nothing but copying.
5. Single PUT: 3 GB×8/120 Mbit = **200 s**, and a failure at 95% restarts from zero. Multipart: **94 parts** of 32 MB; the link still caps throughput so wall time is **≈200 s**, but a failed part costs only **32 MB** of retransmission and the upload is resumable. (Parallelism helps when the bottleneck is per-stream latency, not link bandwidth — state which you have.)
6. Each shard = 40/6 = **6.67 MB**; 9 shards = **60 MB** stored; overhead **1.5×**; survives **3** shard losses.
7. Bucket only: 90,000 GB×$0.09 = **$8,100**. CDN: user egress 90,000×$0.08 = $7,200 + origin fetches 3%×90,000 = 2,700 GB at $0 (free to CDN) → **$7,200**. Difference **$900/mo**, plus a large latency win — and the gap widens as the hit ratio rises.
8. You get **503 SlowDown** until the partition splits (30–60 min). Need 5,200/(3,500×0.6) = 2.5 → **at least 3, use 16 hash prefixes** so headroom survives skew.
9. 4.1 M×300 KB = **1.23 TB per night**; ×30 = **36.9 TB of noncurrent versions** in a month ≈ **$849/mo** in Standard, growing forever. This is Topic 82's trap with numbers.

C. What to verify

1. Application time per upload should be **1–5 ms** and **constant in file size**. If it scales with size, bytes are still passing through your process.
2. Without preconditions: **one write wins, no error anywhere**. With `If-Match`: the loser gets **HTTP 412 Precondition Failed**; its retry must re-read the current ETag, not reuse the stale one.
3. Expect the second upload to **silently replace** the first user's object. After the fix, keys differ by UUID and both objects exist — confirm by listing the prefix.
4. The job must report exactly **50 orphans and 50 broken links**. Verify it uses the **inventory manifest**, not a live `LIST`; at 40 M keys the live listing is hours of requests.
5. `list-multipart-uploads` should show the upload with its parts, while `head-object` returns **404** — billed bytes for an object that does not exist. After the rule, the upload disappears within the configured days.
6. Without versioning: **unrecoverable**. With versioning: delete markers removed, objects back, and you should record **time-to-restore for the whole prefix**, not for one object.
7. Origin requests should fall to roughly **20 (one per distinct object)** out of 1,000. With 200 random widths, the hit ratio should collapse toward **0.1** and worker CPU should spike — the allow-list restores both.

HANDNOTE SUMMARY CARD — PART 7

```
================================================================================
 FILE AND OBJECT STORAGE -- ONE PAGE                              Topics 73-82
================================================================================
 CHOOSE
   block  offsets, one host, fixed size      -> DB files, VM disks   0.1-1 ms
   file   POSIX, shared, chatty metadata     -> legacy apps          1-10 ms
   object PUT/GET whole objects by key, HTTP -> uploads, media, logs 20-100 ms
   no rename (= copy+delete), no partial write, no locks, no directories
--------------------------------------------------------------------------------
 KEYS
   {hash4}/{row_id}/{uuid}-original.jpg      entropy FIRST (timestamp = 503)
   ~3,500 PUT/s and 5,500 GET/s per prefix; splits take 30-60 min
   key is immutable; never encode mutable state (a move is a full copy)
   store the key in the DB row -- the bucket is not an index (LIST is a scan)
--------------------------------------------------------------------------------
 DURABILITY vs PROTECTION
   6+3 erasure code = 1.5x overhead, survives 3 shard losses, >=3 AZs
   3x replication   = 3.0x overhead, survives 2
   11 nines = disks. It does NOT cover: your DELETE, a bad overwrite,
   stolen credentials, a region outage. Versioning + object lock cover those.
--------------------------------------------------------------------------------
 CONSISTENCY
   strong read-after-write + list-after-write since Dec 2020 (delete S3Guard)
   NOT atomic across objects; concurrent PUT = last writer wins, silently
   CAS:  PUT If-None-Match: *        create-only / lock / claim   -> 412
         PUT If-Match: ""      overwrite-if-unchanged       -> 412
   HEAD-then-PUT is a race. Preconditions are not.
--------------------------------------------------------------------------------
 UPLOAD
   NEVER proxy bytes: 900/min x 12 MB @ 6 Mbit = 240 blocked workers, 1.9 GB
   presign: validate -> INSERT pending row -> sign ONE key, 15 min, size cap,
            content-type -> browser PUTs -> bucket event -> queue -> mark ready
   key generated SERVER-side, always
   multipart: 5 MB..5 GB parts, max 10,000, Complete = the atomic commit
   multipart ETag != file MD5 -> store SHA-256 in the row
   lifecycle AbortIncompleteMultipartUpload: 7 days   (invisible bill)
--------------------------------------------------------------------------------
 COST ($/GB/mo, retrieval, min days)
   Standard .023 free -- | IA .0125 .01/GB 30 | 1Zone-IA .010 .01 30 (1 AZ)
   Glacier Instant .004 .03 90 | Flexible .0036 .01 90 (min..12h)
   Deep Archive .00099 .02 180 (12-48 h)   | EGRESS ~.09/GB = usually biggest
   IA/Glacier bill a 128 KB MINIMUM per object -> small files cost MORE
   derivatives EXPIRE (regenerable), originals tier down, archives leave the
   read path entirely
--------------------------------------------------------------------------------
 SERVING
   CDN in front (hit 0.95-0.99) -> origin sees 1-5%
   public: content-hash key + max-age=31536000, immutable
   private: short-lived signed URL (bearer capability, unrevocable until exp)
   fixed allow-list of sizes -- open ?w= parameters destroy the hit ratio
--------------------------------------------------------------------------------
 TWO STORES, ONE TRUTH
   create: row(pending) -> object -> row(ready)
   delete: row(deleting) -> object -> row gone       rows first, rows last
   weekly reconcile from INVENTORY manifest LEFT JOIN files:
     object, no row -> orphan (delete path bug, invisible, GDPR risk)
     row, no object -> broken link (upload path bug, users see 404)
--------------------------------------------------------------------------------
 TRAPS
   versioning without NoncurrentVersionExpiration -> 4-10x bill, flat obj count
   replication without versioning -> corruption replicated faithfully
   compliance-mode object lock -> nobody can delete, including you, for years
   restore drill measures ONE object -> real incident restores 1M (17 min+)
================================================================================
```

PART EIGHT

Observability and monitoring

Every previous part described something that fails silently: a stale cache entry, a lagging replica, a poison message, an orphaned object. This part is how you find out. It is also, for most teams, the third-largest infrastructure bill — so half of it is about what not to collect.

Part 8 · Observability and monitoring

83. Monitoring, observability and the four signals

Monitoring answers questions you wrote down in advance. Observability is whether you can answer a question you had not thought of, using data already collected, without shipping code. The difference shows up at 3am.

```
MONITORING     "CPU > 80% for 5 min"  -- a known failure mode,
               a threshold, an alert. Necessary, insufficient.
OBSERVABILITY  "which tenant, on which shard, using which endpoint,
               is responsible for the p99 that started at 09:14?"
               -- answerable only if the data carries enough
               dimensions, which is a design decision made earlier.

THE FOUR SIGNALS
  METRICS   numbers over time, pre-aggregated. Cheap, bounded,
            queryable in milliseconds. Cannot answer "which user".
            Cost model: per unique time series (Topic 84).
  LOGS      discrete events with full context. Expensive per GB,
            answers anything -- if you can find the line.
            Cost model: per GB ingested and per day retained.
  TRACES    one request's path across services, with timing per hop.
            Answers "where did the 800 ms go". Sampled.
            Cost model: per span retained.
  PROFILES  which function consumed the CPU or allocated the memory,
            continuously, in production. The fourth signal; the
            OpenTelemetry profiles signal reached public alpha in
            March 2026 (Topic 90).

HOW THEY COMPOSE -- the standard incident path
  1. an SLO burn-rate alert fires                        (Topic 93)
  2. a RED dashboard shows which endpoint            (Topic 91)
  3. an exemplar on the latency histogram jumps you to one slow
     trace                                           (Topic 87)
  4. the trace shows 780 ms in one span: a cache miss storm
  5. logs for that trace id give the key and the tenant
  6. a profile shows the CPU is in JSON serialisation
  Each signal hands off to the next by a shared identifier. That
  handoff is what you are buying; four disconnected tools are not
  observability.
```

```
METRIC  aggregate, always on   -> "something is wrong, roughly here"
TRACE   sampled, per request   -> "the 780 ms is in service C"
LOG     per event, searchable  -> "tenant 4821, key trip:v3:88"
PROFILE per process, always on -> "inside json_encode, 61% of CPU"
        joined by: trace_id, span_id, service.name, tenant.id
```

THE KEY INSIGHT

Metrics tell you *that,* traces tell you *where,* logs tell you *what,* profiles tell you *why*. Budget accordingly: metrics on everything, traces sampled, logs structured and sampled, profiles always-on but cheap.

THE TRAP

Buying three tools that do not share identifiers. Symptom: an incident spent copying timestamps between tabs, guessing which log line belongs to which trace. If your logs do not carry `trace_id`, you have three archives, not one observability system.

```
the four signals — GAINS: an incident path from symptom to root cause in minutes · COSTS:
instrumentation discipline and a real budget line · ACCEPTS: no signal alone is sufficient; the
joins are the product
```

84. Metric types, and the cardinality that decides your bill

A metric costs nothing per data point and everything per unique label combination. One badly chosen label can multiply your time-series count by ten thousand and take down the metrics backend — which is how a monitoring system becomes an outage.

```
TYPES
  counter    only increases (requests_total). Query with rate().
  gauge      goes up and down (queue_depth, memory_bytes).
  histogram  bucketed observations -> quantiles, averages, and the
             only correct way to measure latency.
  summary    client-side quantiles; CANNOT be aggregated across
             instances. Avoid unless you know exactly why.

CARDINALITY = product of all label value counts, per metric name
  http_requests_total{method, route, status, instance}
    7 methods x 40 routes x 12 statuses x 60 instances
    = 201,600 series. Fine. Prometheus handles millions.

  ADD ONE BAD LABEL:
    ...{user_id}      x 400,000 users  = 80 BILLION series
    ...{booking_id}   unbounded, grows forever
    ...{url}          /trip/9001 as a distinct value: unbounded
  Each series costs ~1-3 KB of RAM in the TSDB plus index. 1 M
  series ~ 2-4 GB. 80 billion is not a bigger server; it is a
  different architecture (or a mistake).

  RULE: a label value must come from a SMALL, BOUNDED SET known at
  code-review time. Route TEMPLATE (/trip/{id}), never the path.
  High-cardinality identity belongs in traces and logs, which are
  indexed differently and priced per event.

HISTOGRAMS
  classic: one series PER BUCKET.
    12 buckets x 40 routes x 60 instances = 28,800 series for ONE
    latency metric. Buckets are fixed at instrumentation time, so a
    p99 outside your top bucket is unmeasurable.
  native (Prometheus 3.x, stable in 3.8; OTLP exponential
  histograms): one series with dynamic, exponentially spaced
  buckets. ~10x fewer series, resolution wherever the data lands.
  Use them for new latency metrics.

  histogram_quantile(0.99,
    sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))
```

THE KEY INSIGHT

Never average a percentile and never average an average across instances. Sum the histogram buckets first, then compute the quantile — averaging p99 across 60 instances produces a number that corresponds to no real request and always understates the tail.

THE TRAP

A label containing a user id, a request id, an email or a raw URL. Symptom: the metrics backend OOMs an hour after a deploy, and its own dashboards are the first casualty. Catch it in review; recovery means dropping the series and losing the history.

```
metrics — GAINS: millisecond queries over months of history at a few KB per series · COSTS:
cardinality is a hard architectural limit · ACCEPTS: metrics can never answer "which user" — that
is what traces and logs are for
```

85. Prometheus: scrape, query, and what changed in 3.x

Prometheus pulls metrics over HTTP on a schedule, stores them locally, and evaluates rules. The pull model is the unusual part and the source of most of its operational properties.

```
# prometheus.yml
global:
  scrape_interval: 15s          # resolution AND cost multiplier
  evaluation_interval: 15s
scrape_configs:
  - job_name: api
    kubernetes_sd_configs: [{role: pod}]
    relabel_configs:            # drop noisy series BEFORE storage
      - source_labels: [__name__]
        regex: 'go_gc_duration_seconds.*'
        action: drop
rule_files: [/etc/prometheus/rules/*.yml]

WHAT 3.x CHANGED (and why it matters to a design)
  - native OTLP ingestion: an OTel SDK can push straight to
    Prometheus, no Collector required for the simple case
  - UTF-8 metric and label names: OTel's http.server.request.duration
    is now a legal name, so the translation layer disappears
  - Remote Write 2.0: native histograms, exemplars and metadata
  - native histograms stable (3.8), scrape must still be enabled
    explicitly

THE FOUR QUERIES YOU WILL WRITE MOST
  # request rate by route
  sum by (route) (rate(http_requests_total[5m]))
  # error ratio -- the SLI of Topic 92
  sum(rate(http_requests_total{status=~"5.."}[5m]))
    / sum(rate(http_requests_total[5m]))
  # p99 latency, aggregated correctly (Topic 84)
  histogram_quantile(0.99, sum by (le)
    (rate(http_request_duration_seconds_bucket[5m])))
  # saturation: queue depth per consumer (Topic 67)
  sum(kafka_consumergroup_lag) by (group)
    / count(up{job="workers"}) by (group)

RECORDING RULES -- precompute what dashboards ask for constantly
  - record: job:http_requests:rate5m
    expr: sum by (job) (rate(http_requests_total[5m]))
  A dashboard querying 30 days of raw series takes 20 s; the same
  panel on a recording rule takes 40 ms.

STORAGE MATH
  1.2 M series, 15 s scrape, ~1.7 B/sample compressed:
  1.2e6 x (86400/15) x 1.7 B = 11.7 GB/day = 351 GB/month.
  Halve the scrape interval, double the bill. 60 s for slow-moving
  infrastructure metrics is usually correct.
```

THE KEY INSIGHT

Pull means the monitoring system decides what exists and can tell you a target is *missing* (`up == 0`) — a push system only knows what arrived, so a dead instance looks identical to a quiet one. That single property is why `up == 0` is the most valuable alert in most clusters.

THE TRAP

Alerting on a raw `rate()` over a short window on a low-traffic service. Symptom: pages every night at 04:00 because two requests arrived and one failed, producing a 50% error rate. Require a minimum request volume in the alert expression, or alert on burn rate over multiple windows (Topic 93).

```
Prometheus — GAINS: cheap, local, fast queries; missing targets are detectable · COSTS:
cardinality limits, local retention, federation needed to scale · ACCEPTS: resolution is your
scrape interval — sub-15 s events are invisible
```

86. Logs: structure, volume and sampling

Logs are the only signal that can answer an arbitrary question, and the only one whose cost is linear in traffic. Both facts follow from the same property, and the engineering is entirely about keeping the second one bounded.

```
UNSTRUCTURED (unsearchable at scale)
  [2026-08-13 09:14:22] Booking failed for user 4821 on trip 88

STRUCTURED (one JSON object per event, indexed fields)
  {"ts":"2026-08-13T09:14:22.481Z","level":"error",
   "msg":"booking_failed","service":"booking-api",
   "trace_id":"4bf92f3577b34da6a3ce929d0e0e4736",
   "span_id":"00f067aa0ba902b7","tenant_id":"4821","trip_id":88,
   "reason":"seat_taken","attempt":2,"duration_ms":412}

  The two fields that make it observability rather than archaeology:
  trace_id (join to Topic 87) and a stable msg key you can count.

VOLUME MATH
  34,000 req/s x 3 log lines x 400 B = 40.8 MB/s
  = 3.5 TB/day = 105 TB/month.
  At $0.50-2.50 per GB ingested, that is $52,000-260,000 per month
  to log three lines per request. This is why teams sample.

WHAT TO DO INSTEAD
  1. one structured event per request at the edge (access log),
     plus events for state changes -- not per-function tracing
  2. sample the boring: keep 100% of errors and slow requests,
     1-5% of successful fast ones. Same policy as tail sampling
     (Topic 88), and the ratio should be a runtime config, not a
     deploy.
  3. move debug detail into span attributes (bounded, sampled)
  4. tiered retention: 7 days hot/searchable, 90 days in object
     storage (Topic 79) queried rarely and slowly, then delete
  5. never log secrets, tokens, full request bodies or PII --
     log storage is rarely as protected as your database, and it
     is replicated to three vendors

  1% sampling of successes on the numbers above: 3.5 TB/day becomes
  ~90 GB/day, and you still keep every error.
```

THE KEY INSIGHT

A log line without `trace_id` is an orphan. Emitting it costs the same as emitting a useful one, so make the trace context part of your logger setup once, globally, rather than a field people remember to add.

THE TRAP

Debug logging left enabled on a hot path after an incident. Symptom: the logging bill triples the following month, and the ingestion pipeline starts dropping events — including the errors you actually need. Log level should be runtime-configurable per service, with an automatic revert.

```
logs — GAINS: answer any question, with full context · COSTS: linear in traffic; 3.5 TB/day at
34k QPS unsampled · ACCEPTS: sampling means some individual requests are unexplainable
```

87. Distributed tracing: spans, context and exemplars

A trace is one request's journey as a tree of spans across every service it touched. It is the only signal that shows where time went in a system where no single machine sees the whole request.

```
TRACE 4bf92f35...  total 812 ms
 [api-gateway ................................ 812 ms]
   [booking-api .............................. 795 ms]
     [valkey GET trip:v3:88 .. 0.4 ms]  MISS
     [mysql SELECT trips ................ 24 ms]
     [payment-gw POST /charge ........................ 690 ms] <-- here
     [kafka produce booking.created .. 3 ms]
 Without a trace: five services each reporting "we were fast",
 and an 812 ms p99 nobody owns.

PROPAGATION -- W3C Trace Context, one header
  traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
               ^v  ^trace-id (16 B)                ^span-id  ^flags
                                                              01=sampled
  Every hop must forward it: HTTP clients, queue message headers
  (Topic 64 -- put traceparent in the Kafka record headers or SQS
  message attributes), cron jobs, webhooks. One service that drops
  it splits every trace in two and the tree is useless downstream.

SPAN CONTENT
  name (low cardinality: "GET /trip/{id}", never the concrete path)
  start/end, status, and attributes: db.system, http.status_code,
  messaging.destination, tenant.id, retry.count

EXEMPLARS -- the join from metrics to traces
  a histogram bucket carries a sample trace_id for an observation
  in that bucket. Click the spike on the p99 panel, land on a trace
  that WAS that spike. Prometheus supports exemplars via Remote
  Write 2.0 and OTLP; enable them, they are nearly free and they
  remove the hardest step of every investigation.

COST: a span is 300 B - 2 KB. 34,000 req/s x 8 spans unsampled
  = 272,000 spans/s = ~40 TB/day. Nobody keeps that: see Topic 88.
```

THE KEY INSIGHT

Tracing's value is in the *gaps*, not the spans: the 690 ms in which nothing was instrumented is the finding. Instrument boundaries first (inbound handler, outbound client, database, cache, broker) and stop — deep per-function spans multiply cost without changing conclusions.

THE TRAP

Context lost across an async boundary. Symptom: producer traces end at "message sent" and consumer traces start with no parent, so the queue looks instantaneous and the two-hour backlog of Topic 67 is invisible in tracing. Propagate `traceparent` in message headers and link the consumer span to the producer.

```
tracing — GAINS: per-request latency attribution across services; exemplars jump from metric to cause · COSTS: 300 B–2 KB per span, propagation discipline everywhere · ACCEPTS: sampled, so a specific request may not be there
```

88. Sampling: head, tail, and what you lose either way

You cannot keep every trace, so you choose which to keep and when to decide. That timing is the entire trade-off.

```
HEAD SAMPLING -- decide at the root, before the request runs
  "keep 2% of traces", propagated via the sampled flag in
  traceparent so every service agrees.
  + trivially cheap: unsampled requests cost nothing anywhere
  + decision is consistent across services
  - you keep 2% of errors too. A 1-in-50,000 failure is invisible.

TAIL SAMPLING -- decide after the trace completes, in the Collector
  processors:
    tail_sampling:
      decision_wait: 10s
      num_traces: 100000
      policies:
        - name: errors        { type: status_code,
                                status_code: {status_codes: [ERROR]} }
        - name: slow          { type: latency,
                                latency: {threshold_ms: 1000} }
        - name: baseline      { type: probabilistic,
                                probabilistic: {sampling_percentage: 2} }
  + keep 100% of errors and slow requests, 2% of the boring ones
  - the Collector must BUFFER every span for decision_wait seconds
    until the trace is complete: memory = span rate x 10 s x size
    272,000 spans/s x 10 s x 800 B = 2.2 GB of buffer, and every
    span for one trace must reach the SAME collector instance
    (load-balancing exporter, keyed by trace id)

WHAT SAMPLING BREAKS -- and the fix
  metrics derived from sampled traces are wrong by construction.
  Never compute request rate or error rate from traces. Compute
  them from metrics (always-on, aggregated) and use traces for
  explanation only. Some pipelines generate span metrics BEFORE
  sampling, which is the correct order.

RULE OF THUMB
  < 1,000 req/s: keep everything, it is affordable
  1k-50k req/s: tail sampling, 100% errors + 1-5% baseline
  > 50k req/s: tail sampling plus per-route rates, and generate
               span metrics pre-sampling
```

THE KEY INSIGHT

Head sampling optimises cost; tail sampling optimises usefulness. Since the traces you want are precisely the anomalous ones, and anomalies are rare, uniform head sampling throws away almost exactly the data you will need.

THE TRAP

Tail sampling with collectors behind a normal round-robin load balancer. Symptom: traces are permanently incomplete — three spans here, five there — because the decision was made on a partial trace. Tail sampling requires trace-id-aware routing to a single collector instance.

```
sampling — GAINS: 98% cost reduction while keeping every error · COSTS: buffering memory, trace-aware routing, a stateful collector tier · ACCEPTS: rates and counts must never be derived from sampled data
```

89. OpenTelemetry: one SDK, one protocol, one collector

OTel is the vendor-neutral standard for producing and shipping all four signals. Its practical value is that instrumentation is written once and the backend becomes a configuration line rather than a rewrite.

```
THE THREE PIECES
  SDK           in your process: creates spans, metrics, logs,
                propagates context. Stable across every major
                language for traces, metrics and logs.
  OTLP          the wire protocol (gRPC or HTTP), one format for
                all signals.
  COLLECTOR     a separate process (DaemonSet or gateway) that
                receives, processes and exports. This is where the
                leverage is.

# otel-collector.yaml
receivers:
  otlp: { protocols: { grpc: {}, http: {} } }
  prometheus: { config: { scrape_configs: [...] } }
processors:
  memory_limiter: { check_interval: 1s, limit_mib: 4096 }
  k8sattributes: {}            # add pod, node, namespace
  attributes:                  # redact before it leaves the cluster
    actions:
      - { key: http.request.header.authorization, action: delete }
      - { key: user.email, action: hash }
  tail_sampling: { ... }       # Topic 88
  batch: { timeout: 5s, send_batch_size: 8192 }
exporters:
  otlphttp/vendor: { endpoint: "https://ingest.example.com" }
  prometheusremotewrite: { endpoint: "http://prom:9090/api/v1/write" }
service:
  pipelines:
    traces:  { receivers: [otlp], processors: [memory_limiter,
               k8sattributes, attributes, tail_sampling, batch],
               exporters: [otlphttp/vendor] }
    metrics: { receivers: [otlp, prometheus],
               processors: [memory_limiter, batch],
               exporters: [prometheusremotewrite] }

SEMANTIC CONVENTIONS are the other half of the value: http.route,
db.system, messaging.destination.name, service.name. Standard names
mean a dashboard or an alert works against any service without
custom mapping -- and they are why Prometheus 3.x added UTF-8 names
(Topic 85).

WHAT THE COLLECTOR BUYS YOU
  change vendors without redeploying services
  redact PII in one place, provably
  sample, batch and compress before egress ($ per GB, Topic 5)
  add infrastructure attributes the app cannot know
```

THE KEY INSIGHT

The Collector is the seam that makes observability replaceable. Instrument with OTel, export to whatever you can afford, and switch backends by editing one exporter block — instead of re-instrumenting 40 services, which is why teams stay on tools they have outgrown.

THE TRAP

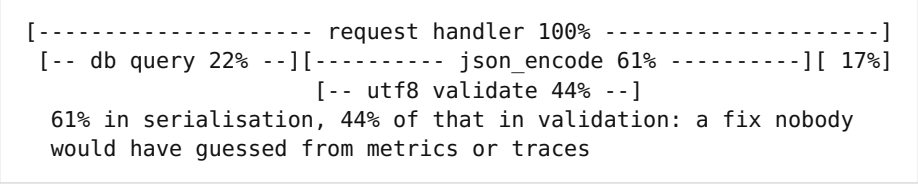
A Collector with no `memory_limiter` and no backpressure handling. Symptom: the vendor endpoint slows down, the Collector queues, exhausts memory, is OOM-killed, and you lose telemetry exactly during the incident that produced it. Always set the limiter, a bounded sending queue, and alert on `otelcol_exporter_send_failed_spans`.

```
OpenTelemetry — GAINS: instrument once, portable across backends, redact and sample centrally · COSTS: a collector tier to run and monitor · ACCEPTS: the collector is now in your telemetry path and can fail
```

90. Continuous profiling and eBPF

Metrics say the service is slow; traces say which span; profiles say which function. Continuous profiling runs a sampling profiler in production permanently, at a cost low enough to leave on.

```
WHAT IT IS
  every ~10 ms, sample the stack of running threads. Aggregate into
  a flame graph over time, tagged by service, version and endpoint.
  Overhead: 0.5-2% CPU. Storage: a few MB/host/day compressed.

WHAT IT ANSWERS THAT NOTHING ELSE DOES
  "p99 rose 40 ms after the deploy" -> diff the flame graph of
  v482 against v481: 38 ms of new time in regex compilation inside
  a validation helper called per row.
  "memory grows 400 MB/day" -> allocation profile names the
  allocation site, not just the total.

eBPF -- profiling without touching the application
  a small verified program runs in the kernel, samples stacks for
  every process on the host, and unwinds them for compiled and
  interpreted runtimes alike. No agent inside your process, no
  restart, no code change. The whole-system eBPF profiler donated
  by Elastic now ships as an official OpenTelemetry Collector
  component, with runtime support for Go, Node.js/V8, Ruby, .NET
  and early BEAM.

STATUS, and why it matters for planning
  the OTLP profiles signal reached PUBLIC ALPHA in March 2026 and
  is on an RC/GA track; the wire format round-trips with pprof and
  is ~40% smaller through a shared string dictionary. Treat it as
  production-viable via vendor agents, and as not-yet-stable if you
  are building tooling directly on the OTLP profile format.

READING A FLAME GRAPH IN ONE LINE
  x-axis is proportion of samples, NOT time order. Width = cost.
  Look for the widest box you did not expect, then look at its
  parent to learn who called it.
```



THE KEY INSIGHT

Profiling is the only signal that points at a line of code. Combined with version tags it turns "the deploy made it slower" from an argument into a diff, which is why continuous profiling belongs in the deploy-verification path (Topic 109), not only in incidents.

THE TRAP

Profiling only when there is a problem. Symptom: you capture a flame graph during an incident and have nothing to compare it against, so 61% in serialisation looks alarming when it has always been 58%. The value is in the baseline; it must be continuous to exist.

```
profiling — GAINS: function-level attribution at 0.5–2% CPU, diffable across versions · COSTS:
symbolisation setup, a few MB/host/day · ACCEPTS: sampled stacks, so very short-lived hot spots
can be missed
```

91. RED and USE: two dashboards that cover almost everything

Two methods, one for the things that serve requests and one for the resources they consume. Between them they produce dashboards that are actually read during an incident, instead of the 60-panel wall nobody understands.

```
RED -- for every service and endpoint (request-driven)
  Rate      sum by (route) (rate(http_requests_total[5m]))
  Errors    sum by (route) (rate(http_requests_total{status=~"5.."}[5m]))
            / sum by (route) (rate(http_requests_total[5m]))
  Duration  histogram_quantile(0.99, sum by (le, route)
              (rate(http_request_duration_seconds_bucket[5m])))

USE -- for every resource (CPU, disk, pool, queue, cache)
  Utilisation  % of time busy      node_cpu, pool_active/pool_size
  Saturation   queued work         run queue, pool_wait_count,
                                   kafka lag, cache evicted_keys
  Errors       failed operations   disk errors, conn refused,
                                   OOM kills

  SATURATION IS THE LEADING INDICATOR. Utilisation at 100% with no
  saturation is a fully used resource, which is fine. Utilisation
  at 70% with a growing queue is a system already failing.

THE DASHBOARD THAT WORKS (one screen, in this order)
  1. SLO burn rate + error budget remaining          (Topic 92)
  2. RED for the top 5 endpoints
  3. saturation of the four resources that fail first for you:
     DB pool wait, cache evictions, queue oldest-message age,
     worker in-flight
  4. dependency health: replica lag, DLQ depth, outbox depth
  5. deploy markers overlaid on all of it
  Everything else belongs in a drill-down, not the first screen.

WHY DEPLOY MARKERS: about half of production incidents begin within
30 minutes of a change. A vertical line on every graph answers the
first question of every incident before anyone asks it.
```

THE KEY INSIGHT

Alert on symptoms (RED, SLO), investigate with causes (USE). A CPU alert pages you for something that may not matter; an error-rate alert pages you only when users are affected, and the CPU panel is one click away when you need it (Topic 94).

THE TRAP

Dashboards built per-tool rather than per-service. Symptom: during an incident, five people open five dashboards and nobody can say whether users are affected. One page per service, owned by that service's team, with the SLO at the top.

```
RED/USE — GAINS: complete coverage from ~8 panels per service · COSTS: consistent metric naming
across services (OTel semconv, Topic 89) · ACCEPTS: utilisation alone is a poor signal;
saturation is the one to watch
```

92. SLIs, SLOs and the error budget

An SLO converts "the site should be up" into a number you can decide with. Its real function is not measurement — it is arbitration between shipping features and improving reliability, using arithmetic instead of opinion.

```
SLI  a measurement of user-visible behaviour, as a ratio:
       good events / valid events
     availability: non-5xx responses / all responses
     latency:      requests faster than 300 ms / all requests
     quality:      searches returning full results / all searches
     freshness:    read models updated within 5 s / all updates

SLO  a target for the SLI over a window:
       99.9% of requests succeed, measured over 30 days

ERROR BUDGET = 1 - SLO, expressed in absolute terms:
  99.9% over 30 days  = 43.2 minutes of full downtime
  99.95%              = 21.6 minutes
  99.99%              =  4.3 minutes   (needs multi-region, T111)
  At 34,000 QPS, 99.9% = 88.1 M requests, budget = 88,128 failures
  per 30 days. A 4-minute total outage costs 8.2 M requests: your
  entire month's budget spent 93 times over -- which is why the
  right conversation is about MINUTES, not percentages.

  A 20-minute partial outage affecting 15% of traffic costs
  0.15 x 20 = 3 minutes of budget. Budgets make partial incidents
  comparable to total ones.

THE POLICY IS THE POINT
  budget remaining > 50%   ship freely, take risks
  budget < 25%             freeze risky changes, fix reliability
  budget exhausted         feature freeze until it recovers
  Agreed in advance, in writing, with the people who own the
  roadmap. Without that agreement an SLO is just another graph.

CHOOSING THE TARGET: measure your current SLI for a month first.
If you are at 99.5%, an SLO of 99.99% is a fantasy that will be
ignored within two weeks. Set it just above where you are, then
tighten. And never set it higher than your dependencies allow:
serial dependencies multiply (Topic 6).
```

THE KEY INSIGHT

100% is the wrong target for everything. It forbids all deploys, all experiments and all dependency changes, and it costs unbounded money to approach. The error budget makes unreliability a resource you spend deliberately rather than a failure you apologise for.

THE TRAP

An SLI measured server-side only. Symptom: your dashboard shows 99.98% while users report a broken site — because the failure is in DNS, the CDN, TLS or the client bundle, none of which appears in your access log. Pair server-side SLIs with synthetic probes and real-user monitoring (Topic 95).

```
SLOs — GAINS: a shared, numeric basis for reliability-versus-features decisions · COSTS:
instrumentation of valid/good events, and a policy people honour · ACCEPTS: you are explicitly
choosing to fail some requests
```

93. Burn-rate alerting on multiple windows

Alerting directly on the SLO is either far too slow or far too noisy. Burn rate — how fast you are consuming the error budget relative to the sustainable pace — fixes both, and it is the standard modern approach.

```
BURN RATE = observed error ratio / (1 - SLO)
  SLO 99.9% -> sustainable error ratio 0.001 = burn rate 1.0
  burn rate 14.4 means: at this pace the 30-day budget is gone in
  30/14.4 = 2.08 days -- and 1 hour at 14.4 spends 2% of the budget.

THE STANDARD MULTI-WINDOW PAIRS (SRE workbook)
  PAGE   burn >= 14.4 over 1 h  AND  >= 14.4 over 5 m
         -> 2% of budget consumed, detected in ~5 minutes
  PAGE   burn >= 6    over 6 h  AND  >= 6    over 30 m
         -> 5% consumed, slower burn still worth waking someone
  TICKET burn >= 3    over 24 h AND  >= 3    over 2 h
  TICKET burn >= 1    over 72 h AND  >= 1    over 6 h

  The SHORT window is what stops a resolved incident from paging
  for another hour; the LONG window is what stops a 30-second blip
  from paging at all. Both must be true.

- alert: HighErrorBudgetBurn
  expr: |
    (
      sum(rate(http_requests_total{status=~"5.."}[1h]))
        / sum(rate(http_requests_total[1h])) > 14.4 * 0.001
    )
    and
    (
      sum(rate(http_requests_total{status=~"5.."}[5m]))
        / sum(rate(http_requests_total[5m])) > 14.4 * 0.001
    )
  for: 2m
  labels: { severity: page }
  annotations:
    summary: "Burning error budget 14.4x -- 2% consumed in 1h"
    runbook: "https://runbooks/internal/booking-api-5xx"
    dashboard: "https://grafana/d/booking-red"

WHY NOT A SIMPLE THRESHOLD
  "error rate > 1% for 5 min" pages at 3am for a 6-minute blip that
  costs 0.02% of the budget, and stays silent through a 0.4% error
  rate that burns the entire month in nine days.
```

THE KEY INSIGHT

Burn rate makes alert urgency proportional to user harm, in one number, for every service. A fast burn wakes someone; a slow burn files a ticket; a blip does nothing. That is the whole design, and it replaces dozens of hand-tuned thresholds.

THE TRAP

Burn-rate alerts on a low-traffic service. Symptom: a 3am page because 1 of 4 overnight requests failed — a 25% error ratio and a burn rate of 250. Add a minimum-volume condition, or aggregate low-traffic endpoints into one SLO with enough events to be statistically meaningful.

```
burn-rate alerting — GAINS: urgency proportional to budget consumed, few false pages · COSTS: an SLI per service and four rules per SLO · ACCEPTS: slow burns are deliberately not paged — they become tickets
```

94. Alerts, runbooks and the on-call human

Every alert costs a human interruption whether or not it was justified. Treat the alert list as a budget: each entry must be actionable, urgent and about a symptom users feel.

```
THE THREE TESTS -- an alert that fails any of them should be a
dashboard panel or a ticket, not a page:
  1. is a user affected right now (or imminently)?
  2. is there something a human can do about it in ten minutes?
  3. would you want to be woken at 3am for it?

SYMPTOM, NOT CAUSE
  page:      error budget burn, latency SLO, queue oldest-age,
             checkout failure rate, up == 0 on a whole service
  ticket:    disk 70% full, one node unhealthy, certificate 20 days
             from expiry, DLQ depth > 0, orphan count rising
  never:     CPU 80%, memory 75%, a single pod restart, a retry
             that succeeded

RUNBOOK -- linked from the alert, not "somewhere in the wiki"
  1. what this alert means, in one sentence
  2. user impact, and how to confirm it (which dashboard, which
     synthetic check)
  3. the three most common causes, most recent first
  4. exact commands for diagnosis (with the actual flags)
  5. mitigation BEFORE root cause: roll back, shed load, scale
     consumers, disable the feature flag
  6. escalation: who, and after how long

FATIGUE, measured
  > 2 pages per on-call shift is unsustainable; the response
  degrades to acknowledging without reading.
  Track: pages per shift, % actioned, % auto-resolved, MTTA.
  A page with a > 60% auto-resolve rate is noise -- delete it or
  raise its threshold. Review the alert list monthly and delete
  aggressively; alerts accumulate, and nobody is ever assigned to
  remove them.

MITIGATE FIRST. Root cause is for the postmortem. Roll back, then
investigate -- the incident ends when users are fine, not when you
understand it.
```

THE KEY INSIGHT

An alert without a runbook link is an alert that will be handled by whoever has the most context, which is one person, who will eventually leave. The runbook is what converts an incident from a personality into a procedure.

THE TRAP

Alerting on every cause you can think of "to be safe." Symptom: 40 alerts fire during one incident, the real signal is buried on page three of the channel, and the responder mutes the whole service. One symptom alert plus good dashboards beats forty cause alerts, every time.

```
alerting — GAINS: fewer, better pages and faster mitigation · COSTS: monthly pruning, runbook
maintenance · ACCEPTS: some causes are deliberately not paged — they surface as tickets
```

95. Outside-in checks, and controlling the bill

Two loose ends: server-side signals cannot see failures that happen before your server, and observability grows to consume whatever budget it is given.

```
OUTSIDE-IN
  HEALTH CHECKS (Topic 15)
    /live   is the process alive? Never check dependencies here --
            a database blip must not restart every pod.
    /ready  can it serve? Check pool, cache, migrations. Failing
            ready removes it from the LB without killing it.
  SYNTHETIC PROBES
    a scripted booking flow from 5 regions every 60 s. Catches DNS,
    TLS expiry, CDN misconfiguration, a broken JS bundle and BGP
    problems -- every one invisible to your access log.
    Cost: trivial. Value: this is what tells you the site is down
    when your own dashboards are green.
  REAL USER MONITORING (RUM)
    actual client timings: DNS, TCP, TLS, TTFB, LCP, INP, plus JS
    errors, by geography and device. This is the only measurement
    of what users experience; server p99 is a component of it, not
    a proxy for it.

BILL CONTROL -- the four levers, in order of effect
  1. cardinality review before merge: no unbounded label (Topic 84)
  2. sample logs and traces: 100% of errors, 1-5% of successes
     (Topics 86, 88)
  3. retention tiers: 7 d hot, 30-90 d in object storage, then gone.
     Ask what you have actually queried older than 14 days -- for
     most teams the honest answer is "nothing except compliance."
  4. aggregate at the edge: recording rules and collector-side
     aggregation, so the vendor is billed for series, not samples

  BUDGET SANITY CHECK
  observability should be roughly 5-15% of the infrastructure it
  observes. At 30%+, you are storing debug logs at full fidelity or
  paying per-host for something you could scrape.
```

THE KEY INSIGHT

The most valuable single check in most systems is a synthetic transaction that exercises the real user path end to end. It is the only signal whose failure means "users cannot use the product" without inference — and it costs less than one dashboard.

THE TRAP

A liveness probe that checks the database. Symptom: a 30-second database blip causes Kubernetes to restart every pod simultaneously, turning a brief degradation into a full outage with cold caches (Topic 43) and a thundering herd on recovery. Liveness means "this process is not wedged," nothing more.

```
outside-in and cost — GAINS: catch failures your servers cannot see; keep telemetry at 5–15% of
infrastructure spend · COSTS: probe maintenance, sampling policy, retention rules · ACCEPTS:
sampled and aggregated data cannot answer every retrospective question
```

PRACTICE SET 8 — Topics 83–95

A. Conceptual

1. State the difference between monitoring and observability in terms of when the question was formulated.
2. Which signal answers "which tenant", and why can metrics never do it?
3. What single field joins logs to traces, and where should it be set?
4. Why is averaging p99 across 60 instances wrong, and what is the correct aggregation?
5. Give three label values that must never appear on a metric, and the shared property that disqualifies them.
6. What do native histograms change about bucket configuration and series count?
7. What can a pull-based collector detect that a push-based one structurally cannot?
8. Why does a low-traffic service page at 04:00 under a naive error-rate rule?
9. Why is head sampling the wrong default for a system with rare errors?
10. What must be true about collector routing for tail sampling to work?
11. Why must request rate and error rate never be computed from sampled traces?
12. What does the OTel Collector let you change without redeploying services? Name three things.
13. What does a profile tell you that a trace cannot?
14. Why is a flame graph captured only during an incident of limited use?
15. Under USE, why is saturation a better leading indicator than utilisation?
16. Write the SLI formula shape, and give a freshness SLI for a CQRS read model.
17. Convert 99.95% over 30 days into minutes, and explain why minutes are the better unit for a conversation with product.
18. Why do burn-rate alerts use two windows rather than one?
19. Give the three tests an alert must pass to be a page.
20. Why must a liveness probe not check the database?

B. Pen-and-paper

1. A metric has labels: 9 methods, 55 route templates, 14 statuses, 80 instances. Compute the series count. Now add `tenant_id` with 12,000 values and recompute. At 2.5 KB per series, what RAM does each version need?
2. Classic histogram with 14 buckets, 55 routes, 80 instances: how many series? If native histograms reduce this by roughly 10×, what do you save in RAM at 2.5 KB/series?
3. 1.6 M series scraped every 15 s at 1.7 B/sample compressed. Compute daily and monthly storage. Recompute at a 60 s interval and state what resolution you lose.
4. 52,000 req/s, 4 log lines per request, 380 B each. Compute MB/s, TB/day and monthly cost at $1.20/GB ingested. Then apply "100% of the 0.4% errors, 2% of successes" and recompute.
5. Same traffic, 9 spans per request at 700 B. Compute unsampled spans/s and TB/day. For tail sampling with a 10 s decision wait, compute the collector buffer memory required.
6. SLO is 99.9% over 30 days at 52,000 QPS. Compute total requests, the error budget in requests and in minutes. A 7-minute outage affecting 40% of traffic consumes what fraction of the budget?

7. Observed error ratio is 0.6% against a 99.9% SLO. Compute burn rate, days to exhaust the budget, and which of the four standard alert pairs fire.
8. A service depends serially on five components each at 99.95%. Compute the best achievable availability and say whether a 99.9% SLO is feasible.
9. Your infrastructure costs $46,000/month and observability costs $19,000. Compute the ratio, compare it to the 5–15% guideline, and list the two levers from Topic 95 that would move it most, with estimated savings.

C. Lab tasks (build → break → observe → fix)

1. **Build.** Instrument one service with OTel: RED metrics, spans across an HTTP call and a database call, and structured logs carrying `trace_id`. Run Prometheus plus a Collector plus any trace backend. Verify you can go from a latency spike to the causing trace to its log lines.
2. **Break: cardinality.** Add `user_id` as a label on a counter and drive 50,000 distinct users through it. Record `prometheus_tsdb_head_series`, the RSS of Prometheus, and query latency before and after. Then drop the label with a `metric_relabel_config` and record how long the series take to be released.
3. **Break: the broken trace.** Remove `traceparent` propagation from one hop (or drop it in a queue producer). Observe the trace tree splitting into two rootless fragments, and confirm the queue delay becomes invisible. Restore propagation via message headers and show the linked span.
4. **Break: head sampling.** Set head sampling to 2% and inject a 1-in-2,000 error. Try to find the failing trace. Switch to tail sampling with an errors policy and confirm you now capture 100% of them; record the collector's memory change.
5. **Break: the liveness probe.** Point `/live` at a database check, then stop the database for 30 s. Record how many pods restart and the recovery time including cold caches. Move the check to `/ready` and repeat — pods should be removed from the load balancer and return without restarting.
6. **Break: the noisy alert.** Configure a naive "error rate > 1% for 5 min" rule and replay a 6-minute blip plus a sustained 0.4% error rate. Record which one paged. Replace with the four burn-rate pairs and replay both again.
7. **Break: the collector.** Point the exporter at an endpoint you can throttle to 1 req/s. Watch queue growth, then memory, then the OOM kill, and confirm telemetry loss during the incident. Add `memory_limiter`, a bounded sending queue and an alert on send failures, then repeat.

ANSWER KEY 8 — Topics 83–95

A. Conceptual

1. Monitoring answers questions **written down in advance**; observability answers questions **first asked during the incident**, from data already collected, without shipping code.
2. **Logs and traces.** Metrics are pre-aggregated by label set, and a per-tenant label would make cardinality unbounded (Topic 84).
3. `trace_id`, set **once in the logger configuration** so it is automatic rather than remembered per call site.
4. A percentile is not linear, so averaging them yields a value matching no real request and understating the tail. Correct: `histogram_quantile(0.99, sum by (le) (rate(..._bucket[5m])))` — sum buckets first, quantile last.
5. **user id, request/booking id, raw URL or email** — all **unbounded or high-cardinality sets not known at review time**.
6. Buckets become **dynamic and exponentially spaced** rather than fixed at instrumentation time, and the metric becomes **one series instead of one per bucket** (~10× fewer).
7. That a target is **missing** (`up == 0`). A push system cannot distinguish a dead instance from a quiet one.
8. Because the ratio has a tiny denominator: **1 failure out of 2 requests is a 50% error rate**. Require a minimum volume, or use multi-window burn rate.
9. Because it keeps a uniform 2% **of errors too**, and the traces worth having are exactly the rare anomalous ones.
10. All spans of a trace must reach **the same collector instance** — trace-id-aware routing (a load-balancing exporter), not round robin.
11. Because the sample is biased by construction (errors over-represented under tail sampling), so counts derived from it are wrong. Rates come from **always-on metrics**, or from span metrics generated **before** sampling.
12. **The backend/vendor, the sampling policy, and PII redaction** (also: batching, compression and added infrastructure attributes).
13. **Which function** consumed the CPU or allocated memory — a trace stops at the span boundary and shows only that 690 ms elapsed inside.
14. Because there is **no baseline to diff against**: 61% in serialisation is only meaningful next to last week's 58%.
15. Utilisation at 100% with no queue is simply a fully used resource; **saturation means work is already waiting**, which is the state that turns into user-visible latency.
16. **good events / valid events.** Freshness: **read-model updates applied within 5 s / all updates** (Topic 55).
17. 0.0005×30×24×60 = **21.6 minutes**. Minutes are concrete and comparable to a real incident; "99.95%" hides that two bad deploys spend the month.
18. The **long window** prevents a brief blip from paging; the **short window** stops the page from persisting after the incident is over. Both must be true simultaneously.
19. **Is a user affected now; can a human act within ten minutes; would you want to be woken for it.**
20. Because a dependency blip would **restart every pod at once**, converting a degradation into an outage with cold caches (Topic 43). Liveness means only "this process is not wedged".

B. Worked

1. 9×55×14×80 = **554,400 series ≈ 1.39 GB**. With tenant_id: ×12,000 = **6.65 billion series ≈ 16.6 TB of RAM** — not a bigger server, a different architecture.
2. 14×55×80 = **61,600 series ≈ 154 MB**. Native: **≈6,160 series ≈ 15 MB**, saving **≈139 MB** for one latency metric — multiply by every service.
3. 1.6e6×5,760×1.7 B = **15.7 GB/day = 470 GB/month**. At 60 s: **3.9 GB/day = 118 GB/month**, losing the ability to see events shorter than a minute — acceptable for infrastructure metrics, not for request latency.
4. 52,000×4×380 B = **79 MB/s = 6.8 TB/day** ≈ 205 TB/month → **$246,000/month**. Sampled: 0.4%+2%×99.6% = 2.39% kept → **163 GB/day ≈ $5,900/month**, a **42×** reduction with every error retained.
5. 52,000×9 = **468,000 spans/s**; ×700 B = 328 MB/s = **28.3 TB/day**. Buffer = 468,000×10×700 B = **3.3 GB** across the collector tier, before overhead — size the collectors for it or decisions will be made on truncated traces.
6. 52,000×2,592,000 = **134.8 billion requests**; budget 0.1% = **134.8 M failed requests = 43.2 minutes**. A 7-minute outage at 40% impact = 2.8 budget-minutes = **6.5% of the budget**.
7. Burn = 0.006/0.001 = **6.0** → budget gone in 30/6 = **5 days**. Fires the **6 h/30 m page pair** (and subsequently the 3× and 1× ticket pairs); it does **not** reach the 14.4 fast-burn page.
8. 0.9995^5 = **99.75%**. A 99.9% SLO is **infeasible** — the dependencies alone spend more than double the budget (Topic 6). Either reduce serial dependencies, add redundancy, or set the SLO at 99.7%.
9. 19/46 = **41%**, roughly **3× the top of the guideline**. Biggest levers: **log sampling** (a 40× reduction on the dominant line, plausibly $10–12k/month) and **retention tiering** to object storage after 7 days (Topic 79), typically another 20–30% of what remains.

C. What to verify

1. The click-through must work: **exemplar → trace → logs filtered by trace_id**. If logs cannot be filtered by trace id, the instrumentation is incomplete regardless of how good the dashboards look.
2. Expect `prometheus_tsdb_head_series` to rise by **~50,000 per metric** and RSS by **100–150 MB**, with query latency degrading noticeably. After relabelling, series are released only **after the retention of the head block** (typically 2–3 hours) — recovery is not immediate.
3. You should see **two disconnected traces**, neither showing the queue wait. After the fix, the consumer span links to the producer and the **broker delay appears as real time**.
4. At 2% head sampling, the chance of catching a 1-in-2,000 error in an hour is low, and you will likely find **zero**. With tail sampling: **100% of errors**, and collector memory up by roughly **span rate × decision_wait × span size**.
5. Expect **every pod to restart** and recovery to exceed the outage itself because caches are cold. With the check on `/ready`, pods are **removed from the LB and restored without restarting**, and recovery is roughly as long as the database blip.
6. The naive rule **pages on the blip and stays silent on the sustained 0.4%**. Burn-rate pairs invert this correctly: **no page for the blip, a page (6× pair) for the sustained burn**.
7. Without limits: queue grows, RSS climbs, **OOM kill, and a gap in telemetry precisely during the incident**. With `memory_limiter`: the collector **refuses and drops deliberately**,

`otelcol_exporter_send_failed_spans` rises, and the process survives — a visible, bounded loss instead of a blind one.

HANDNOTE SUMMARY CARD — PART 8

```
================================================================================
 OBSERVABILITY -- ONE PAGE                                        Topics 83-95
================================================================================
 FOUR SIGNALS (joined by trace_id, service.name)
   metrics  THAT it is wrong   cheap, bounded, per-series cost
   traces   WHERE the time went  sampled, 300 B - 2 KB per span
   logs     WHAT happened        per-GB, linear in traffic
   profiles WHY (which function) 0.5-2% CPU, OTLP profiles alpha 3/2026
--------------------------------------------------------------------------------
 CARDINALITY = product of label value counts, per metric
   9 methods x 55 routes x 14 statuses x 80 instances = 554k series ~1.4 GB
   + tenant_id(12k) = 6.65 BILLION = 16.6 TB      <- the outage, not the bill
   labels must be BOUNDED and known at review time. Route TEMPLATE, not path.
   identity (user, request, url) belongs in traces/logs, never in labels
   native histograms: dynamic buckets, ~10x fewer series -> use for latency
--------------------------------------------------------------------------------
 PROMQL FOUR
   sum by (route) (rate(http_requests_total[5m]))
   sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(...[5m]))
   histogram_quantile(0.99, sum by (le) (rate(..._duration_bucket[5m])))
   NEVER avg() a quantile. Sum buckets first, quantile last.
   storage: series x (86400/interval) x 1.7 B    15s->60s = 4x cheaper
--------------------------------------------------------------------------------
 SAMPLING
   head  decide at root, propagated flag. cheap, keeps only 2% of ERRORS too
   tail  decide after completion in collector: 100% errors + 100% slow + 2%
         buffer = span_rate x decision_wait x span_size  (468k/s x 10 s = 3.3 GB)
         REQUIRES trace-id-aware routing to one collector
   never derive rate/error-rate from sampled traces
   logs: 100% errors + 1-5% successes  -> 42x cheaper, nothing lost
--------------------------------------------------------------------------------
 OTEL COLLECTOR = the seam
   receivers -> memory_limiter, k8sattributes, attributes(redact),
                tail_sampling, batch -> exporters
   swap vendor / redact PII / sample / aggregate WITHOUT redeploying services
   always set memory_limiter + bounded queue; alert send_failed_spans
--------------------------------------------------------------------------------
 DASHBOARD (one screen, this order)
   RED per endpoint  Rate | Errors | Duration(p99)
   USE per resource  Utilisation | SATURATION (leading) | Errors
   1 SLO burn + budget | 2 RED top 5 | 3 saturation: pool wait, evictions,
   queue oldest-age, in-flight | 4 replica lag, DLQ, outbox | 5 deploy markers
--------------------------------------------------------------------------------
 SLO MATH
   SLI = good / valid          budget = 1 - SLO
   99.9%  = 43.2 min / 30 d    99.95% = 21.6 min    99.99% = 4.3 min
   partial: impact_fraction x duration = budget minutes spent
   serial deps multiply: 0.9995^5 = 99.75% -> a 99.9% SLO is impossible
   burn rate = observed_error_ratio / (1 - SLO)
     14.4 over 1h AND 5m   -> PAGE (2% budget in 1 h)
      6.0 over 6h AND 30m  -> PAGE
      3.0 over 24h AND 2h  -> ticket      1.0 over 72h AND 6h -> ticket
--------------------------------------------------------------------------------
 ALERTS
   page: SLO burn, latency, queue oldest-age, checkout failures, up==0
   ticket: disk 70%, cert 20 d, DLQ > 0, orphans rising
   never: CPU 80%, one pod restart, a retry that succeeded
   every alert links a runbook: meaning, impact check, 3 causes, commands,
   MITIGATION FIRST (roll back / shed / scale), escalation
   > 2 pages per shift = unsustainable. Delete alerts monthly.
--------------------------------------------------------------------------------
 OUTSIDE-IN + BILL
   /live = process only (NEVER the DB -> restart storm + cold cache)
   /ready = pool, cache, migrations   synthetics from 5 regions/60 s
   RUM = the only real user measurement (DNS, TLS, TTFB, LCP, INP)
   observability should be 5-15% of the infra it observes; >30% = unsampled logs
================================================================================
```

PART NINE

Failure modes and resilience

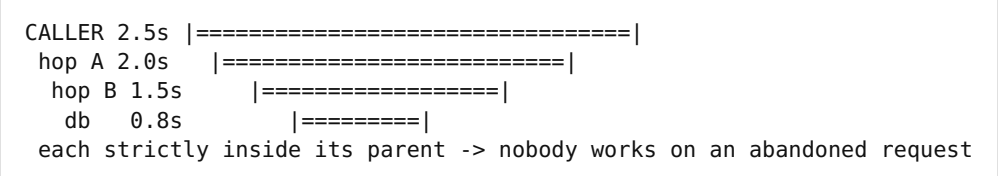
Nothing in this part is a component. It is the behaviour that emerges when the components of Parts 2–8 fail at the same time, under load, while retrying. Most large outages are not a broken machine — they are a healthy system amplifying a small problem until it cannot recover on its own.

Part 9 · Failure modes and resilience

96. Timeouts and the timeout budget

A request with no timeout is a resource leak with a user attached. Timeouts must also be *consistent down the call chain,* or an inner call keeps working on an answer nobody is waiting for — which is how a slow dependency exhausts every pool in the system.

```
THE INVERTED BUDGET, the most common misconfiguration
  client        30 s
  api gateway   60 s      <-- larger than the client
  service       90 s      <-- larger still
  database     none
  At t=30 s the client gives up and (usually) retries. The gateway,
  service and database keep working for another 60 s on a dead
  request, holding a connection each (Topic 45). At 900 req/s that
  is 54,000 abandoned in-flight requests consuming pools.

THE CORRECT SHAPE: each hop gets STRICTLY LESS than its caller
  user-facing SLO        3.0 s
   client HTTP timeout   2.5 s
    gateway              2.0 s
     service handler     1.5 s
      db query timeout   0.8 s   SET statement_timeout = '800ms';
      cache op           0.05 s
      external API       1.0 s   with its own retry INSIDE the budget
  Remaining budget = caller budget - elapsed, propagated as a
  deadline, not a fixed timeout:
    Grpc-Timeout: 1400m        (gRPC does this natively)
    or a custom header: X-Deadline-Ms, decremented per hop

TWO NUMBERS TO SET EVERYWHERE
  connect timeout   1-3 s (fail fast on a dead host, Topic 15)
  read timeout      p99.9 of the operation, not p50
  Postgres: statement_timeout, idle_in_transaction_session_timeout
  MySQL:    max_execution_time (in ms, per SELECT)
  nginx:    proxy_connect_timeout 2s; proxy_read_timeout 5s;
  PHP/curl: CURLOPT_CONNECTTIMEOUT, CURLOPT_TIMEOUT -- both, always

RETRIES CONSUME THE BUDGET
  1.0 s external API budget with 3 retries means each attempt gets
  ~300 ms, not 1 s. Retries multiply time as well as load (T97).
```



THE KEY INSIGHT

Propagate a *deadline,* not a timeout. A deadline is an absolute instant that survives every hop, so the fourth service knows it has 180 ms left rather than starting its own fresh 2-second clock.

THE TRAP

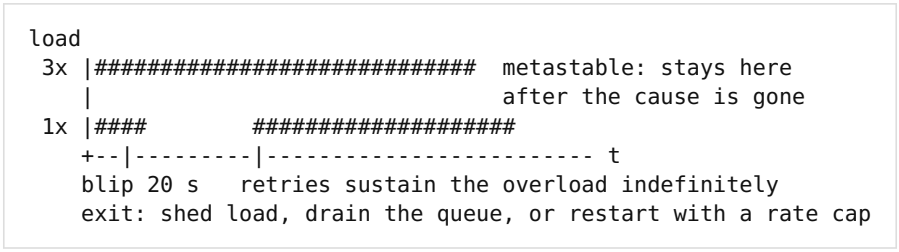
A default HTTP client with no timeout at all — the default in several popular libraries. Symptom: one dependency stops responding (not refusing, responding *slowly*), and within two minutes every worker in the fleet is blocked on it, including on endpoints that never touch that dependency. The pool is the shared resource that transmits the failure.

```
timeouts — GAINS: bounded resource use, failures that fail fast · COSTS: a budget to design and
propagate, some slow-but-valid requests cut off · ACCEPTS: you will occasionally abandon work
that would have succeeded
```

97. Retry amplification and the metastable failure

Retries are the most common way a small problem becomes a large one. The arithmetic is unforgiving, and the resulting state is *stable*: the system stays down after the original cause is gone.

```
AMPLIFICATION THROUGH LAYERS -- each multiplies
  browser retry        x2
  mobile SDK retry     x3
  API gateway retry    x2
  service-to-service   x3
  = 36x the original request rate reaching the failing component,
  produced entirely by "reasonable" per-layer policies.

  1,200 req/s of real traffic -> 43,200 req/s at the database that
  was already too slow. Nothing recovers under 36x load.

THE METASTABLE STATE (why it does not fix itself)
  1. a 20 s blip pushes latency past the client timeout
  2. clients retry: load rises to 3x
  3. at 3x, EVERY request is slow, so every request times out
  4. every timeout produces a retry -> load stays at 3x
  5. the original blip ended at step 1 and is now irrelevant
  The system is in a second stable state: fully loaded, serving
  nothing, and it will stay there until load is REMOVED from
  outside. This is why "wait for it to recover" fails and why the
  runbook step is shed, drain or restart-with-throttle.

DEFENCES, in order of effectiveness
  1. retry budget: total retries <= 10% of successful requests,
     enforced client-side. Caps amplification at 1.1x, not 36x.
  2. retry only at ONE layer -- usually the outermost that can
     still be idempotent. Disable retries in the inner layers.
  3. exponential backoff with FULL JITTER (Topic 68)
  4. circuit breaker (Topic 98): stop sending entirely
  5. deadline propagation (Topic 96): a retry with 40 ms of budget
     left is not attempted at all

  DO NOT RETRY: non-idempotent writes without a key (Topic 63),
  4xx, or anything the deadline cannot accommodate.
```



THE KEY INSIGHT

A retry budget is strictly better than a retry count, because it bounds the *total* extra load regardless of how many clients are failing at once. Counts are per-request and therefore multiply with the size of your fleet; budgets do not.

THE TRAP

Retries configured independently by four teams. Symptom: an incident where the failing service reports 30–40× its normal request rate while upstream traffic is flat, and nobody's own configuration looks unreasonable. Amplification is multiplicative across layers — it must be reviewed as a whole.

```
retry policy — GAINS: transient failures vanish invisibly · COSTS: multiplicative load exactly when capacity is lowest · ACCEPTS: without a budget, retries can convert a 20-second blip into an indefinite outage
```

98. Circuit breakers and bulkheads

Two structural defences: one stops you calling a dependency that is already failing, the other stops one dependency's failure from consuming the resources every other request needs.

```
CIRCUIT BREAKER -- three states
  CLOSED     calls pass. Count failures in a rolling window.
             trip when: >= 50% failures over >= 20 requests in 10 s
             (a percentage AND a minimum volume -- see Topic 93 for
             the same lesson about small denominators)
  OPEN       calls fail IMMEDIATELY, without a network attempt.
             Latency drops from 2,000 ms (timeout) to 0.1 ms.
             stay open 30 s.
  HALF-OPEN  allow N probe requests. All succeed -> CLOSED.
             Any fails -> OPEN for another 30 s (with backoff).

  The value is not just protecting the dependency -- it is
  protecting YOUR threads. An open breaker returns a fallback in
  microseconds instead of holding a worker for the full timeout.

  WHAT TO RETURN WHEN OPEN, in preference order:
    cached/stale data (Part 4)      -> degraded but useful
    a partial response with the section missing
    a clear 503 with Retry-After    -> honest and fast

BULKHEAD -- separate pools per dependency
  ONE shared pool of 200 workers:
    the recommendation service (nice-to-have) hangs
    -> all 200 workers block on it
    -> checkout, which never calls it, also stops. Total outage
       caused by an optional feature.
  PARTITIONED:
    checkout       120 workers    (or 120 pool slots / semaphore)
    search          50
    recommendations 20            <-- can only ever consume 20
    admin           10
  The failure is now bounded to 10% of capacity and the product
  degrades instead of stopping.

  Same idea at every layer: separate connection pools per database
  role, separate queues per class (Topic 71), separate node pools
  per workload, separate cache instances (Topic 42).
```

```
shared pool          bulkheaded
 [200 workers]        [120 checkout][50 search][20 rec][10 admin]
  rec hangs ->            rec hangs ->  only 20 blocked
  200 blocked             checkout unaffected: 94% of value preserved
```

THE KEY INSIGHT

A circuit breaker converts a slow failure into a fast one, and fast failures do not accumulate. Almost every cascading outage is powered by slowness, not by errors — errors return resources immediately, timeouts hold them for seconds.

THE TRAP

A breaker that trips on absolute counts with no minimum volume. Symptom: an endpoint receiving 3 requests per minute opens its breaker after two unrelated failures and stays open for half an hour, disabling a working feature. Always require both a failure percentage and a minimum request count.

```
breakers and bulkheads — GAINS: failure isolated to one dependency; workers returned in µs not
seconds · COSTS: fallback paths to write and test, tuning per dependency · ACCEPTS: an open
breaker rejects requests that might have succeeded
```

99. Load shedding and graceful degradation

When demand exceeds capacity, you serve a subset well or everyone badly. Serving everyone badly is a choice most systems make by default, and it is always the worse one.

```
THE ARITHMETIC OF NOT SHEDDING
  capacity 3,000 rps, arrivals 9,000 rps
  no shedding: all 9,000 queue. Latency = queue/service rate and
    grows without bound; at 30 s client timeout, EVERY request
    fails after consuming a full unit of work. Goodput = 0.
  shedding: accept 3,000, reject 6,000 immediately with 503.
    Goodput = 3,000 rps = 100% of capacity, and the 6,000 fail in
    2 ms instead of 30 s.
  Shedding converts total failure into partial success.

WHERE TO SHED -- as early and as cheaply as possible
  edge/LB (Topic 20)  cheapest: never enters the system
  gateway             per-tenant and per-endpoint quotas (T103)
  service             admission control on queue depth or latency
  Rejecting at the database is the worst place: you have already
  paid for parsing, auth, serialisation and a connection.

WHAT TO SHED FIRST -- priority classes, decided in advance
  1 payments, checkout, auth        never shed
  2 core reads (trip search)        shed at 90% saturation
  3 personalisation, recommendations shed at 75%
  4 analytics, prefetch, background  shed at 60%
  Encode the class in a request header at the edge so every layer
  can act on it consistently.

GRACEFUL DEGRADATION -- shed WORK, not requests
  serve cached/stale results (stale-while-revalidate, Topic 23)
  drop the personalised ranking, return the default order
  skip the thumbnail, return a placeholder
  disable search-as-you-type, keep search
  defer analytics events to a queue with a longer SLA
  A degraded response that arrives is worth far more than a perfect
  one that times out.

ADAPTIVE ADMISSION (better than a fixed threshold)
  measure your own queue wait; if it exceeds X ms, reject the
  lowest class. Self-tuning as capacity changes, and it works when
  the bottleneck is a dependency you cannot see.
```

THE KEY INSIGHT

Goodput, not throughput, is the metric. A system doing 9,000 requests per second of work that all times out is doing zero useful work while burning full capacity — and it looks busy on every dashboard that measures requests rather than successful responses.

THE TRAP

Shedding without a class header, so the 503s are random. Symptom: your load shedding works perfectly and payments fail anyway, because a uniform 30% rejection rate hits checkout as hard as it hits recommendations. Decide the priority order before the incident and encode it in the request.

```
load shedding — GAINS: goodput stays at 100% of capacity under 3× overload · COSTS: a priority scheme, fallback content, an honest 503 path · ACCEPTS: some users are refused service deliberately so others are served
```

100. Thundering herds, composed

The same pathology has appeared in six previous topics. Seeing it as one pattern is the point of this topic: whenever many independent actors are synchronised by a shared clock or a shared event, they arrive together.

```
THE SAME BUG, SIX PLACES
  Topic 15  health checks aligned to the same second
  Topic 21  a CDN purge invalidates 40,000 objects at once
  Topic 35  10,000 cache entries with an identical TTL
  Topic 38  one hot key expires, 1,500 concurrent rebuilds
  Topic 43  cache restart: full traffic hits a cold backend
  Topic 68  900 failed jobs retry at exactly t+2, t+4, t+8
  Topic 49  every client reconnects the instant a primary is
            promoted

WHAT SYNCHRONISES THEM
  a shared expiry time | a shared cron schedule (00:00:00 exactly)
  a shared failure event | a shared deploy | a shared backoff curve

THE FOUR FIXES -- all of them are the same fix
  1. JITTER: add rand(0, 10-30%) to every TTL, interval, backoff
     and cron. One line, removes the correlation.
  2. COALESCE: one actor does the work, the rest wait or serve
     stale -- request coalescing at the CDN, the SET NX mutex of
     Topic 38, singleflight in Go.
  3. STAGGER: warm/refresh in waves. Reintroduce a restarted
     service at 5% of traffic, then 25%, then 100%.
  4. RATE LIMIT THE RECOVERY: cap concurrent rebuilds and
     reconnections. Recovery is a workload and must be
     capacity-planned like any other.

THE RECOVERY IS THE OUTAGE -- worked
  cache cluster restarts. 34,000 rps, hit ratio drops to 0.
  backend capacity 3,000 rps. Without control: 34,000 rps of
  rebuild queries, none complete inside the timeout, retries make
  it 60,000, the cache never fills, and the outage is STABLE
  (Topic 97).
  With control: admit 2,000 rps of rebuilds, shed the rest with a
  fast 503 and stale content where available. Cache reaches 0.9 hit
  ratio in ~4 minutes and load falls away on its own.
```

THE KEY INSIGHT

Any fixed interval shared by N actors is a scheduled outage waiting for N to grow. Randomise every periodic thing you write, including cron minutes — the cost is one function call and the alternative is an incident whose cause is "midnight."

THE TRAP

A daily job scheduled at `0 0 * * *` across 60 instances. Symptom: a latency spike at exactly 00:00 every night that nobody can attribute, because each instance's own logs look unremarkable. Stagger by instance index or add a random delay at start-up.

```
herd control — GAINS: removes an entire class of periodic and recovery outages · COSTS: jitter
everywhere, a coalescing primitive, staged recovery tooling · ACCEPTS: refreshes are slightly
less punctual and recovery is deliberately slower
```

101. Anatomy of a cascading failure

One narrative, using the components of this book, showing how each layer passes the failure to the next. Read it once and the shape becomes recognisable in real incidents.

```
09:14:02  a schema migration adds an index on bookings. The DDL
          takes a metadata lock for 40 s on the busiest table.
09:14:12  queries queue behind the lock. p99 goes 40 ms -> 8 s.
          Connection pool (200) is fully occupied.       (T45)
09:14:20  app workers block waiting for a connection. Threads that
          need no database also stall: one shared pool. (T98 -- no
          bulkhead)
09:14:26  LB health checks time out. Instances are marked
          unhealthy and REMOVED.                          (T15)
          Remaining instances now carry more load and fail faster.
09:14:31  clients see 504s and retry. Mobile SDK retries 3x, the
          gateway 2x: incoming load is now 6x.            (T97)
09:14:40  the migration finishes. THE ORIGINAL CAUSE IS GONE.
09:14:41  it does not recover: 6x load on 40% of the fleet, every
          request times out, every timeout is retried. Metastable.
09:19:00  cache entries begin expiring with no successful
          rebuilds; hit ratio falls; database load rises further.
                                                          (T38, T43)
09:22:00  queue consumers, sharing the same database, stall.
          Queue depth climbs; SMS and tickets stop.        (T67)
09:26:00  operator restarts app servers. They start with cold
          caches and reconnect simultaneously into the
          overload.                                       (T100)
          Restarting does not help; it briefly makes it worse.
09:31:00  RECOVERY: rate limit at the edge to 25% of normal, let
          caches refill, then ramp 25 -> 50 -> 100% over 10 min.

WHAT WOULD HAVE STOPPED IT, in order of cheapness
  health check that fails on saturation, not on latency alone,
    with a floor: never remove more than 30% of instances    (T15)
  bulkheads so non-database work continues                   (T98)
  retry budget capped at 10%                                 (T97)
  load shedding by class at the edge                          (T99)
  online DDL (pt-online-schema-change / gh-ost) so no lock    (T44)
  Any ONE of these would have limited it to a 40-second blip.
```

THE KEY INSIGHT

Cascades propagate through *shared, exhaustible resources*: a connection pool, a thread pool, a queue, a cache. Find the shared resources in your architecture and you have found the transmission paths — partitioning them is what stops a local failure from becoming a global one.

THE TRAP

Health checks that remove instances during a global overload. Symptom: capacity falls exactly when it is needed, and the fleet shrinks toward zero as each survivor is overwhelmed in turn. Cap removals (never eject more than a fraction of the fleet) and prefer failing *ready* over failing *live* (Topic 95).

```
cascades — GAINS: understanding one shape explains most large outages · COSTS: partitioning
shared resources costs utilisation · ACCEPTS: any shared pool is a transmission path you have
chosen to keep
```

102. Gray failure: the worst kind

Binary failures are easy: the process is dead, the check fails, traffic moves. Gray failures are partial, inconsistent and often invisible to the health check — the system reports healthy while users are broken.

```
EXAMPLES YOU WILL MEET
  a node whose disk is failing: 200x slower, still returns 200 OK
  a NIC dropping 3% of packets: TCP retransmits, latency doubles,
    every application-level check passes
  one Kafka partition whose leader is on a degraded broker
  a replica applying binlog at half speed: correct data, 400 s old
  a JVM in a GC death spiral: alive, responding, 40 s pauses
  a dependency returning 200 with an empty body

WHY MONITORING MISSES IT
  the health check is cheaper than real work: GET /health returns
  a static 200 while every real query times out.
  averages hide it: one bad node in 60 moves the fleet average by
  1.7% and the p99 by a lot -- so watch PER-INSTANCE percentiles,
  not fleet aggregates.

DETECTION THAT WORKS
  1. deep health checks: /ready must exercise a real dependency
     (one cheap query), not return a constant
  2. per-instance outlier detection: compare each instance's error
     rate and p99 against the fleet median; eject outliers
     automatically (Envoy outlier_detection, T15)
  3. client-side observation: the caller knows better than the
     callee. Aggregate per-upstream success rate at the client.
  4. end-to-end synthetics (Topic 95): the only check that fails
     when the answer is wrong rather than absent
  5. differential comparison: two replicas answering the same
     query differently is a gray failure by definition

THE RESPONSE IS ALWAYS THE SAME: eject fast, investigate later.
A suspect instance costs you 1/60th of capacity; leaving it in
costs you the p99 of every request routed to it.
```

THE KEY INSIGHT

The observer inside a failing component is the least reliable judge of it. Health signals should come from the caller and from outside the system, because a node with a failing disk will cheerfully report that it is fine — its check never touched the disk.

THE TRAP

Alerting exclusively on fleet-wide aggregates. Symptom: users complain for days about intermittent slowness while every dashboard is green, because one instance in sixty is responsible and the aggregate absorbs it. Alert on the *spread* — max instance p99 divided by median instance p99 — not only on the mean.

```
gray failure — GAINS: detection of the failure mode that evades every binary check · COSTS: per-instance metrics (more series, Topic 84) and automatic ejection · ACCEPTS: you will sometimes eject a healthy instance, which is cheap
```

103. Rate limiting, distributed

Rate limiting is admission control with a per-actor memory. The algorithm choice determines burst behaviour; the distribution problem determines whether the limit means anything when you run 40 instances.

```
ALGORITHMS
  FIXED WINDOW   count per minute, reset on the minute.
    Flaw: 100/min allows 100 at 09:00:59 and 100 at 09:01:00 --
    200 in one second. Cheap; only acceptable for coarse quotas.
  SLIDING WINDOW LOG   store every timestamp, count those in the
    window. Exact, and memory is O(requests) -- expensive.
  SLIDING WINDOW COUNTER  weighted blend of current and previous
    window. ~0.003% error, O(1) memory. The usual production
    choice.
  TOKEN BUCKET   capacity C, refill R per second. Allows a burst
    of C, then a sustained R. Matches how APIs actually want to
    behave.
      allow = tokens >= 1; tokens = min(C, tokens + elapsed*R) - 1
  LEAKY BUCKET   constant output rate, queues the excess. Use when
    the downstream needs smooth input (an SMS gateway at 30/s).

DISTRIBUTED: 40 instances, a global limit of 1,000/min
  naive per-instance 25/min: an unbalanced LB or a sticky client
  sees 25 while the global count is 300. Wrong in both directions.
  CENTRAL COUNTER in Valkey -- atomic, exact:
    -- Lua, one round trip, no race (Topic 37 gives atomicity free)
    local t = redis.call('INCR', KEYS[1])
    if t == 1 then redis.call('EXPIRE', KEYS[1], ARGV[1]) end
    return t
    Cost: +0.4 ms per request, and the limiter is now a dependency
    that must not use the eviction-enabled cache (Topic 42).
  LOCAL + SYNC (for very high rates): each instance holds a local
    bucket and reconciles with the central count every 100-500 ms.
    Slight over-admission during the sync interval, ~50x cheaper.

WHAT TO RETURN -- the headers are part of the contract
  HTTP/1.1 429 Too Many Requests
  Retry-After: 12
  RateLimit-Limit: 1000
  RateLimit-Remaining: 0
  RateLimit-Reset: 12
  Without Retry-After, well-behaved clients retry immediately and
  you have built Topic 97.

DIMENSION MATTERS: limit per API key, per user, per IP, per tenant
AND globally. An IP-only limit punishes an entire office behind
one NAT and does nothing against a distributed client.
```

THE KEY INSIGHT

Token bucket is the right default because real clients are bursty and real limits are about sustained rate. Capacity C is your tolerance for burstiness and R is the actual policy — two numbers that map directly onto how the product should behave.

THE TRAP

A rate limiter whose state lives in the shared cache with `allkeys-lru`. Symptom: during an eviction storm the counters vanish and the limiter admits everything, precisely when the system is already overloaded. Limiter state belongs in a `noeviction` instance (Topic 42).

```
rate limiting — GAINS: a hard ceiling on any one actor's consumption · COSTS: +0.4 ms and a
stateful dependency, or approximation via local buckets · ACCEPTS: distributed limits are either
exact-and-slower or fast-and-approximate
```

104. Capacity planning, autoscaling and the lag that defeats it

Autoscaling is not capacity planning; it is a control loop with a delay. The delay is the whole subject, because traffic can rise faster than instances can boot.

```
THE SCALE-UP TIMELINE, honestly measured
  t+0     traffic rises
  t+60 s  metric is scraped and aggregated       (Topic 85)
  t+90 s  autoscaler evaluates (stabilisation window)
  t+95 s  instance requested from the provider
  t+150 s instance boots
  t+210 s container pulled and started
  t+260 s application warm: JIT, connection pools, local caches
  t+270 s passing readiness, receiving traffic
  ------- 4.5 MINUTES from need to help. -------
  A burst that arrives in 90 seconds (Topic 60) is entirely over
  before autoscaling contributes anything.

WHAT TO DO INSTEAD -- combine three things
  1. HEADROOM: run at 50-65% of capacity, not 85%. The headroom is
     what serves the burst while scaling happens. Cheaper than the
     outage, and it is a decision, not an accident.
  2. PREDICTIVE / SCHEDULED: your traffic is not random. Scale up
     at 08:45 because the 09:00 release happens every day. This is
     the single highest-value scaling rule most teams never write.
  3. SCALE ON THE RIGHT SIGNAL: queue oldest-age or in-flight
     requests per instance, not CPU (Topic 67). I/O-bound work
     never moves CPU.

SCALE-DOWN IS DANGEROUS TOO
  aggressive scale-in during a lull removes warm caches and
  connections; the next rise hits cold instances. Use a long
  scale-down stabilisation window (5-10 min) and a small step.

WHAT DOES NOT AUTOSCALE (plan these manually)
  database primaries, cache clusters (rebalancing is not instant),
  anything with a leader election, licence-limited components,
  and your dependencies' quotas -- a payment gateway that allows
  200 rps does not care how many pods you have.

CAPACITY MODEL, one line per resource
  peak_qps x work_per_request / capacity_per_instance x (1/target
  utilisation)
  34,000 x 0.012 CPU-s / 4 cores / 0.6 = 170 instances
  Then verify against the SLOWEST dependency, not the app tier --
  the app is almost never the binding constraint.
```

THE KEY INSIGHT

Autoscaling handles the daily curve; headroom handles the burst. Teams that replace headroom with autoscaling discover that the control loop is slower than their traffic, and the discovery happens during the launch that mattered.

THE TRAP

Scaling the stateless tier into a fixed-capacity dependency. Symptom: you double the pods and error rates get *worse*, because 340 instances × 20 connections exceeded the database's limit (Topic 45) or the third-party quota. Every scaling rule needs a ceiling derived from what is downstream.

```
autoscaling — GAINS: follows the daily curve, removes idle spend overnight · COSTS: 4–5 minutes
of lag, cold starts, downstream limits · ACCEPTS: bursts are served by headroom or not at all
```

105. Chaos engineering and the game day

Every defence in this part is untested code until it has run in anger. Chaos engineering is the practice of running it deliberately, in business hours, with everyone awake.

```
THE METHOD
  1. state the steady-state metric (goodput, SLI) and its normal
     range -- not "the system works"
  2. form a hypothesis: "killing one cache node will not move the
     error rate above 0.1%"
  3. define the blast radius and the abort condition IN ADVANCE
  4. inject, in production if you can, staging if you must
  5. compare, then fix what you learned, then repeat

THE EXPERIMENTS, in the order most teams should run them
  kill one app instance         (does the LB drain properly? T18)
  kill the database primary     (failover time, DNS TTL, retries?
                                 T49 -- almost always the biggest
                                 surprise)
  FLUSHALL the cache            (cold-start survivable? T43)
  add 200 ms latency to one     (timeouts inside budget? T96)
    dependency
  make a dependency return 503  (breaker trips? fallback works?
                                 T98)
  partition two AZs             (split brain? quorum? T49)
  fill a disk to 100%           (does it fail cleanly?)
  expire a TLS certificate in   (does anything detect it before
    staging                      the browser does?)
  pause a Kafka consumer 30 min (lag alerts? DLQ? drain math? T67)

RULES THAT MAKE IT SAFE
  business hours, announced, one variable at a time
  a documented abort that anyone in the room can trigger
  start in staging, then a single production instance, then a
  percentage. Never begin at "region failure."
  measure everything you expected AND the recovery time

WHAT YOU ARE REALLY TESTING
  not the software -- the runbook, the alerting, the dashboards
  and the humans. Most game days find that the mitigation works
  and nobody could locate the runbook, or the alert fired to a
  channel nobody reads.

CADENCE: the failover drill (T49) and the restore drill (T82)
quarterly, minimum. Untested recovery is not recovery; it is a
plan to discover the problem during the incident.
```

THE KEY INSIGHT

The finding is almost never "the system did not survive." It is "the system survived and nobody could tell," or "the alert fired 20 minutes late," or "the runbook referenced a command that no longer exists." Chaos engineering tests the socio-technical system, which is the one that actually responds at 3am.

THE TRAP

A first experiment with an unbounded blast radius. Symptom: the exercise becomes a real incident, the practice is banned for two years, and the organisation learns exactly the wrong lesson. Start with one instance and an abort condition anyone can invoke.

```
chaos engineering — GAINS: failure modes discovered while everyone is awake and prepared · COSTS: engineering time, a real risk budget, organisational buy-in · ACCEPTS: you will occasionally cause a small self-inflicted incident, on purpose
```

PRACTICE SET 9 — Topics 96–105

A. Conceptual

1. Why is a timeout budget that grows inward worse than having no timeouts at all in one specific respect?
2. What is the difference between propagating a timeout and propagating a deadline?
3. Why do retries multiply rather than add across layers?
4. Define a metastable failure and say why the original cause is irrelevant to recovery.
5. Why is a retry budget better than a retry count in a large fleet?
6. Why does a circuit breaker protect your own threads more than it protects the dependency?
7. Why must a breaker require both a failure percentage and a minimum request count?
8. Give the bulkhead argument in one sentence, using an optional feature as the example.
9. Define goodput and explain why throughput dashboards look healthy during a total failure.
10. Why is shedding at the database the most expensive place to shed?
11. Name the single property shared by the six thundering-herd examples.
12. Why can restarting app servers make a cascading failure worse?
13. Why do health checks that eject instances make a global overload worse, and what is the mitigation?
14. Why is the caller a better judge of a node's health than the node?
15. Why does alerting on fleet averages miss gray failure, and what statistic catches it?
16. What is wrong with a fixed-window rate limiter at the window boundary?
17. Why must rate-limiter state not live in an eviction-enabled cache?
18. Why does a 429 without `Retry-After` create the problem of Topic 97?
19. Why does autoscaling not help with a 90-second burst?
20. What does a game day actually test, beyond the software?

B. Pen-and-paper

1. Client timeout 20 s, gateway 45 s, service 70 s, database unbounded. At 1,400 req/s, how many abandoned in-flight requests exist at steady state after the client gives up? Then design a correct budget for a 2.5 s user-facing SLO with four hops plus a database call.
2. Layers retry 2×, 3×, 2× and 2×. Compute the amplification factor and the load at the failing component when real traffic is 2,200 req/s. Recompute with a 10% retry budget enforced at one layer only.
3. Capacity 4,000 rps, arrivals 11,000 rps, client timeout 25 s, mean service time 250 ms. Compute goodput with no shedding (assume every queued request eventually times out) and with shedding at capacity. What fraction of work is wasted in the first case?
4. A shared pool of 240 workers; the recommendation dependency hangs and holds a worker for the full 8 s timeout at 60 req/s. How long until the pool is exhausted? With a 24-worker bulkhead, what is the maximum impact?
5. Breaker config: trip at 50% failures over a 10 s window with a minimum of 20 requests, open for 30 s, 3 half-open probes. A dependency fails for 4 minutes at 300 req/s with a 2 s timeout. How many requests reach it, versus without a breaker? How many worker-seconds are saved?

6. 34,000 rps, hit ratio drops to 0, backend capacity 3,200 rps, mean rebuild 40 ms, and each cached entry serves 60 requests before expiry. With admission capped at 2,000 rps of rebuilds, estimate the time to reach a 0.9 hit ratio.
7. Token bucket with C = 60 and R = 10/s. A client sends 200 requests instantaneously, then 12/s for a minute. How many are accepted in the burst, and what is the steady-state rejection rate?
8. Global limit 1,200/min across 48 instances. Compute the naive per-instance limit, then the error if one instance receives 22% of traffic. What does a central counter cost at 34,000 rps in extra latency and in Valkey operations per second?
9. Peak 52,000 QPS, 14 ms of CPU per request, 4-core instances, target utilisation 60%. Compute instance count. If scale-up takes 4.5 minutes and traffic rises from 20% to 100% of peak in 80 s, how much headroom must exist to avoid shedding?

C. Lab tasks (build → break → observe → fix)

1. **Build.** Wire two services with an explicit deadline header, inner timeouts strictly smaller than outer, and a database `statement_timeout`. Verify by logging remaining budget at each hop.
2. **Break: the missing timeout.** Make the downstream service sleep 40 s. Call it with a default HTTP client (no timeout) at 50 req/s and record how long until every worker is blocked and unrelated endpoints stop responding. Add connect and read timeouts and repeat.
3. **Break: retry amplification.** Enable 3 retries at two layers and make the dependency fail for 60 s. Count requests at the dependency (`nginx` access log) versus requests entering the system. Then apply a retry budget and full jitter and compare peaks.
4. **Break: no bulkhead.** Give one endpoint a dependency that hangs. Show unrelated endpoints failing. Add a semaphore capping that dependency to 10% of workers and show the unrelated endpoints surviving.
5. **Break: metastability.** Drive load to 1.5× capacity with client retries enabled, then remove the extra load. Observe whether the system recovers on its own. Record the time. Then recover by rate limiting at the edge to 25% and ramping, and record that time.
6. **Break: gray failure.** Add 300 ms of latency and a 3% error rate to one instance of three (`tc netem` or a sleep in a feature flag). Confirm the fleet-average dashboard looks fine and the per-instance p99 does not. Add outlier ejection and verify it removes the node.
7. **Break: the limiter.** Implement a fixed-window limiter of 100/min and send 100 requests at 09:00:59 and 100 at 09:01:00. Record the observed one-second rate. Replace with a token bucket and repeat.

ANSWER KEY 9 — Topics 96–105

A. Conceptual

1. Because **inner hops keep working on requests nobody is waiting for**, consuming pools while the client has already retried — you pay for the work twice and get credit for neither.
2. A timeout is a **fresh duration at each hop** (so total time is the sum); a deadline is an **absolute instant** that every hop shares, so hop four knows it has 180 ms left.
3. Because each layer retries **each attempt of the layer above it**: 2×3×2×3 = **36×**, not 10×.
4. A second **stable** state in which retries generated by timeouts sustain the overload that causes the timeouts. Removing the original cause changes nothing; **load must be removed from outside**.
5. A count is per-request, so total extra load **scales with fleet size**. A budget caps **aggregate** retries (e.g. 10% of successes) regardless of how many clients fail simultaneously.
6. An open breaker returns in **microseconds instead of holding a worker for the full timeout**. Slow failures accumulate; fast failures release resources immediately.
7. Otherwise a **tiny denominator** trips it: 2 failures out of 3 requests opens the breaker and disables a working feature (same lesson as Topic 93).
8. **An optional feature must never be able to consume the resources that checkout needs** — a shared pool lets recommendations take the whole fleet down.
9. Goodput = **successful, useful responses per second**. Throughput counts requests *started*, so a system where all 9,000 rps time out looks maximally busy while delivering zero.
10. Because you have already paid for **parsing, authentication, serialisation and a connection** before rejecting. Shed at the edge, where a rejection costs microseconds.
11. **Synchronisation** — a shared clock, TTL, deploy or failure event that makes many independent actors act at the same instant.
12. Restarted servers come back with **cold caches and simultaneous reconnections** into an already-overloaded system (Topic 100), so the first minutes after restart are worse than before.
13. Ejection **removes capacity exactly when capacity is short**, so survivors fail faster in turn. Mitigation: **cap the fraction of the fleet that can be ejected** and fail readiness rather than liveness.
14. Because a degraded component's own check **usually does not touch the degraded resource** — a failing disk still returns a static 200 from `/health`.
15. One bad instance in sixty moves the mean by ~1.7%, invisible. Catch it with **per-instance percentiles and the spread** (max instance p99 / median instance p99).
16. The boundary allows **2× the limit in one second** — 100 at 09:00:59 and 100 at 09:01:00.
17. Because an **eviction storm deletes the counters** and the limiter admits everything, precisely during overload. It needs `noeviction`.
18. Well-behaved clients **retry immediately**, so the rejection itself generates the retry storm the limiter was meant to prevent.
19. Because the loop takes **~4.5 minutes** from metric to warm instance; the burst is over before the first new instance serves traffic. Headroom covers it.
20. **The runbooks, alerts, dashboards and the humans** — most findings are "it survived and nobody could tell" or "the runbook command no longer exists."

B. Worked

1. The client abandons at 20 s; inner hops continue to 70 s, so **50 s of orphan work** × 1,400/s = **70,000 abandoned in-flight requests**. Correct budget: client 2.5 s → gateway 2.2 → service A 1.8 → service B 1.4 → DB **0.7 s**, each strictly inside its parent, propagated as a deadline.
2. 2×3×2×2 = **24×** → 2,200×24 = **52,800 req/s** at the failing component. With a 10% budget at one layer: **≈2,420 req/s**, a **21.8×** reduction.
3. No shedding: every request occupies capacity and then times out → **goodput ≈ 0**, and **100% of work is wasted**. Shedding: **4,000 rps of goodput** (36% of arrivals served properly), 7,000 rejected in ~2 ms.
4. 60 req/s × 8 s = 480 workers demanded > 240 available → pool exhausted in **240/60 = 4 seconds**. With a 24-worker bulkhead the impact is capped at **10% of capacity, permanently**.
5. Without a breaker: 300×240 s = **72,000 requests**, each holding a worker 2 s = **144,000 worker-seconds**. With the breaker: ~20 requests to trip, then 3 probes per 30 s cycle → 20 + 8×3 = **≈44 requests** and **≈88 worker-seconds** — a **1,600×** reduction in blocked capacity.
6. Admitted rebuilds 2,000/s × 60 requests served per entry means cache coverage grows at 2,000 entries/s of demand-weighted traffic; reaching 0.9 coverage of 34,000 rps requires ~30,600 rps of traffic to be covered → 30,600/2,000 = **≈15 s of admitted rebuilds**, but bounded by backend service time and skew — in practice **2–4 minutes**. The key point: it is **finite and monotonic**, which it is not without admission control.
7. Burst: **60 accepted** (bucket capacity), 140 rejected. Steady state at 12/s against R = 10/s: **≈17% rejected** (2 of every 12).
8. Naive per-instance: **25/min**. The 22% instance wants 264/min and is throttled at 25 — a **10× under-admission** for those clients while the global total is far under 1,200. Central counter: **+0.4 ms per request and 34,000 INCR/s**, well within one node's ~120,000 ops/s (Topic 42).
9. 52,000×0.014 = 728 CPU-seconds/s; /4 cores = 182; /0.6 = **304 instances** at peak. Traffic rises 20%→100% in 80 s while scaling takes 270 s, so the fleet must already hold peak demand: run at **≈61 instances×5 = 304**, i.e. **headroom must cover the entire 5× rise** — or you shed (Topic 99) for the first 4.5 minutes.

C. What to verify

1. Each hop should log a **strictly decreasing remaining budget**, and the database call should be rejected with a `statement_timeout` **error** rather than running to completion when the budget is exhausted.
2. Without timeouts, expect **full worker exhaustion within seconds** (workers = concurrency × hold time) and unrelated endpoints returning 502/504. With timeouts, failures are confined to the slow endpoint and the rest of the service keeps its normal p99.
3. Expect the dependency to see **$(1+r_1)(1+r_2)$×** the entering rate — roughly **16×** for two layers of 3 retries. With a budget plus jitter, the peak should fall by **an order of magnitude** and arrivals should look scattered, not spiked.
4. Before: unrelated endpoints fail. After: the semaphore rejects fast, the hung dependency's endpoint degrades, and **everything else keeps its SLO**.
5. Expect the system **not** to recover after the extra load is removed — that is the finding. Recovery requires the edge cap; record both "time to recover unaided" (often never) and "time to recover with a ramp" (typically minutes).

6. Fleet mean error rate moves by **~1%** (invisible); per-instance p99 for the bad node should be **3–10×** the median. Outlier ejection should remove it within its configured interval; confirm the fleet p99 returns to baseline immediately after.
7. Fixed window: observe **200 requests in one second** against a "100/min" limit. Token bucket: **at most C accepted instantaneously** and a sustained rate of R thereafter.

HANDNOTE SUMMARY CARD — PART 9

```
================================================================================
 FAILURE MODES AND RESILIENCE -- ONE PAGE                       Topics 96-105
================================================================================
 TIMEOUT BUDGET (each hop STRICTLY less than its caller)
   SLO 3.0s > client 2.5 > gw 2.0 > svc 1.5 > db 0.8 > cache 0.05
   propagate a DEADLINE, not a timeout (Grpc-Timeout / X-Deadline-Ms)
   set BOTH connect (1-3 s) and read (p99.9) timeouts. statement_timeout on
   every DB. Inverted budget = orphan work: 50 s x 1,400/s = 70,000 in flight
--------------------------------------------------------------------------------
 RETRY AMPLIFICATION
   layers multiply: 2 x 3 x 2 x 3 = 36x   ->  1,200/s becomes 43,200/s
   METASTABLE: retries sustain the overload after the cause is gone.
     exit only by REMOVING load: shed, drain, restart with a rate cap
   fixes: retry BUDGET <= 10% of successes | retry at ONE layer | full jitter
          | circuit breaker | deadline says "no time left, don't try"
--------------------------------------------------------------------------------
 BREAKER / BULKHEAD
   CLOSED -> (>=50% fail AND >=20 reqs in 10 s) -> OPEN 30 s -> HALF-OPEN(3)
   open = fail in 0.1 ms instead of holding a worker 2 s  (1,600x less
   blocked capacity over a 4-min outage)
   fallback order: stale cache > partial response > fast 503 + Retry-After
   bulkhead: checkout 120 | search 50 | recs 20 | admin 10
     shared pool: 60/s x 8 s timeout exhausts 240 workers in 4 SECONDS
--------------------------------------------------------------------------------
 SHEDDING (goodput, not throughput)
   4,000 capacity vs 11,000 arrivals: no shed -> goodput ~0, 100% wasted
                                      shed    -> goodput 4,000 (full capacity)
   shed at the EDGE (cheapest); at the DB is the most expensive place
   classes: 1 payments/auth never | 2 core reads @90% | 3 personalisation @75%
            | 4 analytics/prefetch @60%   -- carry the class in a header
   degrade WORK: stale results, default ranking, placeholder image, defer
--------------------------------------------------------------------------------
 THUNDERING HERD = one bug in six places (T15 T21 T35 T38 T43 T68)
   cause: a shared clock / TTL / deploy / failure event
   fix:  JITTER everything (incl. cron) | COALESCE (SET NX, singleflight)
         | STAGGER 5->25->100% | RATE LIMIT THE RECOVERY
   recovery is a workload -- capacity plan it
--------------------------------------------------------------------------------
 CASCADE PATH (memorise the shape)
   lock/slow dep -> pool exhausted -> unrelated work blocked (no bulkhead)
   -> health checks eject instances -> less capacity -> clients retry 6x
   -> cache expires with no rebuilds -> consumers stall -> restart = cold
   any ONE of: capped ejection, bulkhead, retry budget, edge shedding,
   online DDL  would have kept it a 40-second blip
--------------------------------------------------------------------------------
 GRAY FAILURE (healthy check, broken service)
   slow disk still returns 200 | 3% packet loss | GC pauses | stale replica
   detect: deep /ready, PER-INSTANCE p99 and spread (max/median), client-side
   success rate, synthetics, replica differential.  Response: EJECT FAST.
   fleet averages hide 1 bad node in 60 (mean moves 1.7%)
--------------------------------------------------------------------------------
 RATE LIMITING
   fixed window: 2x at the boundary | sliding counter: O(1), ~0.003% error
   token bucket: burst C, sustain R  <- default | leaky bucket: smooth output
   distributed: central INCR+EXPIRE (+0.4 ms, noeviction instance!) or
                local buckets synced every 100-500 ms
   return 429 + Retry-After + RateLimit-* or you built T97
--------------------------------------------------------------------------------
 CAPACITY
   scale-up lag ~4.5 min (scrape 60 + evaluate 30 + boot 60 + pull 60 + warm 50)
   run at 50-65%, schedule for known peaks, scale on QUEUE AGE not CPU
   instances = peak_qps x cpu_s_per_req / cores / target_util
   never autoscale into a fixed dependency (DB conns, gateway quota)
 GAME DAY: kill instance | kill primary | FLUSHALL | +200 ms | 503 | partition
   business hours, one variable, documented abort. Quarterly failover+restore.
================================================================================
```

PART TEN

The platform layer

The boxes in Parts 2–9 have to run somewhere, and the software that puts them there has its own failure modes. This part is the layer between your design and the machines: proxies, ingress, deploys, regions and the managed-versus-self-hosted decision that quietly determines how much of this book you have to operate yourself.

Part 10 · The platform layer

106. Reverse proxy, load balancer, API gateway, service mesh

Four names for things that all sit in front of a service and forward traffic. They differ in what they add, and teams routinely buy all four and run three of them for no reason.

```
REVERSE PROXY      forwards, terminates TLS, caches, rewrites,
                   serves static. One process, one config file.
LOAD BALANCER      a reverse proxy whose job is choosing among N
                   backends: health checks, algorithms, draining
                   (Part 2). Every real LB is also a reverse proxy.
API GATEWAY        an LB plus API concerns: authn/authz, rate
                   limiting (T103), quotas, request/response
                   transformation, API keys, per-route policy,
                   sometimes aggregation of several backends.
                   NORTH-SOUTH traffic: outside -> in.
SERVICE MESH       a sidecar proxy beside EVERY service instance,
                   handling service-to-service concerns: mTLS,
                   retries with budgets, circuit breaking (T98),
                   outlier ejection (T102), traffic splitting,
                   and per-hop telemetry, all without touching
                   application code.
                   EAST-WEST traffic: service -> service.

WHAT A MESH COSTS -- the honest table
  +1 hop per call: ~0.3-1.0 ms p50, 2-5 ms p99
  ~50-150 MB RAM and 0.1-0.5 cores per pod (times every pod)
  a control plane to run, upgrade and debug
  a second place where routing can be wrong, and the failure mode
  ("mTLS cert rotation failed") is unfamiliar to most teams
  ambient/sidecar-less modes reduce the per-pod cost but not the
  conceptual one

WHEN A MESH PAYS FOR ITSELF
  30+ services, several languages (so libraries cannot be shared),
  a compliance requirement for mTLS everywhere, or you need traffic
  splitting for canaries across the whole fleet.
WHEN IT DOES NOT
  under ~15 services in one or two languages. A shared HTTP client
  library with timeouts, retries, breakers and OTel does 90% of it
  for none of the operational cost.
```

```
north-south:  internet -> [CDN] -> [LB] -> [API gateway] -> service
east-west:    service A [sidecar] <--mTLS--> [sidecar] service B
              mesh = the sidecars + a control plane telling them the rules
```

THE KEY INSIGHT

Ask which direction the traffic flows. North-south concerns (who is this caller, what are they allowed, how fast may they go) belong in a gateway; east-west concerns (is this hop encrypted, retried, broken-circuited) belong in a mesh or a shared client library. Putting east-west policy in the gateway forces service-to-service traffic through the edge, which is a common and expensive mistake.

THE TRAP

Adopting a mesh to get retries and circuit breakers. Symptom: the mesh now retries, the client library also retries, and the gateway retries too — Topic 97's amplification, delivered by the tool you bought to prevent it. Decide which single layer owns retries and disable them everywhere else.

```
the proxy layer — GAINS: cross-cutting policy without touching application code · COSTS: +0.3–
1 ms per hop, a control plane, per-pod memory · ACCEPTS: another place routing can be wrong, and
unfamiliar failure modes
```

107. The configs that actually matter

Most proxy configuration is defaults. Ten directives carry nearly all the behaviour that shows up in an incident, and their defaults are usually wrong for a system under load.

```
# nginx -- the ten lines that matter
upstream api {
    least_conn;                       # T13, better than round robin
                                      # for uneven request cost
    server 10.0.1.11:8080 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=10s;
    keepalive 64;                     # reuse upstream connections --
                                      # without it every request is a
                                      # new TCP+TLS handshake (T18)
}
server {
    proxy_connect_timeout 2s;         # fail fast on a dead host
    proxy_read_timeout    5s;         # MUST be < caller's timeout (T96)
    proxy_next_upstream error timeout http_502 http_503;
    proxy_next_upstream_tries 2;      # bounded retry -- T97
    proxy_http_version 1.1;           # required for keepalive
    proxy_set_header Connection "";   # ditto
    proxy_set_header X-Request-Id $request_id;   # join to traces (T87)
    client_max_body_size 10m;         # or uploads die at 1m default
    limit_req zone=api burst=40 nodelay;         # T103
}
limit_req_zone $binary_remote_addr zone=api:20m rate=50r/s;

# HAProxy equivalents worth knowing
  balance leastconn
  option httpchk GET /ready          # readiness, not liveness (T95)
  http-check expect status 200
  timeout connect 2s / client 30s / server 5s
  server api1 10.0.1.11:8080 check inter 2s fall 3 rise 2 slowstart 30s
    # slowstart ramps a returning server's share over 30 s -- the
    # single best defence against a cold instance being flooded (T100)

# Envoy concepts (the mesh data plane, and most modern gateways)
  clusters + endpoints, outlier_detection (eject 5xx outliers, T102),
  circuit_breakers (max_connections, max_pending_requests),
  retry_policy with retry_budget, and xDS for dynamic config

THE FIVE DEFAULTS THAT BITE
  no upstream keepalive          -> TLS handshake per request
  proxy_read_timeout 60s         -> longer than the client's (T96)
  client_max_body_size 1m        -> silent 413 on uploads
  unbounded proxy_next_upstream  -> retry storm across every backend
  health check on / (the app     -> ejects healthy nodes under load
    homepage, which hits the DB)
```

THE KEY INSIGHT

Upstream keepalive is the highest-value line in a proxy config. Without it every request pays a TCP handshake and, for TLS upstreams, a full handshake — 1–2 extra round trips per request, which is often the majority of p50 for a fast endpoint.

THE TRAP

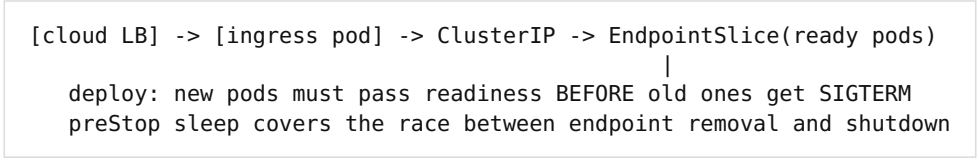
`proxy_next_upstream` left at its permissive default with an expensive endpoint. Symptom: one slow backend causes each request to be replayed against every other backend, so a partial failure becomes N× the load on the healthy remainder — a retry storm generated by the load balancer itself.

```
proxy config — GAINS: keepalive removes 1–2 RTT per request; bounded retries prevent LB-generated
storms · COSTS: config that must be reviewed like code · ACCEPTS: defaults are tuned for safety
in demos, not for load
```

108. The Kubernetes request path

Knowing exactly where a request goes between the internet and your process is what makes Kubernetes debuggable. There are five hops and each one can drop traffic.

```
internet
  -> cloud LB (L4)                    provisioned by a Service of
                                      type LoadBalancer
  -> Ingress / Gateway API controller  (nginx, Envoy, Traefik) --
                                      the L7 hop: TLS, host/path
                                      routing, rewrites
  -> Service (ClusterIP)               a stable VIP; kube-proxy or
                                      eBPF programs the node's
                                      dataplane
  -> Endpoints/EndpointSlice           the list of READY pod IPs --
                                      this is where readiness
                                      matters (T95)
  -> Pod                               your container

WHERE TRAFFIC IS LOST
  1. pod terminating: kubelet sends SIGTERM AND endpoint removal in
     parallel, with no ordering guarantee. In-flight requests are
     killed unless you add:
       lifecycle: preStop: exec: {command: ["sleep","10"]}
       terminationGracePeriodSeconds: 40
     and your app handles SIGTERM by draining (T18).
  2. readiness probe too optimistic: pods receive traffic before
     pools and caches are warm -> a burst of errors on every deploy
  3. liveness probe checking a dependency -> restart storm (T95)
  4. no PodDisruptionBudget -> a node drain removes every replica

resources:
  requests: {cpu: "250m", memory: "512Mi"}   # scheduling AND
  limits:   {memory: "1Gi"}                  # the QoS class
# CPU limits cause THROTTLING, not killing: a pod at its CPU limit
# is paused for the rest of each 100 ms period, which shows up as
# unexplained p99 spikes. Memory limits cause OOMKill (exit 137).
# Common advice: set memory limits, be careful with CPU limits.

HPA scales on a metric; see Topic 104 for why CPU is usually the
wrong one and why the loop is slower than your burst.
```



THE KEY INSIGHT

Readiness is a routing decision and liveness is a restart decision. Nearly every "we drop requests on deploy" and "one dependency restarted the fleet" problem is these two being confused — the same distinction as Topic 95, here with real consequences for every rollout.

THE TRAP

A CPU limit set equal to the request on a latency-sensitive service. Symptom: p99 spikes in exact multiples of 100 ms with no corresponding load, and CPU usage that looks comfortably under the limit on average. That is CFS throttling; check `container_cpu_cfs_throttled_seconds_total` before you tune anything else.

```
the k8s path — GAINS: declarative rollouts, self-healing, standard building blocks · COSTS: five hops to understand, probe semantics to get right · ACCEPTS: default settings drop in-flight requests on every deploy
```

109. Deploys: blue-green, canary, flags

About half of production incidents start within 30 minutes of a change (Topic 91). The deployment strategy decides how much of that risk reaches users and how quickly it can be withdrawn.

```
ROLLING             replace pods in batches. Default, cheapest.
                    Both versions serve simultaneously -> the
                    database schema must be compatible with BOTH.
                    Rollback = another rolling deploy (minutes).
BLUE-GREEN          two full environments; switch traffic at once.
                    Rollback is instant (switch back). Costs 2x
                    infrastructure and does not reduce blast radius:
                    100% of users get the new version at once.
CANARY              1% -> 5% -> 25% -> 100%, watching SLIs at each
                    step. Blast radius bounded by the percentage.
                    Needs: per-version metrics, an automated abort,
                    and enough traffic for 1% to be statistically
                    meaningful (at 200 rps, 1% is 2 rps -- you
                    cannot detect a 0.5% error rate change there).
FEATURE FLAG        deploy the code dark, enable per user/tenant/%.
                    Decouples deploy from release; rollback is a
                    config change (seconds), no rebuild.
                    Cost: dead code paths accumulate. Every flag
                    needs a removal date, or you get a combinatorial
                    test matrix nobody can reason about.

EXPAND-CONTRACT -- how to change a schema without downtime
  1. EXPAND: add the new column, nullable. Deploy code that writes
     BOTH old and new, reads old.
  2. BACKFILL in batches (T53), throttled to protect replica lag.
  3. Deploy code that reads NEW, still writes both.
  4. CONTRACT: stop writing old, then drop the column -- days
     later, after you are sure no rollback will need it.
  Never ship a schema change and the code that requires it in the
  same deploy: that deploy cannot be rolled back.

AUTOMATED ABORT -- the part that makes canaries real
  if error_rate(canary) > 1.5 x error_rate(baseline) over 5 min
     or p99(canary) > 1.3 x p99(baseline)
  then rollback, page, and stop the pipeline.
  A canary a human must watch is a canary that ships at 17:55 on a
  Friday because everyone went home.
```

THE KEY INSIGHT

Separate deploy from release. Once shipping code and enabling behaviour are different actions, rollback stops being a build-and-deploy cycle and becomes a flag flip — which turns a 20-minute incident into a 40-second one.

THE TRAP

A migration that drops or renames a column shipped with the code that stops using it. Symptom: the deploy fails for an unrelated reason, you roll back the code, and the old version crashes on a column that no longer exists — you now have no working version. Expand-contract exists precisely to keep rollback possible.

```
deploy strategy — GAINS: canary bounds blast radius to 1%; flags make rollback instant · COSTS: per-version metrics, flag hygiene, multi-step schema changes · ACCEPTS: two versions run simultaneously, so both must tolerate one schema
```

110. Managed or self-hosted

Every component in this book can be rented. The decision is rarely about features and almost always about which failures you want to be responsible for at 3am, and at what multiple of the raw cost.

```
THE MULTIPLE, roughly, for the same workload
  managed database        2.5-4x the raw compute cost
  managed cache           2-3x
  managed Kafka           3-5x
  managed search          3-4x
  object storage          ~1x (nobody self-hosts this well)
  Kubernetes control plane ~free to $150/month -- always managed

WHAT YOU ARE ACTUALLY BUYING
  backups that are tested, point-in-time recovery, automated minor
  upgrades, failover tooling that someone else debugged (T49),
  monitoring integration, and an on-call team for the storage
  engine's internals -- which for a database is a specialist skill
  your team probably does not have and should not need.

WHAT YOU GIVE UP
  extensions and versions the vendor has not blessed
  superuser access, some kernel and filesystem tuning
  price control (the multiple applies to every future GB)
  portability: a proprietary variant is not a drop-in Postgres

THE DECISION RULE THAT HOLDS UP
  Is operating this component a source of competitive advantage?
  If no -- and for 95% of teams the database, cache and broker are
  not -- rent it, and spend the saved engineering time on the
  parts of Parts 1-9 that are specific to your product.

  Self-host when: you have a genuine scale or cost inflection
  (dozens of nodes where the multiple is millions), a strict data
  residency or air-gap requirement, or in-house expertise you are
  already paying for.

A HONEST TCO LINE ITEM PEOPLE OMIT
  self-hosting a database costs the licence-free compute PLUS
  roughly 0.2-0.5 FTE of engineer time per critical stateful
  system, permanently. At most salaries that is $30-90k/year --
  usually more than the managed multiple on a mid-size cluster.
```

THE KEY INSIGHT

The managed premium buys back the operational topics of this book — failover, replication, backups, upgrades — but not the design ones. You still choose the shard key, the consistency level, the retry policy and the cache strategy; renting the software does not rent the judgement.

THE TRAP

Assuming managed means backed up *and restorable*. Symptom: the restore you have never run turns out to take 6 hours for 4 TB, or the point-in-time window is 7 days and the corruption is 9 days old. Test the restore quarterly (Topic 82) — the vendor guarantees a snapshot exists, not that your RTO is met.

```
managed services — GAINS: operational topics outsourced, 0.2–0.5 FTE per system reclaimed · COSTS: 2–5× the raw resource price, reduced control · ACCEPTS: vendor limits become your architectural limits
```

111. Multi-AZ, multi-region, RPO and RTO

Redundancy topology is a cost decision expressed as two numbers. Fix the numbers first; the architecture follows from them mechanically.

```
RPO  recovery POINT objective -- how much data may be lost,
     in time. Set by replication mode (T47).
RTO  recovery TIME objective -- how long until service resumes.
     Set by failover automation and DNS/routing (T49, T11).

TIER      TOPOLOGY                      RPO      RTO     COST
single AZ one AZ, backups only          24 h     4-8 h   1.0x
multi-AZ  sync replicas in 3 AZs        0        30-90 s 1.4x
          <- THE DEFAULT. Cheap, automatic, covers the failure
             that actually happens (an AZ event).
warm      cross-region async replica,   1-5 min  10-30   1.7x
standby   scaled-down standby                    min
hot       full stack both regions,      <1 min   1-5 min 2.2x
standby   traffic switched by DNS/GSLB
active-   both regions serving          0        seconds 2.5x+
active    (T58's conflict problem)

WHAT DECIDES THE TIER: multiply revenue-per-hour by the RTO and
compare with the annual cost difference. A shop doing $40k/hour
loses $320k in an 8-hour recovery; the jump from single-AZ to
multi-AZ costs far less than that once.

THE PARTS PEOPLE FORGET IN A REGIONAL FAILOVER
  DNS TTL (T11) -- a 300 s TTL is a 300 s floor on your RTO, and
    resolvers ignore short TTLs anyway; prefer anycast/GSLB health
    failover (T12)
  the standby's CAPACITY: a region running at 20% cannot take 100%
    of traffic. Either pre-scale (cost) or accept degraded service
    plus the 4.5-minute scale-up lag (T104)
  cold caches in the new region (T43) -- plan for load shedding
    during the first minutes
  the object storage bucket (T82: cross-region replication is
    async), the queue backlog, and any third-party allow-list
    pinned to the old region's egress IPs
  and the runbook, which must be readable from a laptop while the
    primary region -- including your wiki -- is down

TEST IT: an unrehearsed regional failover has roughly a 50%
chance of working. Quarterly, in business hours (T105).
```

THE KEY INSIGHT

Multi-AZ is close to free and covers the failure that genuinely occurs; multi-region is expensive and covers a rarer one while adding the consistency problems of Topic 58 to every write path. Most teams should be excellent at multi-AZ before they consider multi-region.

THE TRAP

A disaster-recovery plan whose runbook lives in a wiki hosted in the region that just failed. Symptom: the failover is technically possible and nobody can remember the steps. Keep an offline copy, printed or in a second region, and include the credentials path.

```
redundancy topology — GAINS: RPO 0 and RTO under 90 s for ~1.4× at multi-AZ · COSTS: 2.2–2.5× for cross-region, plus consistency complexity · ACCEPTS: an untested failover is a plan, not a capability
```

112. Infrastructure as code, drift and the runbook layer

The last platform topic is the least glamorous and the one that determines whether any of the previous 111 can be reproduced after an incident.

```
WHY IaC IS A RELIABILITY CONTROL, not a convenience
  a hand-configured production environment cannot be rebuilt, cannot
  be reviewed, and differs from staging in ways nobody can enumerate
  -- which is why "it works in staging" is usually true and useless.

DRIFT -- the gap between the code and reality
  someone raises max_connections during an incident and does not
  commit it. Six weeks later a routine apply reverts it at 09:00 on
  a Monday and reproduces the original incident.
  Controls: terraform plan in CI on every PR AND on a schedule
  (drift detection), alert on non-empty plans against main, and
  make emergency changes through a documented break-glass path
  that files a follow-up automatically.

WHAT BELONGS IN CODE
  every resource, every alert rule, every dashboard, every SLO
  definition, every DNS record, every IAM policy. Dashboards and
  alerts as code is what keeps them from silently diverging from
  the services they watch (T94).

THE RUNBOOK LAYER -- the human interface to all of it
  per service: what it does, its SLO, its dependencies, its
  dashboards, its alerts, and the five things most likely to go
  wrong with the exact commands to diagnose them (T94)
  per incident type: mitigation FIRST -- roll back, shed, scale,
  flag off -- then diagnosis
  per quarter: failover drill (T49), restore drill (T82), game day
  (T105); each updates the runbook it exercised

THE ARCHITECTURE DECISION RECORD
  one page per significant choice: context, options considered, the
  decision, the consequences accepted. Six months later this is the
  only artefact that answers "why is it sharded by user_id" -- and
  the cost strips in this book are exactly the "consequences
  accepted" line, written in advance.
```

THE KEY INSIGHT

The system that recovers quickly is not the one with the best architecture; it is the one where a tired engineer can find the right command at 3am. Runbooks, dashboards-as-code and decision records are reliability infrastructure, and they decay unless a drill exercises them.

THE TRAP

Emergency changes made by hand and never committed. Symptom: an incident that recurs weeks later at the exact moment of a routine deploy, with a diff nobody recognises. Every manual production change needs a follow-up commit or an alert that one is missing.

```
IaC and runbooks — GAINS: reproducible environments, reviewable changes, faster human response · COSTS: real effort to maintain and drill · ACCEPTS: drift is inevitable, so it must be detected rather than prevented
```

PRACTICE SET 10 — Topics 106–112

A. Conceptual

1. Which concerns belong to a gateway and which to a mesh? Give the directional rule.
2. Name three costs of a service mesh that do not appear in its feature list.
3. Under roughly what conditions is a shared HTTP client library the better choice than a mesh?
4. Why is upstream keepalive the highest-value line in a proxy config?
5. What does `slowstart` protect against, and which earlier topic is it an instance of?
6. Why is an unbounded `proxy_next_upstream` dangerous?
7. List the five hops from the internet to your process in Kubernetes.
8. Why does a `preStop` sleep prevent dropped requests on deploy?
9. What is the observable signature of CPU throttling, and which metric confirms it?
10. Why can a canary at 1% of 200 rps not detect a small error-rate regression?
11. Why must a schema change and the code requiring it never ship together?
12. State the decision rule for managed versus self-hosted in one sentence.
13. What does the managed premium *not* buy you?
14. Define RPO and RTO and name the mechanism that sets each.
15. Why does a 300 s DNS TTL put a floor under your RTO, and what avoids it?
16. Name three things people forget when planning a regional failover.
17. What is configuration drift, and why is a scheduled `plan` more useful than one run only on pull requests?

B. Pen-and-paper

1. A mesh adds 0.6 ms p50 per hop. A request traverses 5 services. Compute added p50 latency, then the added RAM and CPU across 420 pods at 100 MB and 0.2 cores per sidecar.
2. Without upstream keepalive, each request costs one TCP handshake (1 RTT) and a TLS handshake (1 RTT) at 0.6 ms RTT. If the backend work is 4 ms, compute p50 with and without keepalive and the percentage improvement.
3. A rolling deploy replaces 40 pods in batches of 4, each taking 25 s to become ready, with a 10 s preStop. Compute total deploy time and the window during which both versions serve traffic.
4. Canary at 5% of 9,000 rps for 10 minutes. Baseline error rate is 0.20%. How many canary requests are observed, and roughly what error-rate increase is detectable given the sample size?
5. Self-hosted Postgres: $2,400/month compute plus 0.35 FTE at $95,000/year. Managed equivalent: $7,900/month. Compute annual totals and the crossover FTE fraction at which self-hosting wins.
6. Revenue is $28,000/hour. Compare single-AZ (RTO 6 h) with multi-AZ (RTO 60 s) for one incident per year. Multi-AZ costs 1.4× a $19,000/month infrastructure bill. Compute expected annual loss and the annual cost difference.
7. A standby region runs at 25% of primary capacity. Traffic fails over instantaneously. With a 4.5-minute scale-up lag, what fraction of requests must be shed, and for how long, if scaling is linear to full capacity over 9 minutes?
8. Cross-region async replication lags 90 s at 700 writes/s. Compute the number of writes at risk, and state the RPO you would publish.

C. Lab tasks (build → break → observe → fix)

1. **Build.** Put nginx in front of two backends with `least_conn`, keepalive 64, bounded `proxy_next_upstream_tries`, and a `/ready` health check. Verify with `ss -tan` that upstream connections are reused across requests.
2. **Break: no keepalive.** Remove `keepalive` and `proxy_http_version 1.1`. Measure p50 and connections/s with a load generator, then restore them and compare. Confirm the delta matches your prediction from B.2.
3. **Break: the deploy that drops requests.** Roll a deployment under constant load with no `preStop` and an application that ignores SIGTERM. Count 502s. Add the preStop sleep and SIGTERM draining, and show the count reach zero.
4. **Break: the liveness restart storm.** Set liveness to check a dependency, stop the dependency for 40 s, and record how many pods restart and the total recovery time. Move it to readiness and repeat.
5. **Break: CPU throttling.** Set a CPU limit at the request value on a service doing bursty work. Record p99 and `container_cpu_cfs_throttled_seconds_total`. Raise or remove the limit and compare the latency histogram.
6. **Break: the irreversible migration.** Ship a migration dropping a column together with the code that stops using it. Roll the code back and observe the failure. Redo it expand-contract style and prove rollback works at each step.
7. **Break: drift.** Change a setting by hand on a resource managed by Terraform. Run `terraform plan` and record the diff. Then run `apply` and observe the revert — this is the Monday-morning incident, reproduced deliberately.

ANSWER KEY 10 — Topics 106–112

A. Conceptual

1. **North-south (outside→in) to the gateway**: authn, quotas, rate limits, API keys. **East-west (service→service) to the mesh**: mTLS, retries, breakers, per-hop telemetry.
2. **+0.3–1 ms per hop, 50–150 MB and 0.1–0.5 cores per pod, and a control plane to run and upgrade** — plus unfamiliar failure modes such as certificate rotation.
3. **Under ~15 services in one or two languages**, with no mesh-specific compliance requirement — the library delivers ~90% of the value at none of the operational cost.
4. Without it every request pays a **TCP handshake and, on TLS upstreams, a full handshake** — often 1–2 RTT, which can exceed the backend work itself.
5. It ramps a returning server's traffic share over N seconds, protecting a **cold instance from being flooded**. It is the staggered-recovery fix from **Topic 100**.
6. A slow or failing backend causes each request to be **replayed against every other backend**, so the load balancer itself manufactures the retry storm of Topic 97.
7. **cloud LB → ingress/gateway controller → Service ClusterIP → EndpointSlice (ready pods) → pod**.
8. Because SIGTERM and endpoint removal happen **in parallel with no ordering guarantee**; the sleep keeps the process serving until every proxy has observed the removal.
9. **p99 spikes in multiples of ~100 ms** with average CPU well under the limit. Confirm with `container_cpu_cfs_throttled_seconds_total`.
10. 1% of 200 rps is **2 rps**; over 10 minutes that is 1,200 requests, so anything below roughly a **1% absolute error-rate change** is statistical noise.
11. Because the deploy then **cannot be rolled back** — the old code requires a column the migration removed. Expand-contract keeps both versions runnable.
12. **Rent it unless operating it is a competitive advantage** (or you have a genuine scale, cost or residency inflection).
13. **The design decisions** — shard key, consistency level, retry policy, cache strategy. It rents the operations, not the judgement.
14. RPO = **data loss tolerance**, set by **replication mode** (Topic 47). RTO = **time to restore service**, set by **failover automation and routing** (Topics 49, 11).
15. Resolvers cache for the TTL (and often ignore short ones), so traffic keeps arriving at the dead region for up to that long. Use **anycast/GSLB health-based failover** (Topic 12).
16. **Standby capacity, cold caches, and async-replicated buckets/queues** — also third-party IP allow-lists and a runbook hosted in the failed region.
17. Drift is the **divergence between committed code and live configuration**. A scheduled plan catches **out-of-band manual changes**, which by definition never appear in a pull request.

B. Worked

1. 5 hops, 2 sidecars per hop in the general case, but at minimum **5×0.6 = 3.0 ms** added p50. Fleet: 420×100 MB = **42 GB RAM** and 420×0.2 = **84 cores** spent on proxying.
2. Without: 2 RTT×0.6 = 1.2 ms + 4 ms = **5.2 ms**. With: **4.0 ms**. Improvement **23%**, achieved by one config line.

3. 10 batches × (25 s ready + 10 s preStop) ≈ **350 s ≈ 5.8 minutes**, and both versions serve concurrently for **essentially the whole deploy** — which is why the schema must satisfy both.
4. 5%×9,000×600 = **270,000 canary requests**, expected 540 errors at baseline. With n that large, a change of roughly **0.05 percentage points** is detectable — a well-sized canary.
5. Self-hosted: $28,800 + $33,250 = **$62,050/year**. Managed: **$94,800/year**. Self-hosting wins here; crossover is at **≈0.7 FTE** ($66,000), above which the managed option is cheaper.
6. Single-AZ: 6 h×$28,000 = **$168,000 expected annual loss**. Multi-AZ: 60 s ≈ **$467**. Extra infrastructure cost: 0.4×$19,000×12 = **$91,200/year** — comfortably worth it on one incident per year.
7. At switchover the region serves 25%, so **75% must be shed** initially. Scaling linearly to 100% over 9 minutes after a 4.5-minute lag means shedding for **≈13.5 minutes**, with the shed fraction falling from 75% to 0 — which is exactly why priority classes (Topic 99) must exist beforehand.
8. 90×700 = **63,000 writes at risk**. Publish an RPO of **2 minutes** (round up from observed lag, and alert when lag exceeds half of it).

C. What to verify

1. `ss -tan` should show a **stable, small set of ESTABLISHED upstream connections** rather than a churn of TIME_WAIT sockets.
2. Expect p50 to rise by **1–2 RTT** and connections/s to approach requests/s. TIME_WAIT count is the clearest confirmation.
3. Without draining: a **non-zero 502/504 count** proportional to in-flight requests at each pod termination. With preStop and SIGTERM handling: **zero**.
4. Liveness on a dependency: **every pod restarts**, and recovery exceeds the outage because caches are cold (Topic 43). Readiness: pods are removed and restored **without restarting**.
5. Throttled-seconds should climb steadily and p99 should show **quantised spikes near multiples of the 100 ms CFS period**. Removing the limit should flatten the histogram tail immediately.
6. The rollback should **fail with a missing-column error** — a state with no working version. Expand-contract must allow rollback at **every** intermediate step; verify by rolling back at each.
7. `plan` must show the manual change as a diff; `apply` reverts it silently. Note the timestamp — in production this is the incident that "started for no reason" during a routine deploy.

HANDNOTE SUMMARY CARD — PART 10

```
================================================================================
 THE PLATFORM LAYER -- ONE PAGE                                  Topics 106-112
================================================================================
 WHICH BOX
   reverse proxy  forward, TLS, cache, rewrite
   load balancer  + backend choice, health, draining          (Part 2)
   API gateway    + authn, quotas, rate limit, transform   NORTH-SOUTH
   service mesh   sidecar per pod: mTLS, retry budget, breaker, outlier
                  ejection, traffic split, per-hop telemetry  EAST-WEST
   mesh pays off: 30+ services, several languages, mTLS mandate
   mesh costs: +0.3-1 ms/hop, 50-150 MB + 0.1-0.5 core PER POD, control plane
   ONE layer owns retries. Mesh + library + gateway all retrying = T97.
--------------------------------------------------------------------------------
 THE TEN NGINX LINES
   least_conn | keepalive 64 + proxy_http_version 1.1 + Connection ""
   proxy_connect_timeout 2s | proxy_read_timeout 5s  (< caller's, T96)
   proxy_next_upstream_tries 2      (unbounded = LB-generated retry storm)
   client_max_body_size 10m         (default 1m = silent 413 on uploads)
   limit_req zone=api burst=40 nodelay
   X-Request-Id -> joins to traces
   HAProxy: option httpchk GET /ready | check inter 2s fall 3 rise 2
            slowstart 30s   <- staggered recovery (T100)
   no keepalive = +2 RTT per request: 4 ms work becomes 5.2 ms (23%)
--------------------------------------------------------------------------------
 K8S REQUEST PATH
   cloud LB -> ingress -> Service(ClusterIP) -> EndpointSlice(READY) -> pod
   drops traffic when: no preStop sleep (SIGTERM races endpoint removal),
     readiness too optimistic, liveness checks a dependency (restart storm),
     no PodDisruptionBudget on node drain
   readiness = ROUTING decision | liveness = RESTART decision
   CPU limit -> CFS THROTTLING: p99 spikes in ~100 ms multiples;
     check container_cpu_cfs_throttled_seconds_total. Memory limit -> OOMKill 137
--------------------------------------------------------------------------------
 DEPLOYS
   rolling    cheap, both versions live -> schema must fit BOTH
   blue-green instant rollback, 2x infra, 100% blast radius
   canary     1->5->25->100% with AUTOMATED abort:
                err(canary) > 1.5x baseline OR p99 > 1.3x  -> rollback + page
              needs volume: 1% of 200 rps cannot detect anything
   flags      deploy != release; rollback in seconds; give every flag a
              removal date
   EXPAND-CONTRACT: add column -> write both -> backfill -> read new ->
     stop writing old -> drop (days later). Never ship schema + dependent
     code together: that deploy cannot be rolled back.
--------------------------------------------------------------------------------
 MANAGED vs SELF-HOSTED
   multiples: DB 2.5-4x | cache 2-3x | Kafka 3-5x | search 3-4x | S3 ~1x
   + 0.2-0.5 FTE per critical stateful system if self-hosted ($30-90k/yr)
   rent unless operating it is a competitive advantage
   managed does NOT buy: shard key, consistency level, retry policy, or a
   TESTED restore -- run it quarterly (T82)
--------------------------------------------------------------------------------
 RPO / RTO
   single AZ  RPO 24 h  RTO 4-8 h   1.0x       multi-AZ RPO 0 RTO 30-90 s 1.4x
   warm standby RPO 1-5 min RTO 10-30 min 1.7x hot standby RPO <1m RTO 1-5m 2.2x
   active-active RPO 0 RTO seconds 2.5x+ (and T58's conflicts)
   choose by revenue/hour x RTO vs annual delta
   failover forgets: DNS TTL floor, standby CAPACITY, cold caches, async
   bucket/queue replication, third-party IP allow-lists, the runbook itself
--------------------------------------------------------------------------------
 IaC + RUNBOOKS
   everything in code: resources, alerts, dashboards, SLOs, DNS, IAM
   drift: plan in CI AND on a schedule; alert on non-empty plan vs main
   break-glass path must auto-file the follow-up commit
   runbook per service: SLO, deps, dashboards, 5 likely failures + commands,
   MITIGATION FIRST. ADR per decision = the cost strip, written in advance.
================================================================================
```

PART ELEVEN

Putting it together

Six designs, each assembled entirely from Parts 1–10. Nothing new is introduced. The skill being practised is choosing which of the components you already know, sizing it with arithmetic, and naming the one thing that breaks first — which is what a design interview and a design review are both actually asking.

Part 11 · Putting it together

113. The method: seven steps, in order

A system design conversation goes wrong in the first three minutes, when someone starts drawing boxes before the numbers exist. The order below is what separates a design from a diagram, and it is identical whether the audience is an interviewer or your own team.

```
1 REQUIREMENTS (5 min)   -- ask, do not assume
    functional: what must it do? Write 4-6 bullets, no more.
    non-functional: how many users, read:write ratio, latency
      target, consistency requirement, retention, availability.
    OUT OF SCOPE: say it out loud. "No payments, no moderation."

2 ESTIMATION (5 min)     -- Part 1, every number shown
    DAU -> QPS -> peak QPS (x2-3)
    storage/day, storage/5 years
    bandwidth in and out
    A design without these is a drawing.

3 API (3 min)            -- the contract fixes the data model
    POST /v1/bookings {trip_id, seats[]} -> 201 {id, status}
    GET  /v1/trips?from=&to=&date=  -> 200 {trips[]}
    Say which are idempotent and which carry an Idempotency-Key.

4 DATA MODEL (5 min)     -- entities, access patterns, THEN engine
    for each query: which index, which shard key (T51), how many
    rows examined. Choose SQL/NoSQL from the access pattern (T56).

5 HIGH-LEVEL DESIGN (8 min)
    client -> CDN -> LB -> service(s) -> cache -> DB, plus queue
    and object storage where they belong. Say why each box exists;
    delete any box you cannot justify.

6 DEEP DIVE (10 min)     -- the interesting half
    the interviewer picks, or you offer the hardest part: the hot
    partition, the fan-out, the consistency boundary, the
    contention point.

7 BOTTLENECK AND SCALE (5 min)
    "At 10x, X breaks first, because Y. Then I would do Z."
    Name the single point of failure. Name what you would monitor
    (T92) and what you would shed first (T99).

WHAT MAKES AN ANSWER GOOD, in one line each
  every choice has a number attached
  every choice names what it gives up (the cost strip habit)
  you say when a simpler design is sufficient
  you know which component fails first and what the symptom is
```

THE KEY INSIGHT

Estimation before architecture is not ceremony — it decides the architecture. 400 QPS and 400,000 QPS produce genuinely different diagrams, and a candidate who draws Kafka before knowing which one it is has guessed.

THE TRAP

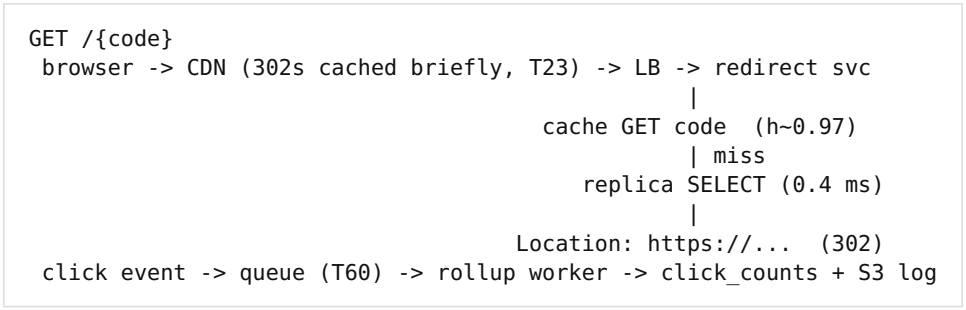
Designing for a scale nobody asked for. Symptom: sharding, a mesh and an event bus for a system doing 300 QPS that one Postgres instance and a cache would serve at 4 ms. Saying "one database is enough here, and here is the number at which that stops being true" is a stronger answer than any diagram.

```
the method — GAINS: a defensible design in 40 minutes · COSTS: discipline to postpone the drawing
· ACCEPTS: you will spend a third of the time before any box is on the board
```

114. Design: URL shortener

The classic, because it is small enough to finish and every interesting decision is about scale, not features. Read-dominated, tiny records, and one genuine question: how to generate ids.

```
REQUIREMENTS
  create a short link, optionally custom; redirect; expiry; basic
  click analytics. Out of scope: auth, ownership UI.
  100 M new links/month, 10:1 read:write, redirect p99 < 50 ms,
  links live 5 years.

ESTIMATION                                                (Part 1)
  writes 100e6 / 2.6e6 s   = 38 wps      peak x3 = 115 wps
  reads  38 x 10           = 380 rps     peak x3 = 1,150 rps
  storage/row: short_code 7 B + url 200 B + meta 40 B ~ 500 B
    with index overhead ~ 700 B
  5 years: 100e6 x 60 months x 700 B = 4.2 TB
  clicks at 10:1 = 60e9 events over 5 years -> NOT in the OLTP
    store: aggregate counters + a log to object storage (T79)
  bandwidth: 1,150 rps x 500 B = 0.6 MB/s. Trivial.

  CONCLUSION FROM THE NUMBERS: 1,150 rps and 4.2 TB. This is ONE
  Postgres instance with a read replica and a cache. No sharding.
  Saying that is the correct answer; the rest is what changes at
  100x.

API
  POST /v1/links {url, custom_code?, ttl_days?} -> 201 {code}
  GET  /{code} -> 301/302 + Location
  (302 if you want click analytics; 301 is cached by browsers
   forever and your counter stops working -- a real trade-off)

ID GENERATION -- the actual design question
  a) random 7-char base62 (62^7 = 3.5e12). Check-and-insert races
     -> rely on a UNIQUE constraint and retry on conflict (T63).
     Collision probability at 6e9 rows: ~0.17% per insert. Fine.
  b) counter + base62 encode. Sequential -> guessable, and a hot
     shard if you ever shard by code (T51).
  c) hash(url) truncated: same URL gives the same code (nice), but
     collisions must still be resolved, and custom codes conflict.
  CHOOSE (a) + UNIQUE + retry. Simple, unguessable, no coordinator.

DATA MODEL
  links(code PK, url, user_id, expires_at, created_at)
  click_counts(code, day, count)     -- rolled up, not per click
  shard key if ever needed: code (hash) -- every read has it.
```




```
DEEP DIVE -- the redirect path at p99 < 50 ms
  cache hit ratio here is exceptional: links are Zipf-distributed
  (T31) and small. 4.2 TB of links, but the hot 1% is 42 GB and
  the hot 0.01% serves most traffic. 8 GB cache -> h ~ 0.97.
  effective latency = 0.4 + 0.03 x 25 = 1.15 ms server-side.
  Redirects are the ideal CDN candidate: cache the 302 for 60 s
  with stale-while-revalidate (T23) and origin traffic drops
  another 20x -- at the cost of 60 s of staleness on deletion,
  which matters for abuse takedowns. State the trade-off.

BOTTLENECK AT 100x (11.5k wps, 115k rps)
  writes first: 11,500 wps on one primary is beyond a single
  node's comfortable range. Shard by hash(code) (T52) -- every
  read and write carries the code, so there are NO cross-shard
  queries. This is the rare system that shards perfectly.
  Then: counters. 115k click events/s must never touch the OLTP
  store; they go to a queue and a columnar store (T60, T56).
  SPOF: the id-generation retry loop under collision pressure --
  at 6e9 rows extend to 8 characters before the retry rate rises.
```

THE KEY INSIGHT

A URL shortener is a cache with a database behind it. Once you see that, every decision follows: optimise the read path to the edge, make ids cache-friendly and shard-friendly, and keep the write path boring.

THE TRAP

Counting clicks synchronously in the redirect path. Symptom: p99 tracks database write latency, and a slow counter update delays every user's navigation. Emit an event and roll up asynchronously — the count may be 30 seconds stale and nobody cares.

```
URL shortener — GAINS: sub-5 ms redirects, near-perfect shardability · COSTS: analytics must be a
separate pipeline · ACCEPTS: CDN-cached redirects are stale for their TTL, which delays takedowns
```

115. Design: news feed

The canonical fan-out problem. Its whole content is one decision made twice: whether to do the work at write time or read time, and what to do about the accounts that make the answer different.

```
REQUIREMENTS
  post; follow; home timeline of followees, reverse-chronological;
  p99 < 200 ms; a few seconds of staleness is fine.
  40 M DAU, average 200 follows, 2 posts/user/day, 30 feed
  refreshes/user/day.

ESTIMATION
  writes: 40e6 x 2 / 86400 = 926 posts/s   peak x3 = 2,800/s
  reads:  40e6 x 30 / 86400 = 13,900 feed reads/s  peak = 42,000/s
  read:write = 15:1 -> favour precomputation at write time
  fan-out cost: 926 posts/s x 200 followers = 185,000 timeline
    inserts/s at average. Feasible in a KV store; not in an RDBMS.
  storage: timeline entry = post_id 8 B + author 8 B + ts 8 B = 24 B
    keep 800 entries/user: 40e6 x 800 x 24 B = 768 GB. Fits in a
    sharded cache/KV tier.

THE TWO MODELS
  FAN-OUT ON WRITE (push)
    on post, insert the post id into every follower's timeline list
    read = LRANGE timeline:{user} 0 49    -> 0.4 ms, one call
    cost = one write x followers. Celebrity with 30 M followers =
    30 M inserts for ONE post: minutes of work, a hot shard, and
    everyone's timeline write-amplified.
  FAN-OUT ON READ (pull)
    read = fetch recent posts of 200 followees and merge
    cost = 200 lookups + a k-way merge per read, 13,900 times/s
    = 2.8 M lookups/s. Feed latency 80-300 ms. No write cost.
  HYBRID -- what every real system does
    push for normal accounts (< ~100k followers)
    pull for the few hundred celebrity accounts, merged at read
    time into the pushed timeline.
    Reads: one LRANGE + (usually 0-3) celebrity fetches + merge.

DATA MODEL
  posts(post_id PK, author_id, body, created_at)   sharded by
    post_id (T51) -- Snowflake-style ids sort by time
  follows(follower_id, followee_id)  sharded by follower_id so
    "who do I follow" is single-shard
  timeline:{user_id} -> capped list of 800 post_ids (KV/cache tier)
```

```
POST -> posts store -> [queue: fanout job]        (T60, T70)
                              |
        normal author: LPUSH into 200 timelines, LTRIM to 800
        celebrity:     skip fan-out entirely, mark author hot

READ /feed -> LRANGE timeline:{u} 0 49  (0.4 ms)
           +  pull recent posts of the <=3 celebrities I follow
           +  merge by timestamp -> hydrate post bodies from cache
```

```
DEEP DIVE -- fan-out as a queue problem
  185,000 inserts/s average, but posting is bursty: an evening peak
  of 2,800 posts/s x 200 = 560,000 inserts/s.
  Little's Law (T60): this MUST be asynchronous. The post returns
  as soon as it is durable; the timeline insert is a job.
  Backlog tolerance: the SLO is "your followers see it within 30 s",
  so oldest-message age (T67) is the alert, not queue depth.
  Ordering: fan-out jobs are keyed by author (T69) so a user's own
  posts cannot be reordered; across authors, order is by timestamp
  at read time anyway.
  Idempotency: LPUSH is not idempotent -> use a sorted set keyed by
  post_id (ZADD), which is (T63).

BOTTLENECK AT 10x
  fan-out write amplification is the first wall: 5.6 M inserts/s.
  Raise the push/pull threshold (more accounts become pull), shrink
  the stored timeline (800 -> 200 entries), and consider ranking,
  which changes the problem entirely: a ranked feed cannot be a
  precomputed reverse-chronological list, and becomes a retrieval +
  scoring pipeline at read time.
  SPOF: the timeline KV tier. Its loss is not data loss (timelines
  are derivable, T55) but rebuilding 40 M timelines is hours -- so
  degrade to pull-only mode during rebuild rather than showing
  empty feeds.
```

THE KEY INSIGHT

Fan-out on write trades storage and write amplification for read latency; it is the right default at a 15:1 read ratio. The hybrid exists because the cost is proportional to follower count, and follower counts are power-law distributed — the same skew that makes caching work in Topic 31 makes uniform fan-out fail.

THE TRAP

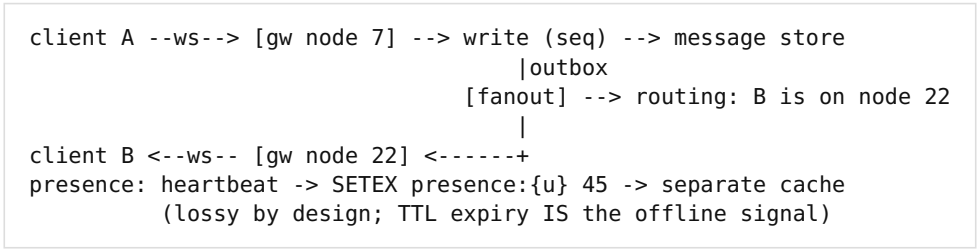
Treating the timeline store as a source of truth. Symptom: a fan-out bug corrupts timelines and there is no way to rebuild them because the follow graph at posting time was not recorded. Timelines are a read model (Topic 55); posts and follows are truth, and rebuild must be a supported operation.

```
news feed — GAINS: 0.4 ms feed reads at 42,000 rps · COSTS: 185,000–560,000 async inserts/s,
768 GB of derived data · ACCEPTS: feeds are seconds stale, and celebrity posts arrive by a
different path
```

116. Design: chat and presence

Two systems wearing one name. Messaging is a durable, ordered, per-conversation log; presence is high-volume, ephemeral and lossy. Designing them the same way is the most common mistake.

```
REQUIREMENTS
  1:1 and group chat (<= 500 members), delivery receipts, history,
  online status, typing indicators. Delivery within 500 ms.
  8 M DAU, 60 messages/user/day, 40% concurrently connected at peak.

ESTIMATION
  messages: 8e6 x 60 / 86400 = 5,555/s      peak x3 = 16,700/s
  connections: 8e6 x 0.40 = 3.2 M concurrent WebSockets
    per connection ~10 KB kernel + app state -> 32 GB RAM
    at 60k connections per node: 54 nodes (memory- and
    file-descriptor-bound, not CPU-bound)
  storage: 5,555/s x 86400 x 400 B = 192 GB/day = 70 TB/year
    -> hot 30 days in the OLTP store, older to object storage (T79)
  presence: 3.2 M users x 1 heartbeat/30 s = 107,000 writes/s of
    data that may be lost freely -> NEVER the same store as
    messages.

CONNECTION LAYER
  WebSocket (or long-lived HTTP/2) terminated on gateway nodes.
  A user's socket lives on ONE node, so delivery requires knowing
  which: a routing table user_id -> node_id in the cache tier,
  TTL 60 s, refreshed on heartbeat.
  Sending to a user on another node = publish to that node
  (pub/sub, T61) or to a per-node queue.
  LB must support sticky, long-lived connections and long drain
  times (T18): a deploy that kills 60,000 sockets at once produces
  a reconnect storm (T100) -- drain over minutes, with jitter on
  the client's reconnect backoff.

MESSAGE PATH (durability first, delivery second)
  1. client sends {conversation_id, client_msg_id, body}
  2. service assigns a monotonic per-conversation seq, writes to
     the store, returns ACK to the sender
  3. outbox/CDC (T59, T70) -> fan out to recipients' gateway nodes
  4. recipients ack; unacked -> push notification after N seconds
  client_msg_id makes retries idempotent (T63) -- a flaky mobile
  network retries constantly and must not duplicate messages.

  ORDERING: per conversation, by seq. Global ordering is not
  required and is expensive; say so explicitly.
  DATA MODEL: messages sharded by conversation_id (T51) so
  "load this conversation" is single-shard and ordered.
```



```
DEEP DIVE -- presence, and why it is the expensive part
  naive: broadcast every status change to every contact.
  8 M users x 60 contacts x a few transitions/hour = tens of
  millions of messages/s. Impossible and pointless.
  Real approach:
    - presence is a TTL key, written by heartbeat, read on demand
    - clients SUBSCRIBE only to the contacts in the visible list
      (typically 10-30), not their whole address book
    - typing indicators are pub/sub with no persistence and a
      client-side rate limit of 1 per 3 s
    - accept staleness: 45 s TTL means "online" can be 45 s wrong.
      Nobody has ever filed a bug about that.

BOTTLENECK AT 10x (32 M sockets)
  the connection tier: 540 nodes, and the routing table becomes the
  hot component (32 M keys, 1 M writes/s of heartbeats) -> shard
  it, and move heartbeats to a per-node aggregate rather than
  per-user writes.
  Then group fan-out: a 500-member group at 16,700 msg/s worst case
  is 8.3 M deliveries/s. Cap group size, or switch large groups to
  a pull model (clients poll the conversation log) -- the same
  push/pull hybrid as Topic 115.
  SPOF: the routing table. Its loss disconnects delivery but not
  durability; clients reconnect and resync from seq, which is why
  step 2 writes before it delivers.
```

THE KEY INSIGHT

Acknowledge on durability, not on delivery. Once the message has a sequence number and is stored, every other step — fan-out, push notification, receipt — can retry freely, and a client that reconnects simply resumes from its last seq. That single ordering makes the whole system resumable.

THE TRAP

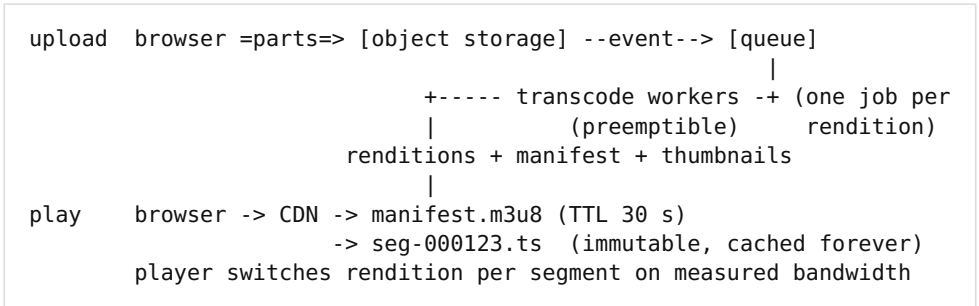
Running presence through the message path. Symptom: 107,000 heartbeat writes per second land in the durable store, replication lag rises, and message delivery slows — degraded by data that nobody would miss. Presence belongs in an ephemeral TTL store with its own capacity.

```
chat — GAINS: sub-500 ms delivery, resumable clients, ordered history · COSTS: 54+ stateful
connection nodes, a routing table, long drains · ACCEPTS: presence is up to 45 s stale and
deliberately lossy
```

117. Design: video upload and streaming

The system that exercises Part 7 hardest. Almost all of the engineering is in getting bytes out of the request path and into a pipeline, and almost all of the cost is egress.

```
REQUIREMENTS
  upload up to 4 GB; transcode to several renditions; adaptive
  playback; thumbnails; private and public videos.
  20,000 uploads/day, 3 M views/day, average view 8 minutes.

ESTIMATION
  uploads: 20,000/day x 1.2 GB avg = 24 TB/day ingest
  storage after transcode: original 1.2 GB + renditions ~0.9 GB
    = 2.1 GB/video -> 42 TB/day -> 15 PB/year. Lifecycle is not
    optional (T79): originals to IA at 30 days, archive at 1 year.
  egress: 3e6 views x 8 min x 2.5 Mbps / 8 = 450 TB/day
    at $0.08/GB CDN = $36,000/day. THE BILL IS EGRESS.
    Every design decision that reduces bitrate is worth more than
    every decision about compute.
  transcode compute: 20,000 videos x ~0.5x realtime x 4 renditions
    ~ 33,000 CPU-minutes/day = ~23 machines, easily preemptible.

UPLOAD PATH                                              (T77, T78)
  1. POST /v1/videos -> row status=pending, multipart upload id
  2. client uploads parts DIRECTLY to object storage, resumable
  3. CompleteMultipartUpload -> bucket event -> queue
  4. transcode workers: probe, then one job per rendition
     (240p/480p/720p/1080p) + thumbnail sprite + captions
  5. write HLS/DASH manifests; row status=ready
  NEVER through the app servers: 24 TB/day would need a fleet
  sized entirely by copying (T77).

PLAYBACK PATH
  manifest (short TTL, 10-60 s) + segments (immutable, long TTL)
    -- exactly the split from Topic 28
  segments are 2-6 s; a 2-hour video is ~2,400 objects per
  rendition. Object COUNT, not size, is what makes listing and
  lifecycle rules expensive.
  private video: signed CDN URLs (T80) scoped to the manifest path,
  short expiry, re-signed per session. DRM if the content demands
  it, which changes the vendor conversation entirely.
```



```
DEEP DIVE -- the pipeline as a queue system         (Part 6)
  a 4 GB video is 30-90 minutes of transcode. That means:
    visibility timeout must exceed p99.9 duration or the job runs
    twice concurrently (T66) -> use heartbeat extension
    workers are preemptible/spot: jobs MUST be idempotent and
    resumable per rendition (T63), keyed by (video_id, rendition)
    priority queues (T71): a paying customer's upload and a
    backfill re-encode do not share a queue
    DLQ with the failure reason: corrupt input is the most common
    permanent failure, and it must not retry forever (T68)
  Progress must be visible: users tolerate 20 minutes of processing
  and do not tolerate an unexplained blank page (T60's trap).

BOTTLENECK AT 10x
  egress cost, not capacity: $360,000/day. The levers, in order of
  effect: better codec (AV1/HEVC saves 30-50% bitrate), per-title
  encoding, capping default rendition on mobile, and negotiating
  committed CDN rates. None of these is an architecture change --
  which is the lesson.
  Then object COUNT: 10x means ~50 M new segment objects/day;
  lifecycle rules and inventory reconciliation (T81) become
  serious operations in their own right.
  SPOF: the transcode queue's DLQ being unwatched -- videos that
  silently never become playable, discovered by users (T68).
```

THE KEY INSIGHT

In media systems the interesting numbers are on the bill, not the latency graph. A design that halves bitrate is worth more than one that halves server count, and recognising which resource dominates is the whole skill.

THE TRAP

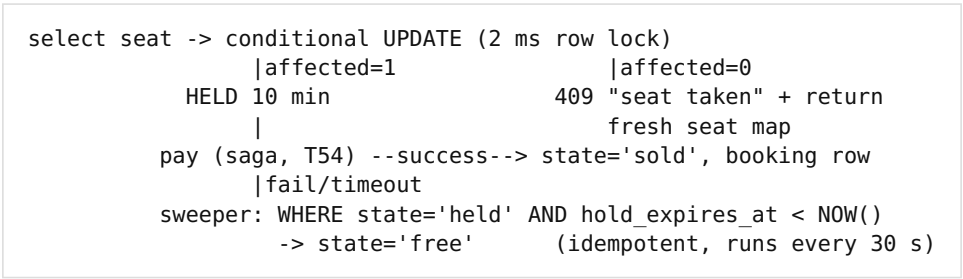
Long-TTL caching of the manifest. Symptom: a newly added 1080p rendition or a fixed caption track does not appear for users for hours, and purging every manifest is expensive. Manifests are the mutable pointer and must be short-lived; segments are immutable and cached forever (Topic 25).

```
video — GAINS: uploads cost the app nothing; playback served ~99% from the edge · COSTS: egress
dominates at $36k/day, a large transcode pipeline · ACCEPTS: processing latency of minutes, and
manifest staleness up to its TTL
```

118. Design: seat reservation under contention

The design where most of this book's advice inverts. Caching is wrong, eventual consistency is wrong, and the interesting resource is not throughput but a single contended row — which is exactly why it is the best final exercise.

```
REQUIREMENTS
  a bus/ferry trip has 40 numbered seats. A seat may be sold once.
  Users pick seats, hold them while paying, and the hold expires.
  Release day: 6,000 concurrent users on ~200 popular trips,
  4,000 seat-selection attempts/s, 300 confirmed bookings/s.
  Overselling is unacceptable. Underselling (holds that expire
  unused) is merely annoying.

WHY THE USUAL TOOLS FAIL HERE
  cache the availability count (Part 4)? A 500 ms stale "3 left"
  sells seat 14A twice. Topic 43 case 3 -- do not cache money.
  eventual consistency / active-active (T58)? No CRDT enforces
  "only 40 exist". The constraint is global.
  a queue in front of booking (T60)? Helps with load levelling,
  but the contention just moves to the consumer. It does not
  remove the need for a serialisation point.

THE CORE: ONE ROW PER SEAT, AND A CONDITIONAL UPDATE
  seats(trip_id, seat_no, state, held_by, hold_expires_at,
        booking_id)   PK (trip_id, seat_no)

  -- the entire correctness argument is this statement
  UPDATE seats
     SET state='held', held_by=?, hold_expires_at=NOW()+INTERVAL 10 MINUTE
   WHERE trip_id=? AND seat_no=?
     AND (state='free'
          OR (state='held' AND hold_expires_at < NOW()));
  -- affected_rows = 1 -> you hold it. 0 -> someone else does.
  No SELECT-then-UPDATE (that is a race), no application lock, no
  distributed lock service. This is Topic 63's conditional update,
  and the database's row lock is the serialisation point.

CONTENTION MATH -- the number that decides the design
  a row lock is held for the transaction: ~2 ms (single-row update,
  local commit).
  one seat row therefore supports ~500 attempts/s.
  one trip = 40 rows -> ~20,000 attempts/s IF demand is spread.
  Demand is NOT spread: everyone wants the window seats. Assume
  the hottest 4 seats take 60% of attempts on a hot trip:
    4,000/s x (200 hot trips) is spread across trips, but per trip
    the hot seats see maybe 40-120 attempts/s -> comfortably
    within 500/s. No sharding needed.
  Under distributed SQL (T57) with a 4 ms consensus round, one row
  supports only ~250/s -- worth knowing before choosing the engine.
```



```
THE HOLD IS A SAGA                                          (T54)
  reserve seat -> charge card -> issue ticket
  compensator on payment failure: release the seat.
  The intermediate state is VISIBLE and must be named in the UI:
  "held for 10 minutes". That is not a workaround -- it is the
  product design that makes a distributed transaction unnecessary.
  Expiry must be enforced by the CONDITION in the UPDATE, not only
  by the sweeper: if the sweeper is down, an expired hold must
  still be claimable. Never depend on a background job for
  correctness.

WHAT THE REST OF THE BOOK STILL DOES HERE
  CDN + cache: trip metadata, route names, schedules -- everything
    except the seat state (T43)
  queue: post-booking work only -- SMS, PDF, email, analytics (T60)
  outbox: booking.created published in the same transaction (T59)
  idempotency key on the payment call (T63): the mobile client
    WILL retry
  rate limit per user (T103): 20 seat attempts/min stops both
    scrapers and a stuck client's retry loop
  read replica: the seat MAP for browsing (stale by 200 ms is fine)
    but the UPDATE always goes to the primary (T48)
  load shedding (T99): if the system saturates, shed browsing
    before booking. Never the reverse.

BOTTLENECK AT 10x (40,000 attempts/s)
  not the seat rows -- it is the primary's write throughput and
  connection pool (T45), and the payment gateway's quota (T104).
  Shard by trip_id (T51): all seat rows for a trip are on one
  shard, so the conditional UPDATE stays single-shard and
  transactional. This system shards cleanly because the constraint
  is per-trip, never global.
  SPOF: the database primary. Its failover (T49, ~25 s) is a
  visible booking outage -- so publish it as an SLO (T92), make the
  client retry idempotently, and rehearse it (T105).
```

THE KEY INSIGHT

When a constraint is global and small ("40 seats exist"), the correct answer is a single serialisation point with a conditional write — not a distributed protocol. Find the smallest thing that must be serialised, make it one row, and let the database do the hard part it has been doing correctly for forty years.

THE TRAP

`SELECT ... WHERE state='free'` followed by an `UPDATE`. Symptom: two users are told they got seat 14A, both pay, and the incident is discovered at the boarding gate. The check and the write must be one statement (or one `SELECT ... FOR UPDATE` transaction); anything else is a race that appears only under the load of release day.

```
seat reservation — GAINS: overselling impossible, ~500 attempts/s per row, clean sharding by trip
· COSTS: writes must hit the primary; no caching of state; a visible hold state in the product ·
ACCEPTS: primary failover is a booking outage, and expired holds briefly undersell
```

PRACTICE SET 11 — Topics 113–118

A. Conceptual

1. Why does estimation come before architecture rather than after it?
2. What is the strongest thing you can say about a system doing 300 QPS?
3. Why is a URL shortener best described as a cache with a database behind it?
4. Why does choosing 301 over 302 break click analytics?
5. Why does random id generation with a unique constraint beat a central counter?
6. State the fan-out-on-write versus fan-out-on-read trade in terms of where the work happens.
7. Why does the hybrid feed model exist at all — what property of follower counts forces it?
8. Why must a timeline store never be treated as a source of truth?
9. Why is `ZADD` keyed by post id preferable to `LPUSH` for fan-out?
10. Why must a chat system acknowledge on durability rather than on delivery?
11. Why does presence belong in a different store from messages, and what does its TTL represent?
12. Why is the connection tier memory-bound rather than CPU-bound?
13. In video, why is the manifest short-TTL while segments are immutable?
14. Why is object count, not object size, the operational problem in video storage?
15. Why does the biggest lever in a video system live on the bill rather than in the architecture?
16. Give three reasons caching seat availability is wrong.
17. Why is a conditional `UPDATE` sufficient where a distributed lock is not needed?
18. Why must hold expiry be enforced in the `WHERE` clause and not only by the sweeper?
19. Why does the seat system shard cleanly while a feed system does not?
20. In each of the five designs, name the component that fails first at 10×.

B. Pen-and-paper

1. A shortener handles 240 M new links/month at 12:1 read:write. Compute average and peak (×3) write and read QPS, five-year storage at 700 B per row, and state whether sharding is required.
2. Same system: hot 0.02% of 8 B rows. Compute the cache size needed at 700 B/row and the effective latency at h = 0.97 with `t_cache` = 0.4 ms and `t_db` = 20 ms.
3. A feed system has 70 M DAU, 300 average follows, 3 posts/user/day, 25 refreshes/user/day. Compute post rate, feed read rate, average fan-out inserts per second, and peak at ×3.
4. Same system: 900 accounts exceed 5 M followers and post 4 times a day each. Compute the daily insert volume those accounts alone would generate under pure push, and the read-side cost of switching them to pull for a user following 3 of them.
5. Chat: 14 M DAU, 45% concurrent at peak, 10 KB per connection, 70,000 connections per node. Compute concurrent sockets, total memory and node count. Then compute heartbeat write rate at one per 30 s.
6. Chat storage: 90 messages/user/day at 380 B. Compute daily and annual storage, and the cost difference between keeping 12 months in Standard versus 30 days hot with the remainder in Standard-IA.
7. Video: 45,000 uploads/day averaging 900 MB, renditions adding 75% of the original size. Compute daily and annual storage. Then compute egress cost per day for 7 M views averaging 6 minutes at 2.2 Mbps at $0.08/GB.

8. Video: how many 4-second segments does a 90-minute video produce across 4 renditions? At 45,000 uploads/day, how many new objects per day?
9. Seats: a row lock is held 2.3 ms. Compute maximum attempts/s for one seat row. On release day the hottest seat on a trip receives 180 attempts/s — state whether it holds and what the margin is. Recompute under a distributed SQL engine with a 5 ms consensus round.
10. Seats: 10-minute holds, 4,500 attempts/s, 22% of holds abandoned. How many seats are held-but-unpaid at steady state, and how much inventory is invisible to other buyers at any moment?

C. Design tasks (build → break → observe → fix)

1. **Build.** Implement the seat-reservation core: seats table, conditional `UPDATE` hold, payment stub, sweeper. Drive it with 200 concurrent clients all targeting the same 4 seats and record the affected-rows distribution.
2. **Break: the check-then-act race.** Replace the conditional update with `SELECT` then `UPDATE`. Run the same load and count how many seats were sold twice. Record the exact concurrency at which the first double-sale appears, then restore the single statement and show it reach zero.
3. **Break: caching the count.** Add a 500 ms cached `seats_left` and drive the last 3 seats with 100 concurrent buyers. Count how many users were told a seat was available that was not, and how many paid.
4. **Break: the sweeper dependency.** Remove the `hold_expires_at < NOW()` clause from the `WHERE`, then stop the sweeper. Show that expired holds become permanently unclaimable — inventory lost to a background job outage.
5. **Build and break a shortener.** Implement random base62 with a unique constraint. Artificially shrink the code space to 3 characters, insert until collisions dominate, and plot insert latency and retry rate against row count. Extend the code length and show the curve reset.
6. **Break: synchronous click counting.** Increment a counter row inside the redirect transaction and drive 500 rps at one popular code. Record p99 and lock waits, then move counting to a queue and compare.
7. **Break: fan-out for a celebrity.** Simulate an account with 2 M followers posting once, with fan-out on write. Measure job duration, queue depth and the delivery latency of ordinary users' posts during that window. Then implement the pull path for that account and re-measure.

ANSWER KEY 11 — Topics 113–118

A. Conceptual

1. Because **the numbers choose the architecture**: 400 QPS and 400,000 QPS produce different diagrams, and drawing first means guessing which.
2. **"One database and a cache are sufficient, and here is the QPS at which that stops being true."** Naming the limit is stronger than any diagram.
3. Because the workload is **read-dominated, tiny records and highly skewed** — 97%+ of requests never reach the database, so the design work is all on the read path.
4. A 301 is cached by browsers **indefinitely**, so subsequent visits never reach your server and the click is never counted.
5. It needs **no coordinator**: the unique index resolves collisions (Topic 63), ids are unguessable, and nothing becomes a hot shard the way a sequential counter would.
6. Write-time: **work per post × follower count**, reads are one lookup. Read-time: **no write cost**, work per read × followee count.
7. Follower counts are **power-law distributed**, so a uniform policy is either wasteful for everyone or catastrophic for the top 0.001% (30 M inserts for one post).
8. Because it is a **read model** (Topic 55) — derived data that must be rebuildable from posts and follows. Treating it as truth means a fan-out bug is unrecoverable.
9. `ZADD` keyed by post id is **idempotent**; a redelivered fan-out job re-adds the same member instead of duplicating the entry (Topic 63).
10. Because once the message has a sequence number and is stored, **every later step can retry and any client can resume from its last seq**. Acking on delivery makes the system unresumable.
11. Presence is **high-volume, lossy and ephemeral** (107,000 writes/s of disposable data). Its TTL expiry **is** the offline signal — no explicit "went offline" event is needed.
12. Each idle socket costs **~10 KB of kernel and application state** and a file descriptor while consuming almost no CPU, so nodes run out of memory and descriptors long before cycles.
13. The manifest is the **mutable pointer** (renditions and captions change); segments are **content-addressed and immutable**, so they can be cached forever (Topic 25).
14. Because a single 2-hour video is **~2,400 objects per rendition**; lifecycle rules, inventory reconciliation and listing costs all scale with count, not bytes.
15. Because **egress dominates** ($36,000/day at the stated scale), and bitrate reduction — codec, per-title encoding, mobile caps — beats any server-side optimisation.
16. **It is money under contention; staleness sells the same seat twice; and the constraint is global**, so no cache or CRDT can enforce it (Topics 43, 58).
17. Because the database's **row lock already provides the serialisation point**, and the condition makes check-and-act a single atomic statement — an external lock adds a dependency and a new failure mode for no gain.
18. Because **correctness must not depend on a background job**: if the sweeper is down, an expired hold must still be claimable by the next buyer.
19. Because its constraint is **per-trip** — all rows that must be serialised together live on one shard. A feed's constraint spans arbitrary follower sets, which no shard key can co-locate.
20. Shortener: **write throughput, then the analytics path**. Feed: **fan-out write amplification**. Chat: **the connection tier and routing table**. Video: **egress cost, then object count**. Seats: **the primary's write throughput and the payment gateway quota**.

B. Worked

1. Writes 240e6/2.6e6 = **92 wps**, peak **277**. Reads **1,108 rps**, peak **3,323**. Storage: 240e6×60×700 B = **10.1 TB** over five years. **No sharding** — one primary plus replicas and a cache serves this comfortably.
2. Hot rows 8e9×0.0002 = 1.6 M × 700 B = **1.1 GB**. Effective latency 0.4 + 0.03×20 = **1.0 ms**.
3. Posts 70e6×3/86400 = **2,431/s** (peak **7,292**). Feed reads 70e6×25/86400 = **20,255/s** (peak **60,764**). Fan-out 2,431×300 = **729,000 inserts/s**, peak **2.19 M/s**.
4. 900×4×5e6 = **18 billion inserts/day** under pure push — alone an order of magnitude above the rest of the system. Pull side: a user following 3 celebrities pays **3 extra lookups plus a merge** per feed read, roughly **+1–2 ms**.
5. Sockets 14e6×0.45 = **6.3 M**; memory 6.3e6×10 KB = **63 GB**; nodes 6.3e6/70,000 = **90**. Heartbeats 6.3e6/30 = **210,000 writes/s**.
6. 14e6×90×380 B = **479 GB/day ≈ 175 TB/year**. All Standard: 175,000×0.023 = **$4,025/mo**. 30 days hot (14.4 TB) + rest IA (160 TB): 331 + 2,000 = **$2,331/mo**, a **42%** saving.
7. Storage 45,000×0.9 GB×1.75 = **70.9 TB/day ≈ 25.9 PB/year**. Egress 7e6×360 s×2.2 Mbit/8 = **693 TB/day → $55,400/day**.
8. 90 min = 5,400 s / 4 s = 1,350 segments × 4 renditions = **5,400 objects per video** (plus manifests and thumbnails). At 45,000/day: **243 M new objects/day**.
9. 1/0.0023 = **435 attempts/s per row**. 180/s **holds**, with a **2.4× margin**. Under distributed SQL at 5 ms: **200/s** — only a **1.1×** margin, which is too thin for release day.
10. Held-but-unpaid at steady state = hold rate × hold duration. If 300 holds/s convert and 22% are abandoned, abandoned holds ≈ 85/s × 600 s = **≈51,000 seats held-but-unpaid** across the system at any instant — inventory invisible to other buyers, which is why the hold window is a product decision with a revenue cost.

C. What to verify

1. Across 200 concurrent attempts on 4 seats, exactly **4 should report affected_rows = 1** and the rest 0. Any other distribution means the statement is not atomic as written.
2. With check-then-act, double-sales should appear at **low concurrency — often under 20 clients** — and rise with load. With the single statement: **zero, at any concurrency**. This is the most important experiment in the part.
3. Expect a number of users to receive "available" for a sold seat roughly proportional to **cache TTL × sell rate**. Any non-zero count of completed payments for an unavailable seat is a production incident in miniature (Topic 43).
4. Expired holds become **permanently unclaimable** while the sweeper is down — verify the seats never return to sale. This demonstrates why the condition, not the job, is the correctness mechanism.
5. Insert latency and retry rate should stay flat until occupancy approaches the code space, then **rise sharply** (retries per insert grow roughly as 1/(1−occupancy)). Extending the code length must return both to baseline.
6. Synchronous counting: p99 tracks the counter's lock wait and should degrade markedly at 500 rps on one row (Topic 118's contention math applies here too). Asynchronous: **p99 returns to the cache-hit path** and counts lag by seconds.
7. The celebrity fan-out job should show **queue depth spiking and ordinary users' delivery latency rising in proportion** — head-of-line effects across shared workers (Topic 71). With

the pull path, ordinary delivery latency must be **unaffected**, and the read side should gain only a few milliseconds.

HANDNOTE SUMMARY CARD — PART 11

```
================================================================================
 PUTTING IT TOGETHER -- ONE PAGE                                Topics 113-118
================================================================================
 THE SEVEN STEPS
   1 requirements (functional + non-functional + OUT of scope)      5 min
   2 estimation   DAU -> QPS -> peak x3 | storage/5y | bandwidth    5 min
   3 API          endpoints + which are idempotent                  3 min
   4 data model   entities, access patterns, index, SHARD KEY       5 min
   5 HLD          client-CDN-LB-svc-cache-DB (+queue, +blob)        8 min
   6 deep dive    the hard part: fan-out / contention / consistency 10 min
   7 bottleneck   "at 10x, X breaks first, because Y" + the SPOF    5 min
   every choice gets a number AND a stated cost. Say when one DB suffices.
--------------------------------------------------------------------------------
 URL SHORTENER          cache with a database behind it
   38 wps / 380 rps / 4.2 TB -> ONE Postgres + replica + cache. No shard.
   random base62(7) + UNIQUE + retry (no coordinator, unguessable)
   302 not 301 if you need click counts; CDN the redirect 60 s (takedown lag)
   clicks -> queue -> rollup. NEVER in the redirect transaction.
   10x: shard by hash(code) -- every query has the code = perfect shardability
--------------------------------------------------------------------------------
 NEWS FEED              fan-out on write, hybrid for celebrities
   926 posts/s x 200 followers = 185k inserts/s (peak 560k) -> async, keyed
   read = LRANGE timeline:{u} (0.4 ms) + pull <=3 celebrities + merge
   push < ~100k followers, pull above it (follower counts are power-law)
   ZADD by post_id (idempotent), LTRIM to 800, timelines are a READ MODEL
   alert on oldest-message age, not depth.  10x: raise the pull threshold
--------------------------------------------------------------------------------
 CHAT + PRESENCE        two systems, one name
   3.2 M sockets / 60k per node = 54 nodes, MEMORY-bound (10 KB each)
   ACK ON DURABILITY: assign per-conversation seq, store, then fan out
   client_msg_id = idempotency (mobile retries constantly)
   routing table user -> gw node, TTL 60 s, in the cache tier
   presence = SETEX 45 s in a SEPARATE store; TTL expiry IS "offline"
   shard messages by conversation_id; drain sockets slowly + client jitter
--------------------------------------------------------------------------------
 VIDEO                  the bill is egress
   24 TB/day in, 450 TB/day out = $36k/day. Bitrate beats servers.
   presigned multipart direct to storage -> event -> queue -> transcode
   manifest short TTL (mutable) | segments immutable, cached forever
   visibility timeout > p99.9 job (30-90 min) or heartbeat-extend it
   idempotent per (video_id, rendition); preemptible workers; DLQ watched
   10x: object COUNT (243 M/day) makes lifecycle + reconciliation real work
--------------------------------------------------------------------------------
 SEAT RESERVATION       where this book inverts
   DO NOT cache availability. No CRDT. Constraint is global and small.
   UPDATE seats SET state='held', hold_expires_at=NOW()+10m
    WHERE trip_id=? AND seat_no=?
      AND (state='free' OR (state='held' AND hold_expires_at < NOW()));
   affected_rows 1 = yours, 0 = taken.  NEVER select-then-update.
   row lock ~2 ms -> ~500 attempts/s PER SEAT ROW (250/s on distributed SQL)
   hold = saga; name the state in the UI; expiry lives in the WHERE clause,
   the sweeper is only an optimisation
   shard by trip_id (constraint is per-trip). SPOF: primary failover ~25 s
--------------------------------------------------------------------------------
 WHAT BREAKS FIRST AT 10x
   shortener: write throughput | feed: fan-out amplification
   chat: connection tier + routing table | video: egress cost, object count
   seats: primary write throughput + payment gateway quota
================================================================================
```

Cheatsheets, patterns and revision

CONSOLIDATED CHEATSHEET — PARTS 1–3

```
================================================================================
 FOUNDATIONS + LOAD BALANCER + CDN                              Topics 1-30
================================================================================
 ESTIMATION CHAIN (write these five lines before any box)
   MAU x DAU/MAU x actions/day / 86,400          = avg QPS
   x peak factor (3x diurnal, 10-40x event)      = peak QPS
   peak x r/(1+r) reads, x 1/(1+r) writes        r = read:write
   DB load = peak reads x (1 - cache hit ratio)
   storage = bytes/record x records/day x retention x replication
   bandwidth = QPS x payload;  EGRESS $/GB is the invoice
 LATENCY  L1 0.5 ns | RAM 100 ns | NVMe 50-150 us | same-AZ 0.5 ms
   cross-AZ 1-2 ms | cross-region 110-130 ms | count RTT first, CPU last
 AVAILABILITY serial A1 x A2 x A3 | parallel 1-(1-A1)(1-A2)
   99.9 = 43.8 min/mo | 99.95 = 21.9 | 99.99 = 4.4 | cheapest nine = remove
   a component from the critical path
 TAIL  P(slow) = 1-(1-p)^fanout: 20 @1% = 18%, 100 @1% = 63%
   fixes: batch, hedge at p95 (+5% load), deadline + partial response
--------------------------------------------------------------------------------
 LOAD BALANCER
   L4 5-tuple 0.08 ms, no retries | L7 path/header 0.5 ms, retries, canary
   algorithms: RR ignores cost | least_conn attracts fast failures
     | P2C near-optimal O(1) | consistent hash pins keys (modulo remaps
     (N-1)/N, ring remaps 1/(N+1), ~150 vnodes, bounded load 1.25x mean)
   ALWAYS pair a load metric with an ERROR metric or you route toward broken
   health: liveness = process ONLY | readiness = deps, hysteresis
     detect = interval x threshold (ALB default 60 s); fail OPEN when all down
   drain: unready -> sleep for propagation -> finish in flight -> GOAWAY -> exit
     grace MUST exceed p99.9 duration
   TLS: RSA-2048 1.5k hs/s/core, ECDSA P-256 10k, resumption ~1/20
     cert expiry = correlated total outage; alert at 30 d and 7 d
   failover ladder: client ms -> LB 3-30 s -> anycast 5-60 s -> DNS TTL+minutes
   admission: reject at CDN 0.01 ms > LB 0.05 > app 3-10 > DB 20+; 429+Retry-After
--------------------------------------------------------------------------------
 CDN
   origin fraction = (1-h_edge)(1-h_shield); 0.90/0.80 -> 2% reaches origin
   no shield: 300 PoPs each miss the same object -> a CDN can RAISE origin load
   hashed asset  public,max-age=31536000,immutable
   index.html    no-cache          authed  private,no-store   money  no-store
   swr=N -> instant response + ONE background refresh (= T38's coalescing)
   key = scheme+host+path; every dimension divides hit ratio
     Vary: Accept-Encoding ok | Language risky | Cookie/User-Agent fatal
   rename > purge: content hash is instant and free; purge is 5-90 s and paid
   RTT: TLS1.2 3 | TLS1.3 2 | resumed 1 | QUIC 1 | QUIC resumed 0 (replayable)
   10k viewers x 5 Mbps = 50 Gbps = 22.5 TB/h ~ $1,900/h
   keep a TESTED origin bypass hostname -- the CDN is a global SPOF
================================================================================
```

CONSOLIDATED CHEATSHEET — PARTS 4–6

```
================================================================================
 CACHE + DATABASE SCALING + QUEUES                              Topics 31-72
================================================================================
 CACHE
   T = t_cache + (1-h) t_db     backend_qps = (1-h) x front_qps
   0.4 + (1-h)25: h=.90 2.9 ms | .95 1.65 | .99 0.65  (10x less DB load)
   cache-aside default (fails safe). write-behind loses data. write-around for
     write-heavy. TTL + rand(0,20%) ALWAYS.
   maxmemory + allkeys-lru (lfu if batch jobs scan cold keys); volatile-* with
     no TTLs = OOM with free RAM
   single-threaded: KEYS/HGETALL on 1 M keys = 200 ms of TOTAL blocking; SCAN
   stampede: SET lock NX EX / XFetch / background refresh
   penetration: negative TTL 30 s + Bloom (m = -n ln p/(ln2)^2, k=(m/n)ln2)
   invalidate: TTL floor + event DEL or version key INCR (O(1) for all derived)
   race: reader reads DB, writer commits+DELs, reader SETs stale -> wrong for a
     full TTL, self-heals at expiry. Fix: short TTL, SET NX, or CDC.
   cluster: slot=CRC16 mod 16384; multi-key needs {hash tags}; replicas async
   DO NOT CACHE: read-once keys, write-heavy, money/inventory, or when the DB
     cannot survive h=0 (cold start is a stable outage)
--------------------------------------------------------------------------------
 DATABASE
   LADDER: index -> vertical -> pool -> cache -> replicas -> federate -> shard
     one index = 2,333x rows examined; one shard split = 8x. Do them in order.
   pool: useful ~ 2 x cores + spindles; 48 apps x 25 = 1,200 wanted -> 40 seen
     transaction mode breaks search_path/temp tables/advisory locks
   replication: async (tail loss) | semi-sync (+1 RTT) | sync (writes STOP)
     lag 0.2 s normal, minutes on batch; replicas scale READS only, ceiling ~1/w
     read-your-writes: sticky 5 s | LSN/GTID fence (correct) | write-through
   failover: detect -> elect -> highest LSN -> FENCE -> promote -> reroute (~25 s)
     quorum odd, >= 3. Fence BEFORE promote or split brain.
   shard: usable = min(N, 1/largest_share). created_at = worst key.
     hash even/no ranges | range scans/skews | directory placement/lookup
     reshard: dual-write -> backfill -> verify(days) -> cutover 1/10/100% -> stop
   after sharding: app-side joins, scatter-gather latency = MAX of shards,
     2PC (3-10x slower) or SAGA (no isolation -- name the state in the UI)
   derived state: OUTBOX in the same transaction, or CDC on binlog/WAL
   multi-region: single write region (right answer 90%) | partition by region |
     active-active (LWW = silent loss; CRDTs never enforce "only N exist")
--------------------------------------------------------------------------------
 QUEUES
   L = lambda W;  ETA = depth/(mu - lambda);  mu <= lambda -> never drains
   sync workers = peak x job_time; async = mean x job_time (5.7x cheaper)
   task queue (delete, per-msg retry) | log (retained, REPLAYABLE) | pub/sub
     (lossy) | ack BEFORE = at-most-once, AFTER = at-least-once. No third option.
   effectively-once = at-least-once + idempotent consumer
     INSERT processed(idem_key) PK | UPDATE ... WHERE status='pending'
     key identifies the ATTEMPT; window >= worst DLQ replay
   Kafka: order only within a partition; classic group parallelism <= partitions;
     share groups (4.2) = per-record ack, elastic consumers, NO order
     acks=all + min.insync.replicas=2 for anything that matters
   visibility timeout > p99.9 job duration (else concurrent double-run)
   retry: random(0, min(cap, base 2^n)) FULL JITTER; budget <= 10% of successes
     never retry 400/401/403/404/422. DLQ depth > 0 = ALERT.
   alert on oldest-message AGE and the sign of (mu - lambda), never depth alone
================================================================================
```

CONSOLIDATED CHEATSHEET — PARTS 7–9

```
============================================================================
 OBJECT STORAGE + OBSERVABILITY + RESILIENCE                    Topics 73-105
============================================================================
 OBJECT STORAGE
   no rename (copy+delete), no partial write, no locks, no directories
   key: {hash4}/{row_id}/{uuid} -- entropy FIRST (timestamp prefix = 503 SlowDown
     at ~3,500 PUT/s per prefix; splits take 30-60 min)
   6+3 erasure = 1.5x overhead, survives 3 losses. 11 nines covers DISKS only --
     not your DELETE, a bad overwrite, stolen credentials or a region outage
   strong read-after-write since Dec 2020; still last-writer-wins on concurrent
     PUT -> If-None-Match: * (create-only) / If-Match: etag (CAS), 412 on conflict
   NEVER proxy uploads: 900/min x 12 MB @ 6 Mbit = 240 blocked workers
     presign one key, size cap, content-type, SERVER-generated key
   multipart: Complete = the atomic commit; ETag != MD5; ALWAYS set
     AbortIncompleteMultipartUpload (invisible permanent bill)
   IA/Glacier bill a 128 KB MINIMUM per object -> small files cost MORE cold
   derivatives EXPIRE (regenerable); originals tier down; archives leave the
     read path (12-48 h)
   two stores one truth: create row(pending)->object->row(ready);
     delete row(deleting)->object->row.  Reconcile weekly from the INVENTORY
     manifest: object-no-row = orphan (delete-path bug, GDPR risk);
     row-no-object = broken link (users see 404)
   versioning WITHOUT NoncurrentVersionExpiration = 4-10x bill, flat object count
----------------------------------------------------------------------------
 OBSERVABILITY
   metrics THAT | traces WHERE | logs WHAT | profiles WHY -- joined by trace_id
   cardinality = product of label values. + user_id(12k) = 6.65 BILLION series.
     route TEMPLATE never path; identity belongs in traces/logs
   never avg() a percentile: sum buckets, then histogram_quantile
   sampling: head is cheap and keeps only 2% of ERRORS; tail keeps 100% of
     errors + slow + 2% baseline, needs trace-id-aware routing and
     span_rate x decision_wait x size of buffer
   logs: 100% errors + 1-5% successes = ~42x cheaper, nothing lost
   collector = the seam: swap vendor, redact, sample centrally.
     ALWAYS memory_limiter + bounded queue.
   RED per endpoint, USE per resource; SATURATION is the leading indicator
   SLO: 99.9 = 43.2 min/30 d | partial = impact_fraction x duration
     serial deps multiply: 0.9995^5 = 99.75%
     burn = err_ratio/(1-SLO): 14.4 over 1h AND 5m -> page; 6 over 6h AND 30m ->
     page; 3 over 24h AND 2h -> ticket
   page on symptoms only; every alert links a runbook; > 2 pages/shift = broken
   /live = process ONLY (DB check = restart storm). Synthetics + RUM see what
     your access log cannot. Telemetry should be 5-15% of infra spend.
----------------------------------------------------------------------------
 RESILIENCE
   timeout budget strictly decreasing; propagate a DEADLINE, not a timeout
     inverted budget = 50 s of orphan work x 1,400/s = 70,000 in flight
   retries multiply: 2x3x2x3 = 36x. METASTABLE: retries sustain the overload
     after the cause is gone -- exit only by REMOVING load
   breaker: >=50% fail AND >=20 reqs -> OPEN 30 s -> half-open probes
     turns a 2 s hold into 0.1 ms; bulkheads stop recs from eating checkout
   shed by class at the EDGE: goodput 4,000 vs 0 under 11,000 arrivals
   thundering herd = one bug in six places -> jitter, coalesce, stagger,
     rate-limit the recovery
   gray failure: per-instance p99 and SPREAD (max/median), client-side success
     rate, deep /ready. Response: eject fast.
   limits: token bucket (burst C, sustain R); central INCR on a NOEVICTION
     instance; 429 + Retry-After or you built the retry storm
   scale-up lag ~4.5 min -> run at 50-65%, schedule known peaks, scale on
     queue AGE not CPU; never autoscale into a fixed dependency
============================================================================
```

CONSOLIDATED CHEATSHEET — PARTS 10–11

```
================================================================================
 PLATFORM + THE DESIGNS                                          Topics 106-118
================================================================================
 PLATFORM
   north-south (authn, quotas, rate limit) -> API gateway
   east-west (mTLS, retry, breaker, telemetry) -> mesh or a shared client lib
   mesh pays off at 30+ services / several languages / mTLS mandate;
     costs +0.3-1 ms per hop and 50-150 MB + 0.1-0.5 core PER POD
   ONE layer owns retries (mesh + lib + gateway all retrying = 36x)
   nginx: least_conn | keepalive 64 + http_version 1.1 + Connection ""
     (no keepalive = +2 RTT: 4 ms of work becomes 5.2 ms)
     proxy_read_timeout < caller's | proxy_next_upstream_tries 2 (unbounded =
     LB-generated retry storm) | client_max_body_size (default 1m = silent 413)
   HAProxy slowstart 30s = staggered recovery for a cold instance
   k8s path: cloud LB -> ingress -> ClusterIP -> EndpointSlice(READY) -> pod
     readiness = ROUTING, liveness = RESTART. preStop sleep or you drop
     in-flight requests every deploy. CPU limit -> CFS throttling: p99 spikes
     in ~100 ms multiples (container_cpu_cfs_throttled_seconds_total)
   deploys: canary 1/5/25/100 with AUTOMATED abort (err > 1.5x or p99 > 1.3x)
     flags decouple deploy from release -> rollback in seconds
     EXPAND-CONTRACT: never ship a schema change with the code that needs it
   managed multiples: DB 2.5-4x, cache 2-3x, Kafka 3-5x, +0.2-0.5 FTE if self
     hosted. Managed does NOT buy shard key, consistency level or a TESTED restore
   RPO/RTO: single-AZ 24h/4-8h 1.0x | multi-AZ 0/30-90 s 1.4x <- default
     warm 1-5m/10-30m 1.7x | hot <1m/1-5m 2.2x | active-active 0/seconds 2.5x+
     failover forgets: DNS TTL floor, standby CAPACITY, cold caches, async
     bucket/queue replication, and the runbook hosted in the dead region
   IaC: plan in CI AND on a schedule; break-glass must auto-file the commit
--------------------------------------------------------------------------------
 THE METHOD  requirements -> ESTIMATION -> API -> data model -> HLD -> deep
   dive -> "at 10x, X breaks first, because Y" + the SPOF
   every choice gets a number and a stated cost; say when one DB suffices
--------------------------------------------------------------------------------
 THE FIVE DESIGNS, one line of design + one of failure
   SHORTENER  a cache with a DB behind it. random base62 + UNIQUE + retry;
     302 if you count clicks; clicks -> queue.  10x: shard by hash(code)
     (every query carries the key = perfect shardability)
   FEED  fan-out on write (185k-560k inserts/s) + PULL for celebrities,
     because follower counts are power-law. Timelines are a READ MODEL.
     10x: raise the pull threshold, shrink stored timeline
   CHAT  ack on DURABILITY (seq -> store -> fan out); client_msg_id =
     idempotency; presence in a SEPARATE TTL store (TTL expiry = offline).
     Connection tier is MEMORY-bound.  10x: routing table becomes the hot spot
   VIDEO  presigned multipart -> event -> transcode queue; manifest short TTL,
     segments immutable. The bill is EGRESS ($36k/day) -- bitrate beats servers.
     10x: object COUNT (243 M/day) makes lifecycle real work
   SEATS  UPDATE ... WHERE state='free' OR hold_expires_at < NOW();
     affected_rows 1 = yours. NEVER select-then-update. ~500 attempts/s per row
     (250 on distributed SQL). Do not cache money. Shard by trip_id.
     10x: primary write throughput + the payment gateway's quota
================================================================================
```

MASTER CHEATSHEET — ALL 118 TOPICS

```
================================================================================
 HIGH LEVEL DESIGN -- THE WHOLE BOOK ON TWO CARDS                      (1/2)
================================================================================
 THE THREE QUESTIONS, asked of every component you add
   1. WHAT NUMBER justifies it?      QPS, GB, ms, $/month -- shown, not implied
   2. WHAT DOES IT COST?             latency, consistency, an operator, a bill
   3. WHAT FAILS, AND HOW DOES IT LOOK?  the symptom, not the cause
   A box that cannot answer all three is decoration. Delete it.
--------------------------------------------------------------------------------
 THE FIVE FORMULAS WORTH MEMORISING
   QPS      = MAU x DAU/MAU x actions/day / 86,400, then x peak factor 3
   cache    = t_cache + (1-h) x t_db ;  backend = (1-h) x front
   queue    = L = lambda W ;  ETA = depth/(mu-lambda) ;  mu<=lambda -> never
   avail    = serial multiply ; parallel 1-(1-A)^n ; fanout tail 1-(1-p)^n
   shard    = usable capacity = min(N, 1/largest_key_share)
--------------------------------------------------------------------------------
 THE FIVE NUMBERS WORTH MEMORISING
   same-AZ RTT 0.5 ms | cross-region 110-130 ms | NVMe read ~100 us
   cache GET 0.4 ms | indexed DB point query 0.2-2 ms, unindexed = seconds
   one Valkey node ~120k ops/s | one Kafka partition 10-50 MB/s
   one contended DB row ~500 txn/s | one Postgres primary ~2-5k writes/s
   99.9% = 43 min/month | egress ~$0.09/GB | storage ~$0.023/GB/month
--------------------------------------------------------------------------------
 SECTION MAP
   1-8    foundations: the arithmetic that chooses the architecture
   9-20   load balancer: L4/L7, algorithms, health, drain, TLS, admission
   21-30  CDN: hierarchy, headers, keys, invalidation, edge, protocol, economics
   31-43  cache: hit-ratio math, patterns, eviction, stampede, invalidation
   44-59  database: the ladder, pooling, replication, failover, sharding, CDC
   60-72  queues: load levelling, semantics, idempotency, Kafka, retries, DLQ
   73-82  object storage: keys, durability, CAS, uploads, tiering, orphans
   83-95  observability: signals, cardinality, sampling, RED/USE, SLO, alerts
   96-105 resilience: budgets, amplification, breakers, shedding, herds, chaos
   106-112 platform: proxies, k8s path, deploys, managed vs self, RPO/RTO, IaC
   113-118 designs: method, shortener, feed, chat, video, seats
--------------------------------------------------------------------------------
 THE FOUR TEMPLATES WORTH MEMORISING
   1 CACHE-ASIDE            2 IDEMPOTENT CONSUMER
     hit? return              BEGIN
     row = db.get()             INSERT processed(idem_key)  -- dup = done, ack
     if null: setex 30 'null'   side_effect()
     setex ttl+rand(20%)      COMMIT; ack   -- ack AFTER commit, always
   3 CONDITIONAL CLAIM      4 OUTBOX
     UPDATE t SET state=..    BEGIN
      WHERE id=? AND           INSERT business_row
       (state='free' OR        INSERT outbox(idem_key, topic, payload)
        expires < NOW());    COMMIT
     affected_rows 1 = mine   relay: SELECT ... FOR UPDATE SKIP LOCKED
--------------------------------------------------------------------------------
 THE UNIVERSAL TEST MATRIX (run against any design)
   empty          zero users, zero rows, cold cache -- does anything divide by 0
   one            single item, single shard, single AZ
   duplicate      the same message twice; the same request retried
   concurrent     two writers on one row/key/object; two consumers on one job
   10x            which component saturates FIRST, and what is the symptom
   cold start     cache empty, pools empty, JIT cold -- can the origin survive
   partial        one AZ, one shard, one dependency gone -- degrade or die
   restore        delete it and put it back; time the whole prefix, not one item
================================================================================
```

```
================================================================================
 HIGH LEVEL DESIGN -- THE WHOLE BOOK ON TWO CARDS                      (2/2)
================================================================================
 THE SAME IDEA, WEARING TEN HATS (this is what makes 118 topics learnable)
   CONDITIONAL WRITE / CAS
     If-Match on an object (76) | UPDATE WHERE state=... (63, 118)
     version key INCR (40) | optimistic retry in distributed SQL (57)
   JITTER
     TTLs (35) | backoff (68) | health checks (15) | cron (100) | reconnects (116)
   COALESCE ONE REFILL
     stale-while-revalidate (23) | SET NX rebuild lock (38) | singleflight (100)
   AT-LEAST-ONCE + IDEMPOTENCE
     queue consumers (62,63) | outbox relay (59) | webhook retries | fan-out (115)
   DERIVED STATE, REBUILDABLE FROM A LOG
     read models/CQRS (55) | timelines (115) | search index | cache (40) | CDC (59)
   ONE OWNER FOR A GLOBAL CONSTRAINT
     seat rows (118) | inventory (58) | leader election (49) | sequence (116)
   SEPARATE THE HOT MINORITY
     celebrity pull path (115) | hot key fanout (39) | hot shard (51) | tenant
     quotas (71,103) | priority classes (99)
   BOUND EVERYTHING
     timeouts (96) | retry budget (97) | pool size (45) | prefetch (66)
     | queue depth (67) | cardinality (84) | blast radius (109)
--------------------------------------------------------------------------------
 WHEN THE ANSWER IS "DON'T"
   don't cache      read-once keys, write-heavy, money/inventory, or when the
                    origin cannot survive h=0                        (43,118)
   don't shard      before indexes, pooling, cache and replicas       (44)
   don't queue      when the user waits for the result, or mu < lambda (72)
   don't mesh       under ~15 services in 1-2 languages               (106)
   don't multi-     until multi-AZ is excellent; active-active adds
     region         conflict resolution to every write               (58,111)
   don't retry      non-idempotent writes, 4xx, or without a budget   (68,97)
   don't alert      on causes, on CPU, or without a runbook           (94)
--------------------------------------------------------------------------------
 THE SYMPTOM INDEX (what you will actually see at 3am)
   one row wrong, self-heals after exactly the TTL      cache race        41
   OOM error while RAM is free                          volatile-lru      36
   p99 spikes whenever an admin saves                   KEYS in a helper  37
   hit ratio collapses at 02:04 nightly                 batch scan + LRU  36
   duplicates correlate with job DURATION               visibility timeout 66
   one partition's lag climbs, others at zero           poison record     69
   site down after a cache restart, and stays down      cache load-bearing 43
   failing component sees 30x its normal request rate   retry amplification 97
   p99 spikes in multiples of ~100 ms                   CFS throttling    108
   users complain, every dashboard is green             gray failure      102
   storage grows, object count is flat                  noncurrent versions 82
   bill goes UP after a cost optimisation               128 KB minimum    79
   metrics backend OOMs an hour after a deploy          cardinality       84
   "it didn't save", only under load                    replica lag       46,48
   two people bought seat 14A                           select-then-update 118
--------------------------------------------------------------------------------
 THE INTERVIEW SENTENCES THAT SCORE
   "At this scale one Postgres and a cache is enough; it stops being enough at
    roughly N QPS, and then I would do X."
   "That buys us A and costs us B; the failure mode I am accepting is C."
   "X breaks first at 10x, and the symptom will be Y."
   "This is the same discipline as the earlier decision about Z."
   "I would not cache this, because it is money under contention."
================================================================================
```

PATTERN RECOGNITION TABLE

The phrasing on the left is what a requirement, a ticket or an interviewer actually says. The right-hand column is where to start — not the whole answer, but the topic that owns the decision.

When the problem says…	Reach for
"how many servers do we need"	peak QPS chain, then capacity per node — T3, T104
"it's slow for users in another country"	RTT is the floor: CDN/edge, not more CPU — T2, T21
"reads vastly outnumber writes"	cache, then read replicas — T31, T46
"static assets", "images", "video"	object storage + CDN, content-hash keys — T25, T80
"the same expensive query over and over"	cache the result, not the rows — T32
"stale data is unacceptable here"	do not cache; conditional write on the source — T43, T118
"a few keys get most of the traffic"	Zipf skew: hot-key fanout or in-process cache — T39
"everything expired at once"	TTL jitter + coalescing — T35, T38
"the database is at 100% CPU"	EXPLAIN first, index first — T44
"too many connections"	transaction-mode pooler — T45
"I saved it and it showed the old value"	replica lag: LSN fence or sticky window — T48
"writes exceed one machine"	shard — but only after the ladder — T44, T51
"which shard key"	the key every hot query carries; check 1/N skew — T51
"we need to add a shard"	consistent hashing or a directory, never mod N — T52
"migrate without downtime"	dual-write → backfill → verify → cutover — T53, T109
"a transaction across two services"	saga with a named intermediate state — T54
"this join is too slow at scale"	read model / CQRS, rebuildable from a log — T55
"update the cache and the DB together"	you cannot: outbox or CDC — T59, T41
"a burst we cannot provision for"	queue + workers sized for the mean — T60
"another team needs these events too"	a log, not a task queue — T61
"it must happen exactly once"	at-least-once + idempotency key — T62, T63
"these must be processed in order"	partition key = the entity; accept the parallelism cap — T69
"we need more workers than partitions"	share groups, and give up ordering — T65
"the queue is backing up"	oldest-age + sign of (μ−λ); shed or scale — T67
"users upload large files"	presigned multipart direct to storage — T77, T78
"storage costs too much"	lifecycle tiers, but check the 128 KB minimum — T79
"we found files nobody references"	inventory reconciliation; fix the delete order — T81
"someone deleted the bucket"	versioning + object lock + a tested restore — T82
"which user caused this"	traces and logs; never a metric label — T84
"we can't afford to keep all traces"	tail sampling: all errors, 2% baseline — T88
"how do we know it's working"	SLI/SLO + burn-rate alerts — T92, T93
"we get paged too much"	symptom alerts only, with runbooks — T94
"it got slower after the deploy"	profile diff by version — T90
"one slow dependency took us down"	timeout budget + bulkhead + breaker — T96, T98

"it didn't recover when the cause cleared"	metastable: remove load, ramp back — T97, T100
"we're over capacity right now"	shed by class at the edge; goodput — T99
"healthy dashboards, unhappy users"	gray failure: per-instance spread, synthetics — T102, T95
"one customer is using everything"	per-tenant quota + fair queueing — T71, T103
"traffic doubles in ninety seconds"	headroom, not autoscaling (4.5 min lag) — T104
"we drop requests on every deploy"	readiness, preStop, drain — T108, T18
"we need to survive a region loss"	fix RPO/RTO first, then pick the tier — T111
"can we roll this back"	expand-contract + feature flags — T109
"only N of these exist"	one owner, one row, conditional update — T118, T58
"one post reaches millions of feeds"	hybrid push/pull — T115
"millions of open connections"	memory-bound tier + routing table — T116

REVISION TRACKER

Fill this in by hand as you go. The schedule is Day 0 (first learn), +1 day, practice set, +7 days, +30 days, +90 days. A part is "done" when you can reproduce its summary card from memory and answer group B without the key — not when you have read it. If a +30 review takes more than ten minutes, you skipped the practice set.

Topic group	First	Rev 1 +1d	Practice	Rev 2 +7d	Rev 3 +30d	Rev 4 +90d
1. Foundations (1–8)						
2. Load balancer (9–20)						
3. CDN and edge (21–30)						
4. Cache layer (31–43)						
5. Database scaling (44–59)						
6. Queue systems (60–72)						
7. Object storage (73–82)						
8. Observability (83–95)						
9. Resilience (96–105)						
10. Platform layer (106–112)						
11. The designs (113–118)						
Master cheatsheet						

Drill (quarterly)	Q1	Q2	Q3	Q4	Time taken	Runbook fixed?
Database failover (T49)						
Restore from versioning (T82)						
Cache cold start (T43)						
Regional failover (T111)						
Game day (T105)						

AFTERWORD

Nearly everything in this book is one of three moves: put a copy closer to the reader, put a buffer between two rates that do not match, or put a single owner in front of something that must not be done twice. Caches, replicas, CDNs and read models are the first. Queues, pools, buffers and headroom are the second. Row locks, partition leaders, consistent-hash slots and shard keys are the third.

What separates a design that survives from one that merely works is the second half of every cost strip in these pages — the failure mode you accepted. A cache accepts staleness. Async replication accepts a lost tail. A saga accepts a visible intermediate state. Load shedding accepts refusing some users so the rest are served. None of those is a flaw; each is a decision, and the systems that fail badly are usually the ones where nobody knew they had made it.

The practice sets matter more than the prose. You will not remember that a visibility timeout shorter than p99.9 causes concurrent double-execution because you read it here; you will remember it because you set one to 30 seconds, watched two workers claim the same message, and saw the log lines overlap. Break things on purpose while it is cheap.

Finally, the strongest sentence you can say in a design review remains the one that argues against the design: *one database and a cache are enough here, and here is the number at which that stops being true.* Everything in Parts 2–11 exists for the moment after that number is crossed — and not one moment before.